

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

NGUYỄN NGỌC MINH

LƯƠNG CÔNG DUẤN

BÀI GIẢNG
HỆ THỐNG NHÚNG

HÀ NỘI – 10.2019

MỤC LỤC

MỤC LỤC	2
DANH MỤC CÁC HÌNH.....	5
DANH MỤC BẢNG BIỂU	9
CHƯƠNG 1 - GIỚI THIỆU CHUNG VỀ HỆ THỐNG NHÚNG	10
1.1 Khái niệm Hệ thống nhúng (Embedded system)	10
1.2 Lịch sử phát triển của hệ thống nhúng	11
1.3 Các đặc điểm hệ thống nhúng	11
1.3.1 Giao diện.....	12
1.3.2 Kiến trúc CPU.....	12
1.4 Kiến trúc điển hình của hệ thống nhúng	14
Một số kiến trúc phần mềm hệ thống nhúng.....	15
1.5 Phân loại các hệ thống nhúng.....	18
1.6 Phạm vi ứng dụng của hệ thống nhúng.....	18
1.7 Các yêu cầu về kĩ năng trong thiết kế hệ thống nhúng	18
1.7.1 Quản lý, tích hợp, thiết kế hệ thống:.....	21
1.7.2 Thiết kế, phát triển phần mềm ứng dụng	22
1.7.3 Thiết kế firmware.....	22
1.7.4 Thiết kế mạch, PCB:	23
1.7.5 Thiết kế vi điện tử: Linh kiện, IP, IC, phụ kiện	23
Câu hỏi ôn tập	25
CHƯƠNG 2: CÁC THÀNH PHẦN HỆ THỐNG.....	26
2.1 Các thành phần phần cứng.....	26
2.1.1 Bộ xử lý nhúng.....	26
2.1.2 Bộ nhớ.....	35
2.1.3 Bảng mạch Vào/Ra	37
2.1.5 Hệ thống Bus.....	46
2.2 Các thành phần phần mềm của hệ thống:	56
2.2.1. Trình điều khiển thiết bị.....	56
2.2.2. Hệ điều hành thời gian thực	58
2.2.3. Middleware	59
2.2.4 Phần mềm ứng dụng	62
Câu hỏi ôn tập	64
CHƯƠNG 3 - HỆ ĐIỀU HÀNH THỜI GIAN THỰC DÙNG CHO CÁC HỆ THỐNG NHÚNG	66
3.1 Yêu cầu chung cho các hệ điều hành thời gian thực.....	66
3.2 Các chức năng chính của phần lõi trong hệ điều hành thời gian thực	67
3.2.1. Kernel.....	67
3.2.2 Tác vụ và Multi-tasking	68
3.3.3 Lập lịch thời gian thực (Real-time Scheduling).....	71
3.3.4 Đồng bộ.....	73
3.2.5 HAL (Hardware Abstraction Layer).....	75
3.3 Giới thiệu các hệ điều hành thời gian thực	76

3.3.1 FreeRTOS:	76
3.3.2 Windows CE:	88
3.3.3 Hệ điều hành Embedded Linux:	90
3.3.4 Hệ điều hành uCLinux:	91
Câu hỏi ôn tập	92
CHƯƠNG 4: THIẾT KẾ VÀ CÀI ĐẶT CÁC HỆ THỐNG NHÚNG	93
4.1 Thiết kế hệ thống	93
4.1.1 Xác định yêu cầu.....	93
4.1.2 Đặc tả	95
4.1.3 Phân hoạch phần cứng - phần mềm	98
4.1.4 Thiết kế hệ thống	109
4.2 Cài đặt và thử nghiệm hệ thống nhúng	129
Câu hỏi ôn tập	132
CHƯƠNG 5: PHÁT TRIỂN HỆ THỐNG NHÚNG DỰA TRÊN HỆ VI XỬ LÝ NHÚNG	133
5.1 Giới thiệu chung.....	133
5.2 Kiến trúc của hệ vi xử lý nhúng ARM	133
Lõi ARM.....	133
Thanh ghi và các chế độ hoạt động.....	134
Pipeline	136
Cấu trúc bus:	138
Tập lệnh ARM	138
Các lệnh xử lý dữ liệu	139
Các lệnh rẽ nhánh.....	141
Các lệnh chuyển dữ liệu Load- Store.....	142
Tập lệnh Thumb.....	147
5.3 Giới thiệu về dòng vi xử lý ARM Cortex và ARM Cortex M3	148
Các dòng ARM Cortex	148
Vi điều khiển STM32F1	150
Lập trình các thanh ghi	153
Chuẩn CMSIS	153
Thư viện Standard Peripheral Library (SPL).....	155
Quy trình lập trình chương trình STM32F1.....	157
Bộ thanh ghi RCC (Register Clock Control)	158
Lập trình điều khiển IO với STM32F1 sử dụng thanh ghi	159
Lập trình điều khiển IO với STM32F1 sử dụng SPL	168
Lập trình SysTick.....	175
Lập trình điều khiển Timer	180
Lập trình điều chế độ rộng xung - PWM (Pulse-width modulation)	186
Lập trình điều khiển bộ UART	191
Lập trình điều khiển SPI	199
Lập trình ADC	211
Lập trình DAC	216
Lập trình với FreeRTOS	220
5.4 Thiết lập hệ điều hành nhúng trên nền ARM.....	240
Firmware và Bootloader	240

Hệ thống file (Filesystem)	241
Thiết lập nhân (kernel).....	242
PHỤ LỤC	247
TÀI LIỆU THAM KHẢO	276

PDF

DANH MỤC CÁC HÌNH

Hình 1. 1: Mô hình chung hệ thống nhúng	15
Hình 1. 2: Cấu trúc của thiết bị điện tử.....	20
Hình 1. 3: Các mảng công việc trong thiết kế hệ thống nhúng.....	21
Hình 1. 4: Các yêu cầu về kỹ năng tương ứng.....	24
Hình 2. 1: Bảng mạch Encore 400 của Ampro	26
Hình 2. 2: Các thao tác ISA đơn giản	28
Hình 2. 3: Ví dụ bảng mạch TV tương tự với sự thi hành bộ điều khiển ISA.....	28
Hình 2. 4: Board ví dụ với điện thoại di động thực hiện kỹ thuật số ISA đường dữ liệu ..	29
Hình 2. 5: Board ví dụ với máy quay kỹ thuật số FSMD ISA.....	29
Hình 2. 6: Ví dụ thực hiện JVM ISA	30
Hình 2. 7: Ví dụ thực hiện CISC ISA	31
Hình 2. 8: Ví dụ thực hiện RISC ISA	31
Hình 2. 9: Ví dụ thực hiện SIMD ISA	32
Hình 2. 10: Ví dụ thực hiện siêu vô hướng ISA	33
Hình 2. 11: Ví dụ thực hiện VLIW ISA.....	33
Hình 2. 12: Sự phân cấp bộ nhớ.....	36
Hình 2. 13: Ví dụ DIP	37
Hình 2. 14: Ví dụ SIMM 30 chân	37
Hình 2. 15: Ví dụ DIMM 168 chân.....	37
Hình 2. 16: Sơ đồ khối I/O cơ bản của kiến trúc Von Newman.....	38
Hình 2. 17: Các cổng và bộ điều khiển thiết bị điều khiển trên một bảng mạch nhúng ...	39
Hình 2. 18: Bảng mạch I/O phức tạp	39
Hình 2. 19: Bảng mạch I/O đơn giản.....	40
Hình 2. 20: Ví dụ sơ đồ truyền đơn giản	41
Hình 2. 21: Ví dụ sơ đồ truyền bán song công	41
Hình 2. 22: Ví dụ sơ đồ truyền song công	41
Hình 2. 23: Mô hình OSI	42
Hình 2. 24: Sơ đồ mạng nối tiếp	42
Hình 2. 25: Sơ đồ khối những thành phần nối tiếp	42
Hình 2. 26: Các tín hiệu RS-232 và đầu nối DB25.....	43
Hình 2. 27: Các tín hiệu RS-232 và đầu nối DB9.....	43
Hình 2. 28: Các tín hiệu RS-232 và đầu nối RJ45	44
Hình 2. 29: Hệ thống con I/O mẫu.....	44
Hình 2. 30: Phương tiện truyền tin dùng dây.....	44
Hình 2. 31: Phương tiện truyền tin không dây.....	45
Hình 2. 32: Giao diện cổng truyền tin tới I/O tấm bảng mạch khác	45
Hình 2. 33: Cấu trúc bus chung	46
Hình 2. 34: Kiến trúc MPC620 với cầu	47
Hình 2. 35: Sự phân xử song song tập trung động.....	49
Hình 2. 36: Sự phân xử trên nền tảng FIFO.....	49
Hình 2. 37: Sự phân xử trên nền quyền ưu tiên	49

Hình 2. 38: Sự phân xử tuần tự/chuỗi tập trung.....	50
Hình 2. 39: Sự phân xử phân tán qua sự tự chọn.....	50
Hình 2. 40: Bảng mạch mẫu TV	52
Hình 2. 41: Các điều kiện START và STOP của I2C.....	53
Hình 2. 42: Ví dụ truyền dữ liệu trên I2C.....	53
Hình 2. 43: Sơ đồ truyền dữ liệu hoàn chỉnh trên I2C.....	54
Hình 2. 44: Bus PCI.....	54
Hình 2. 45: IC tương thích chuẩn PCI	55
Hình 2. 46: Sơ đồ khung RS-232.....	58
Hình 2. 47: Middleware trong mô hình hệ nhúng.....	60
Hình 2. 48: Mô hình OSI và middleware.....	61
Hình 2. 49: Sơ đồ khối mô hình OSI, TCP/IP và mô hình hệ nhúng	62
Hình 2. 50: Sơ đồ khối mô hình TCP/IP và các giao thức.....	62
Hình 2. 51: Lớp ứng dụng và mô hình hệ nhúng.....	63
Hình 3. 1: Kernel trong hệ thống	67
Hình 3. 2: Cấu trúc của tác vụ.....	70
Hình 3. 3: Các trạng thái của tác vụ.....	72
Hình 3. 4: Cơ chế truyền tin trong mailbox	75
Hình 3. 5: Cấu trúc thư mục FreeRTOS	79
Hình 4. 1: Thiết kế trên nền (platform).....	94
Hình 4. 2: Sơ đồ trạng thái với ngoại lệ k.....	96
Hình 4. 3: Tổng quan về phân hoạch phần cứng/phần mềm	100
Hình 4. 4: Hợp nhất các nút nhiệm vụ được ánh xạ đến cùng một thành phần phần cứng	102
Hình 4. 5: Đồ thị nhiệm vụ	106
Hình 4. 6: Không gian thiết kế cho phòng thí nghiệm âm thanh.....	109
Hình 4. 7: Môi trường phát triển.....	110
Hình 4. 8: IDE.....	111
Hình 4. 9: Ví dụ trình mô phỏng PSpice CAD	112
Hình 4. 10: Ví dụ mạch PSpice CAD	112
Hình 4. 11: Sơ đồ biên dịch	114
Hình 4. 12: Các bước biên dịch/liên kết và tập tin đối tượng kết quả, thực hiện trong C	115
Hình 4. 13: Sơ đồ sự phiên dịch.....	116
Hình 4. 14: Sơ đồ giải thích	128
Hình 4. 15: Sơ đồ giải thích	128
Hình 4. 16: Ma trận mô hình thử nghiệm	130
Hình 5. 1: Cấu trúc của thanh ghi trạng thái chương trình hiện tại	134
Hình 5. 2: Các thanh ghi của lõi ARM	135
Hình 5. 3: Các chế độ hoạt động và các thanh ghi.....	136
Hình 5. 4: Dòng chảy lệnh 3 tác vụ áp dụng trong trường hợp 1 lệnh có nhiều chu kỳ máy	137
Hình 5. 5: Sơ đồ bộ nhớ của ARM M3.....	150
Hình 5. 6: hiệu năng, chức năng, hoạt động của các dòng vi điều khiển STM32 Cortex M	151

Hình 5. 7: Vai trò của CMSIS và SPL trong phát triển phần mềm cho vi điều khiển ARM	156
Hình 5. 8: Kiến trúc các ngoại vi của STM32	157
Hình 5. 9: Thanh ghi RCC_AHBENR.....	158
Hình 5. 10: RCC_APB2ENR.....	158
Hình 5. 11: Thanh ghi RCC_APB1ENR	159
Hình 5. 12: Cấu trúc của các chân điều khiển thông dụng	160
Hình 5. 13: Cấu trúc của các chân IO hỗ trợ giao tiếp 5V	161
Hình 5. 14: Thông tin cấu hình IO	161
Hình 5. 15: Các chế độ tốc độ đầu ra số	161
Hình 5. 16: Thanh ghi GPIOx_CRL	162
Hình 5. 17: Thanh ghi GPIOx_CRH.....	162
Hình 5. 18: Thanh ghi GPIOx_IDR.....	163
Hình 5. 19: Thanh ghi GPIOx_ODR	163
Hình 5. 20: Thanh ghi Lập/Xóa GPIOx_BSRR.....	164
Hình 5. 21: Thanh ghi Xóa GPIOx_BRR	164
Hình 5. 22: Thanh ghi khóa cấu hình GPIOx_LCKR.....	164
Hình 5. 23: Cấu hình các package sử dụng cho Project với KeilC 5.....	169
Hình 5. 24: Thông tin các gói sau khi cấu hình	171
Hình 5. 25: Hoạt động của SysTick	175
Hình 5. 26: Cấu trúc bộ Timer Advanced-Control	180
Hình 5. 27: Cấu trúc bộ Timer General-purpose	181
Hình 5. 28: Hoạt động của Timer	182
Hình 5. 29: Điều chế độ rộng xung PWM	186
Hình 5. 30: Thông tin cấu hình PWM cho các Timer.....	189
Hình 5. 31: Mức điện thế biểu diễn tín hiệu nhị phân	192
Hình 5. 32: Cổng DB25	193
Hình 5. 33: Cổng DB-9.....	193
Hình 5. 34: Kết nối tối thiểu giữa PC với vi điều khiển trên giao tiếp UART	194
Hình 5. 35: Định dạng 8-N-1	194
Hình 5. 36: Thông tin các chân giao tiếp UART	194
Hình 5. 37: Cấu hình thông tin các chân của giao tiếp UART	196
Hình 5. 38: Kết nối trong giao tiếp SPI	201
Hình 5. 39: Màn hình PCD8544 sử dụng giao tiếp SPI.....	207
Hình 5. 40: Sơ đồ khối bộ ADC	212
Hình 5. 41: Sơ đồ khối bộ DAC	216
Hình 5. 42: Kiến trúc RTOS	220
Hình 5. 43: Hoạt động của CPU và quan sát của người dùng với Multi Thread.....	221
Hình 5. 44: Các trạng thái của Task trong RTOS	221
Hình 5. 45: Cơ chế Round-Robin	222
Hình 5. 46: Cơ chế Priority Base	222
Hình 5. 47: Cơ chế Priority-based pre-emptive	223
Hình 5. 48: Sử dụng Signal Event trong đồng bộ thông tin các Task.....	224
Hình 5. 49: Trao đổi dữ liệu giữa các Task sử dụng Queue	225
Hình 5. 50: Trao đổi dữ liệu giữa các Task sử dụng Mail Queue.....	225
Hình 5. 51: Binary semaphore và Counting semaphore	226

Hình 5. 52: Sử dụng chung tài nguyên hệ thống với Mutex	227
Hình 5. 53: Thêm FreeRTOS vào Project.....	227



DANH MỤC BẢNG BIỂU

Bảng 2. 1: Các bộ xử lý và kiến trúc thực tế.....	27
Bảng 3. 1: So sánh giữa FreeRTOS và OpenRTOS	76
Bảng 4. 1: Các giải pháp khả dĩ của vấn đề IP đang được trình bày	103
Bảng 4. 2: Thời gian thực hiện của các nhiệm vụ từ T1 đến T5 trên các thành phần	107
Bảng 4. 3: Những công cụ gỡ lỗi.....	125
Bảng 5. 1: Bảng các chân của cổng DB-9	193
Bảng 5. 2: Các thông tin cấu hình giao tiếp SPI	203
Bảng 5. 3: Thông tin cấu hình các chân giao tiếp SPI	204
Bảng 5. 4: Các thông tin cấu hình ADC	214

CHƯƠNG 1 - GIỚI THIỆU CHUNG VỀ HỆ THỐNG NHÚNG

1.1 *Khái niệm Hệ thống nhúng (Embedded system)*

Hệ thống nhúng (Embedded system) là một thuật ngữ để chỉ một hệ thống có khả năng tự trị được nhúng vào trong một môi trường hay một hệ thống mẹ. Đó là các hệ thống tích hợp cả phần cứng và phần mềm phục vụ các bài toán chuyên dụng trong nhiều lĩnh vực công nghiệp: điện tử, viễn thông, công nghệ thông tin, tự động hoá điều khiển, quan trắc và truyền tin. Đặc điểm của các hệ thống nhúng là hoạt động ổn định và có tính năng tự động hoá cao.

Hệ thống nhúng thường được thiết kế để thực hiện một chức năng chuyên biệt nào đó. Khác với các máy tính đa chức năng, chẳng hạn như máy tính cá nhân, một hệ thống nhúng chỉ thực hiện một hoặc một vài chức năng nhất định, thường đi kèm với những yêu cầu cụ thể và bao gồm một số thiết bị máy móc và phần cứng chuyên dụng mà ta không tìm thấy trong một máy tính đa năng nói chung. Vì hệ thống chỉ được xây dựng cho một số nhiệm vụ nhất định nên các nhà thiết kế có thể tối ưu hóa nó nhằm giảm thiểu kích thước và chi phí sản xuất. Các hệ thống nhúng thường được sản xuất hàng loạt với số lượng lớn. Hệ thống nhúng rất đa dạng, phong phú về chủng loại. Đó có thể là những thiết bị cầm tay nhỏ gọn như đồng hồ kỹ thuật số và máy chơi nhạc MP3, hoặc những sản phẩm lớn như đèn giao thông, bộ kiểm soát trong nhà máy hoặc hệ thống kiểm soát các máy năng lượng hạt nhân. Xét về độ phức tạp, hệ thống nhúng có thể rất đơn giản với một vi điều khiển hoặc rất phức tạp với nhiều đơn vị, các thiết bị ngoại vi và mạng lưới được nằm gọn trong một lớp vỏ máy lớn.

Các thiết bị PDA hoặc máy tính cầm tay cũng có một số đặc điểm tương tự với hệ thống nhúng như các hệ điều hành hoặc vi xử lý điều khiển chúng nhưng các thiết bị này không phải là hệ thống nhúng thật sự bởi chúng là các thiết bị đa năng, cho phép sử dụng nhiều ứng dụng và kết nối đến nhiều thiết bị ngoại vi.

Cho đến nay, khái niệm hệ thống nhúng được nhiều người chấp nhận nhất là: hệ thống thực hiện một số chức năng đặc biệt có sử dụng vi xử lý. Không có hệ thống nhúng nào chỉ có phần mềm.

1.2 Lịch sử phát triển của hệ thống nhúng

Hệ thống nhúng đầu tiên là Apollo Guidance Computer (Máy tính Dẫn đường Apollo) được phát triển bởi Charles Stark Draper tại phòng thí nghiệm của trường đại học MIT. Hệ thống nhúng được sản xuất hàng loạt đầu tiên là máy hướng dẫn cho tên lửa quân sự vào năm 1961. Nó là máy hướng dẫn Autonetics D-17, được xây dựng sử dụng những bóng bán dẫn và một đĩa cứng để duy trì bộ nhớ. Khi Minuteman II được đưa vào sản xuất năm 1996, D-17 đã được thay thế với một máy tính mới sử dụng mạch tích hợp. Tính năng thiết kế chủ yếu của máy tính Minuteman là nó đưa ra thuật toán có thể lập trình lại sau đó để làm cho tên lửa chính xác hơn, và máy tính có thể kiểm tra tên lửa, giảm trọng lượng của cáp điện và đầu nối điện.

Từ những ứng dụng đầu tiên vào những năm 1960, các hệ thống nhúng đã giảm giá và phát triển mạnh mẽ về khả năng xử lý. Bộ vi xử lý đầu tiên hướng đến người tiêu dùng là Intel 4004, được phát minh phục vụ máy tính điện tử và những hệ thống nhỏ khác. Tuy nhiên nó vẫn cần các chip nhớ ngoài và những hỗ trợ khác. Vào những năm cuối 1970, những bộ xử lý 8 bit đã được sản xuất, nhưng nhìn chung chúng vẫn cần đến những chip nhớ bên ngoài.

Vào giữa thập niên 80, kỹ thuật mạch tích hợp đã đạt trình độ cao dẫn đến nhiều thành phần có thể đưa vào một chip xử lý. Các bộ vi xử lý được gọi là các vi điều khiển và được chấp nhận rộng rãi. Với giá cả thấp, các vi điều khiển đã trở nên rất hấp dẫn để xây dựng các hệ thống chuyên dụng. Đã có một sự bùng nổ về số lượng các hệ thống nhúng trong tất cả các lĩnh vực thị trường và số các nhà đầu tư sản xuất theo hướng này. Ví dụ, rất nhiều chip xử lý đặc biệt xuất hiện với nhiều giao diện lập trình hơn là kiểu song song truyền thống để kết nối các vi xử lý. Vào cuối những năm 80, các hệ thống nhúng đã trở nên phổ biến trong hầu hết các thiết bị điện tử và khuynh hướng này vẫn còn tiếp tục cho đến nay.

1.3 Các đặc điểm hệ thống nhúng

Hệ thống nhúng thường có một số đặc điểm chung như sau:

- Các hệ thống nhúng được thiết kế để thực hiện một số nhiệm vụ chuyên dụng chứ không phải đóng vai trò là các hệ thống máy tính đa chức năng. Một số hệ thống đòi hỏi ràng buộc về tính hoạt động thời gian thực để đảm bảo độ an toàn và tính ứng dụng; một số hệ thống không đòi hỏi hoặc ràng buộc chặt chẽ, cho phép đơn giản hóa hệ thống phần cứng để giảm thiểu chi phí sản xuất.
- Một hệ thống nhúng thường không phải là một khối riêng biệt mà là một hệ thống phức tạp nằm trong thiết bị mà nó điều khiển.

- Phần mềm được viết cho các hệ thống nhúng được gọi là firmware và được lưu trữ trong các chip bộ nhớ chỉ đọc (read-only memory) hoặc bộ nhớ flash chứ không phải là trong một ổ đĩa. Phần mềm thường chạy với số tài nguyên phần cứng hạn chế: không có bàn phím, màn hình hoặc có nhưng với kích thước nhỏ, bộ nhớ hạn chế. Sau đây, ta sẽ đi sâu, xem xét cụ thể đặc điểm của các thành phần của hệ thống nhúng.

1.3.1 Giao diện

Các hệ thống nhúng có thể không có giao diện (đối với những hệ thống đơn nhiệm) hoặc có đầy đủ giao diện giao tiếp với người dùng tương tự như các hệ điều hành trong các thiết bị để bàn. Đối với các hệ thống đơn giản, thiết bị nhúng sử dụng nút bấm, đèn LED và hiển thị chữ cỡ nhỏ hoặc chỉ hiển thị số, thường đi kèm với một hệ thống menu đơn giản.

Còn trong một hệ thống phức tạp hơn, một màn hình đồ họa, cảm ứng hoặc có các nút bấm ở lề màn hình cho phép thực hiện các thao tác phức tạp mà tối thiểu hóa được khoảng không gian cần sử dụng; ý nghĩa của các nút bấm có thể thay đổi theo màn hình và các lựa chọn. Các hệ thống nhúng thường có một màn hình với một nút bấm dạng cần điều khiển (joystick button). Sự phát triển mạnh mẽ của mạng toàn cầu đã mang đến cho những nhà thiết kế hệ nhúng một lựa chọn mới là sử dụng một giao diện web thông qua việc kết nối mạng. Điều này có thể giúp tránh được chi phí cho những màn hình phức tạp nhưng đồng thời vẫn cung cấp khả năng hiển thị và nhập liệu phức tạp khi cần đến, thông qua một máy tính khác. Điều này là hết sức hữu dụng đối với các thiết bị điều khiển từ xa, cài đặt vĩnh viễn. Ví dụ, các router là các thiết bị đã ứng dụng tiện ích này.

1.3.2 Kiến trúc CPU

Các bộ xử lý trong hệ thống nhúng có thể được chia thành hai loại: vi xử lý và vi điều khiển. Các vi điều khiển thường có các thiết bị ngoại vi được tích hợp trên chip nhằm giảm kích thước của hệ thống. Có rất nhiều loại kiến trúc CPU được sử dụng trong thiết kế hệ nhúng như ARM, MIPS, Coldfire/68k, PowerPC, x86, PIC, 8051, Atmel AVR, Renesas H8, SH, V850, FR-V, M32R, Z80, Z8 ... Điều này trái ngược với các loại máy tính để bàn, thường bị hạn chế với một vài kiến trúc máy tính nhất định. Các hệ thống nhúng có kích thước nhỏ và được thiết kế để hoạt động trong môi trường công nghiệp thường lựa chọn PC/104 và PC/104++ làm nền tảng. Những hệ thống này thường sử dụng DOS, Linux, NetBSD hoặc các hệ điều hành nhúng thời gian thực như QNX hay VxWorks. Còn các hệ thống nhúng có kích thước rất lớn thường sử dụng một cấu hình thông dụng là hệ thống on chip (System on a chip – SoC), một bảng mạch tích hợp cho một ứng dụng cụ thể (an application-specific integrated circuit – ASIC). Sau đó nhân CPU được mua và thêm vào như một phần của thiết kế chip. Một chiến lược tương tự là

sử dụng FPGA (field-programmable gate array) và lập trình cho nó với những thành phần nguyên lý thiết kế bao gồm cả CPU.

Thiết bị ngoại vi

Hệ thống nhúng giao tiếp với bên ngoài thông qua các thiết bị ngoại vi, ví dụ như:

- Serial Communication Interfaces (SCI): RS-232, RS-422, RS-485...
- Synchronous Serial Communication Interface: I2C, JTAG, SPI, SSC và ESSI
- Universal Serial Bus (USB)
- Networks: Controller Area Network, LonWorks...
- Bộ định thời: PLL(s), Capture/Compare và Time Processing Units
- Discrete IO: General Purpose Input/Output (GPIO)

Công cụ phát triển

Tương tự như các sản phẩm phần mềm khác, phần mềm hệ thống nhúng cũng được phát triển nhờ việc sử dụng các trình biên dịch (compilers), chương trình dịch hợp ngữ (assembler) hoặc các công cụ gỡ rối (debuggers). Tuy nhiên, các nhà thiết kế hệ thống nhúng có thể sử dụng một số công cụ chuyên dụng như:

- Bộ gỡ rối mạch hoặc các chương trình mô phỏng (emulator)
- Tiện ích để thêm các giá trị checksum hoặc CRC vào chương trình, giúp hệ thống nhúng có thể kiểm tra tính hợp lệ của chương trình đó.
- Đối với các hệ thống xử lý tín hiệu số, người phát triển hệ thống có thể sử dụng phần mềm workbench như MathCad hoặc Mathematica để mô phỏng các phép toán.
- Các trình biên dịch và trình liên kết (linker) chuyên dụng được sử dụng để tối ưu hóa một thiết bị phần cứng.
- Một hệ thống nhúng có thể có ngôn ngữ lập trình và công cụ thiết kế riêng của nó hoặc sử dụng và cải tiến từ một ngôn ngữ đã có sẵn.

Các công cụ phần mềm có thể được tạo ra bởi các công ty phần mềm chuyên dụng về hệ thống nhúng hoặc chuyển đổi từ các công cụ phát triển phần mềm GNU. Đôi

khi, các công cụ phát triển dành cho máy tính cá nhân cũng được sử dụng nếu bộ xử lý của hệ thống nhúng đó gần giống với bộ xử lý của một máy PC thông dụng.

Độ tin cậy

Các hệ thống nhúng thường nằm trong các cỗ máy được kỳ vọng là sẽ chạy hàng năm trời liên tục mà không bị lỗi hoặc có thể khôi phục hệ thống khi gặp lỗi. Vì thế, các phần mềm hệ thống nhúng được phát triển và kiểm thử một cách cẩn thận hơn là phần mềm cho máy tính cá nhân. Ngoài ra, các thiết bị rời không đáng tin cậy như ổ đĩa, công tắc hoặc nút bấm thường bị hạn chế sử dụng. Việc khôi phục hệ thống khi gặp lỗi có thể được thực hiện bằng cách sử dụng các kỹ thuật như watchdog timer – nếu phần mềm không đều đặn nhận được các tín hiệu watchdog định kì thì hệ thống sẽ bị khởi động lại.

Một số vấn đề cụ thể về độ tin cậy như:

- Hệ thống không thể ngừng để sửa chữa một cách an toàn, ví dụ như ở các hệ thống không gian, hệ thống dây cáp dưới đáy biển, các đèn hiệu dẫn đường,... Giải pháp đưa ra là chuyển sang sử dụng các hệ thống con dự trữ hoặc các phần mềm cung cấp một phần chức năng.
- Hệ thống phải được chạy liên tục vì tính an toàn, ví dụ như các thiết bị dẫn đường máy bay, thiết bị kiểm soát độ an toàn trong các nhà máy hóa chất,... Giải pháp đưa ra là lựa chọn backup hệ thống.
- Nếu hệ thống ngừng hoạt động sẽ gây tổn thất rất nhiều tiền của ví dụ như các dịch vụ buôn bán tự động, hệ thống chuyển tiền, hệ thống kiểm soát trong các nhà máy ...

1.4 Kiến trúc điển hình của hệ thống nhúng

Kiến trúc của một hệ thống nhúng là một sự trừu tượng hóa thiết bị nhúng, điều đó có nghĩa là một sự tổng quát hóa của một hệ thống mà không chỉ rõ các thông tin thực thi chi tiết của hệ thống như mã nguồn hoặc thiết kế mạch phân cứng.

Các thành phần phần cứng và phần mềm ở mức kiến trúc trong một hệ thống nhúng được đại diện bởi các phần tử có tác động lẫn nhau. Các phần tử là đại diện của phần cứng hoặc phần mềm nhưng chi tiết đã được trừu tượng hóa. Do đó, chỉ có thông tin về các mối quan hệ qua lại và các hoạt động của chúng. Các phần tử này có thể được tích hợp bên trong thiết bị nhúng hoặc tồn tại bên ngoài hệ thống nhúng và tương tác với các phần tử bên trong. *Tóm lại, một kiến trúc hệ thống nhúng bao gồm các phần tử của hệ thống nhúng, các phần tử tương tác với một hệ thống nhúng, các tính chất của mỗi phần tử riêng biệt và mối quan hệ tương tác giữa các thành phần.*

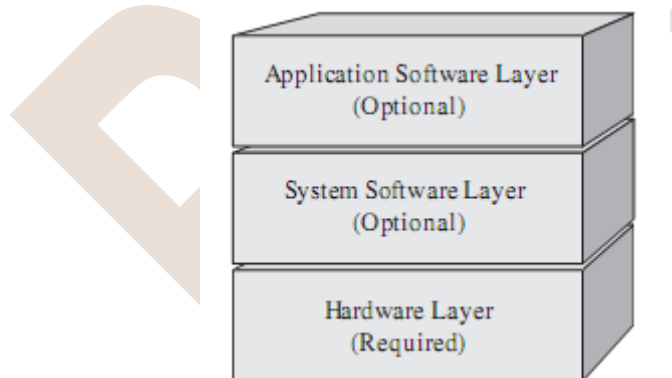
Các thông tin ở mức kiến trúc được mô tả theo dạng cấu trúc. Một cấu trúc sẽ bao gồm tập hợp của các phần tử, các tính chất và thông tin về các mối quan hệ qua lại. Do đó, một cấu trúc là một hình ảnh của phần cứng và phần mềm của hệ thống tại thời điểm thiết kế hoặc thời điểm chạy.

Do hệ thống thường có cấu trúc phức tạp, một kiến trúc thường là sự kết hợp của nhiều cấu trúc khác nhau. Tất cả các cấu trúc trong một kiến trúc có mối quan hệ thừa kế qua lại với nhau. Một số kiểu cấu trúc như sau:

- Cấu trúc theo dạng module: Theo dạng này, các phần tử là các thành phần có chức năng khác nhau của hệ thống (Ví dụ như các phần cứng hoặc phần mềm căn bản để cho hệ thống hoạt động được) trong một thiết bị nhúng. Cấu trúc này thường được trình bày theo dạng lớp (layers), theo các phần mềm dịch vụ cho nhân (kernel services) ...
- Cấu trúc theo dạng thành phần và kết nối: Cấu trúc này là sự kết hợp của các thành phần (VD: Phần cứng, Phần mềm, CPU) và các kết nối như bus phần cứng, các bản tin của phần mềm, các process trong hệ thống.

Mô hình hệ thống nhúng

Trên thực tế, có nhiều cấu trúc kiến trúc được sử dụng. Tuy nhiên, ở mức căn bản cao nhất, mô hình hệ thống nhúng thường được trình bày như hình sau:



Hình 1. 1: Mô hình chung hệ thống nhúng

Từ mô hình trên, có thể thấy rằng tất cả hệ thống nhúng đều có chung sự tương tự ở mức cao nhất. Cụ thể, chúng đều có các lớp (Phần cứng, phần mềm hệ thống và phần mềm ứng dụng). Lớp phần cứng bao gồm các thành phần vật lý trên bo mạch nhúng. Lớp phần mềm hệ thống và phần mềm ứng dụng bao gồm các phần mềm trên hệ thống nhúng.

Một số kiến trúc phần mềm hệ thống nhúng

Một số loại kiến trúc phần mềm thông dụng trong các hệ thống nhúng như sau:

Vòng lặp kiểm soát đơn giản

Theo thiết kế này, phần mềm được tổ chức thành một vòng lặp đơn giản. Vòng lặp gọi đến các chương trình con, mỗi chương trình con quản lý một phần của hệ thống phần cứng hoặc phần mềm.

Hệ thống ngắt điều khiển

Các hệ thống nhúng thường được điều khiển bằng các ngắt. Có nghĩa là các tác vụ của hệ thống nhúng được kích hoạt bởi các loại sự kiện khác nhau. Ví dụ, một ngắt có thể được sinh ra bởi một bộ định thời sau một chu kỳ được định nghĩa trước, hoặc bởi sự kiện khi cổng nối tiếp nhận được một byte nào đó.

Loại kiến trúc này thường được sử dụng trong các hệ thống có bộ quản lý sự kiện đơn giản, ngắn gọn và cần độ trễ thấp. Hệ thống này thường thực hiện một tác vụ đơn giản trong một vòng lặp chính. Đôi khi, các tác vụ phức tạp hơn sẽ được thêm vào một cấu trúc hàng đợi trong bộ quản lý ngắt để được vòng lặp xử lý sau đó. Lúc này, hệ thống gần giống với kiểu nhân đa nhiệm với các tiến trình rời rạc.

Đa nhiệm tương tác

Một hệ thống đa nhiệm không ưu tiên cũng gần giống với kỹ thuật vòng lặp kiểm soát đơn giản ngoại trừ việc vòng lặp này được ẩn giấu thông qua một giao diện lập trình API. Các nhà lập trình định nghĩa một loạt các nhiệm vụ, mỗi nhiệm vụ chạy trong một môi trường riêng của nó. Khi không cần thực hiện nhiệm vụ đó thì nó gọi đến các tiến trình con tạm nghỉ (bằng cách gọi “pause”, “wait”, “yield” ...).

Ưu điểm và nhược điểm của loại kiến trúc này cũng giống với kiểm vòng lặp kiểm soát đơn giản. Tuy nhiên, việc thêm một phần mềm mới được thực hiện dễ dàng hơn bằng cách lập trình một tác vụ mới hoặc thêm vào hàng đợi thông dịch (queue-interpreter).

Đa nhiệm ưu tiên

Ở loại kiến trúc này, hệ thống thường có một đoạn mã ở mức thấp thực hiện việc chuyển đổi giữa các tác vụ khác nhau thông qua một bộ định thời. Đoạn mã này thường nằm ở mức mà hệ thống được coi là có một hệ điều hành và vì thế cũng gặp phải tất cả những phức tạp trong việc quản lý đa nhiệm.

Bất kỳ tác vụ nào có thể phá hủy dữ liệu của một tác vụ khác đều cần phải được tách biệt một cách chính xác. Việc truy cập tới các dữ liệu chia sẻ có thể được quản lý bằng một số kỹ thuật đồng bộ hóa như hàng đợi thông điệp (message queues),

semaphores ... Vì những phức tạp nói trên nên một giải pháp thường được đưa ra đó là sử dụng một hệ điều hành thời gian thực. Lúc đó, các nhà lập trình có thể tập trung vào việc phát triển các chức năng của thiết bị chứ không cần quan tâm đến các dịch vụ của hệ điều hành nữa.

Vi nhân (Microkernel) và nhân ngoại (Exokernel)

Khái niệm vi nhân (microkernel) là một bước tiếp cận gần hơn tới khái niệm hệ điều hành thời gian thực. Lúc này, nhân hệ điều hành thực hiện việc cấp phát bộ nhớ và chuyển CPU cho các luồng thực thi. Còn các tiến trình người dùng sử dụng các chức năng chính như hệ thống file, giao diện mạng lưới,... Nói chung, kiến trúc này thường được áp dụng trong các hệ thống mà việc chuyển đổi và giao tiếp giữa các tác vụ là nhanh.

Còn nhân ngoại (exokernel) tiến hành giao tiếp hiệu quả bằng cách sử dụng các lời gọi chương trình con thông thường. Phần cứng và toàn bộ phần mềm trong hệ thống luôn đáp ứng và có thể được mở rộng bởi các ứng dụng.

Nhân khối (monolithic kernels)

Trong kiến trúc này, một nhân đầy đủ với các khả năng phức tạp được chuyển đổi để phù hợp với môi trường nhúng. Điều này giúp các nhà lập trình có được một môi trường giống với hệ điều hành trong các máy để bàn như Linux hay Microsoft Windows và vì thế rất thuận lợi cho việc phát triển. Tuy nhiên, nó lại đòi hỏi đáng kể các tài nguyên phần cứng làm tăng chi phí của hệ thống. Một số loại nhân khối thông dụng là Embedded Linux và Windows CE. Mặc dù chi phí phần cứng tăng lên nhưng loại hệ thống nhúng này đang tăng trưởng rất mạnh, đặc biệt là trong các thiết bị nhúng mạnh như Wireless router hoặc hệ thống định vị GPS. Lý do của điều này là:

- Hệ thống này có cổng để kết nối đến các chip nhúng thông dụng
- Hệ thống cho phép sử dụng lại các đoạn mã sẵn có phổ biến như các trình điều khiển thiết bị, Web Servers, Firewalls, ...
- Việc phát triển hệ thống có thể được tiến hành với một tập nhiều loại đặc tính, chức năng còn sau đó lúc phân phối sản phẩm, hệ thống có thể được cấu hình để loại bỏ một số chức năng không cần thiết. Điều này giúp tiết kiệm được những vùng nhớ mà các chức năng đó chiếm giữ.
- Hệ thống có chế độ người dùng để dễ dàng chạy các ứng dụng và gỡ rối. Nhờ đó, qui trình phát triển được thực hiện dễ dàng hơn và việc lập trình có tính linh động hơn.

- Có nhiều hệ thống nhúng thiếu các yêu cầu chặt chẽ về tính thời gian thực của hệ thống quản lý. Còn một hệ thống như Embedded Linux có tốc độ đủ nhanh để trả lời cho nhiều ứng dụng. Các chức năng cần đến sự phản ứng nhanh cũng có thể được đặt vào phần cứng.

1.5 Phân loại các hệ thống nhúng

Hiện nay có nhiều cách phân loại hệ thống nhúng khác nhau. Tùy theo cách phân chia có thể phân loại các hệ thống nhúng như sau:

- Hệ thống phân phối và hệ thống không phân phối:

Các hệ thống không phân phối thường hoạt động riêng biệt. Ngược lại, hệ thống phân phối phối kết các thiết bị được kết nối với nhau, ví dụ như các vi điều khiển nhúng, các thiết bị mạng, các máy tính nhúng, các hệ thống cảm biến không dây. Sự kết hợp các thiết bị nhúng này thường được gặp trong các thiết bị điều khiển ô tô hoặc hàng không, các thiết bị giám sát môi trường hoặc các thiết bị quản lý dây chuyền sản xuất tự động.

- Hệ thống dữ liệu và hệ thống điều khiển:

Các hệ thống dữ liệu dùng để xử lý dữ liệu, xử lý hoặc cung cấp các dữ liệu thông tin cần thiết khi có yêu cầu. Còn các hệ thống điều khiển dùng để điều khiển hệ thống, điều khiển các quy trình trong sản xuất hoặc trong các thiết bị.

1.6 Phạm vi ứng dụng của hệ thống nhúng

Ngày nay, hệ thống nhúng được ứng dụng rộng rãi ở khắp mọi nơi. Hầu hết những thiết bị điện tử đều là các hệ thống nhúng.

Một số ví dụ như sau:

- Các hệ thống điều khiển ô tô, tàu, máy bay
- Các hệ thống y tế
- Các hệ thống quân sự
- Các hệ thống giám sát, cảnh báo.
- Các hệ thống cảm ứng
- Các thiết bị điện tử dân dụng

Ngoài ra, theo ước tính, mỗi năm lượng phần mềm nhúng được phát triển lớn gấp năm lần lượng phần mềm thường. Đối với phần cứng, đa số CPU được sản xuất là cho thị trường nhúng. Chỉ có một phần nhỏ CPU là dùng trong các hệ thống máy tính.

1.7 Các yêu cầu về kỹ năng trong thiết kế hệ thống nhúng

Tổng quan về thiết kế các hệ nhúng

Thiết kế các hệ thống nhúng là thiết kế phần cứng và phần mềm phối hợp bao gồm những bước sau:

- Mô hình hoá hệ thống: Mô tả các khối chức năng với các đặc tính và thuật toán xử lý.
- Chi tiết hoá các khối chức năng
- Phân bố chức năng cho phần cứng và mềm (HW-SW)
- Đồng bộ hoạt động của hệ thống
- Cài đặt các chức năng thiết kế vào phần cứng (hardware) và phần mềm (software) hoặc phần nhào (firm-ware).

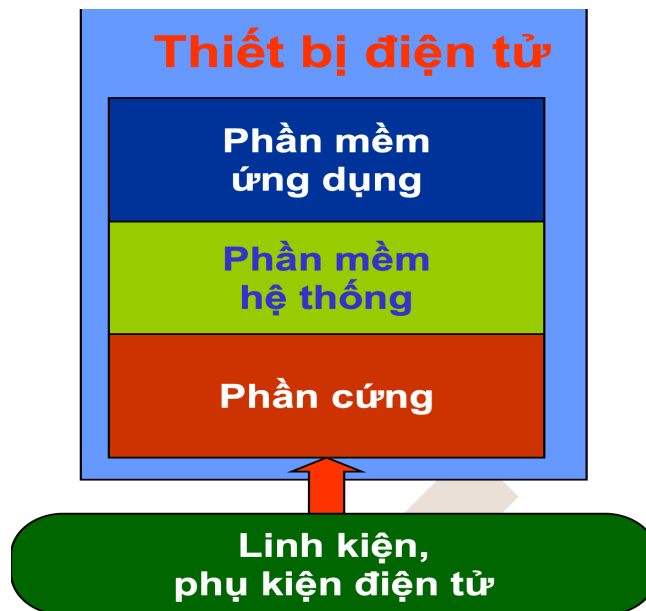
Cách thiết kế cổ điển là các chức năng phần mềm (SW) và phần cứng (HW) được xác định trước rồi sau đó các bước thiết kế chi tiết được tiến hành một cách độc lập ở hai khối. Hiện nay đa số các hệ thống tự động hoá thiết kế (CAD) thường dành cho thiết kế phần cứng. Các hệ thống nhúng sử dụng đồng thời nhiều công nghệ như vi xử lý, DSP, mạng và các chuẩn phối ghép, protocol, do vậy xu thế thiết kế các hệ nhúng hiện nay đòi hỏi có khả năng thay đổi mềm dẻo hơn trong quá trình thiết kế 2 phần HW và SW. Để có được thiết kế cuối cùng tối ưu quá trình thiết kế SW và HW phải phối hợp với nhau chặt chẽ và có thể thay đổi sau mỗi lần thử chức năng hoạt động tổng hợp

Thiết kế các hệ nhúng đòi hỏi kiến thức đa ngành về điện tử, xử lý tín hiệu, vi xử lý, thuật điều khiển và lập trình thời gian thực.

Đối với mỗi một ngành nghề đào tạo, mục đích cơ bản đó là xây dựng tập các kỹ năng cần phát triển cho đối tượng được đào tạo, để sau khi kết thúc khóa đào tạo, đối tượng được đào tạo về cơ bản phải đạt được những kỹ năng đã đặt ra. Tập các kỹ năng này được xây dựng dựa trên tính chất, đặc trưng của nội dung đào tạo.

Trong thiết kế hệ thống nhúng, nội dung đào tạo liên quan đến việc thiết kế các hệ thống nhúng hay các thiết bị điện tử thông minh hoặc còn được gọi tắt là hệ thống được điều khiển bởi các linh kiện bán dẫn khả trình như bộ vi xử lý – microprocessor, bộ vi điều khiển – microcontroller, FPGA, CPLD,

Để đảm bảo tính đồng nhất, từ giờ chúng ta sẽ gọi tên gọi chung cho các thiết bị điện tử hay các hệ thống nhúng là các hệ thống nhúng.



Hình 1. 2: Cấu trúc của thiết bị điện tử

Cấu trúc của một hệ thống nhúng về cơ bản được mô tả như trong hình 1, nó gồm các thành phần:

- Phần mềm ứng dụng: là các phần mềm được thiết kế để thực thi một tác vụ thực tế dựa trên tài nguyên do nền phần cứng cung cấp. Ví dụ như các phần mềm chơi nhạc trên các máy MP3, ứng dụng game trên các máy PS2, bộ công cụ Microsoft Office trên các PC...
- Phần mềm hệ thống: ví dụ như các hệ điều hành Windows, Linux, Unix, và các chương trình hỗ trợ như trình biên dịch, loader, linker, debugger... giúp quản lý các tài nguyên phần cứng ở mức thấp. Về cơ bản nó cho phép các phần của hệ thống làm việc với nhau, cấp phát các tài nguyên cho các phần mềm ứng dụng.
- Phần cứng : chỉ các thành phần vật lý của hệ thống, được cấu tạo về cơ bản từ các linh kiện vật lý. Ví dụ như với một PC, phần cứng gồm các thành phần bo mạch chủ, Ram, ổ cứng, nguồn nuôi, ... Các bo mạch chủ thì được cấu tạo từ các linh kiện bán dẫn, các linh kiện thụ động như điện trở, tụ điện, cuộn cảm,...

Từ cấu trúc trên của hệ thống nhúng, có thể phân chia việc thiết kế một hệ thống thành các mảng công việc như trong hình 2:



Hình 1. 3: Các mảng công việc trong thiết kế hệ thống nhúng

Nó gồm các mảng khác nhau đó là:

- Quản lý, tích hợp và thiết kế hệ thống
- Thiết kế, phát triển phần mềm ứng dụng
- Thiết kế firmware (Device Driver, OS, Middleware)
- Thiết kế mạch, PCB
- Thiết kế vi điện tử: linh kiện, IC, IP, Phụ kiện

Đối với từng mảng công việc, đòi hỏi người thực hiện cần có các kỹ năng tương ứng.

1.7.1 Quản lý, tích hợp, thiết kế hệ thống:

Đây là một trong những mặt có vai trò quan trọng đối với sự thành công của một dự án thiết kế một hệ thống nhúng. Nó bao gồm các công việc:

- Hoạch định các yêu cầu của hệ thống, từ đó xây dựng kết cấu chung của hệ thống.
- Xác định các tài nguyên có sẵn bao gồm nhân lực và vật lực.
- Lên kế hoạch các bước thực hiện.
- Giám sát quá trình thực hiện.

Để đảm nhiệm được công việc này, yêu cầu các kỹ năng sinh viên cần có:

- Có kiến thức về quản lý dự án thiết kế, kiến trúc hệ thống: bao gồm kiến thức các mô hình trong phân tích hệ thống và triển khai các dự án thiết kế; kiến trúc hệ

thống nhúng, có cái nhìn tổng quan với các khía cạnh của hệ thống nhúng gồm phần mềm, phần cứng...

- Có kĩ năng quản lý nhóm, đây là một yêu cầu quan trọng có tính chất quyết định
- Có khả năng sáng tạo, ứng dụng các phương thức quản lý tiên tiến nhằm đạt hiệu năng cao trong thiết kế hệ thống.
- Kĩ năng về phân tách, tích hợp hệ thống, kiểm tra hệ thống... đảm bảo hệ thống hoạt động ổn định, đáp ứng được các yêu cầu về hiệu năng, giá thành, tuổi thọ....

1.7.2 Thiết kế, phát triển phần mềm ứng dụng

Đối với các hệ thống nhúng, điều quan trọng là ứng dụng thực tiễn của nó trong đời sống, quan trọng hơn, chức năng được quyết định bởi phần mềm ứng dụng được cài đặt trên hệ thống. Vì vậy, các kĩ năng trong thiết kế, phát triển phần mềm ứng dụng là không thể thiếu đối với thiết kế hệ thống nhúng hiện nay. Các kĩ năng yêu cầu gồm:

- Có kiến thức đối với khoa học lập trình: bao gồm kiến thức về xây dựng cấu trúc dữ liệu và giải thuật; cơ sở dữ liệu; các phương thức lập trình hướng cấu trúc, hướng đối tượng; đồ họa; đa phương tiện; kiến thức liên quan đến xử lý tín hiệu...
- Kĩ năng thực thi các ứng dụng trên các ngôn ngữ lập trình: yêu cầu có kĩ năng về các ngôn ngữ lập trình ứng dụng như C/C++, VC++, Python, Java, VB, Delphi, ASP, PHP, JAVA.
- Có kiến thức lập trình trên các nền tảng khác nhau: PC, smartphone, ...

1.7.3 Thiết kế firmware

Điều quan trọng của các ứng dụng hệ thống nhúng đó là tận dụng được các tài nguyên của hệ thống: CPU, memory, các ngoại vi, các giao diện... Hầu hết quá trình xử lý bên trong của hệ thống là sự giao tiếp giữa các thành phần phần cứng bên trong. Vì vậy đòi hỏi xây dựng firmware để quản lý các tài nguyên này một cách hữu hiệu, cung cấp các giao diện truy cập các tài nguyên này cho lớp cao hơn (lớp ứng dụng).

Nó gồm các lĩnh vực về thiết kế trình điều khiển thiết bị - Device Driver, thiết kế OS – hệ điều hành, thiết kế phần mềm Middleware.

Để thực hiện được, yêu cầu các kĩ năng sinh viên cần có:

- Hiểu biết về hệ điều hành, hệ điều hành thời gian thực... như khái niệm hệ điều hành, các thành phần hệ điều hành, các khái niệm về scheduling, process, thread, inter-process....
- Kiến thức về kiến trúc máy tính, các kĩ thuật ứng dụng trong kiến trúc máy tính như pipeline, DMA, caching, buffering...
- Kĩ năng lập trình hệ thống, hệ thống nhúng, lập trình vi xử lý, bảo mật, mạng...

- Thành thạo các ngôn ngữ lập trình hệ thống như assembly, C/C++; các ngôn ngữ mô tả phần cứng như VHDL, Verilog.
- Sử dụng thành thạo các công cụ phần mềm hỗ trợ lập trình firmware trình biên dịch, IDE, ...
- Với xu thế hiện nay, các kiến thức về phần mềm mã nguồn mở, hệ điều hành mã nguồn mở sẽ đem lại những lợi ích to lớn trong phát triển phần mềm hệ thống nhúng, ngoài vấn đề về giá thành, khi sử dụng các phần mềm mã nguồn mở, sinh viên sẽ được sự hỗ trợ to lớn từ cộng đồng mã nguồn mở, các kỹ thuật lập trình tiên tiến trong các phần mềm... nên đây cũng là một kỹ năng sinh viên nên chú ý phát triển.

1.7.4 Thiết kế mạch, PCB:

Phần cứng có thể coi như phần xác của hệ thống, cũng là một phần không thể thiếu của hệ thống nhúng, và là kỹ năng yêu cầu bắt buộc đối với các sinh viên điện tử viễn thông. Nó gồm các kỹ năng về:

- Hiểu biết về mạch điện tử, phần cứng vi xử lý, vi điều khiển, IC chức năng, FPGA, CPLD... khả năng kết hợp các thành phần để tạo lên 1 hệ thống hoàn chỉnh.
- Các khái niệm trong thiết kế mạch PCB như footprint, layer, path, SMD, SMT..
- Có kiến thức về thiết kế các mạch ổn định, khả năng chống nhiễu, bố trí hợp lý, khoa học, và thẩm mỹ. Đặc biệt là kinh nghiệm làm việc với các mạch hoạt động ở tần số cao.
- Sử dụng thành thạo các công cụ hỗ trợ thiết kế mạch, mô phỏng mạch như Altium, ISE, proteus...

1.7.5 Thiết kế vi điện tử: Linh kiện, IP, IC, phụ kiện

Vi điện tử là các vấn đề liên quan đến nghiên cứu, chế tạo các linh kiện điện tử. Những linh kiện này làm từ các chất bán dẫn. Chúng là những thành phần cơ bản trong các thiết kế điện tử như transistor, tụ điện, điện trở, diode, IC, ...

Có thể nói các linh kiện, IP, IC, phụ kiện... là những “viên gạch” xây lên “ngôi nhà” phần cứng hệ thống. Kiến thức thiết kế ở mặt này là những kiến thức cơ bản, nền tảng cho các thiết kế nâng cao hơn trong thiết kế phần cứng.

Các kỹ năng được yêu cầu:

- Kiến thức về vật lý bán dẫn, nguyên lý mạch tích hợp tương tự, số, mạch RF và cao tần, điện tử ứng dụng...
- Các công nghệ chế tạo chất bán dẫn như NMOS, MOSFET, CMOS,

- Thành thạo thiết kế layout, ASIC, VLSI... sử dụng các công cụ như MentorGraphic, Cadence, ADS, ...

Tổng kết lại, có thể tóm tắt các kỹ năng yêu cầu đối với từng mặt trong thiết kế hệ thống nhúng như trong bảng 1.



Hình 1. 4: Các yêu cầu về kỹ năng tương ứng

Câu hỏi ôn tập

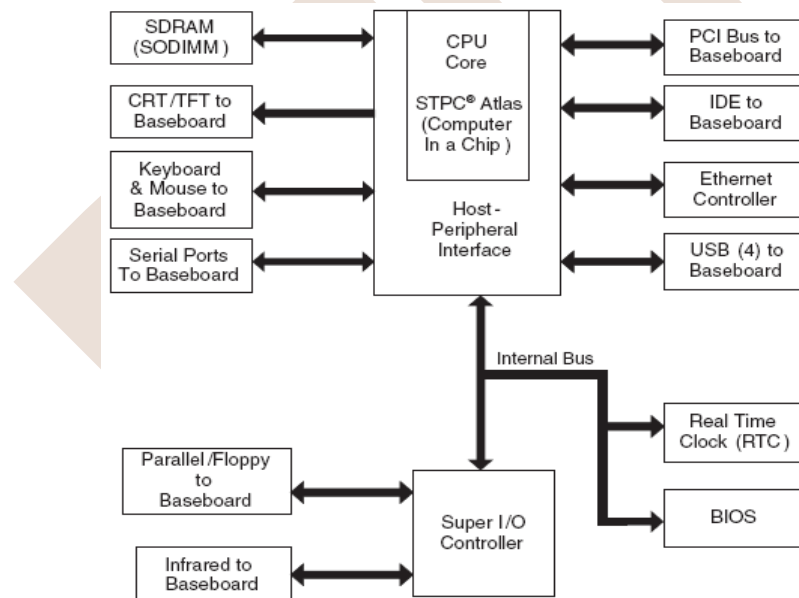
1. Lấy ví dụ 3 thị trường sử dụng hệ thống nhúng
Mỗi thị trường lấy 5 ví dụ thiết bị.
2. Các đặc tính của hệ thống nhúng bao gồm những đặc tính nào
3. Nêu các thành phần căn bản của kiến trúc hệ thống nhúng
4. Các kiến trúc CPU nhúng căn bản bao gồm những kiến trúc nào
5. Liệt kê các giao tiếp sử dụng trong hệ thống nhúng và giải thích hoạt động của chúng
6. Liệt kê các loại thiết bị lưu trữ thường sử dụng trong hệ thống nhúng.
7. So sánh các loại thiết bị lưu trữ thường sử dụng trong hệ thống nhúng.
8. Để phát triển một hệ thống nhúng, cần những kỹ năng nào?

CHƯƠNG 2: CÁC THÀNH PHẦN HỆ THỐNG

2.1 Các thành phần phần cứng

2.1.1 Bộ xử lý nhúng.

Bộ xử lý là đơn vị chức năng chính của một hệ thống nhúng, và chịu trách nhiệm trong việc xử lý lệnh và dữ liệu. Một thiết bị điện tử có chứa ít nhất một bộ xử lý chủ (Master Processor), có thể bổ sung các bộ xử lý tớ (Slave Processors) cùng làm việc và điều khiển bởi bộ xử lý chủ. Slave processors có thể tham gia và các chỉ lệnh của master processors hoặc thi hành quản lý bộ nhớ, các bus và các thiết bị vào ra. Trong sơ đồ khối của một board X86, Atlas STPC là một master processor còn bộ điều khiển I/O, bộ điều khiển ethernet là các Slave processor.



Hình 2. 1: Bảng mạch Encore 400 của Ampro

Hệ thống nhúng được thiết kế xung quanh bộ xử lý chủ. Các bộ xử lý chủ phức tạp thường được xác định cho dù nó được phân loại là vi xử lý hay vi điều khiển. Thông thường các bộ vi xử lý chứa một lượng nhỏ bộ nhớ tích hợp và thành phần vào ra (I/O), trong khi các bộ vi điều khiển có phần lớn bộ nhớ hệ thống và các thành phần vào ra tích hợp trên chip. Tuy nhiên hãy nhớ rằng các định nghĩa truyền thống này có thể không còn chính xác cho các thiết kế bộ xử lý thời gian gần đây. Ví dụ các bộ xử lý hiện nay ngày càng được tích hợp nhiều lên.

Các bộ xử lý nhúng có thể tách thành các nhóm dựa trên kiến trúc. Khác biệt giữa các nhóm kiến trúc này là tập hợp các chỉ lệnh mã máy mà các bộ xử lý trong các nhóm kiến trúc có thể thực thi. Bộ xử lý được xem như kiến trúc tương tự nhau khi chúng có thể thực thi cùng tập lệnh. Dưới đây là bảng vài ví dụ về các bộ xử lý thực tế và họ kiến trúc của chúng.

Architecture	Processor	Manufacturer
AMD	Au1xxx	Advanced Micro Devices, ...
ARM	ARM7, ARM9, ...	ARM, ...
C16X	C167CS, C165H, C164CI, ...	Infineon, ...
ColdFire	5282, 5272, 5307, 5407, ...	Motorola/Freescale, ...
I960	I960	Vmetro, ...
M32/R	32170, 32180, 32182, 32192, ...	Renesas/Mitsubishi, ...
M Core	MMC2113, MMC2114, ...	Motorola/Freescale
MIPS32	R3K, R4K, 5K, 16, ...	MTI4kx, IDT, MIPS Technologies, ...
NEC	Vr55xx, Vr54xx, Vr41xx	NEC Corporation, ...
PowerPC	82xx, 74xx, 8xx, 7xx, 6xx, 5xx, 4xx	IBM, Motorola/Freescale, ...
68k	680x0 (68K, 68030, 68040, 68060, ...), 683xx	Motorola/Freescale, ...
SuperH (SH)	SH3 (7702, 7707, 7708, 7709), SH4 (7750)	Hitachi, ...
SHARC	SHARC	Analog Devices, Transtech DSP, Radstone, ...
strongARM	strongARM	Intel, ...
SPARC	UltraSPARC II	Sun Microsystems, ...
TMS320C6xxx	TMS320C6xxx	Texas Instruments, ...
x86	X86 [386, 486, Pentium (II, III, IV)...]	Intel, Transmeta, National Semiconductor, Atlas, ...
TriCore	TriCore1, TriCore2, ...	Infineon, ...

Bảng 2. 1: Các bộ xử lý và kiến trúc thực tế

2.1.1.1 Các mô hình kiến trúc tập lệnh (ISA architecture models)

ISA (instruction set architecture) định nghĩa các tính năng như các thao tác (Operations) có thể sử dụng bởi người lập trình để tạo chương trình, các toán hạng (Operands), lưu trữ (Storage), các kiểu định địa chỉ để truy xuất và xử lý toán hạng và xử lý các ngắt. Các tính năng được mô tả chi tiết hơn dưới đây, bởi vì thực thi ISA là một nhân tố xác định đặc điểm của một hệ thống nhúng như là hiệu suất, thời gian thiết kế, chức năng và giá thành.

- Các thao tác.

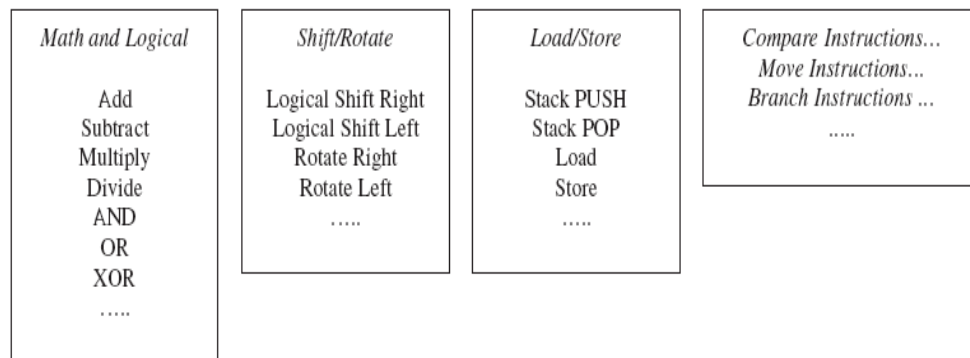
Các thao tác được tạo ra từ một hay nhiều chỉ lệnh. Các bộ xử lý khác nhau có thể thực thi các thao tác tương tự nhau nhờ sử dụng một số lượng và kiểu chỉ lệnh khác nhau.

Các kiểu thao tác.

Các thao tác là các chức năng có thể thực hiện trên dữ liệu, chúng thường bao gồm các phép toán số học, các thao tác chuyển dữ liệu từ bộ nhớ hoặc thanh ghi đến các vị trí

khác, các rẽ nhánh (có điều kiện hoặc không điều kiện), hoạt động truyền nhận dữ liệu, chuyển đổi phép toán...

Chỉ lệnh trên một bộ vi xử lý cấp thấp phổ biến 8051, bao gồm hơn 100 chỉ lệnh toán học, truyền dữ liệu, phép toán logic, thao tác bit, rẽ nhánh và điều khiển. Trong khi đó ở cấp cao hơn MPC-823 có tập chỉ lệnh lớn hơn 8051, nhưng cùng với nhiều kiểu hoạt động tương tự bao gồm trong 8051 thiết lập cùng với một bổ sung, bao gồm phép toán số nguyên, dấu chấm động, thao tác nạp, lưu trữ, các thao tác rẽ nhánh và điều khiển, thao tác điều khiển bộ xử lý, đồng bộ bộ nhớ. Ví dụ về các thao tác phổ biến quy định trong một ISA (Hình 2-2).



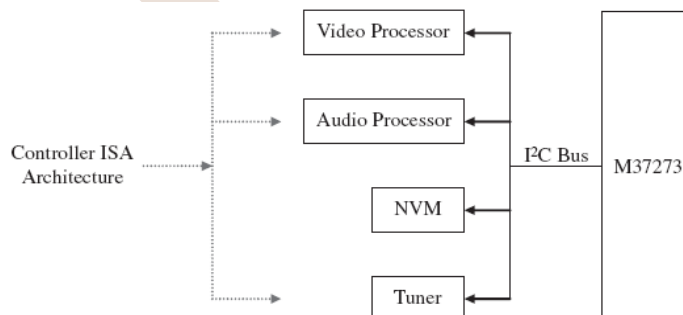
Hình 2. 2: Các thao tác ISA đơn giản

2.1.1.2 Ứng dụng cụ thể của mô hình ISA

Ứng dụng cụ thể của mô hình ISA xác định các bộ xử lý dành cho các ứng dụng nhúng cụ thể, ví dụ các bộ xử lý cho TV. Một số ứng dụng cụ thể của mô hình ISA được thực hiện bởi các bộ xử lý nhúng, các mô hình phổ biến nhất là:

Mô hình điều khiển – Controller model

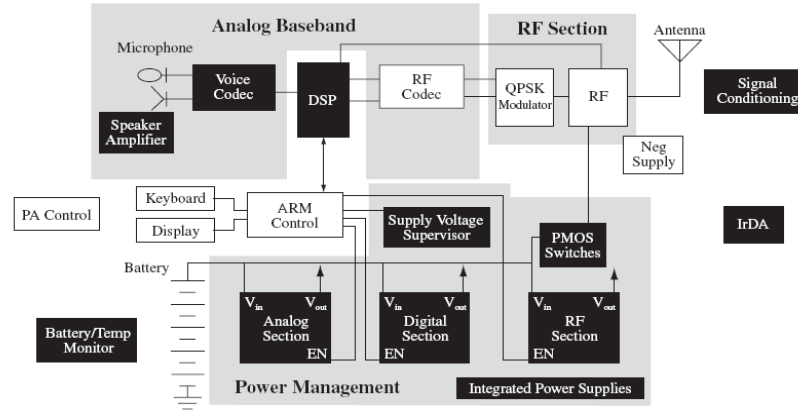
Các ISA controller được thực hiện trong các bộ xử lý mà không yêu cầu các thao tác dữ liệu phức tạp. Ví dụ như bộ xử lý audio và video được sử dụng như các slave processors trong bảng mạch TV, xem Hình 2-3.



Hình 2. 3: Ví dụ bảng mạch TV tương tự với sự thi hành bộ điều khiển ISA

Mô hình đường dữ liệu – Datapath model

Datapath model được thực hiện trong các bộ xử lý với mục đích thực hiện nhiều lần các phép toán cố định trên các bộ dữ liệu khác nhau. Ví dụ điển hình là bộ xử lý tín hiệu số (DSP), thể hiện trong Hình 2-4.



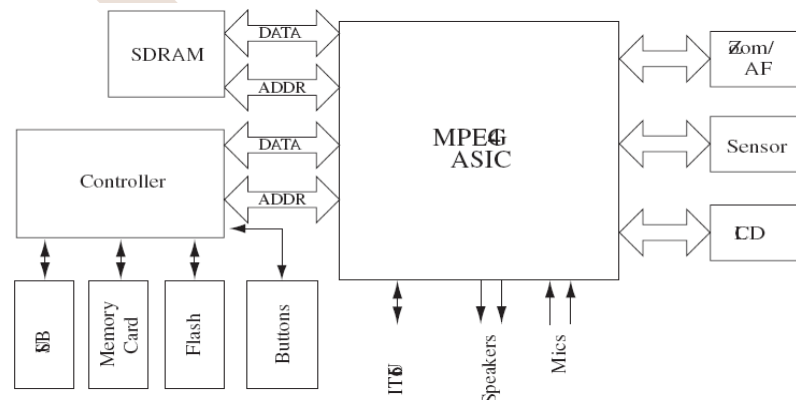
Hình 2. 4: Board ví dụ với điện thoại di động thực hiện kỹ thuật số ISA đường dữ liệu

Finite State Machine with Datapath (FSMD) Model

FSDM ISA là sự kết hợp của Datapath ISA và Controller ISA cho các bộ xử lý, không yêu cầu thực hiện các thao tác dữ liệu phức tạp và lặp lại việc tính toán các phép toán cố định trên các bộ dữ liệu khác nhau.

Ví dụ của FSMD thực hiện các ứng dụng cụ thể:

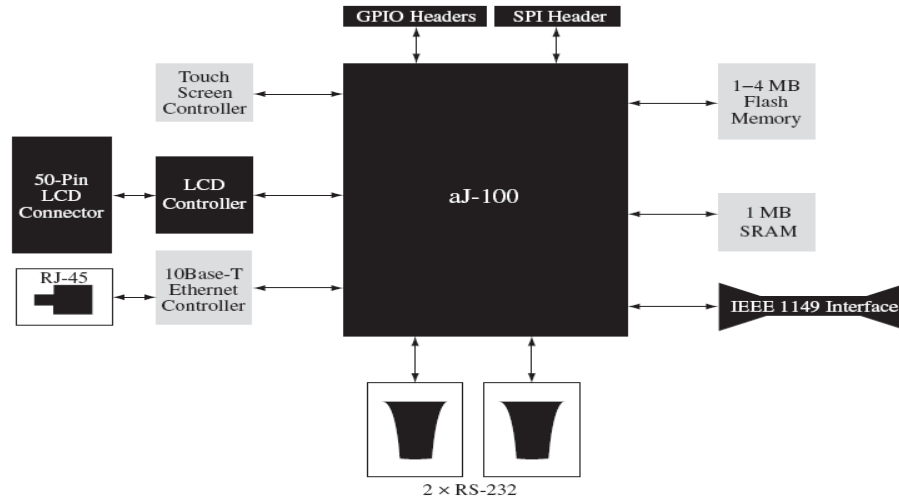
Mạch tích hợp ASICs thể hiện trên hình 2-5, các thiết bị logic khả trình (PLD), FPGA



Hình 2. 5: Board ví dụ với máy quay kỹ thuật số FSMD ISA

Mô hình máy ảo Java (Java Virtual Machine: JVM)

ISA JVM dựa trên một trong các tiêu chuẩn Java Virtual Machine. Trong thế giới thực, JVM có thể được thực hiện trong một hệ thống nhúng thông qua phần cứng. Ví dụ trong Hình 2-6.



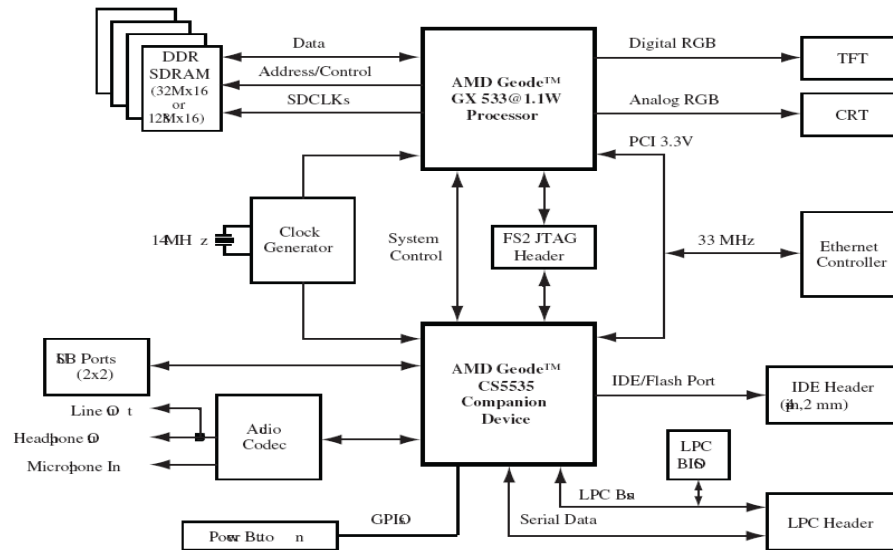
Hình 2. 6: Ví dụ thực hiện JVM ISA

2.1.1.3 Các mô hình ISA đa năng (General-Purpose ISA models)

General-Purpose ISA models thường được thực hiện trong các bộ xử lý với mục đích sử dụng rộng rãi trong nhiều hệ thống chứ không chỉ trong một dạng hệ thống nhúng. Các dạng phổ biến nhất của General-Purpose ISA models trong các bộ xử lý nhúng là:

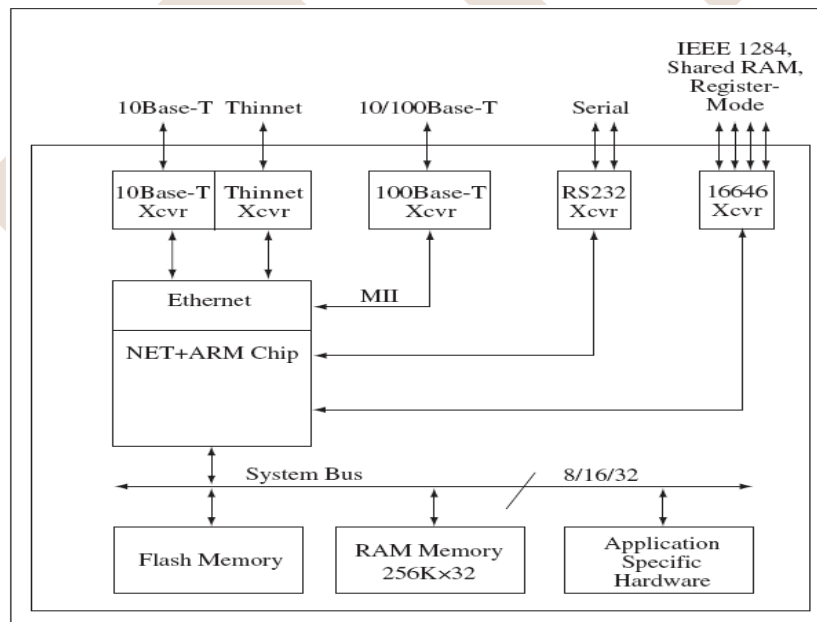
Mô hình tính toán với tập lệnh phức tạp (Complex Instruction Set Computing: CISC)

CISC ISA, như tên của nó xác định các hoạt động phức tạp tạo ra từ nhiều lệnh. Ví dụ phổ biến về kiến trúc thực hiện CISC ISA là các họ bộ xử lý intel x86 và Motorola/Freescale 68000



Hình 2. 7: Ví dụ thực hiện CISC ISA

Mô hình tính toán với tập lệnh rút gọn (Reduced Instruction Set Computing: RISC)



Hình 2. 8: Ví dụ thực hiện RISC ISA

Trái ngược với CISC, RISC ISA thường định nghĩa:

- Một kiến trúc đơn giản và/hoặc hoạt động ít hơn gồm ít lệnh hơn.
- Một kiến trúc giảm số chu kỳ hoạt động.

- Các bộ xử lý RISC chỉ có một chu kì hoạt động trong khi CISC thường có nhiều chu kì.
- ARM, PowerPC, SPARC, MIPS là một vài ví dụ về kiến trúc RISC cơ bản.

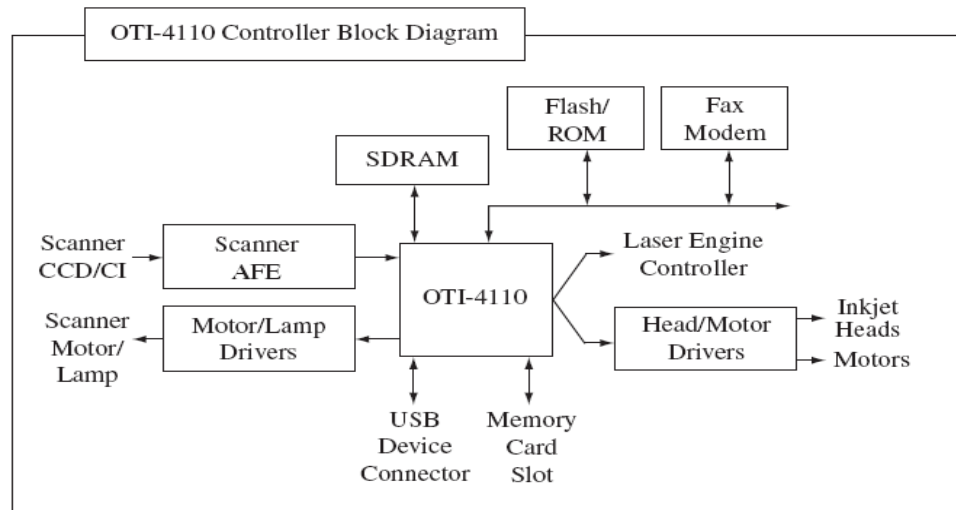
Trong lĩnh vực tính toán đa năng, lưu ý rằng nhiều mẫu thiết kế bộ vi xử lý hiện nay thuộc thể loại chip CISC hoặc RISC chủ yếu là do tính kế thừa của chúng. Các bộ xử lý RISC đã trở nên phức tạp hơn, trong khi các bộ xử lý CISC đã trở nên hiệu quả hơn để cạnh tranh với chip RISC đối thủ tương ứng của chúng, do đó làm mờ đi các ranh giới giữa các định nghĩa của một kiến trúc RISC so với một kiến trúc CISC. Về mặt kỹ thuật, những bộ xử lý này có cả hai thuộc tính RISC và CISC, bất kể những định nghĩa của chúng.

2.1.1.4 Các mô hình ISA song song mức lệnh (Instruction-Level Parallelism ISA Models)

Kiến trúc Instruction-level Parallelism ISA tương tự như kiến trúc chung của ISA ngoại trừ việc thực hiện nhiều lệnh song song. Trong thực tế, Instruction-level Parallelism ISA được xem như phát triển từ RISC ISA, thường chỉ có một chu kì hoạt động, một trong những lý do chính khiến RISC là cơ sở của việc thực hiện lệnh song song. Ví dụ về Instruction-level Parallelism ISA bao gồm

Mô hình SIMD (Single Instruction Multiple Data)

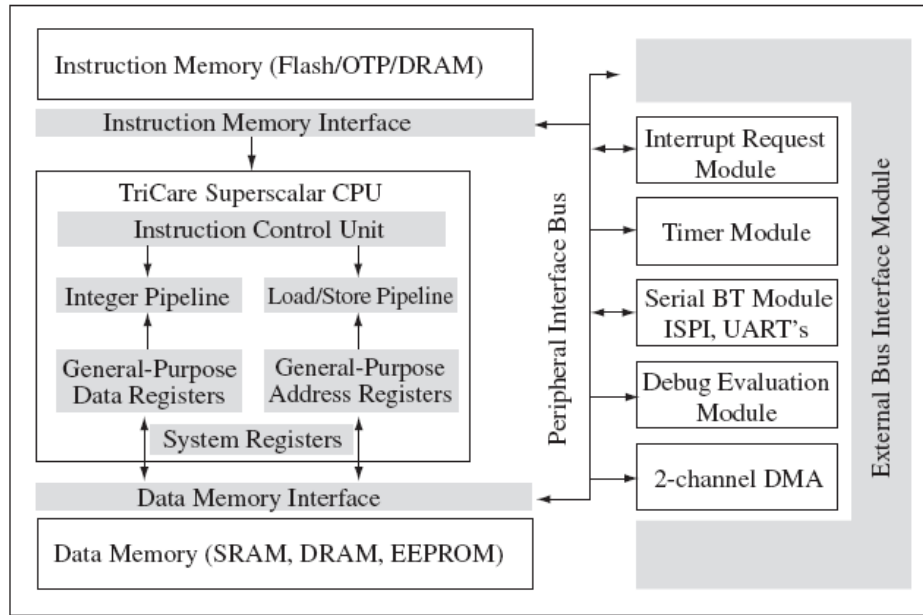
SIMD Machine ISA được thiết kế để xử lý một lệnh đồng thời trên nhiều phần dữ liệu.



Hình 2. 9: Ví dụ thực hiện SIMD ISA

Mô hình máy siêu vô hướng (Superscalar Machine Model)

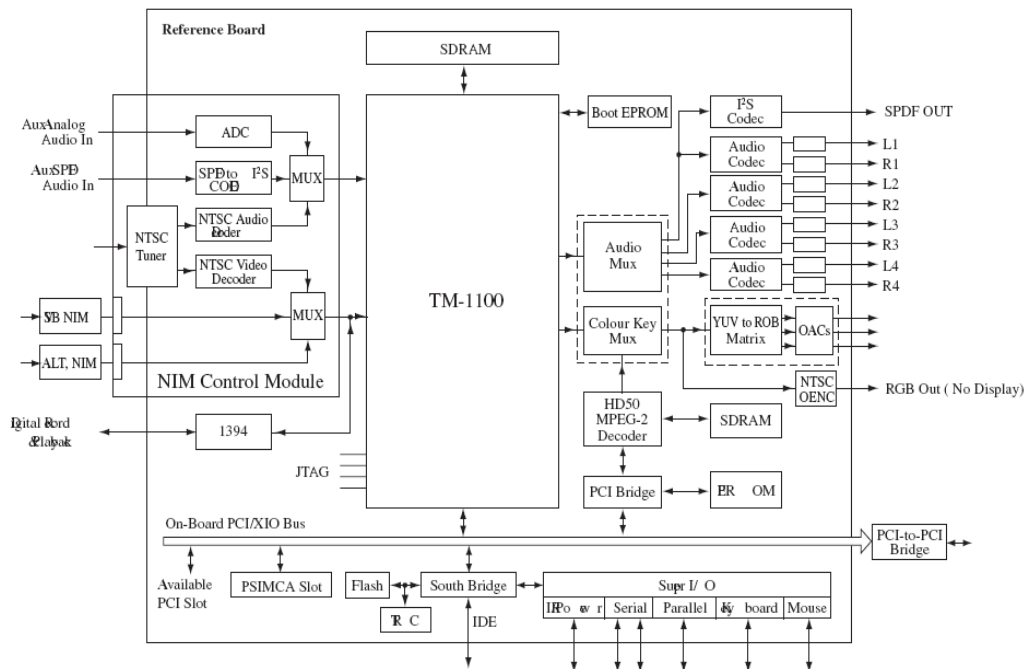
Superscalar ISA có thể thực hiện nhiều lệnh đồng thời trong một chu kì xung nhịp thông qua việc thực hiện nhiều hàm chức năng trong xử lý



Hình 2. 10: Ví dụ thực hiện siêu vô hướng ISA

Mô hình tính toán với từ lệnh rất dài (Very Long Instruction Word Computing: VLIW)

The VLIW ISA là kiến trúc trong đó một từ lệnh dài yêu cầu thực thi nhiều hoạt động. Các hoạt động này được chia nhỏ và xử lý song song bởi nhiều đơn vị thực thi trong bộ xử lý.



Hình 2. 11: Ví dụ thực hiện VLIW ISA

2.1.1.5 Hiệu suất bộ xử lý

Có nhiều cách để đo hiệu suất của một bộ xử lý, nhưng tất cả đều dựa trên hoạt động của bộ xử lý trong một khoảng thời gian nhất định. Một trong những định nghĩa chung nhất về hiệu suất của bộ xử lý đó chính là lưu lượng thông tin đi qua bộ xử lý, số lượng công việc mà CPU hoàn thành trong một khoảng thời gian. Như đã nói đến ở phần 2.1, một sự thực thi của bộ xử lý được đồng bộ bởi một hệ thống bên ngoài hoặc xung nhịp chủ trên bảng mạch.

Sử dụng tốc độ xung nhịp, thời gian thực thi của CPU, là tổng thời gian bộ xử lý xử lý một số chương trình có thể tính toán được. Từ tốc độ xung nhịp, khoảng thời gian CPU hoàn thành một chu kỳ xung nhịp, là nghịch đảo của tốc độ xung nhịp, được gọi là chu kỳ của bộ xử lý. Tốc độ và chu kỳ xung nhịp của bộ xử lý thường có trong tài liệu kỹ thuật của bộ xử lý.

Nhìn vào các lệnh, chỉ số CPI (trung bình số chu kỳ xung nhịp trên một lệnh) có thể xác định theo nhiều cách. Một cách là lấy chỉ số CPI của mỗi lệnh nhân với tần số của lệnh đó.

$$CPI = (CPI \text{ per instruction} * \text{instruction frequency})$$

Tổng thời gian thực thi của các CPU có thể được xác định bởi:

$$\text{Thời gian CPU thực thi mỗi chương trình (tính bằng giây)} = (\text{Tổng số chỉ lệnh của mỗi chương trình hay tổng số chỉ lệnh đếm được}) * (\text{CPI trong số lượng các chu kỳ/chỉ lệnh}) * (\text{số giây trên một chu kỳ xung nhịp}) = ((\text{tổng số chỉ lệnh đếm được}) * (\text{CPI trong số lượng các chu kỳ/chỉ lệnh})) / (\text{tốc độ xung nhịp (tính bằng MHz)})$$

$$\{CPU \text{ execution time in seconds per program} = (\text{total number of instructions per program or instruction count}) * (\text{CPI in number of cycle cycles/instruction}) * (\text{clock period in seconds per cycle}) = ((\text{instruction count}) * (\text{CPI in number of cycle cycles/ instruction})) / (\text{clock rate in MHz}) \}$$

Tỉ lệ thực thi bình quân của bộ xử lý còn gọi là thông lượng hay băng thông, cho biết số lượng công việc CPU thực hiện trong một chu kỳ thời gian và bằng nghịch đảo của thời gian thực thi của CPU

$$CPU \text{ throughput (in bytes/sec or MB/sec)} = 1 / CPU \text{ execution time} = CPU \text{ performance}$$

Các định nghĩa khác về hiệu suất bên cạnh thông lượng bao gồm:

Độ đáp ứng của bộ xử lý hay độ trễ là khoảng thời gian một bộ xử lý cần để đáp ứng một số sự kiện.

Sự sẵn sàng của bộ xử lý, thể hiện qua thời gian bộ xử lý hoạt động bình thường mà không gặp sự cố, hay là độ tin cậy, thời gian trung bình xảy ra sự cố, thời gian CPU cần để khắc phục sự cố.

Thiết kế bên trong của bộ xử lý quyết định xung nhịp và chỉ số CPI của bộ xử lý. Do đó hiệu suất của bộ xử lý phụ thuộc vào mô hình ISA nào được thực hiện và thực hiện như thế nào. Hiệu suất có thể được cải thiện bởi những hiện thực vật lý thực tế của ISA bên trong bộ xử lý, chẳng hạn như sự thực hiện kỹ thuật đường ống trong ALU.

Khoảng cách ngày càng tăng giữa hiệu suất của bộ xử lý và bộ nhớ có thể cải thiện bằng các thuật toán cache thực hiện lệnh và tìm nạp dữ liệu trước (đặc biệt là các thuật toán sử dụng các dự đoán rẽ nhánh để giảm thời gian trì hoãn) và giải phóng bộ nhớ đệm. Về cơ bản bất kì thiết kế tính năng cho phép tăng xung nhịp đồng hồ hoặc giảm chỉ số CPI sẽ tăng hiệu suất tổng thể của một bộ xử lý.

2.1.1.6 Những chuẩn đánh giá (Benchmarks)

Một trong những hiệu suất chung sử dụng bộ xử lý nhưng là hàng triệu lệnh thực hiện trong một giây (MIPS)

$$MIPS = \text{Instruction Count} / (\text{CPU execution time} * 10^6) = \text{Clock Rate} / (\text{CPI} * 10^6)$$

MIPS khiến chúng ta cho rằng bộ xử lý nhanh hơn thì có giá trị MIPS cao hơn, do công thức MIPS tỉ lệ nghịch với thời gian thực hiện của CPU. Tuy nhiên MIPS có thể gây hiểu lầm ví một số lý do:

Các lệnh và hàm phức tạp không được xem xét trong MIPS, do đó MIPS không thể so sánh khả năng của bộ xử lý với các ISA khác nhau.

MIPS có thể biến đổi trên cùng một bộ xử lý khi chạy các chương trình khác nhau. (Với sự thay đổi cách tính các lệnh và các loại lệnh khác nhau)

Chương trình phần mềm chuẩn có thể chạy trên các bộ xử lý để đo hiệu suất của chúng.

2.1.1.7 Đọc datasheet của một bộ xử lý.

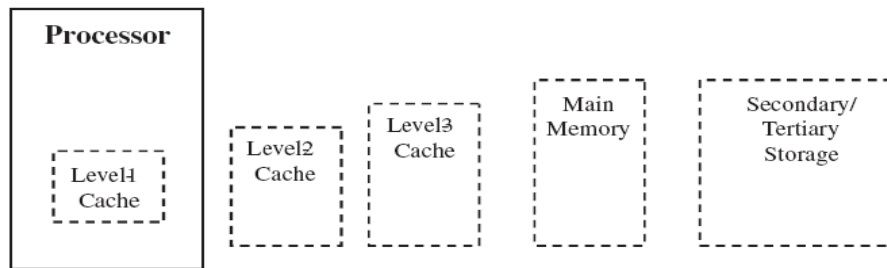
Datasheet của bộ xử lý cung cấp các thành phần chính của bộ xử lý và các thông tin để sử dụng bộ xử lý.

Lưu ý: Đừng cho rằng những gì đọc từ nhà cung cấp là chính xác 100% cho đến khi tự mình chứng kiến bộ xử lý hoạt động và tự mình xem xét các tính năng của chúng.

Datasheet tồn tại ở hầu hết các thành phần, cả phần cứng lẫn phần mềm và thông tin từ các nhà cung cấp khác nhau là khác nhau. Một số datasheet chỉ dài vài trang và chỉ xuất hiện ở những thành phần chính của hệ thống. Trong khi một số khác lại không dưới 100 trang thông số kĩ thuật.

2.1.2 Bộ nhớ

Nền tảng nhưng có sự phân cấp bộ nhớ, 1 tập hợp gồm các loại bộ nhớ khác nhau, mỗi loại đều có tốc độ, kích cỡ và cách sử dụng riêng biệt (Xem Hình 2-12). Một vài bộ nhớ có thể được tích hợp sẵn trong bộ xử lý, như các thanh ghi và các loại bộ nhớ sơ cấp đã biết, là nơi mà bộ nhớ có thể kết nối trực tiếp hay tích hợp trong bộ xử lý như ROM, RAM và level-1 cache. Trong chương này, ta nói đến loại bộ nhớ điển hình bên ngoài bộ xử lý, hay có thể là cả hai cùng được tích hợp bên trong bộ xử lý hay cùng bên ngoài bộ xử lý, đó là vấn đề cần thảo luận. Chương này bao gồm các loại bộ nhớ sơ cấp, như ROM, level-2+ cache, và bộ nhớ chính, và bộ nhớ cấp2, cấp3, đó là những loại bộ nhớ có thể kết nối tới bản mạch nhưng không tới bộ xử lý chủ trực tiếp được, như CD-ROM, ổ mềm, ổ cứng và băng từ.



Hình 2. 12: Sự phân cấp bộ nhớ

Bộ nhớ sơ cấp là một phần đặc trưng của hệ thống con của bộ nhớ, gồm 3 phần:

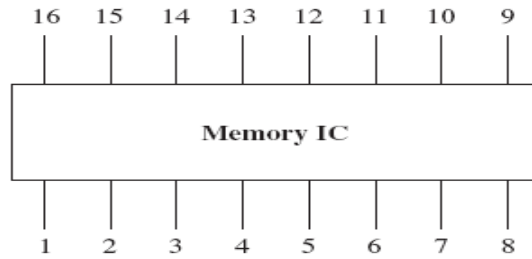
- IC nhớ
- Bus địa chỉ
- Bus dữ liệu

Thông thường, một IC nhớ gồm 3 bộ phận: mảng nhớ, bộ giải mã địa chỉ, và giao diện dữ liệu. Mảng nhớ thực tế là bộ nhớ vật lý để lưu trữ các bit dữ liệu. Trong khi bộ xử lý chủ, và các hệ lập trình, thiết lập bộ nhớ như mảng một chiều, mỗi ô trong mảng là một hàng các byte và số bit trên một hàng có thể biến thiên, trong bộ nhớ vật lý thực tế là mảng 2 chiều được tạo thành bởi các ô nhớ định địa chỉ bởi mỗi hàng và cột duy nhất, mỗi ô có thể lưu trữ 1 bit.

Bộ phận chính còn lại của IC nhớ, bộ giải mã địa chỉ, định vị địa chỉ của dữ liệu trong mảng nhớ, làm nền tảng cho những thông tin nhận được qua bus địa chỉ, và giao diện của dữ liệu sẽ cung cấp các dữ liệu tới bus dữ liệu trong quá trình vận chuyển. Bus địa chỉ và bus dữ liệu đều nhận địa chỉ và dữ liệu từ bộ giải mã địa chỉ và giao diện địa chỉ của IC nhớ.

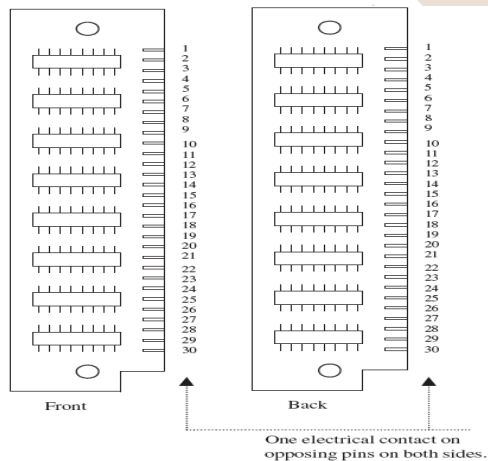
Các IC nhớ có thể kết nối tới bản mạch của nhiều loại gói khác nhau, phụ thuộc vào từng loại bộ nhớ. Các loại gói bao gồm loại vỏ 2 hàng chân (DIP), môđun nhớ từng dòng đơn lẻ (SIMM), và môđun nhớ 2 dòng (DIMM). Như chỉ dẫn ở Hình 2-16a, DIP là các gói bao quanh IC, được làm từ gốm hoặc chất dẻo. Số chốt có thể đa dạng giữa các IC nhớ, nhưng sơ đồ chân thực tế của các IC nhớ khác nhau đều tuân theo chuẩn JEDEC để đơn giản lệnh giao diện bên ngoài của các IC nhớ tới bộ xử lý.

SIMM và DIMM (được nêu ở Hình 2-16b và c) là các môđun nhỏ (PCB) chứa một vài các IC nhớ. SIMM và DIMM có các chân nhô ra từ 1 phía (cả 2 cùng nằm phía trước hoặc sau) của môđun để kết nối tới bản mạch nhúng chính. Cấu hình của SIMM và DIMM có thể cùng thay đổi trong kích cỡ của IC nhớ trên môđun (256KB, 1MB, ...) Ví dụ, 256K x 8 SIMM là một môđun yêu cầu 256K (256*1024) địa chỉ cho mỗi byte. Để hỗ trợ cho bộ xử lý chủ 16-bit, 2 trong số các SIMM sẽ cần dùng; để hỗ trợ cho cấu trúc 32-bit, 4 SIMM của cấu hình cần dùng đến, v.v...

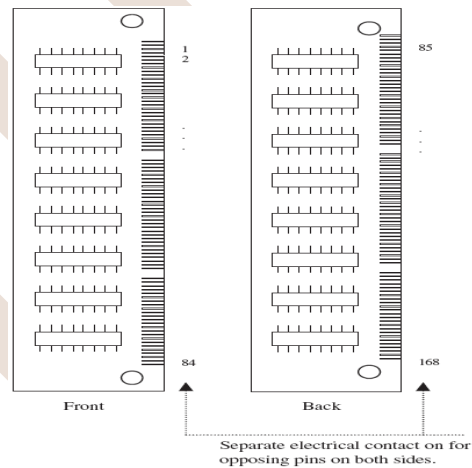


Hình 2. 13: Ví dụ DIP

Số chân nhô ra từ các SIMM và DIMM có thể khác nhau (30 chân, 72 chân, 168 chân, v.v.). Ưu điểm của SIMM và DIMM là có nhiều chân hơn để nó cấp phát cho ít môđun cần đến hỗ trợ cấu trúc rộng hơn. Do vậy, ví dụ như, 1 SIMM 72 chân (256K x32) sẽ thay thế cho 4 SIMM 30 chân (256K x8) cho cấu trúc 32 bit. Vậy, sự khác nhau lớn nhất giữa SIMM và DIMM là đặc trưng của các chân trên môđun thế nào: ở SIMM, loại 2 chân trong cùng bản mạch được kết nối, tạo một tiếp xúc, trái lại, ở DIMM các chân ngược lại đều có các tiếp xúc độc lập.



Hình 2. 14: Ví dụ SIMM 30 chân



Hình 2. 15: Ví dụ DIMM 168 chân

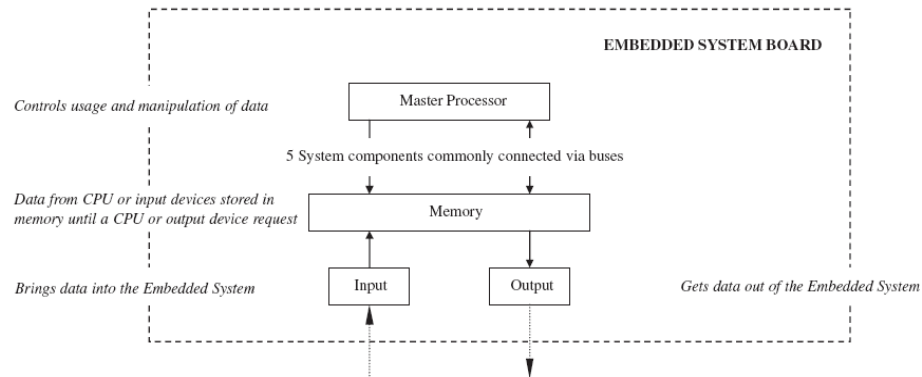
Ở mức cao nhất, cả bộ nhớ sơ cấp và cấp 2 đều có thể được chia thành 2 nhóm, cố định và khả biến. Bộ nhớ không bốc hơi là bộ nhớ có thể lưu trữ dữ liệu sau khi nguồn điện chính cấp cho bảng mạch đã được tắt (thường là do nhỏ, gắn trên bảng mạch, có pin nguồn cấp lâu dài). Bộ nhớ khả biến sẽ làm mất các bit dữ liệu của nó khi nguồn điện chính trên bảng mạch bị tắt. Trên mạch nhúng, có 2 loại bộ nhớ không thay đổi- bộ nhớ chỉ đọc (ROM) và bộ nhớ ngoài- và một họ bộ nhớ thay đổi, bộ nhớ truy cập tức thời (RAM).

2.1.3 Bảng mạch Vào/Ra

Các bộ phận vào/ra trên bảng mạch là thiết bị chịu trách nhiệm di chuyển thông tin đến và đi của bảng mạch đến thiết bị vào/ra kết nối tới một hệ thống nhúng. Bảng mạch vào/ra có thể gồm những bộ phận đầu vào, thiết bị chỉ đưa thông tin từ một thiết bị

đầu vào đến bộ xử lý chính, bộ phận đầu ra là cái lấy thông tin ra của bộ xử lý chính đưa đến 1 thiết bị đầu ra. Hoặc là gồm cả thiết bị vào và thiết bị ra.

Bất kỳ hệ thống điện cơ nào, được nhúng hay không nhúng, cho dù thông thường hay không thông thường, có thể được kết nối tới một bảng mạch nhúng và hoạt động như một thiết bị vào/ra. Vào/ra ở mức cao có thể được chia nhỏ ra thành những bộ nhỏ hơn của thiết bị ra, thiết bị vào, và cả thiết bị mà bao gồm cả thiết bị ra và thiết bị vào. Thiết bị ra nhận dữ liệu từ những bộ phận bảng mạch I/O và hiển thị dữ liệu đó theo một số cách thức, ví dụ : in dữ liệu ra giấy, in ra đĩa, hiển thị lên màn hình hoặc là nhấp nháy đèn LED... Thiết bị vào ví dụ như chuột, bàn phím hoặc là điều khiển từ xa truyền dữ liệu đến bảng mạch I/O. Một số thiết bị vào ra có thể làm cả 2, vừa có thiết bị kết nối vừa có thể truyền dữ liệu đi. Ví dụ một thiết bị vào/ra có thể kết nối tới 1 bảng mạch nhúng thông qua dây nối hoặc môi trường truyền dữ liệu không dây, ví dụ 1 bàn phím hay điều khiển từ xa, hoặc có thể định vị ở trên chính bảng mạch nhúng ví dụ như đèn LED.



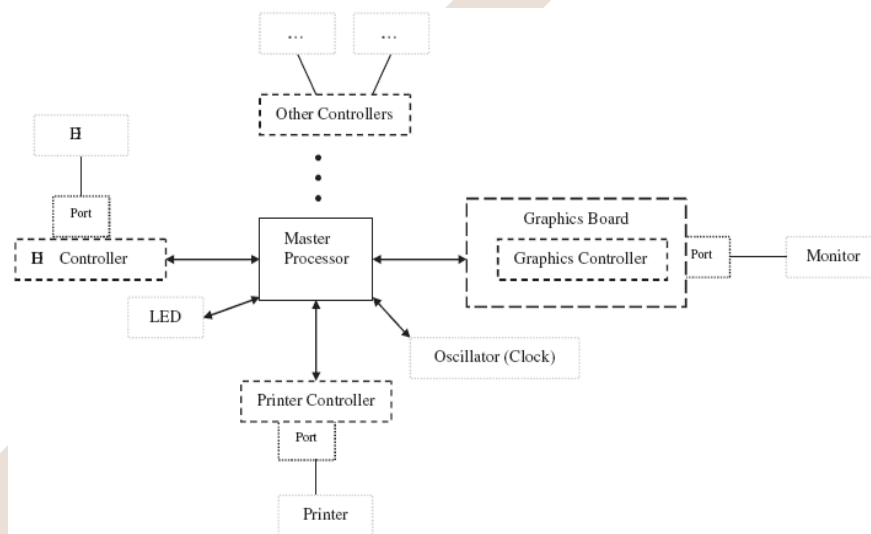
Hình 2. 16: Sơ đồ khối I/O cơ bản của kiến trúc Von Newman

Bởi vì các thiết bị vào/ra rất đa dạng, khoảng cách từ một mạch đơn giản đến hệ thống nhúng hoàn thành, linh kiện của bảng mạch vào ra có thể chênh lệch một hoặc nhiều loại khác nhau. Những điều giống nhau nhất gồm:

- Liên kết mạng và kết nối vào/ra
- Thiết bị vào
- Sơ đồ và thiết bị ra
- Thiết bị điều chỉnh vào /ra
- Thời gian thực và đa dạng vào/ra (Bộ thời gian/đếm, chuyển từ tương tự sang số, chuyển từ số sang tương tự...)

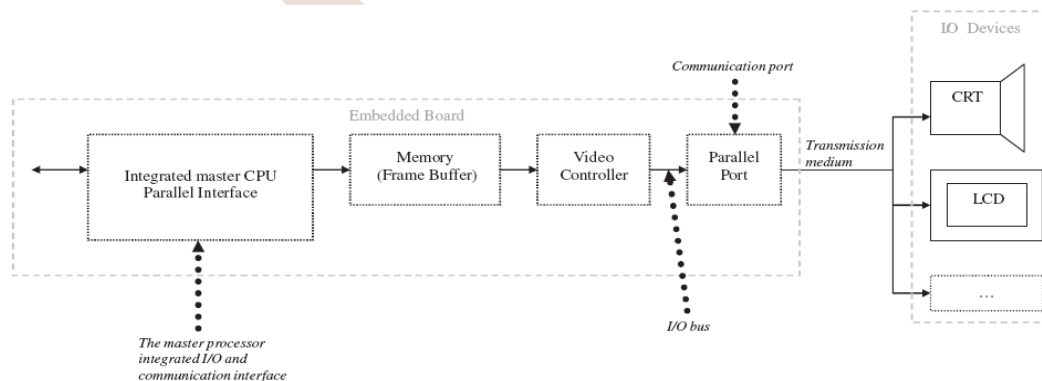
Ban đầu, bảng mạch vào/ra rất đơn giản chủ yếu là mạch điện kết nối với bộ xử lý chính, cổng vào/ra của bộ xử lý chính như là cổng xung nhịp hay đèn LED được định thời trên bảng. Phần cứng của I/O điển hình được làm từ tất cả hoặc một số kết hợp của 6 bộ logic chính sau:

- Phương tiện truyền thông, đường truyền không dây hoặc có dây kết nối với thiết bị vào/ra đến dữ liệu thông tin và chuyển đổi của bảng mạch nhúng.
- Cổng thông tin là phương tiện truyền thông kết nối tới bảng mạch hoặc nếu có hệ thống không dây thì nhận tín hiệu không dây.
- Giao diện truyền thông, bộ phận quản lý dữ liệu thông tin giữa CPU chủ và thiết bị vào/ra hoặc điều khiển vào/ra và nó chịu trách nhiệm mã hoá và giải mã dữ liệu và từ mức logic của một IC và mức logic của một cổng vào/ra. Giao diện này có thể được tích hợp ở trong bộ xử lý chính hoặc có thể là một IC riêng rẽ.
- Một điều khiển vào/ra xử lý quản lý thiết bị vào/ra.
- Đường truyền dẫn vào/ra là cái kết nối giữa bảng mạch vào/ra và bộ xử lý chính
- Bộ xử lý chính tích hợp vào /ra.

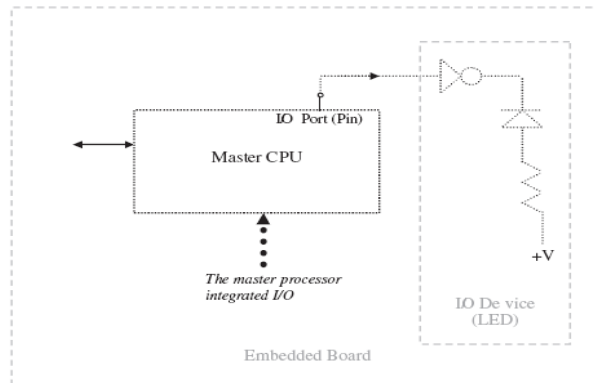


Hình 2. 17: Các cổng và bộ điều khiển thiết bị điều khiển trên một bảng mạch nhúng

Bảng mạch vào/ra có thể giới hạn từ thành phần hệ thống phức tạp Hình 2.18. Một số thành phần bảng mạch vào/ra tích hợp xem Hình 2-19.



Hình 2. 18: Bảng mạch I/O phức tạp



Hình 2. 19: Bảng mạch I/O đơn giản

Hiện tại sự lắp ráp của 1 hệ thống cài đặt vào/ra ở trên 1 bảng mạch nhúng, có thể dùng kết nối và cổng hoặc dùng 1 thiết bị điều khiển vào/ra, là phụ thuộc của thiết bị kết nối vào/ra định vị trên bảng mạch nhúng. Nó có nghĩa là trong khi một số chỉ số như là an toàn và mở rộng rất là quan trọng trong việc thiết kế một hệ thống vào/ra. Đọc các thông số của bộ phận chính sau khi thiết kế hệ thống vào ra chính là đường bao quanh thiết bị vào/ra - giới hạn mục đích -chất lượng.

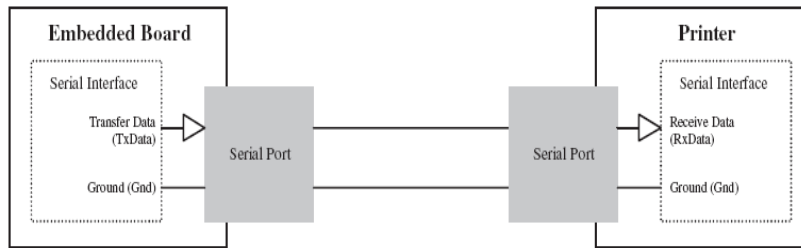
2.1.3.1 Quản lý dữ liệu nối tiếp

Bảng mạch I/O này có thể truyền dữ liệu và nhận dữ liệu nối tiếp. Phần cứng I/O nối tiếp là kiểu được tạo sẵn của hệ thống được kết hợp 6 phần logic chính đã được nói qua ở trên. Sự truyền thông nối tiếp bao gồm hệ thống con I/O bên trong một cổng nối tiếp và một giao diện nối tiếp.

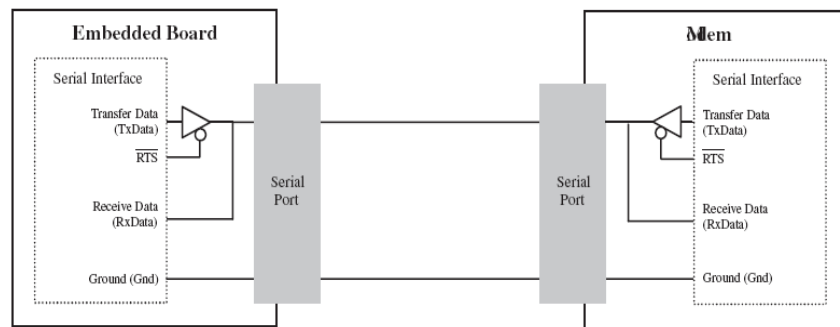
Giao diện nối tiếp là chuỗi dữ liệu được truyền và nhận giữa CPU chính và một số thiết bị I/O hoặc bộ điều khiển của nó. Chúng bao gồm nhận và truyền dữ liệu từ bộ đệm tới bộ lưu trữ và giải mã hoặc mã hoá dữ liệu và chúng có trách nhiệm truyền dữ liệu tới CPU chính khác hoặc thiết bị vào ra khác. Như vậy là sự thật là truyền và nhận dữ liệu trên một từ, dữ liệu truyền và nhận giới hạn bởi ghép nối tiếp.

Dữ liệu có thể truyền giữa 2 thiết bị trong 1 của 3 hướng: trong 1 chiều hướng, trong cả 2 hướng trong thời gian riêng, bởi vì nó có thể chia sẻ những đường dữ liệu giống nhau và cả 2 hướng tương thích. Chuỗi dữ liệu thông tin vào/ra dữ liệu nối tiếp vào/ra có thể dùng sơ đồ đơn giản mô tả dữ liệu nối tiếp có thể truyền hoặc nhận trong 1 bộ xử lý.

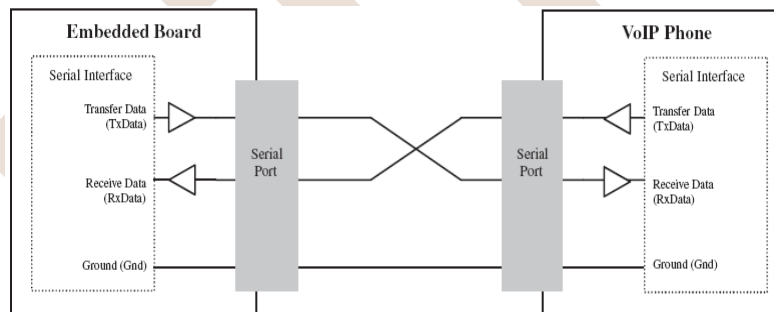
Sơ đồ bán song công mô tả chuỗi dữ liệu có thể truyền hoặc nhận ở một bộ vi xử lý khác nhưng 1 bộ xử lý chỉ truyền 1 lần. Sơ đồ chuỗi song công mô tả chuỗi dữ liệu có thể truyền hoặc nhận ở bộ vi xử lý khác tương thích.



Hình 2. 20: Ví dụ sơ đồ truyền đơn giản



Hình 2. 21: Ví dụ sơ đồ truyền bán song công

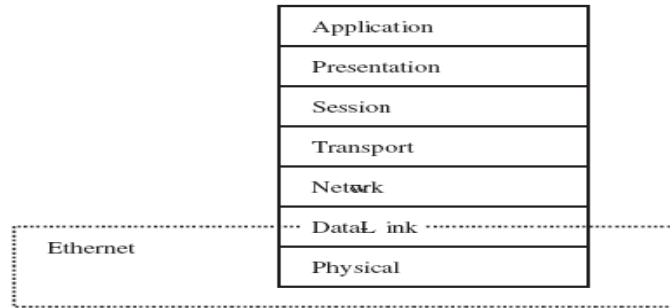


Hình 2. 22: Ví dụ sơ đồ truyền song công

VÍ DỤ: CỔNG NỐI TIẾP VÀO/RA: Liên kết mạng và truyền thông RS232.

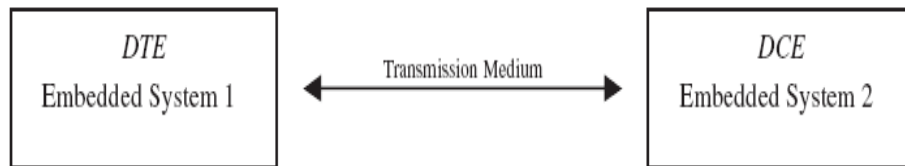
Giao thức vào/ra nối tiếp có thể truyền đồng bộ hoặc không đồng bộ: RS232, EIA232. EIA hiệp hội công nghiệp điện tử. Một số chuẩn định nghĩa những thành phần chính của hệ thống RS_232, những cái được thực hiện trong phần cứng.

Thành phần của phần cứng là tất cả lớp ánh xạ vật lý của OSI phần mềm yêu cầu khởi động chức năng ánh xạ RS232 đến những đoạn dưới của đường dữ liệu, nhưng chúng ta sẽ không thảo luận ở phần này.



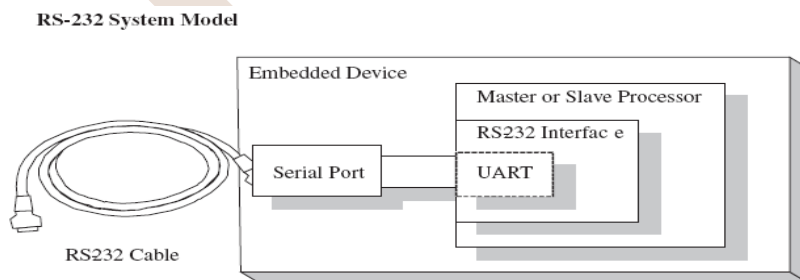
Hình 2. 23: Mô hình OSI

Sự phù hợp của chuẩn EIA232, thiết bị thích hợp của RS232 (Xem Hình 2-19) được gọi là DTE và DCE. Thiết bị DTE là sự khởi đầu của thông tin nối tiếp như là PC hay hệ thống nhúng. DCE là thiết bị DTE muốn truyền qua. Ví dụ như thiết bị vào ra kết nối với hệ thống nhúng.



Hình 2. 24: Sơ đồ mạng nối tiếp

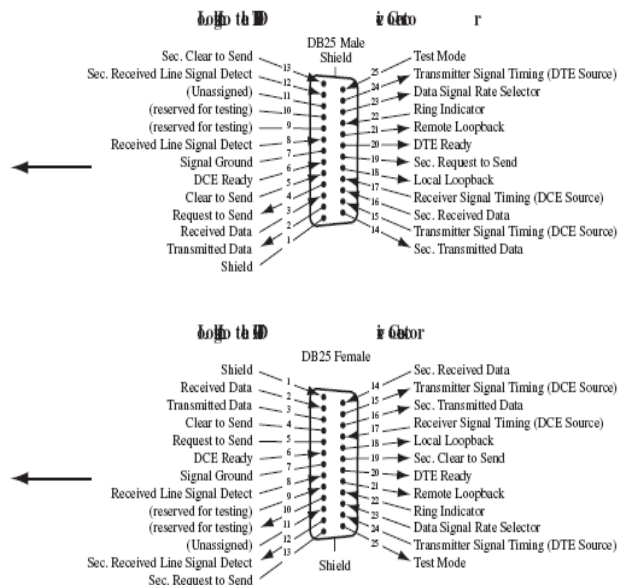
Bản chất đặc tính kỹ thuật của RS232: *RS232 interface*. RS232 interface là định nghĩa chi tiết của cổng nối tiếp và những tín hiệu với những mạch điện được bổ sung là những tín hiệu ánh xạ từ 1 cổng giao diện đồng bộ hoặc không đồng bộ (ví dụ UART) được mở rộng bởi thiết bị vào/ra. RS232 được định nghĩa như phương tiện truyền dữ liệu là những dây nối tiếp. Kết nối RS232 phải có cả hai cổng truyền thông tin (DTE và DCE hoặc hệ thống nhúng và thiết bị vào/ra).



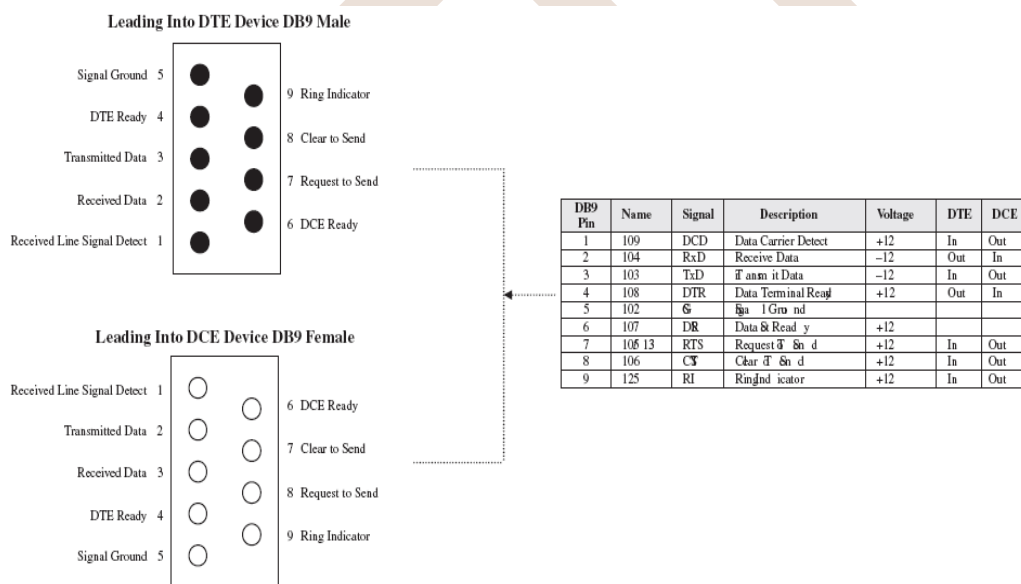
Hình 2. 25: Sơ đồ khối những thành phần nối tiếp

Dùng số tín hiệu để phân biệt giữa chuẩn EIA 232 và chuẩn RS232. Chuẩn RS 232 dùng tất cả 25 tín hiệu gọi là kết nối DB25 xem Hình 2-30a. Chuẩn EIA có 8 tín hiệu phù hợp với kết nối DB9 xem Hình 2-21b. Chuẩn EIA 561 có 8 tín hiệu phù hợp với kết nối RJ45 xem Hình 2-21c.

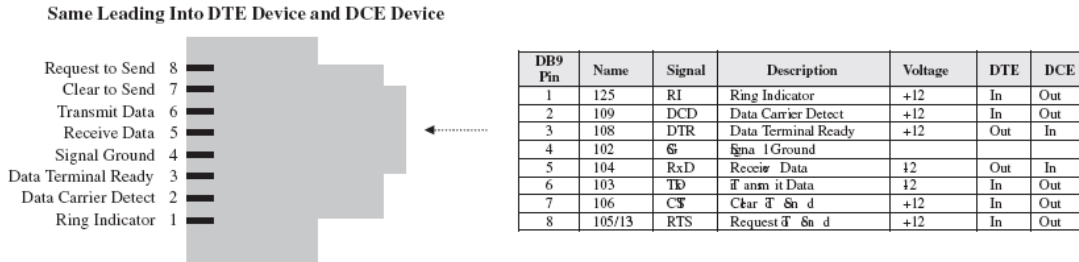
DB9 Pin	Name	Signal	Description	Voltage	DTE	DCE
1		FG	Frame Ground/Shield		Out	In
2	BA	TxD	Transmit Data	-12	In	Out
3	BB	RxD	Receive Data	-12	Out	In
4	CA	RTS	Request To Send	+12	In	Out
5	CB	CTS	Clear To Send	+12	In	Out
6	CC	DSR	Data Set Ready	+12		
7	AB	SG	Signal Ground			
8	CF	DCD	Data Carrier Detect	+12	In	Out
9			Positive Test Voltage			
10			Negative Test Voltage			
11			Not Assigned			
12		sDCD	Secondary DCD	+12	In	Out
13		sCTS	Secondary CTS	+12	In	Out
14		sTxD	Secondary TxD	-12	Out	In
15	DB	TxC	DCE Transmit Clock		In	Out
16		sRxD	Secondary RxD	-12	In	Out
17	DD	RxC	Receive Clock		In	Out
18	LL		Local Loopback			
19		sRTS	Secondary RTS	+12	Out	In
20	CD	DTR	Data Terminal Ready	+12	Out	In
21	RL	SQ	Signal Quality	+12	In	Out
22	CE	RI	Ring Indicator	+12	In	Out
23		SEL	Speed Selector DTE		In	Out
24	DA	TCK	Speed Selector DCE		Out	In
25	TM	TM	Test Mode	+12	In	Out



Hình 2. 26: Các tín hiệu RS-232 và đầu nối DB25



Hình 2. 27: Các tín hiệu RS-232 và đầu nối DB9

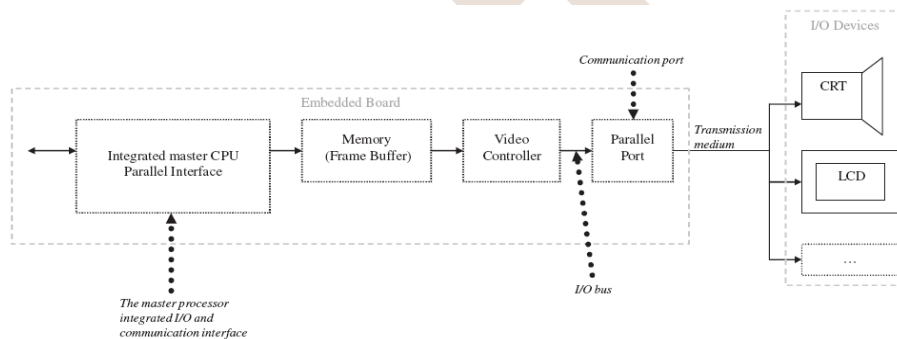


Hình 2. 28: Các tín hiệu RS-232 và đầu nối RJ45

Hai thiết bị DTE có thể kết nối với nhau thông qua *null modem*. DTE có thể truyền và nhận dữ liệu trên 1 chân. Những chân null modem trao đổi nhận và truyền kết nối trên mỗi thiết bị DTE được điều phối.

2.1.3.2 Giao diện các thành phần I/O

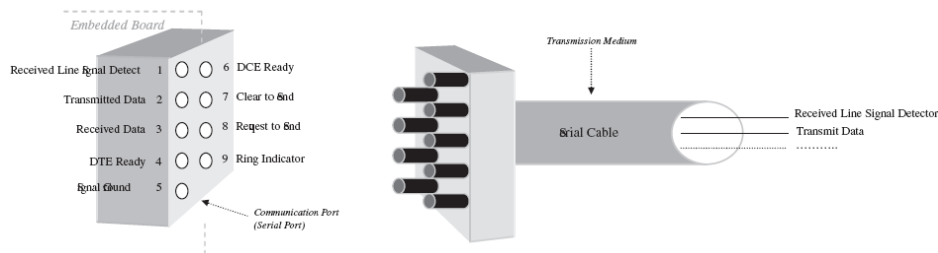
Phần cứng vào/ra được làm từ tất cả hoặc kết hợp của bộ xử lý chính vào/ra hoặc bộ điều khiển vào/ra, bus vào/ra và môi trường truyền dữ liệu. Xem Hình 2-22.



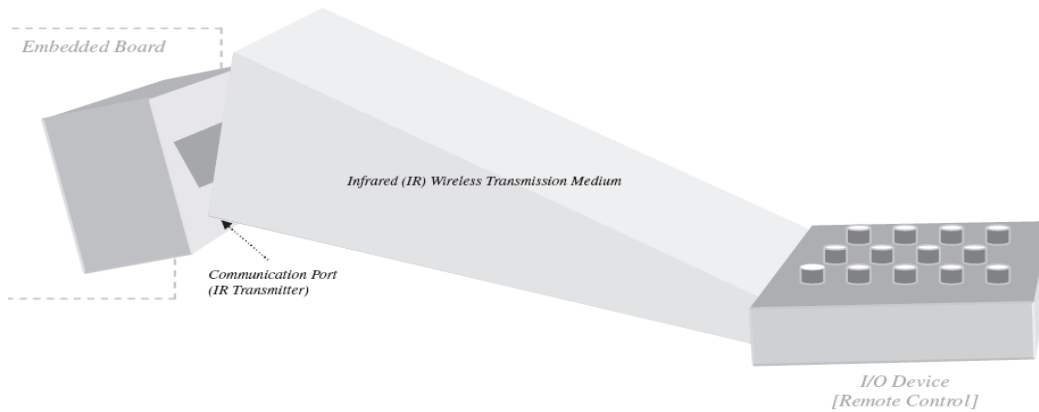
Hình 2. 29: Hệ thống con I/O mẫu

Giao diện thiết bị I/O với bảng mạch nhúng

Thiết bị vào/ra ví dụ như bàn phím, LCD, máy in kết hợp bảng mạch nhúng thông qua cổng thông tin. Môi trường không dây Hình 2.31 hoặc môi trường dây Hình 2.30.



Hình 2. 30: Phương tiện truyền tin dùng dây

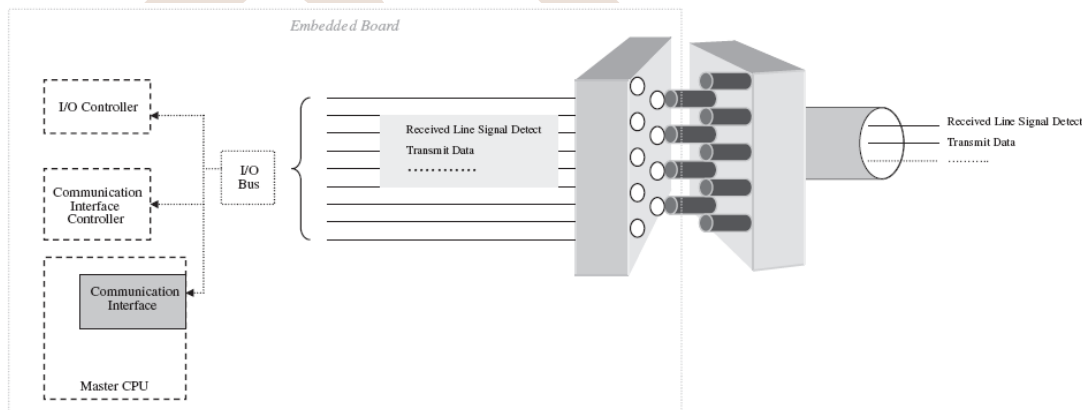


Hình 2. 31: Phương tiện truyền tin không dây

Trong hình 2.30, môi trường truyền dữ liệu giữa thiết bị vào/ra và bảng mạch nhúng bằng dây dẫn. Trong hình 2.31, môi trường truyền dữ liệu là không dây.

Giao diện điều khiển thông tin, bộ xử lý chính thông qua bus vào/ra trên hệ thống nhúng bus vào/ra về bản chất là tập hợp truyền dữ liệu bằng dây dẫn.

Thiết bị vào/ra có thể kết nối bộ xử lý chính thông qua cổng vào/ra. Nếu thiết bị vào/ra định vị trên bảng mạch hoặc có thể kết nối gián tiếp dùng 1 giao diện truyền thông trong bộ xử lý chính hoặc trên 1 IC riêng biệt trên bảng mạch và 1 cổng giao tiếp. Giao diện thông tin kết nối trực tiếp với thiết bị vào/ra hoặc con trỏ điều khiển thiết bị vào/ra. Ngoài tấm mạch của thiết bị vào/ra các thành phần bảng mạch vào/ra có liên quan thông qua bus vào/ra.



Hình 2. 32: Giao diện cổng truyền tin tới I/O tấm bảng mạch khác

Giao diện một bộ điều khiển I/O và CPU chủ

Hệ con bao gồm 1 bộ điều khiển quản lý thiết bị vào/ra, thiết kế giao diện giữa bộ điều khiển I/O và CPU chủ thông qua 1 giao diện thông tin – 4 yêu cầu cơ bản sau:

- Năng lực của máy chủ CPU khởi tạo và điều chỉnh thiết bị vào/ra. Bộ điều khiển vào/ra có thể đặc trưng cho cấu hình thông qua bộ điều khiển bộ ghi và bộ điều

chỉnh thông qua trạng thái bộ ghi. Những bộ ghi được định vị trên bộ điều khiển vào/ra. Dữ liệu bộ ghi được máy chủ xử lý có thể điều chỉnh cấu hình trên bộ điều khiển vào/ra. Trạng thái bộ đếm chỉ đọc trên bộ đếm, máy chủ có thể xử lý thông tin phụ thuộc vào trạng thái của bộ điều khiển vào/ra. Máy chủ CPU dùng trạng thái và bộ điều khiển để nhớ thông tin đến thiết bị vào/ra thông qua bộ điều khiển vào/ra.

- Đường đi của bộ xử lý chính đến yêu cầu thiết bị vào/ra: bộ xử lý chính yêu cầu thiết bị vào/ra thông qua bộ điều khiển vào/ra.
- Đường đi của thiết bị vào/ra kết nối với máy chủ CPU: bộ điều khiển vào ra kết nối với máy chủ CPU thông qua các ngắt.
- Bộ dẫn động thay đổi dữ liệu: trong 1 chương trình bộ xử lý chính nhận dữ liệu từ bộ điều khiển vào/ra trong bộ đếm và CPU truyền dữ liệu đến bộ nhớ.

2.1.3.3 I/O và hiệu suất

Gồm các đặc trưng:

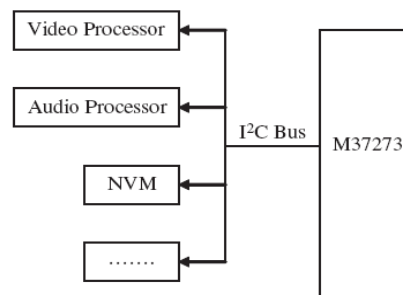
- Tốc độ truyền dữ liệu của thiết bị vào/ra
- Tốc độ của bộ xử lý chính
- Cách đồng bộ hóa giới hạn của bộ xử lý chính và giới hạn của thiết bị vào/ra.
- Cách để vào/ra và bộ xử lý chính liên lạc

Những thành phần để đo đặc trưng của thiết bị vào/ra gồm :

- Lưu lượng của các thành phần vào/ra khác nhau
- Thời gian thực hiện của các thành phần vào/ra
- Thời gian phản hồi hoặc là độ trễ của thiết bị vào/ra.

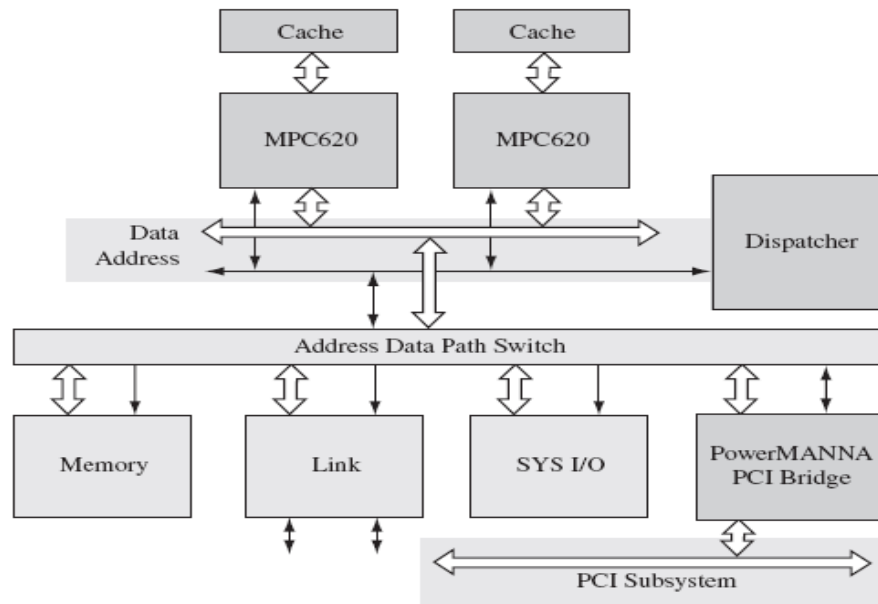
2.1.5 Hệ thống Bus

Tất cả các bộ phận chính khác nhau tạo nên một bảng nhúng- bộ xử lý tổng thể, bộ phận vào ra I/O, bộ nhớ - được kết nối với nhau thông qua các bus trên bảng nhúng. Định nghĩa ban đầu, bus là một kết nối đơn giản của tập hợp các dây mang các tín hiệu khác nhau, địa chỉ, và kiểm soát tín hiệu (tín hiệu đồng hồ, yêu cầu, nhận biết tín hiệu, ...) giữa tất cả các thành phần chính khác trên bảng nhúng, trong đó bao gồm các hệ thống con vào ra I/O, hệ thống bộ nhớ con và bộ xử lý tổng thể. Trên bảng nhúng có ít nhất 1 bus liên kết với các thành phần chính khác trong hệ thống (Hình 2-26).



Hình 2. 33: Cấu trúc bus chung

Trên một bản phức tạp, nhiều bus có thể được tích hợp trên một bảng. Một bảng nhúng với một vài kết nối các thành phần để liên lạc, cầu (bridge) trên các bảng kết nối bus khác nhau và mang thông tin từ bus khác. Trong Hình 2-27 cầu PCI là một trong những ví dụ. Một cầu có thể tự động cung cấp một cách minh bạch bản đồ của địa chỉ thông tin khi dữ liệu được di chuyển từ một bus khác, thực hiện điều khiển tín hiệu yêu cầu cho các bus nhận chu kỳ, ví dụ như: cũng như sửa đổi các dữ liệu được truyền đi nếu chuyển đổi giao thức khác từ bus đến bus. Nếu một byte sắp xếp khác, cầu có thể xử lý việc trao đổi byte.



Hình 2. 34: Kiến trúc MPC620 với cầu

Bảng bus đặc trưng bởi một trong 3 loại: hệ thống bus, bus đa năng, hoặc bus vào ra I/O. Hệ thống bus (Cũng được xem là chính, cục bộ, bộ xử lý, bộ nhớ bus) kết nối bên ngoài bộ nhớ chính và giữ CPU chỉ và/hoặc bất kì bridge đến bus khác. Hệ thống bus thường ngắn hơn, tốc độ cao hơn, tùy chỉnh bus. Backplane Bus đa năng thường nhanh hơn bus mà ở bộ nhớ kết nối, bộ xử lý tổng thể, I/O tất cả trên 1 bus. Các bus I/O cũng được gọi là bus “mở rộng”, “bên ngoài” hoặc “chủ”, kết quả hoạt động như phần mở rộng của hệ thống bus để kết nối các thành phần còn lại cho CPU với nhau, đến hệ thống bus qua bridge và/hoặc đến các hệ thống nhúng của riêng mình, qua cổng thông tin I/O. Các Bus I/O là các bus đặc trưng tiêu chuẩn hóa có thể ngắn hơn, tốc độ bus cao hơn, như PCI và USB hoặc bus dài hơn, chậm hơn ví dụ như SCSI.

Sự khác biệt chính giữ hệ thống bus và bus I/O là có IRQ điều khiển tín hiệu trên bus I/O. Có nhiều cách khác nhau I/O và bộ xử lý chủ có thể giao tiếp và ngắt là một trong những phương thức phổ biến nhất. Một IRQ vào ra trên I/O trên một bus để cho phép đến bộ xử lý trung tâm một sự kiện đã xảy ra hoặc một hành động đã được hoàn thành bởi tín hiệu trên bus IRQ. Khác I/O bus có thể có tác động khác trên sơ đồ ngắt.

Một bus ISA, ví dụ như yêu cầu mỗi thẻ mà tạo ra ngắt phải chỉ định của nó giá trị IRQ duy nhất. Bus PCI cho phép hai hoặc nhiều thẻ I/O phân chia giá trị giống như IRQ.

Trong mỗi loại bus, nhiều bus có thể được chia thành hoặc bus là mở rộng hoặc là không mở rộng. Một bus mở rộng (PCMCIA, PCI, IDE, SCSI, USB....) là một trong những thành phần bổ sung có thể cắm trong bảng on-the-fly, nhưng trái lại một **bus không mở rộng** (DIB, VME, I2C là những ví dụ) là một trong những thành phần bổ sung không thể cắm đơn giản trong bảng và sau đó liên lạc trên bus đến các thành phần khác.

Trong khi thực hiện hệ thống bus được mở rộng linh hoạt hơn bởi vì các thành phần có thể được thêm đặc biệt cho các bus và làm việc “ngoài khung”, mở rộng các bus có xu hướng đắt hơn khi thực hiện. Nếu bảng không được thiết kế ban đầu với tất cả các loại có thể có của các thành phần có thể được thêm vào trong bộ nhớ sau này, chất lượng có thể bị ảnh hưởng xấu bởi việc bổ sung quá nhiều “đường dẫn” hoặc các thành phần được thiết kế kém trên bus mở rộng.

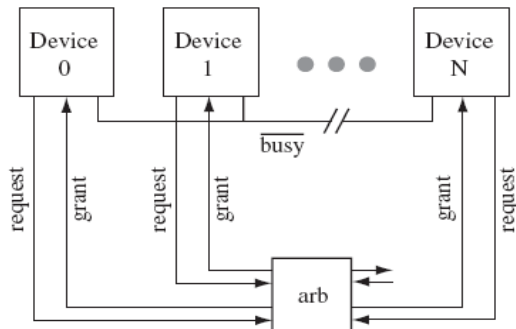
2.1.5.1 TRỌNG TÀI BUS VÀ ĐỊNH THỜI

Được liên kết với bus là một số loại giao thức được định nghĩa cách các thiết bị truy nhập vào bus (trọng tài) các qui định kèm theo các thiết bị phải tuân để giao tiếp trên bus (bắt tay) và các tín hiệu liên kết với các dòng bus khác nhau.

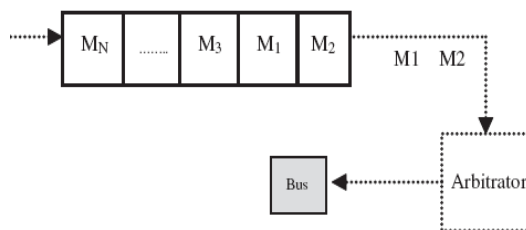
Bảng các thiết bị được truy cập vào một bus đang dùng một hệ thống trọng tài bus. Trọng tài Bus được dựa trên các thiết bị đang được phân loại như là một trong hai thiết bị chủ (các thiết bị có thể bắt đầu một giao dịch bus - transaction) hoặc các thiết bị phụ (là các thiết bị mà có thể truy nhập vào một bus để đáp ứng với yêu cầu của thiết bị chủ). Một hệ thống trọng tài bus đơn giản nhất là chỉ có một thiết bị ở trên bảng (board) – xử lý chủ để được cho phép tới máy chủ, trong khi tất cả các thiết bị khác là thiết bị phụ. Trong trường hợp này, không cần có trọng tài khi có thể chỉ là một máy chủ.

Nhiều bus có thể cho phép nhiều máy chủ, có một số phân xử (phân chia sơ đồ mạch phần cứng) xác định tổng thể một máy chủ được điều khiển của bus. Có một vài sơ đồ trọng tài bus được sử dụng cho bus nhúng đang được phổ biến nhất song song tập trung động, tập trung theo từng chuỗi (daisy-chain) và tự lựa chọn phân phối.

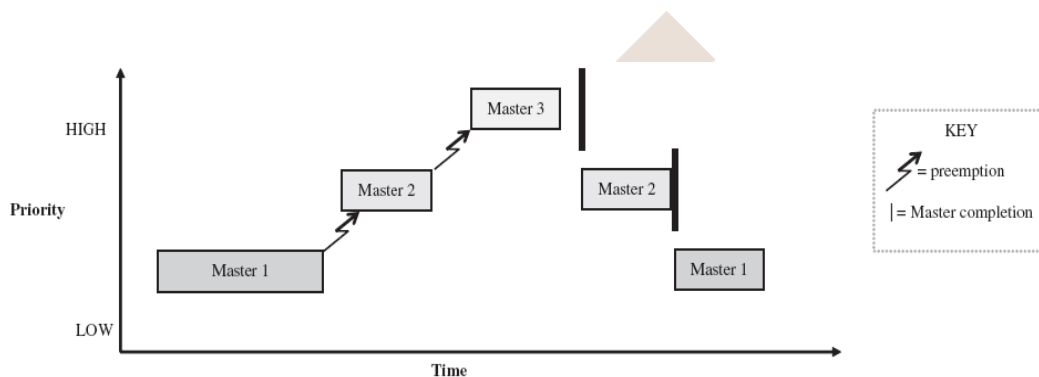
Sự phân xử song song tập trung động là sơ đồ trong đó trọng tài ở vị trí trung tâm. Tất cả các bus chủ kết nối với trọng tài trung tâm. Trong sơ đồ này, các máy chủ sau khi được cấp quyền truy cập đến bus thông qua FIFO hoặc được ưu tiên trên cơ sở hệ thống. Thuật toán FIFO thực hiện một số loại hàng đợi lưu trữ trong danh sách của các thiết bị chủ sẵn sàng sử dụng bus theo thứ tự của yêu cầu bus. Các thiết bị chủ được thêm ở cuối hàng đợi và được phép truy nhập đến bus từ đầu hàng đợi. Một nhược điểm chính là khả năng của trọng tài không can thiệp nếu một máy chủ ở trước hàng đợi duy trì kiểm soát của bus không hoàn thành hoặc không cho phép máy chủ truy cập bus.



Hình 2. 35: Sự phân xử song song tập trung động



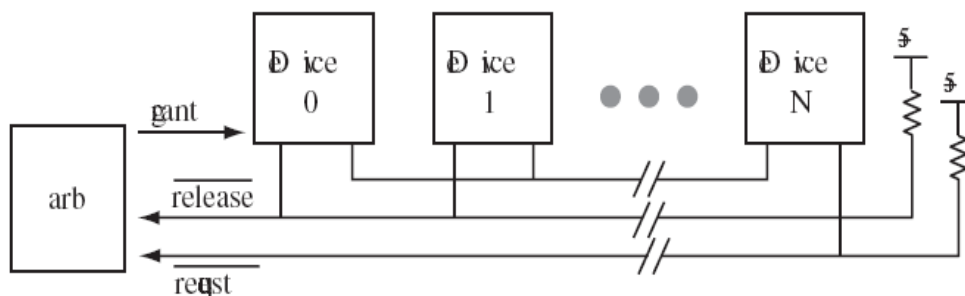
Hình 2. 36: Sự phân xử trên nền tảng FIFO



Hình 2. 37: Sự phân xử trên nền quyền ưu tiên

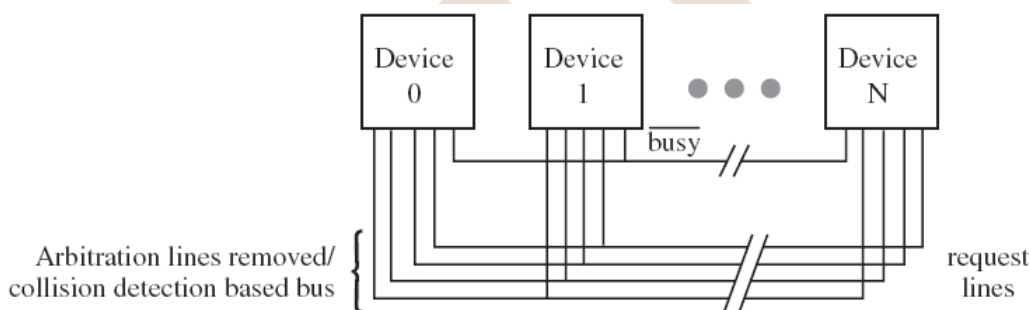
Hệ thống phân xử ưu tiên phân biệt với máy chủ dựa trên tầm quan trọng tương đối với nhau và hệ thống. Về cơ bản thì mỗi thiết bị chủ được ưu tiên gán giá trị, hành động như một chỉ thị của thứ tự ưu tiên trong hệ thống. Nếu bộ phân xử thực hiện quyền ưu tiên trên cơ sở hệ thống, thiết bị chủ với quyền ưu tiên cao nhất luôn luôn có thể chặn trước những thiết bị chủ có ưu tiên thấp hơn khi chúng muốn truy cập đến bus, điều đó có nghĩa là một máy chủ đang truy cập vào bus có thể bị buộc phải giải phóng bởi trọng tài nếu muốn ưu tiên máy chủ hơn bus. Hình 2.37 đã thể hiện 3 thiết bị chủ (1, 2, 3, thiết bị 1 là ưu tiên thấp nhất, ưu tiên cao nhất là thiết bị 3).

Thứ tự phân xử trung tâm, cũng được xem như chuỗi phân xử, là một hệ thống trong đó bộ phân xử được kết nối với các máy chủ và các máy chủ được kết nối thành chuỗi. Không kể đến việc máy chủ tạo yêu cầu cho bus, máy chủ đầu tiên ở trong chuỗi được cấp bus và chuyển sang “cấp bus” cho máy chủ tiếp theo trong chuỗi nếu/khi bus không cần thiết hơn nữa.



Hình 2. 38: Sự phân xử tuần tự/chuỗi tập trung

Các hệ thống phân bố sự phân xử, điều đó có nghĩa là không có bộ phân xử trung tâm, không có mạch bổ xung (Hình 2-30). Trong các hệ thống, máy chủ tự phân xử bằng cách trao đổi ưu tiên thông tin để quyết định nếu máy chủ ưu tiên cao hơn là tạo yêu cầu đến bus, hoặc đều nhau bằng cách loại bỏ tất cả các dòng phân xử và chờ xem nếu có sự xung đột trên bus, có nghĩa là bus đang làm việc với hơn một máy chủ đang cố sử dụng nó.



Hình 2. 39: Sự phân xử phân tán qua sự tự chọn

Mặt khác tùy thuộc vào bus, các bộ trọng tài bus có thể cấp 1 bus đến một máy chủ *atomically* (cho đến khi máy chủ hoàn thành việc truyền dẫn với nó) hoặc cho phép phân chia truyền dẫn, nơi mà bộ phân xử có thể tránh các thiết bị ở giữa việc thực hiện, chuyển đổi giữa các máy chủ để cho phép các máy chủ khác truy cập bus.

Một thiết bị chủ được cấp bus, chỉ có 2 thiết bị một là máy chủ và thiết bị khác phụ thuộc vào kiểu giao tiếp qua bus tại bất cứ thời điểm nào. Chỉ có 2 kiểu thực hiện một kiểu thiết bị có thể được (Hoặc nhận) và/hoặc viết (truyền). Các giao dịch này có thể diễn ra giữa một trong 2 bộ xử lý (Một là máy chủ, một là điều khiển I/O) hoặc bộ xử lý và bộ nhớ (một máy chủ và bộ nhớ). Trong mỗi loại giao dịch, dù đọc hay viết cũng có một vài qui tắc cụ thể mà mỗi thiết bị cần làm theo thứ tự để hoàn thành một giao dịch. Các qui tắc có thể thay đổi nhiều giữa các kiểu thiết bị thông tin, cũng như từ bus đến bus. Bộ qui tắc, thông thường được gọi là bắt tay bus, dạng cơ sở của bất kì giao thức bus nào.

Cơ sở của bất kì bắt tay bus nào là giới hạn sau cùng bởi bus định thời. Các bus dựa trên một hoặc một vài sự kết hợp đồng thời hoặc không đồng thời của hệ thống bus định thời cho phép các thành phần gắn liền với bus đồng bộ truyền hóa nó. Một bus đồng bộ bao gồm một tín hiệu đồng hồ giữa các tín hiệu khác nó truyền tải, chẳng hạn như dữ

liệu, địa chỉ hoặc các thông tin điều khiển khác. Các thành phần sử dụng bus đồng bộ tất cả đều chạy giống như nhịp đồng hồ như bus, (tùy thuộc vào từng bus) và dữ liệu được truyền trên xung cao hoặc thấp của chu kỳ xung nhịp. Thứ tự làm việc của một hệ thống, các thành phần hoặc phải đồng bộ cho tốt độ xung tăng lên, hoặc tốc độ xung phải chậm cho bus lâu hơn. Một bus quá dài với 1 tốc độ xung quá nhanh (hoặc quá nhiều thành phần gắn liền với bus) sẽ gây ra độ lệch trong đồng bộ truyền đi, bởi vì việc truyền đi trong một hệ thống như vậy sẽ không đồng bộ với xung nhịp. Diễn hình các bus nhanh hơn sử dụng đồng bộ hệ thống bus định thời.

Một bus không đồng bộ truyền tín hiệu không có xung đồng hồ, nhưng thay vào đó là truyền tín hiệu “bắt tay”, chẳng hạn như yêu cầu và nhận tín hiệu. Trong khi hệ thống không đồng bộ phức tạp hơn cho các thiết bị có yêu cầu phân phối lệnh, trả lời lệnh, và như vậy một bus không đồng bộ không có vấn đề gì với độ dài của bus hoặc một số lớn hơn của các thành phần giao tiếp trên bus, bởi vì một xung nhịp không phải là cơ sở để giao tiếp đồng bộ. Tuy nhiên một bus không đồng bộ cần một “bộ đồng bộ” khác để quản lý sự trao đổi thông tin và để khóa các thông tin liên lạc.

Có hai giao thức phổ biến nhất để bắt đầu bất cứ bắt tay bus nào là chỉ dẫn hoặc yêu cầu một giao tiếp (hoặc đọc hoặc viết) và phụ thuộc vào đáp ứng đến chỉ dẫn giao tiếp hoặc yêu cầu (ví dụ như một sự thừa nhận ACK hoặc yêu cầu ENQ). Cơ sở của hai giao thức này là điều khiển truyền tín hiệu thông qua điều khiển một dải bus riêng hoặc qua một dải dữ liệu. Dù nó là một yêu cầu dữ liệu tại bộ nhớ hay là giá trị của bộ điều khiển I/O điều khiển hoặc trạng thái đăng ký, nếu phụ thuộc vào đáp ứng trong việc xác định đến yêu cầu truyền của các thiết bị chủ, sau đó hoặc là một địa chỉ của dữ liệu liên quan đến việc truyền được trao đổi qua một dải bus riêng hoặc dải dữ liệu hoặc là địa chỉ này được truyền như một phần của việc truyền giống như yêu cầu truyền ban đầu. Nếu địa chỉ này hợp lệ, sau đó trao đổi dữ liệu trên đường truyền dữ liệu (thêm hoặc bớt một tập các nhận biết qua một dải khác hoặc ghép vào cùng một chuỗi). Chú ý giao thức bắt tay biến đổi khác với bus. Ví dụ, một bus yêu cầu truyền tải của yêu cầu và/hoặc nhận biết với mọi đường truyền, các bus khác chỉ đơn giản là cho phép truyền quảng bá của máy chủ đến tất cả các thiết bị bus, và chỉ phụ thuộc các thiết bị liên quan đến giao dịch truyền dữ liệu lại cho nơi gửi. Một ví dụ khác về sự khác nhau giữa các giao thức bắt tay, thay vì trao đổi phức tạp của điều khiển tín hiệu thông tin đang được yêu cầu một xung đồng hồ có thể là cơ sở của tất cả bắt tay.

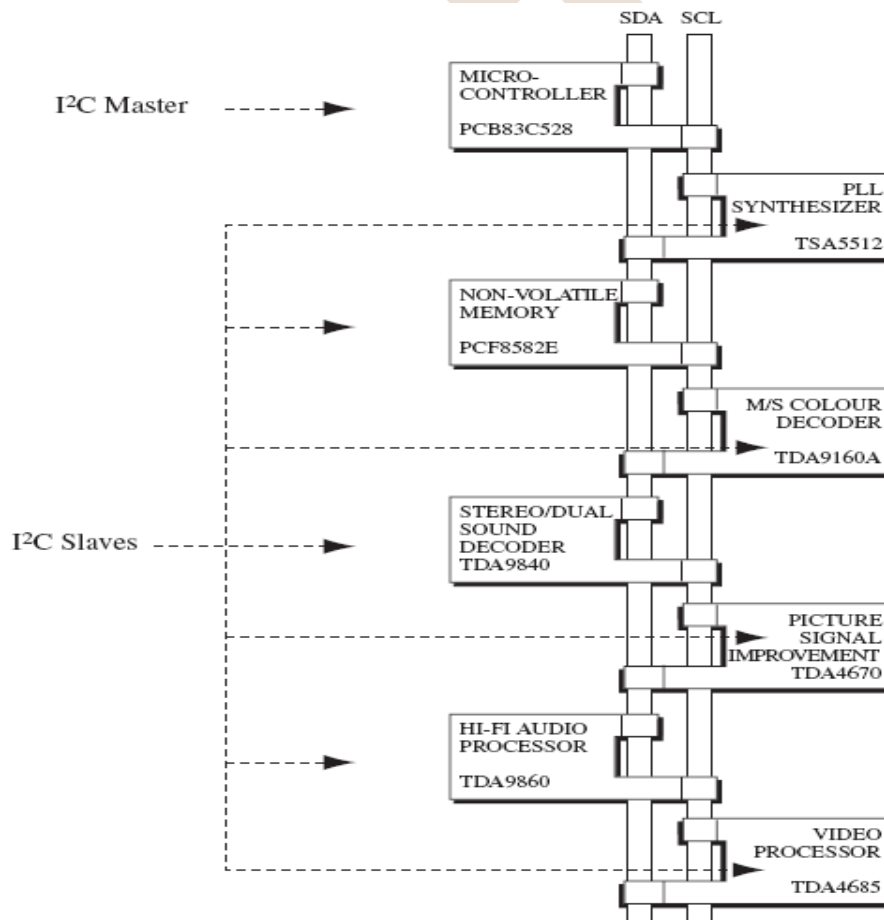
Các bus có thể hợp thành một tập hệ thống truyền, ra lệnh như thế nào để bus truyền dữ liệu. Các hệ thống phổ biến nhất là đơn lẻ, nơi mà một địa chỉ truyền đi trước thông tin truyền của dữ liệu và chặn, nơi địa chỉ được truyền một lần cho nhiều thông tin của dữ liệu. Một hệ thống truyền bị chặn có thể tăng băng thông của bus (không có thêm không gian và thời gian cho việc truyền cùng một địa chỉ) và đôi khi cho phép để tách rời truyền hệ thống. Nó được dùng phổ biến trong các loại giao tiếp bộ nhớ, ví dụ như giao tiếp vùng nhớ đệm. Tuy nhiên, một hệ thống bị chặn, tác động tiêu cực đến hiệu suất của bus trong đó các thiết bị khác có thể phải chờ lâu hơn để truy cập vào bus. Một trong

những thế mạnh của hệ thống truyền đơn lẻ bao gồm các thiết bị không phụ thuộc vào yêu cầu để có bộ đệm để lưu trữ địa chỉ và các thông tin của dữ liệu được liên kết với địa chỉ cũng như không sử dụng bất kỳ vấn đề gì nảy sinh với nhiều thông tin hoặc đến theo thứ tự hoặc không trực tiếp liên kết với một địa chỉ.

Bus không mở rộng:

I²C Bus

Các bus I²C kết nối với các bộ xử lý có kết hợp một I²C trên giao diện chip, cho phép trực tiếp truyền thông tin giữa các bộ xử lý trên bus. Một máy chủ/ máy phụ thuộc có mối quan hệ với các bộ xử lý tồn tại ở tất cả các giai đoạn, với máy chủ hoạt động như máy chủ nhận hoặc truyền. Hình 2-31, bus I²C là bus hai dây với một chuỗi dài dữ liệu (SDA) và một chuỗi dài Clock (SCL). Bộ xử lý được kết nối qua I²C định địa chỉ bởi một địa chỉ duy nhất là một phần của một luồng dữ liệu được truyền giữa các thiết bị.

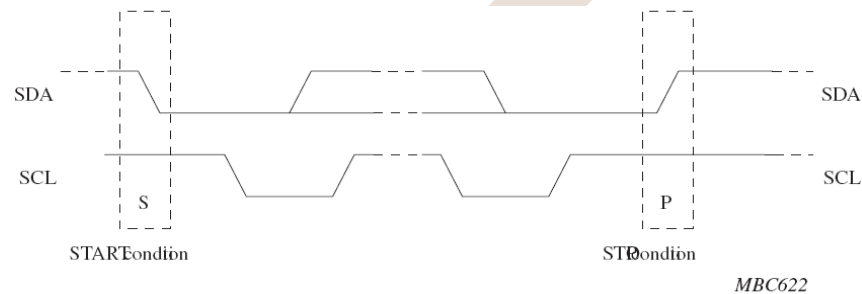


Hình 2. 40: Bảng mạch mẫu TV

Máy chủ I²C bắt đầu truyền dữ liệu và tạo ra các xung nhịp cho phép truyền. Về cơ bản SCL có chu kỳ là mức cao hay thấp.

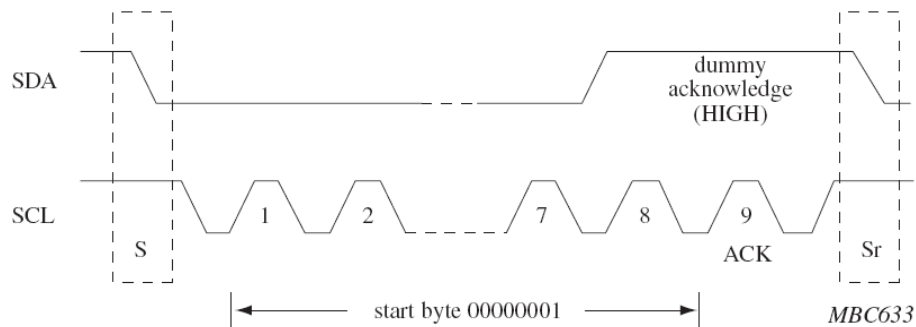
Một máy chủ sau khi sử dụng dải SDA (như SCL là chu kỳ) để chuyển dữ liệu đến các máy phụ, một phiên được bắt đầu và kết thúc như Hình 2-32. “START” là bắt đầu khi máy chủ kéo cổng SDA ở mức thấp trong khi tín hiệu SCL ở mức cao, nhưng trái lại điều kiện “STOP” bắt đầu khi máy chủ kéo SDA ở mức cao khi SCL ở mức cao.

Đối với việc truyền tải dữ liệu, bus I²C là một dây bus 8 bit. Điều này có nghĩa là trong khi không có giới hạn về số lượng byte có thể được truyền đi trong một phiên, chỉ có 1 byte (8 bit) của dữ liệu được truyền đi cùng một lúc, 1 bit tại một thời điểm. Cách dịch vào trong sử dụng các tín hiệu SDA và SCL là một bit dữ liệu “đọc” mỗi lần tín hiệu SCL chuyển từ mức cao xuống mức thấp, sườn đến sườn.

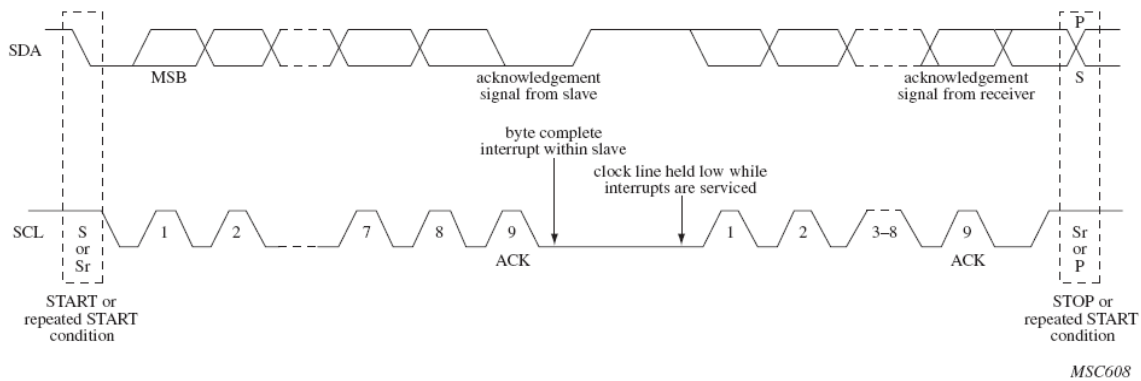


Hình 2. 41: Các điều kiện START và STOP của I²C

Nếu tín hiệu SDA ở mức cao tại điểm của một sườn, sau đó bit dữ liệu sẽ được đọc là “1”. Nếu SDA ở mức thấp tại điểm của một sườn, sau đó bit dữ liệu được đọc là “0”. Một ví dụ về byte “00000001” truyền, Hình 2-33a, trong Hình 2-33b cho thấy ví dụ về hoàn thành một phiên truyền.



Hình 2. 42: Ví dụ truyền dữ liệu trên I²C

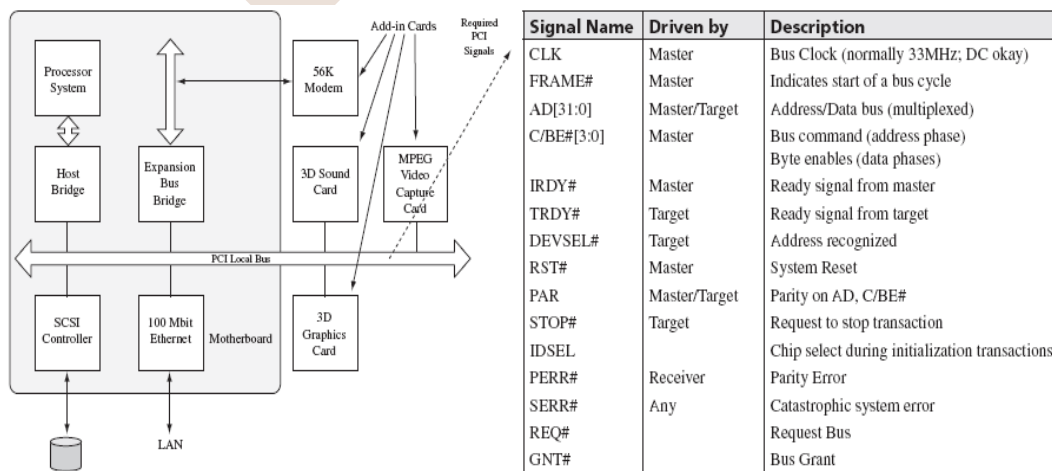


Hình 2. 43: Sơ đồ truyền dữ liệu hoàn chỉnh trên I2C

Bus có thể mở rộng: PCI (Peripheral Component Interconnect):

PCI Local Bus Specification Revision 2.1, xác định yêu cầu (điện, thời gian, giao thức ...) của việc xử lý một bus PCI. PCI là một bus đồng bộ, điều đó có nghĩa là nó đồng bộ hóa thông tin liên lạc bằng cách sử dụng xung clock. Tiêu chuẩn định nghĩa thiết kế một bus PCI với ít nhất 33MHz xung nhịp (lên đến 66 MHz) và độ rộng của một bus ít nhất là 32 bit (lên đến 64 bit) có thể đưa ra thông lượng tối thiểu xấp xỉ khoảng 132 Mbytes/giây và lên đến 528 Mbytes/giây, lớn nhất với 64bit truyền cho 66 MHz xung, PCI chạy ở một trong những tốc độ xung, không kể đến tốc độ xung nhịp mà tại đó các thành phần kèm theo nó đang hoạt động.

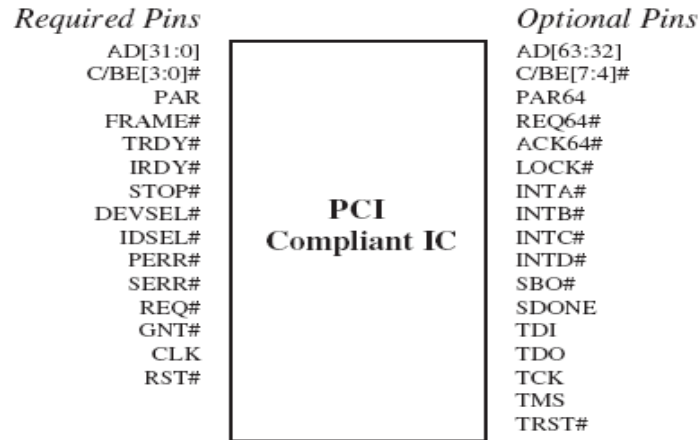
Ở Hình 2-34 bus PCI có 2 giao diện kết nối, một nội bộ giao diện PCI kết nối với bảng chính (để xử lý...) qua kênh EIDE và mở rộng giao diện PCI, bao gồm các khe bên trong PCI để cắm thẻ (audio, video...). Các giao diện mở rộng làm cho PCI mở rộng bus, nó cho phép các phần cứng cắm vào bus, cho toàn bộ hệ thống tự động điều chỉnh và hoạt động chính xác.



Hình 2. 44: Bus PCI

2.1.5.2 GHÉP BUS VỚI CÁC THÀNH PHẦN BẢN MẠCH KHÁC

Các bus thay đổi trong các đặc tính vật lý của nó, các đặc tính này tương ứng với các thành phần mà bus kết nối, chủ yếu là các sơ đồ chân của bộ xử lý và các chip bộ nhớ, nó phản ánh các tín hiệu mà bus có thể truyền (xem Hình 2-35).



Hình 2. 45: IC tương thích chuẩn PCI

Bên trong một kiến trúc, cũng có hỗ trợ chức năng giao thức bus. Như ví dụ, MPC860 bao gồm bus điều khiển tích hợp I2C.

Bus I2C là bus với 2 tín hiệu: SDL và SCL, cả hai tín hiệu này được thể hiện bên trong khối sơ đồ của bộ điều khiển PowerPC I2C. Do I2C là bus đồng bộ, một thiết bị tạo ra tốc độ baud bên trong bộ điều khiển cung cấp xung nhịp nếu PowerPC hoạt động như một máy chủ, theo 2 bộ (nhận và truyền) bao phủ xử lý và điều hành sự truyền tải bus.

Trong bộ điều khiển tích hợp I2C, địa chỉ và thông tin dữ liệu được truyền qua bus thông qua thanh ghi truyền dữ liệu và ra thanh ghi dịch. Khi MPC860 nhận dữ liệu, dữ liệu được truyền vào bên trong thanh ghi nhận dữ liệu qua một thanh ghi dịch.

2.1.5.3 HIỆU SUẤT BUS.

Năng suất của bus được tính qua băng thông của nó, số lượng dữ liệu mà bus có thể truyền trong một khoảng thời gian, một cấu trúc của bus cả cấu trúc vật lý và giao thức liên kết của nó sẽ tác động đến hiệu suất của nó. Trong giới hạn của giao thức, ví dụ, hệ thống bắt tay đơn giản băng thông cao hơn. Thực tế các thiết kế vật lý của bus (độ dài, số dòng, số lượng các thiết bị được hỗ trợ...) giới hạn hoặc tăng hiệu suất của nó. Các bus ngắn hơn, các thiết bị được kết nối ít, nhiều dữ liệu hơn thường nhanh hơn các bus và băng thông cao hơn.

Số lượng các dải bus và cách sử dụng các dải bus – Ví dụ, cho dù có nhiều đường dây riêng cho mỗi tín hiệu hay có nhiều tín hiệu đa bội ít được chia sẻ trên đường là các yếu tố tác động bổ xung cho băng thông của bus. Càng nhiều dải bus thì càng có nhiều dữ liệu có thể được truyền qua cùng một thời điểm. Ít dải hơn có nghĩa là nhiều dữ liệu có

thể chia sẻ vào dải để truyền, kết quả là có ít dữ liệu được truyền trong cùng một thời gian hơn. Liên quan đến chi phí, hãy lưu ý sự gia tăng trong chất dẫn điện trên bảng mạch, trong trường hợp dây của bus làm tăng chi phí trên bảng. Tuy nhiên, chú ý rằng, sự dồn dải sẽ đưa ra độ trễ trên cuối mỗi đường truyền, các yêu cầu ở cuối bus đến tín hiệu dồn kênh và phân kênh được tạo thành từ các tín hiệu thông tin khác.

Các yếu tố khác góp phần vào băng thông của bus là số bit dữ liệu mà một bus có thể truyền trong một chu kỳ bus, đây gọi là độ rộng của bus. Các bus thường có băng thông là số mũ nhị phân 2, chẳng hạn như $1(2^0)$ cho các bus với độ rộng bus nối tiếp, $8(2^3)$ bit, $16(2^4)$ bit, $32(2^5)$ bit... Ví dụ, cho dữ liệu 32 bit cần được truyền, nếu một bus nói riêng có độ rộng là 8bit, sau đó dữ liệu được chia và gửi trong 4 lần truyền riêng; nếu bus có độ rộng 16 bit, thì có 2 gói dữ liệu riêng để truyền đi, bus có 32 bit dữ liệu truyền một gói, một bit bất kể lúc nào cũng có thể được truyền. Độ rộng của bus giới hạn băng thông của bus vì nó giới hạn số bit dữ liệu có thể được truyền trong bất kỳ một giao dịch nào. Độ trễ có thể xảy ra trong mỗi phiên truyền bởi vì giao thức bắt tay, lưu lượng bus và tần số xung nhịp khác các thành phần giao tiếp, đặt các thành phần này trong hệ thống trễ, chẳng hạn như trạng thái chờ. Độ trễ tăng vì số gói dữ liệu cần được truyền cũng tăng. Vì vậy độ rộng của băng thông lớn hơn, độ trễ ít hơn và băng thông lớn hơn.

Đối với bus có giao thức bắt tay phức tạp hơn, hệ thống truyền thực hiện có thể ảnh hưởng lớn đến hiệu suất. Một khối hệ thống truyền tải cho phép băng thông lớn hơn qua hệ thống truyền tải đơn lẻ, bởi vì có ít sự trao đổi giao thức bắt tay trên mỗi khối với những từ đơn lẻ, các byte dữ liệu. Truyền tải khối có thể thêm thời gian chờ để các thiết bị chờ lâu hơn cho việc truy cập bus. Từ khi một khối truyền tải dựa trên chuyển giao một khối kéo dài lâu hơn là truyền tải đơn lẻ cơ sở chuyển giao. Một giải pháp phổ biến cho loại thời gian chờ là bus cho phép chia chuyển giao, là nơi mà bus được hoạt động trong suốt quá trình bắt tay, chẳng hạn như trong khi chờ đợi xác nhận trả lời. Điều này cho phép các giao dịch khác diễn ra và cho phép các bus rồi còn lại chờ thiết bị chuyển giao. Tuy nhiên, nó gắn với độ trễ của các giao dịch bằng cách yêu cầu bus có được nhiều hơn một lần giao dịch duy nhất.

2.2 Các thành phần phần mềm của hệ thống:

2.2.1. Trình điều khiển thiết bị

Các ví dụ về bộ điều khiển cổng I/O

Các bộ phận của hệ thống con I/O yêu cầu một số dạng quản lý phần mềm bao gồm các thành phần được tích hợp sẵn trong bộ xử lý chủ, như bộ điều khiển thụ động I/O. Các bộ điều khiển I/O đều gồm có một bộ trạng thái và các thanh ghi điều khiển được dùng để điều khiển bộ xử lý và tìm hiểu trạng thái của nó. Phụ thuộc vào hệ thống con I/O, thông thường tất cả hoặc một vài hệ thống trong 10 đặc trưng từ danh sách chức

năng bộ điều khiển thiết bị đã được giới thiệu ở phần đầu của chương sẽ thực thi trong bộ điều khiển I/O, bao gồm:

- I/O Startup, là phần khởi động của nguồn cấp cho I/O hoặc khởi động lại.
- I/O Shutdown, thiết lập I/O trở về trạng thái tắt nguồn của nó
- I/O Disable, cấp phát chương trình khác để ngăn chặn hoạt động của I/O
- I/O Enable, cấp phát chương trình khác để kích hoạt hoạt động của I/O
- I/O Acquire, cấp phát chương trình khác để khuếch đại truy cập đơn lẻ (chốt) tới I/O
- I/O Release, cấp phát chương trình khác để nghỉ (mở chốt) I/O
- I/O Read, cấp phát chương trình khác để đọc dữ liệu từ I/O
- I/O Write, cấp phát chương trình khác để viết dữ liệu lên I/O
- I/O Install, cấp phát chương trình khác để cài đặt hoạt động mới của I/O
- I/O Uninstall, cấp phát chương trình khác để gỡ bỏ hoạt động của I/O đã được cài đặt.

Các chương trình con khởi động Ethernet và RS232 I/O cho PowerPC và các cấu trúc ARM được cung cấp như trong ví dụ của hệ thống thiết bị khởi động (thiết lập) I/O. Các ví dụ đó chỉ ra cách I/O được thực thi trong các cấu trúc liên hợp, như PowerPC và ARM, và nó có thể được dùng như một bản chỉ dẫn để nắm bắt cách viết các hệ thống I/O lên các bộ xử lý phức hợp hoặc kém phức hợp hơn PowerPC và cấu trúc ARM.

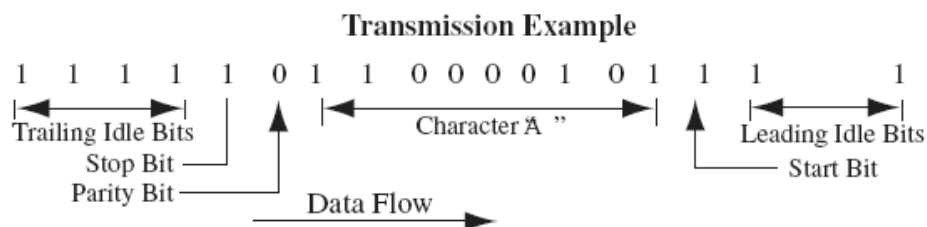
Ví dụ khởi tạo Driver RS-232:

Một trong những chuỗi giao thức I/O không đồng bộ được thiết lập là RS-232 hoặc EIA-232, dựa trên nền tảng chuẩn của hội công nghiệp điện tử. Các chuẩn này xác minh những bộ phận chính của bất cứ hệ thống gốc RS-232 nào được thực thi trong phần cứng.

Firmware (Software) yêu cầu kích hoạt ánh xạ đặc trưng tới phiên thấp hơn của lớp liên kết dữ liệu vật lý. Các bộ phận phần cứng có thể được ánh xạ hết tới lớp vật lý của mô hình OSI, nhưng nó sẽ không được nói đến trong phần này.

Bộ phận RS-232 được tích hợp sẵn trong bộ xử lý chủ được gọi là giao diện RS-232, nó có thể được lập trình truyền dẫn đồng bộ hay không đồng bộ. Ví dụ, loại dẫn tuyến không đồng bộ, chỉ có firmware (software) được thực thi RS-232 nằm trong bộ phận UART, nó sẽ thực thi việc truyền dẫn dữ liệu lần lượt.

Dữ liệu được truyền không đồng bộ qua RS-232 nằm trong một chuỗi các bit mà được chuyển với tốc độ không đổi. Khung truy cập bởi UART nằm trong khuôn mẫu được nêu trong Hình 2-36.



Hình 2. 46: Sơ đồ khung RS-232

2.2.2. Hệ điều hành thời gian thực

Hệ điều hành thời gian thực là một hệ điều hành hỗ trợ cấu trúc của hệ thống thời gian thực.

Trạng thái định thời của OS nên được dự đoán trước:

Đối với mỗi dịch vụ của hệ điều hành (OS), ràng buộc trong thời gian thực hiện cần được đảm bảo. Trên thực tế có các mức độ dự đoán khác nhau. Ví dụ, có thể có bộ của dịch vụ hệ điều hành gọi mà ràng buộc được biết và sự thay đổi của thời gian thực hiện là không đáng kể. Các lệnh gọi như là “đưa cho tôi thời gian trong ngày” có thể ở trong lớp này. Đối với các lệnh gọi khác có thể thay đổi rất lớn. Các lệnh gọi như “đưa cho tôi 4MB bộ nhớ độc lập” có thể rơi ở trong lớp thứ hai này. Đặc biệt, bản liệt kê danh sách của RTOS cần phải được xác định (tiêu chuẩn Java không chú ý về mặt này, như là không có thứ tự thực hiện đối với thực thi “chuỗi” – các luồng được qui định). Một trường hợp đặc biệt khác, chúng ta đề cập đến việc thu thập dữ liệu rác. Trong phạm vi Java, có nhiều cố gắng đã thực hiện để cung cấp thu gom dữ liệu rác.

Cũng có thể trong suốt thời gian lệnh ngắt có thể không có khả năng gây trở ngại phá hủy giữa các nhiệm vụ (nó là một cách rất đơn giản đảm bảo loại trừ lẫn nhau trên một hệ thống bộ vi xử lý đơn). Trong các giai đoạn, các ngắt bị vô hiệu hóa có thể hoàn toàn ngăn để tránh sự chậm trễ không thể dự đoán được trong việc xử lý các biến cố quan trọng.

Đối với RTOS thực hiện hệ thống tập tin, nó có thể cần thiết để thực hiện các tập tin kề nhau (file được lưu trữ kề nhau trong đĩa) để hạn chế sự chuyển động của đầu đĩa không dự đoán trước được.

OS cần phải được quản lý thời gian và lịch trình các nhiệm vụ.

Lịch trình được định nghĩa như sự ánh xạ từ tập các nhiệm vụ đến các khoảng thời gian thực hiện. Bao gồm ánh xạ để bắt đầu như là một trường hợp đặc biệt. Ngoài ra, OS có thể nhận biết được nhiệm vụ đúng thời hạn để OS có thể áp dụng lịch trình kỹ thuật thích hợp (trong đó lịch trình là hoàn toàn thực hiện gián tiếp và OS chỉ cần cung cấp các dịch vụ để bắt đầu nhiệm vụ ở thời điểm cụ thể hoặc các mức độ ưu tiên). OS cần phải cung cấp các dịch vụ thời gian chính xác với độ chính xác cao. Thời gian dịch vụ được

yêu cầu, ví dụ, để phân biệt giữa bản gốc và các lỗi con. Ví dụ, nó có thể giúp xác định năng lượng của máy, để chịu trách nhiệm cho việc mất nguồn.

OS cần phải nhanh

Ngoài việc dự đoán được, OS cần phải có khả năng hỗ trợ các ứng dụng với thời hạn trên tỉ lệ giây. Mỗi RTOS bao gồm một thời gian thực gọi là hệ thống điều hành chính. Bộ phận quản lý chính này là nguồn được tìm thấy trong mỗi hệ thống, bao gồm bộ xử lý, bộ nhớ, và hệ thống giờ. Bộ phận bảo vệ các thiết bị máy (loại từ an toàn, an toàn, hay lí do bảo mật) không cần có mặt.

Chức năng chính trong bộ phận chính bao gồm các quản lí nhiệm vụ, nhiệm vụ đồng bộ và truyền dữ liệu, quản lí thời gian, quản lí bộ nhớ.

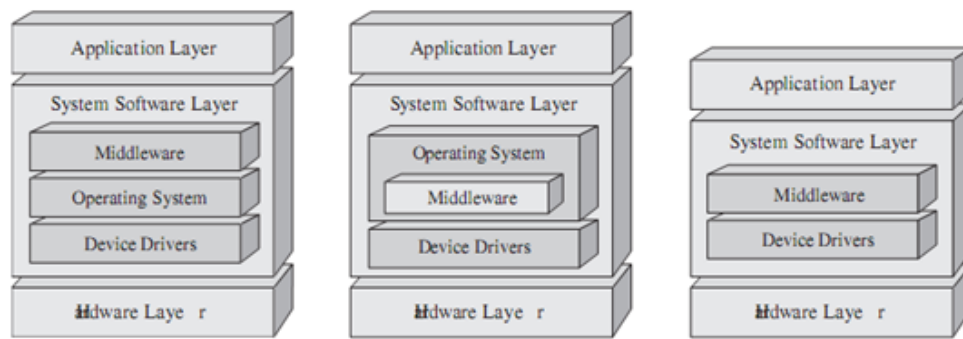
Trong khi một số RTOS được thiết kế cho các ứng dụng gán nói chung, các ứng dụng khác tập trung trên một khu vực cụ thể. Ví dụ, OSEK/VDX OS tập trung điều khiển tự động. Do tập trung này, nó là một hệ điều hành khá nhỏ.

Tương tự, trong một số RTOS cung cấp API chuẩn những cái khác đến cùng với đặc điểm riêng của chúng, API độc quyền. Ví dụ, một vài RTOSs phù hợp với POSIX RT mở rộng cho UNIX, với OSEK/VDX OS hoặc với ITRON đặc biệt phát triển nhiều ở nhật bản. Nhiều loại RT chính của OS có riêng API. ITRON, được đề cập trong trường hợp này là một RTOS hoàn chỉnh sử dụng liên kết cấu hình.

2.2.3. Middleware

Khái quát chung nhất, middleware là phần mềm hệ thống bất kì, nó không chỉ là một bộ phận chính của hệ điều hành, các bộ điều khiển thiết bị, hay các phần mềm ứng dụng. Chú ý rằng một vài OS có thể tích hợp middleware vào trong các bộ thực thi của hệ điều hành. Các middleware hệ thống nhúng thông thường được đặt bên trong các bộ điều khiển thiết bị hay được đặt ở phần trên cùng của hệ điều hành, và thi thoảng có thể được hợp nhất lại bên trong chính hệ điều hành.

Middleware thường là phần mềm trung gian giữa các phần mềm ứng dụng với bộ phận chính hay là phần mềm điều khiển thiết bị. Middleware cũng là phần mềm trung gian và điều khiển các phần mềm ứng dụng khác. Đặc biệt, middleware là một lớp cấu trúc chung được sử dụng trên hầu hết các thiết bị nhúng với 2 hoặc nhiều ứng dụng với mục đích để đáp ứng được tính phức tạp, tính bảo mật, tính linh động, tính liên kết, truyền thông với nhau, tính tương thích giữa các ứng dụng. Một trong các thế mạnh chính khi sử dụng middleware đó là nó cho phép giảm thiểu được sự phức tạp của các ứng dụng bởi cơ sở hạ tầng phần mềm trung tâm. Tuy nhiên, khi giới thiệu về middleware hay giới thiệu về một hệ thống, cái cần giới thiệu đầu tiên là cái nhìn tổng thể, nó có thể tác động rất lớn đến tính mở rộng và hiệu năng. Trong một thời gian ngắn, middleware tác động tới hệ thống nhúng tại tất cả các lớp.



Hình 2. 47: Middleware trong mô hình hệ nhúng

Trên đây có thể kể ra các thành phần middleware khác nhau như phần middleware định hướng thông báo (MOM-message oriented middleware), các thực thể trung gian yêu cầu đối tượng (ORBs-object request brokers), gọi thủ từ xa (remote procedure call-RPCs), cơ sở dữ liệu/truy nhập cơ sở dữ liệu, và các giao thức mạng trên tầng driver thiết bị. Tuy thế, hầu hết các loại middle nhìn chung đều rơi vào một trong 2 loại sau:

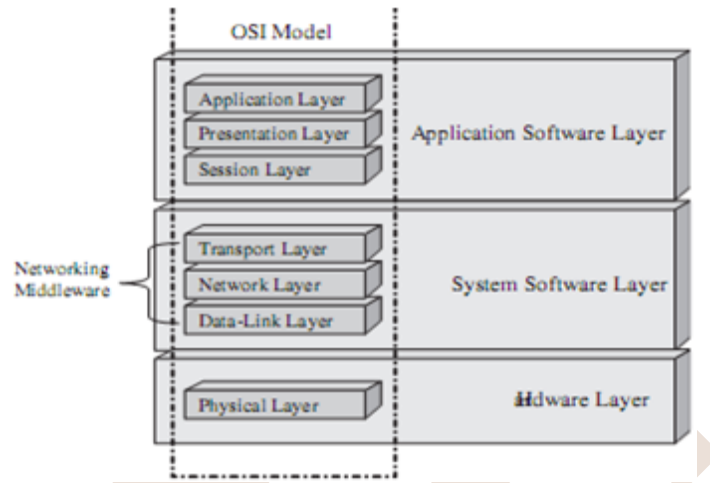
- + **General –purpose**: nghĩa là chúng được thực thi trên các thiết bị khác nhau như giao thức mạng nằm bên trên lớp điều khiển thiết bị và nằm bên dưới lớp ứng dụng trong mô hình OSI, các hệ thống tập tin hay trong một vài thiết bị ảo như JVM.
- + **Market-specific**: nghĩa là chúng là độc nhất trong một gia đình riêng biệt của các hệ thống nhúng như một phần mềm trên nền tiêu chuẩn TV số đặt trên hệ điều hành hay trên JVM.

Dù chúng là general purpose hay market –specific, thì thành phần middleware có thể được phân chia rõ hơn theo “**đóng**”, nó là các phần mềm được cung cấp bởi một công ty với bản quyền sử dụng cho phép ta có thể sử dụng nó, hoặc “**mở**”, nghĩa là nó là một chuẩn qui định bởi các hội đồng công nghiệp và chúng được cài đặt hay cấp phép bởi bất kì bên nào có liên quan.

Khi người ta tìm ra một công nghệ có thể đáp ứng được tất cả các yêu cầu của ứng dụng riêng biệt nào đó, thì lúc này các hệ thống nhúng trở nên phức tạp hơn và đòi hỏi phải có thêm nhiều hơn các linh kiện nhúng. Trong trường hợp đó, các linh kiện middleware cá nhân được chọn lựa tùy thuộc vào khả năng thao tác với các linh kiện khác, để tránh được các vấn đề sau này khi chúng được tích hợp lại. Trong một vài trường hợp, các gói middleware tích hợp của thành phần middleware đã mang sẵn tính thương mại, sử dụng cho hệ thống nhúng như các giải pháp nhúng Java của SUN, Microsoft’s, Framework, và CORBA từ nhóm quản lí đối tượng (OMG), để đặt tên cho một vài hệ thống nhúng. Nhiều nhà cung cấp hệ điều hành nhúng cũng cung cấp các gói middleware tích hợp chạy theo kiểu “out of box” với hệ điều hành tương ứng và nền tảng phần cứng của họ.

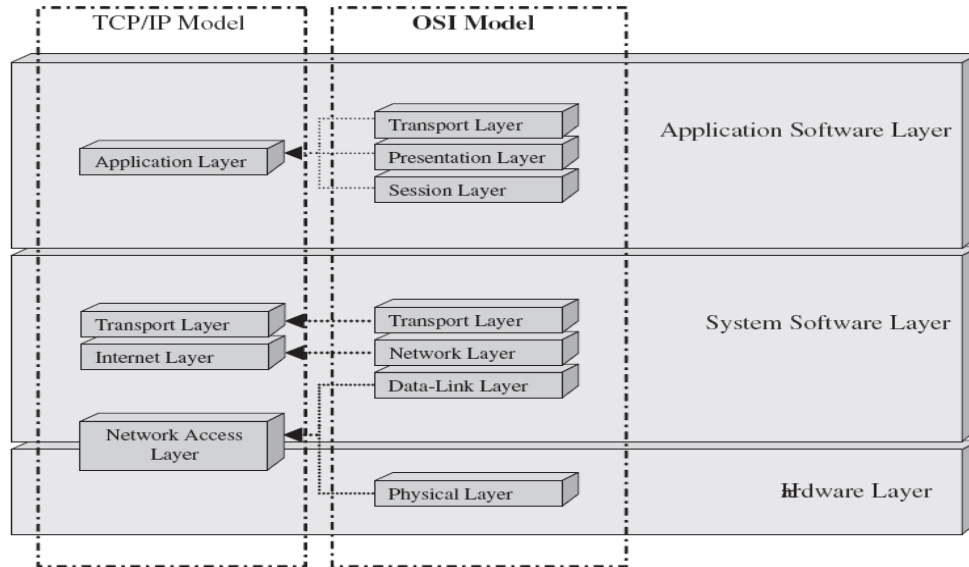
Ví dụ middleware driver mạng:

Một trong các cách đơn giản nhất để có thể hiểu về các thành phần cần có để lắp đặt một mạng trong một thiết bị nhúng để hiển thị hóa các thành phần của mạng theo mô hình OSI và liên quan tới mô hình hệ thống nhúng. Trong Hình 2-38, phần mềm được đặt nằm trên lớp liên kết dữ liệu và lớp phiên, nó có thể được xem như thành phần phần mềm của middleware mạng.

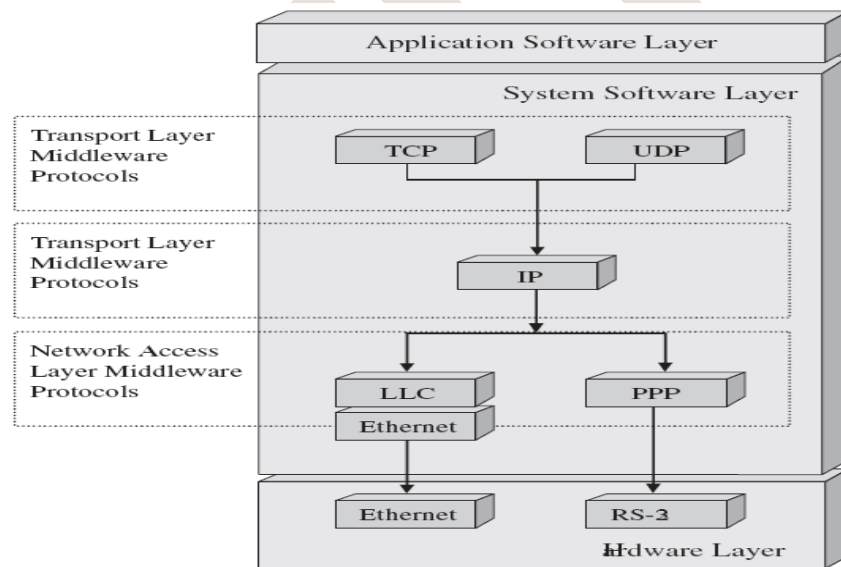


Hình 2. 48: Mô hình OSI và middleware

Trong ví dụ ở phần này, ta sẽ đề cập tới hai giao thức UDP và IP, đây là các giao thức trong chồng giao thức TCP/IP và được thực thi như middleware. Như đã giới thiệu trong chương 2, mô hình TCP/IP được tạo nên từ 4 lớp: lớp truy nhập mạng, lớp mạng, lớp giao vận, và lớp ứng dụng. Lớp ứng dụng của mô hình TCP/IP hợp nhất chức năng của 3 lớp trên cùng trong mô hình OSI (lớp ứng dụng, lớp trình diễn, và lớp phiên), và lớp truy nhập hợp nhất chức năng của hai lớp là lớp vật lý và lớp liên kết dữ liệu. Lớp internet tương đương với lớp mạng trong mô hình OSI và lớp giao vận thì giống nhau cho cả hai mô hình. Điều đó có nghĩa là, khi xem xét mô hình TCP/IP cần chú ý, middleware mạng nằm bên dưới lớp giao vận, lớp internet và nằm bên trên lớp truy nhập mạng.



Hình 2. 49: Sơ đồ khối mô hình OSI, TCP/IP và mô hình hệ nhúng

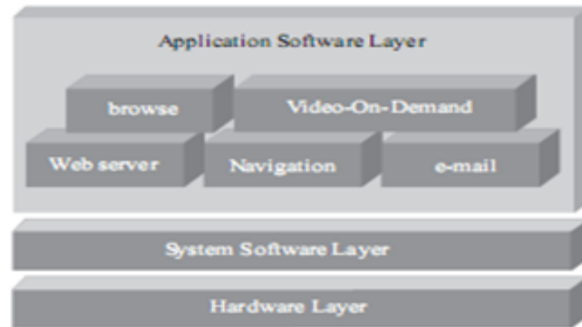


Hình 2. 50: Sơ đồ khối mô hình TCP/IP và các giao thức

2.2.4 Phần mềm ứng dụng

Là loại phần mềm lớp trên cùng trong hệ thống nhúng (*Application Software*). Trong hình 2.51, phần mềm ứng dụng nằm ở phần trên cùng của lớp phần mềm hệ thống và phụ thuộc, chịu sự quản lý và chạy các ứng dụng tùy thuộc vào phần mềm hệ thống. Nó là một phần mềm nằm bên trong lớp ứng dụng, qui định loại thiết bị nào được dùng trong hệ thống nhúng, bởi vì chức năng của một lớp ứng dụng là đặc tả mục đích cao nhất của hệ thống nhúng đó và làm tất cả các công việc liên quan gần nhất với người sử dụng

hoặc quản lý thiết bị đó nếu cái thiết bị nào tồn tại . Khi người sử dụng bấm nút thì có thể kích hoạt chức năng điều khiển của một cách trực tiếp bằng cách đóng/ngắt theo tuần tự, tốt hơn so với một ứng dụng phụ thuộc vào một người lập trình viên làm công việc đó bằng tay.



Hình 2. 51: Lớp ứng dụng và mô hình hệ nhúng

Giống như các tiêu chuẩn nhúng, các ứng dụng nhúng có thể được chia theo market specific và general purpose.

- + Marker specific (Thực thi chỉ trên duy nhất một kiểu thiết bị, như các ứng dụng yêu cầu video trong một TV số tương tác).
- + General purpose (Có thể chỉ được thực thi trên nhiều loại thiết bị khác nhau, như một trình duyệt web).

Câu hỏi ôn tập

1. Sự khác biệt giữa von Neumann model và Harvard model?
2. Vẽ sơ đồ phân cấp bộ nhớ trong hệ thống và giải thích.
3. Bản đồ bộ nhớ là gì?
4. Các loại bộ nhớ nào thường được sử dụng trong hệ thống nhúng.
5. Loại bộ nhớ nào tích hợp trên chip, trên board.
6. Việc quản lý bộ nhớ ngoài có thể thực hiện bằng các phương pháp nào?
7. Mục đích của I/O trên bảng mạch là gì?
8. Định nghĩa bus?
9. Mục đích của bus là gì?
10. Lấy ví dụ một số loại bus phổ biến.
11. Sự khác biệt giữa bus mở rộng được và không mở rộng được
12. Sự khác biệt giữa bus master và slave
13. Tại sao lại cần trọng tài bus?
14. Trọng tài bus làm nhiệm vụ gì?
15. Nêu 3 loại trọng tài bus phổ biến?
16. Vẽ sơ đồ các cơ chế của trọng tài bus và giải thích.
17. Sự khác biệt giữa serial và parallel I/O.
18. Sự khác biệt giữa UART và SPI.
19. Vẽ sơ đồ hoạt động của I2C và giải thích các tín hiệu.
20. Vẽ biểu đồ hoạt động theo thời gian của I2C và giải thích.
21. Vẽ sơ đồ hoạt động của SPI và giải thích các tín hiệu.
22. Vẽ biểu đồ hoạt động theo thời gian của SPI và giải thích.
23. Trình điều khiển thiết bị (device driver) là gì?
24. Vẽ sơ đồ các vị trí có thể của lớp trình điều khiển thiết bị trong kiến trúc hệ thống nhúng.
25. Liệt kê và mô tả chức năng của các hàm trong trình điều khiển thiết bị.
26. Hệ điều hành nhúng làm chức năng gì?
27. Vẽ sơ đồ vị trí có thể của hệ điều hành nhúng trong phân lớp hệ thống nhúng.
28. Nhân hệ điều hành (kernel) làm những nhiệm vụ chính nào?
29. Middleware là gì?
30. Vẽ sơ đồ vị trí có thể của middleware trong phân lớp hệ thống nhúng.
31. Liệt kê các loại middleware

- 32. Định nghĩa phần mềm ứng dụng?
- 33. Phần mềm ứng dụng nằm ở vị trí nào trong phân lớp hệ thống nhúng.
- 34. Phân loại phần mềm ứng dụng?



CHƯƠNG 3 - HỆ ĐIỀU HÀNH THỜI GIAN THỰC DÙNG CHO CÁC HỆ THỐNG NHÚNG

3.1 Yêu cầu chung cho các hệ điều hành thời gian thực

Thời gian thực là một khái niệm rất khó định nghĩa. Theo nghĩa đen, hệ thống phải phản ứng tức thì, thích hợp theo yêu cầu. Theo đúng ý nghĩa, thời gian thực của một sự kiện nghĩa là nó xảy ra ngay lập tức. Một ví dụ khi ta chuyển khoản tiền giữa các tài khoản, đối với một hệ thống thời gian thực, chúng ta sẽ được cập nhật tài khoản ngay lập tức. Tuy nhiên, đối với một hệ thống không phải thời gian thực, việc xử lý sẽ được thực hiện vào cuối mỗi ngày, do đó việc cập nhật có thể phải đợi đến ngày hôm sau. Một ví dụ khác là một hệ thống đa phương tiện, khi chơi các file media, hệ thống này phải đủ khả năng xử lý video, audio đúng theo thời gian của chuẩn dữ liệu, nếu không, sẽ xảy ra hiện tượng giật hình, trễ tiếng, trễ hình hoặc các lỗi khác.

Ngoài việc phản ứng đủ nhanh, hệ thống thời gian thực cần phải tin cậy và chính xác. Đối với các hệ thống điều khiển như trong ô tô, trong dây chuyền sản xuất, độ chính xác rất quan trọng.

Tùy theo mức độ của yêu cầu về thời gian, hệ thống thời gian thực có thể phân thành thời gian thực cứng (hard real-time) hay thời gian thực mềm (soft real-time):

- Thời gian thực cứng (hard real-time): Một hệ thống thời gian thực cứng cần một sự đảm bảo về thời gian đáp ứng trong trường hợp xấu nhất. Cả hệ thống bao gồm hệ điều hành, ứng dụng, phần cứng, phải được thiết kế đảm bảo các yêu cầu về thời gian đáp ứng. Thời gian đáp ứng có thể là ms, ns nhưng chúng phải luôn được đảm bảo. Nếu không có thể sẽ có những hậu quả nghiêm trọng đối với hệ thống. Ví dụ như các hệ thống quốc phòng, các hệ thống điều khiển xe cộ hoặc máy bay, các hệ thống thu thập dữ liệu, các thiết bị y tế...
- Thời gian thực mềm (soft real-time): Đối với hệ thống thời gian thực mềm, việc đáp ứng yêu cầu về thời gian không phải lúc nào cũng cần thiết. Ví dụ như hệ thống xem phim, thỉnh thoảng có thể hệ thống bị treo, hoặc giật hình. Những sự cố này không có hậu quả lớn. Tuy nhiên nếu hệ thống không phản ứng đúng thời gian quá nhiều lần, hệ thống có thể coi như không đạt yêu cầu. Ví dụ như thoại VoIP, hệ thống audio, video...

POSIX 1003.1b định nghĩa thời gian thực cho hệ điều hành là khả năng của hệ điều hành để cung cấp mức độ của dịch vụ trong một thời gian giới hạn.

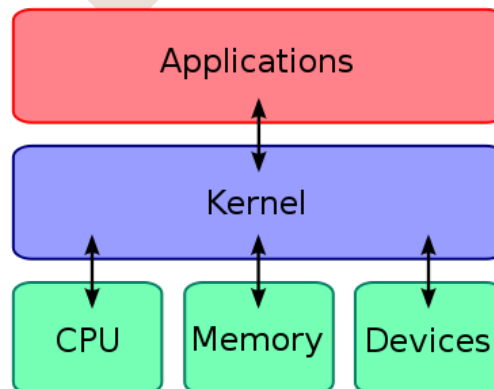
Các đặc tính sau có thể được gắn với hệ thống thời gian thực:

- Đa nhiệm/Đa luồng (Multitasking/multithreading): Một hệ điều hành thời gian thực phải có khả năng xử lý đa nhiệm và đa luồng.
- Cơ chế ưu tiên (Priorities): Hệ điều hành thời gian thực phải có cơ chế ưu tiên cho các tác vụ. Các chức năng có yêu cầu giới hạn về thời gian cao cần được ưu tiên xử lý.
- Thừa kế ưu tiên (Priority inheritance): Hệ điều hành thời gian thực phải có cơ chế thừa kế ưu tiên.
- Chiếm quyền (Preemption): Hệ điều hành thời gian thực cần có tính năng chiếm quyền, có nghĩa là khi một tác vụ có mức ưu tiên cao hơn sẵn sàng chạy, nó có thể chiếm quyền của một tác vụ có mức ưu tiên thấp hơn.
- Trễ ngắt (Interrupt latency): Thời gian trễ ngắt là thời gian từ khi một ngắt từ phần cứng xảy ra đến khi hàm xử lý ngắt chạy. Một hệ điều hành thời gian thực cần có trễ ngắt có thể đoán được và thường càng nhỏ càng tốt.
- Trễ lập lịch (Scheduler latency): Đây là thời gian bắt đầu từ khi tác vụ có khả năng chạy đến khi tác vụ thực sự chạy. Một hệ điều hành thời gian thực cần có trễ lập lịch có thể xác định được.
- Đồng bộ và thông tin giữa các process (Interprocess communication and synchronization): Kiểu thông tin phổ biến nhất giữa các tác vụ trong một hệ thống nhúng là gửi các bản tin. Một hệ điều hành thời gian thực cần có một cơ chế xử lý bản tin ở thời gian không đổi.
- Phân phối bộ nhớ động (Dynamic memory allocation): Một hệ điều hành thời gian thực cần cung cấp các hàm cung cấp bộ nhớ với thời gian cố định.

3.2 Các chức năng chính của phần lõi trong hệ điều hành thời gian thực

3.2.1. Kernel

Trong một hệ điều hành bất kỳ, kernel là thành phần trung tâm nhất của hệ điều hành, nó là cầu nối giữa các ứng dụng và việc xử lý dữ liệu ở mức phần cứng.



Hình 3. 1: Kernel trong hệ thống

Nhiệm vụ chính của kernel là quản lý tài nguyên hệ thống, các tài nguyên này thường bao gồm:

- CPU: Kernel chịu trách nhiệm quyết định chương trình nào sẽ được chạy trên CPU hoặc các CPU tại một thời điểm nhất định.
- Bộ nhớ: Kernel chịu trách nhiệm quyết định mỗi tác vụ sẽ được sử dụng phần bộ nhớ nào, và xác định sẽ làm gì khi không đủ bộ nhớ.
- Các thiết bị vào/ra (I/O devices): Kernel cung cấp các yêu cầu từ các ứng dụng để thực hiện các tính năng truy cập vào/ra cho các thiết bị tương ứng.

3.2.2 Tác vụ và Multi-tasking

Polling và ngắt

Các hệ thống thời gian thực được hiểu là được điều khiển tùy theo sự kiện, có nghĩa là chức năng chính của hệ thống là phản ứng theo sự kiện xảy ra. Có hai cách tiếp cận vấn đề này: dùng polling hoặc ngắt.

Ví dụ vòng lặp polling như sau:

```
int main (void)
{
    sys_init();
    while (TRUE)
    {
        if (event_1)
            service_event_1();
        if (event_2)
            service_event_2();
    }
}
```

Ở đây, chương trình bắt đầu chạy cài đặt cho hệ thống, sau đó vào một vòng lặp vô hạn. Đối với mỗi sự kiện (event) có một hàm (một tác vụ) tương ứng. Mỗi tác vụ được quyền điều khiển lần lượt. Khi tác vụ một hoàn thành, nó chuyển quyền điều khiển cho tác vụ hai, lần lượt như thế đến hết, sau đó lại lặp lại. Phương pháp này còn gọi là lập lịch nối tiếp (sequential scheduling hoặc một tên khác là round-robin scheduling).

Mặc dù phương pháp này hoạt động tốt với các hệ thống đơn giản, tuy nhiên nó cũng có một số mặt hạn chế. Thời gian hệ thống phản ứng lại sự kiện phụ thuộc vào vị trí mà chương trình chạy vòng lặp. Ví dụ nếu sự kiện event_1 xảy ra ngay trước câu lệnh if(event_1) thì thời gian phản ứng sẽ rất ngắn. Tuy nhiên, nếu nó xảy ra ngay sau khi chương trình kiểm tra câu lệnh đó, hệ thống sẽ phải đợi một chu kỳ vòng lặp để thực hiện phản ứng đối với event_1 đó.

Đối với một số hệ thống đơn giản, phương pháp này chấp nhận được. Tuy nhiên đối với các hệ thống phức tạp, khi vòng lặp quá lớn, nếu hệ thống phản ứng không kịp với sự kiện xảy ra, hệ thống có thể bị sự cố. Ví dụ trong một hệ thống điều khiển dây chuyền lắp ráp, nếu hệ thống không phản ứng kịp lại một sự kiện nào quá lâu, cả dây chuyền có thể bị trục trặc.

Một vấn đề nữa là tất cả các tác vụ đều có ưu tiên ngang nhau. Trên thực tế trong một hệ thống bao giờ cũng có những tác vụ có quyền ưu tiên cao hơn các tác vụ khác.

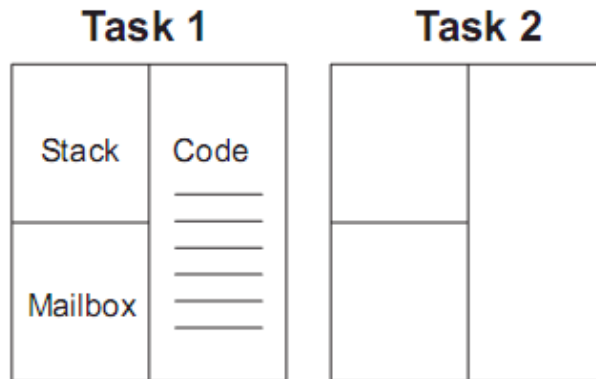
Vấn đề thứ ba là số lượng tác vụ. Nếu số lượng tác vụ trong hệ thống quá lớn, hệ thống sẽ không thể xử lý kịp các yêu cầu, thậm chí mỗi tác vụ chỉ chiếm một lượng thời gian hạn chế.

Phương pháp thứ hai là ngắt. Phương pháp này hiệu quả hơn và được sử dụng rộng rãi trong các hệ điều hành thời gian thực hiện nay. Khi có một sự kiện xảy ra, sự kiện này sẽ ngắt chương trình đang chạy hiện tại và gọi hàm xử lý ngắt. Hàm này sẽ phản ứng lại sự kiện vừa xảy ra. Sau khi hoàn thành, chương trình lại chạy về đoạn chạy dở. Các sự kiện xảy ra sẽ được đáp ứng ngay và không phải chờ đợi cả vòng lặp polling.

Tác vụ

Để hệ thống có thể phản ứng kịp với các sự kiện bên ngoài, các hệ điều hành nhúng đều phải có khả năng đa nhiệm (multi-tasking). Các vấn đề lớn sẽ được chia thành các vấn đề nhỏ hơn, đơn giản hơn. Mỗi một vấn đề con này trở thành một tác vụ. Mỗi tác vụ chỉ làm một thứ và phải đơn giản. Các tác vụ này có thể được cho rằng chạy song song với nhau. Nhưng trên thực tế, trên hệ thống có một bộ VXL, các tác vụ chia nhau tài nguyên của bộ VXL. Mỗi tác vụ sẽ chiếm dùng bộ VXL một khoảng thời gian. Các khoảng thời gian này rất nhỏ, cho nên ta có thể coi như các tác vụ chạy song song.

Giống như một chương trình bất kỳ, một tác vụ chứa mã lệnh để thực hiện tính năng mà tác vụ được thiết kế cho. Mã lệnh này được chứa trong các hàm tương tự hàm main() trong một chương trình C bình thường. Tuy nhiên, sự khác biệt là mỗi tác vụ có ngữ cảnh (context) trong ngăn xếp (stack) của nó.



Hình 3. 2: Cấu trúc của tác vụ

Mỗi tác vụ bao gồm:

- Đoạn mã thực hiện chức năng
- Ngăn xếp đựng ngữ cảnh
- Hộp thư (mailbox) để liên hệ với các tác vụ khác

Các tác vụ khác nhau có thể độc lập với nhau, tuy nhiên chúng thường được tạo ra để thực hiện một nhiệm vụ nào đó theo thiết kế. Do đó, trong tác vụ cần có một cơ chế thông tin để nó có thể liên lạc và đồng bộ với các tác vụ khác. Cơ chế thường được dùng là một hòm thư (mailbox).

Ví dụ một đoạn code tác vụ.

```
void tác_vụ (void *data)
{
    init_task();
    while (TRUE)
    {
        Wait for message at task mailbox();
        switch (message.type)
        {
            case MESSAGE_TYPE_X:
                ...
                break;
            case MESSAGE_TYPE_Y:
                ...
                break;
        }
    }
}
```

Trong đoạn code trên, tác vụ được bắt đầu bằng một hàm khởi tạo. Sau đó, nó sẽ đi vào một vòng lặp vô hạn (VD `while (TRUE)`). Trong vòng lặp đó, tác vụ hầu như không sử dụng tài nguyên, mà chỉ đợi một sự kiện nào đó xảy ra.

Khi có sự kiện xảy ra, tác vụ sẽ thức dậy, và khi nhận được một bản tin, tác vụ sẽ giải mã bản tin đó và xử lý bằng lệnh `switch` như ví dụ trên. Sau khi xử lý bản tin xong, tác vụ sẽ quay lại để đợi sự kiện tiếp theo.

Hệ điều hành thời gian thực theo dõi các tác vụ thông qua khối điều khiển tác vụ (tác vụ control block hoặc TCB). Đây là nơi hệ điều hành thời gian thực lưu các thông tin về các tác vụ. Một khối TCB thường lưu các thông tin sau:

- Tác vụ ID: Thường là số của tác vụ.
- Tình trạng của tác vụ: thường là sẵn sàng, bị chặn...
- Thứ tự ưu tiên của tác vụ: thường là số hoặc mức độ ưu tiên.
- Địa chỉ của tác vụ: Nơi đoạn mã của tác vụ được lưu trong bộ nhớ
- Con trỏ ngăn xếp của tác vụ

Đa nhiệm (Multi-tasking)

Từ trên có thể thấy các tác vụ giành phần lớn thời gian chờ đợi sự kiện. Thời gian xử lý thường rất ít so với thời gian chờ đợi. Đây là lý do để có thể dùng đa nhiệm (multi-tasking).

Đa nhiệm là cách mà nhiều tác vụ có thể chia sẻ tài nguyên xử lý chung (VD như một CPU). Trong trường hợp hệ thống chỉ có một CPU, tại một thời điểm chỉ có một tác vụ chạy. Điều đó có nghĩa là CPU chỉ phục vụ tác vụ đó. Để có thể chạy đa nhiệm, hệ thống cần có cơ chế lập lịch để cho các tác vụ có thể chia sẻ tài nguyên của CPU. Trong khi một tác vụ đang chạy, tác vụ khác sẽ xếp hàng chờ đến lượt. Thông tin chi tiết sẽ được trình bày ở phần sau.

3.3.3 Lập lịch thời gian thực (Real-time Scheduling)

Trong một hệ điều hành thời gian thực, việc lập lịch được thực hiện bởi một bộ phận của nhân (kernel) gọi là bộ lập lịch (scheduler). Bộ lập lịch sẽ quản lý việc phân chia tài nguyên của hệ thống cho các tác vụ khác nhau.

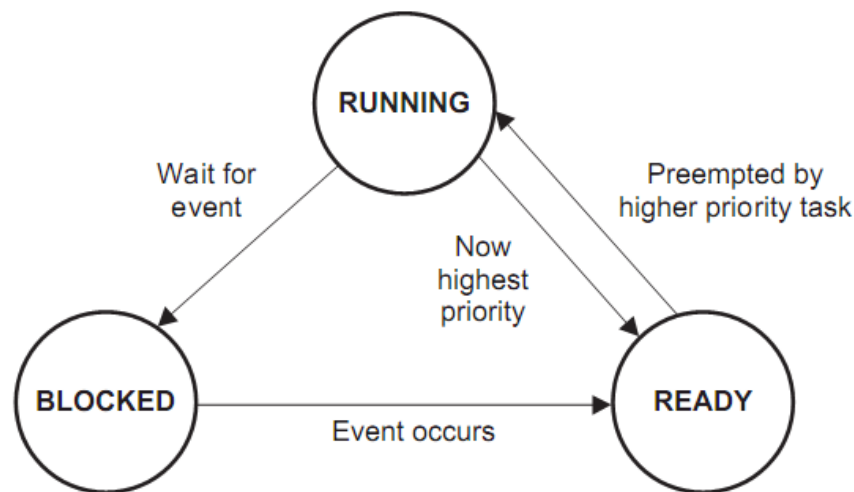
Do yêu cầu nghiêm ngặt về thời gian trong các hệ điều hành thời gian thực, bộ lập lịch phải đảm bảo các tác vụ phải đáp ứng về thời hạn. Do đó, bộ lập lịch phải phân chia mức độ ưu tiên của các tác vụ và đảm bảo rằng chỉ có duy nhất tác vụ với mức ưu tiên cao nhất đang được chiếm dụng tài nguyên CPU.

Trên thực tế, bộ lập lịch xem xét các tác vụ như một hệ thống máy trạng thái (state machine). Các trạng thái như sau:

- **Đang hoạt động (running):** Đây là trạng thái hoạt động của tác vụ. Khi được nhân hệ điều hành trao quyền, tác vụ sẽ chuyển từ trạng thái sẵn sàng sang trạng thái hoạt động. Ở trạng thái này, tác vụ sẽ thực hiện các công việc của mình. Tác vụ có thể chuyển từ trạng thái hoạt động sang khóa trong trường hợp chờ đợi một sự kiện nào đó, hoặc sang trạng thái sẵn sàng nếu có một tác vụ khác với mức độ ưu tiên đã trở thành sẵn sàng. Quá trình này gọi là chiếm quyền thực thi (preemption).
- **Sẵn sàng (ready):** Đây là trạng thái chuẩn bị hoạt động của tác vụ. Khi ở trạng thái sẵn sàng, tác vụ sẽ chuyển sang trạng thái hoạt động khi được cung cấp quyền ưu tiên cao nhất.

Một tác vụ có thể chuyển sang trạng thái sẵn sàng từ một trong hai trường hợp:

- Khi một tác vụ đang bị khóa thì có sự kiện xuất hiện, ví dụ như có một ngắt hoặc một bản tin gửi đến hộp thư.
 - Khi tác vụ đang ở trạng thái hoạt động thì có một tác vụ khác chiếm quyền thực thi.
- **Khóa (blocked):** Đây là trạng thái không hoạt động của tác vụ. Tác vụ chuyển sang trạng thái này trong trường hợp đang chờ đợi một sự kiện xảy ra



Hình 3. 3: Các trạng thái của tác vụ

Lập lịch theo chu kỳ:

Nhiều tác vụ thường cần thức dậy theo chu kỳ và thực thi công việc, sau đó, tác vụ quay lại trạng thái ngủ. Đối với các tác vụ này, việc lập lịch được tính theo chu kỳ. VD: Cứ đến một thời điểm nhất định, tác vụ phải kiểm tra một số tính năng của hệ thống.

Để thực thi, trong các hệ điều hành nhúng thường có hàm trễ `Delay()`, các tác vụ được quản lý để thức dậy và hoạt động trong khoảng thời gian trễ này.

Một phương pháp khác là khai báo cáo tác vụ có tính chất chu kỳ. Trong trường hợp này, bộ lập lịch sẽ đánh thức các tác vụ thức dậy trong mỗi khoảng thời gian mà không phụ thuộc vào thời gian chạy của tác vụ (VD cứ mỗi 4s lại gọi một tác vụ). Một tác vụ có tính chất chu kỳ gọi một hàm tương tự như `WaitUntilNext()`, hàm này sẽ chặn tác vụ đến thời điểm chạy tiếp theo.

Lập lịch không theo chu kỳ:

Khác với các tác vụ có tính chu kỳ, một số tác vụ chỉ chạy ở một thời điểm ngẫu nhiên để phản ứng lại một sự kiện xảy ra. Sự kiện đó có thể là một công tắc đã được bật, một bộ cảm ứng đã đọc dữ liệu. Thường thì các sự kiện này được kết nối với máy tính qua các bộ ngắt (interrupt). Các chương trình con dịch vụ ngắt (ISR) sẽ thông tin về việc xảy ra ngắt với các tác vụ chịu trách nhiệm cho việc xử lý sự kiện.

Lập lịch chiếm quyền thực thi:

Trong một hệ thống thông thường, khi có một ngắt xảy ra, hệ thống gọi chương trình con ISR để phục vụ cho ngắt. Nếu có một tác vụ với mức ưu tiên thấp đang chạy và một tác vụ có mức ưu tiên đang ở trạng thái khóa (block) để chờ sự kiện xảy ra, chương trình con dịch vụ ngắt ISR sẽ chuyển tác vụ với mức ưu tiên cao hơn chuyển sang trạng thái sẵn sàng (ready). Sau đó, bộ lập lịch xác định tác vụ có mức ưu tiên cao hơn đang ở trạng thái sẵn sàng và chuyển nó lên trạng thái hoạt động (running). Như vậy, tác vụ với mức ưu tiên thấp hơn đã bị chiếm quyền thực thi.

Các hệ thống cho phép chiếm quyền thực thi cung cấp cho ta hệ thống với thời gian phản ứng có thể dễ dàng đoán được hơn bởi các sự kiện với mức độ ưu tiên cao được xử lý sớm hơn. Đây là một điều căn bản của hệ thống thời gian thực, nó sẽ có thể đảm bảo thời gian tối đa để xử lý một sự kiện. Trong trường hợp hệ thống không chiếm quyền thực thi, không có sự đảm bảo về thời gian trước khi tác vụ đang chạy sẽ thoát.

3.3.4 Đồng bộ

Trong hệ thống, các tác vụ có thể độc lập với nhau. Tuy nhiên, chức năng của cả hệ thống cho một công việc yêu cầu các tác vụ phải có sự liên hệ, phối hợp. Việc đồng bộ giữa các tác vụ trong hệ điều hành thời gian thực được thực hiện bằng tập hợp các dịch vụ truyền thông giữa các tác vụ.

Một số cơ chế thông tin và đồng bộ hóa thường dùng như sau

- Semaphore: được sử dụng cho việc đồng bộ và khóa tài nguyên
- Cờ sự kiện (Event Flag): Dùng để thể hiện một hay nhiều sự kiện xảy ra. Cờ sự kiện cho phép đồng bộ khóa trong trường hợp có nhiều sự kiện kết hợp.
- Hộp thư (mailbox), hàng đợi (queue) hay đường ống (pipe): Các cơ chế này dùng để chuyển dữ liệu giữa các tác vụ với nhau.

Ngoài ra, trên thực tế có thể sử dụng một số cơ chế khác.

Semaphore:

Về mặt chức năng, semaphore hoạt động giống như một chiếc chìa khóa cho việc truy cập tài nguyên. Tác vụ vào giữ chiếc chìa khóa này mới có quyền sử dụng tài nguyên hệ thống.

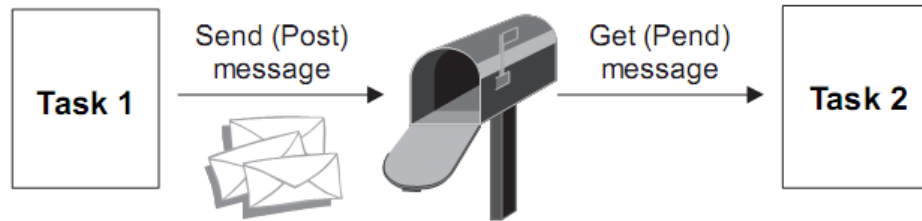
VD: Trong trường hợp có 2 tác vụ cùng truy cập một chiếc máy in. Nếu không có cơ chế đồng bộ giữa 2 tác vụ, máy in sẽ in bất cứ dữ liệu nào mà nó nhận được. Khi đó các bản tin sẽ bị trộn lẫn.

Trên thực tế, để có thể sử dụng tài nguyên hệ thống, tác vụ cần yêu cầu chìa khóa (semaphore) bằng cách gọi tới một dịch vụ thích hợp. Nếu chìa khóa ở trạng thái sẵn sàng (có nghĩa là tài nguyên hiện tại đang không được sử dụng bởi tác vụ nào), tác vụ có thể được phép sử dụng tài nguyên, đồng thời chìa khóa sẽ được giữ lại. Sau khi sử dụng xong, tác vụ đó phải trả lại chìa khóa (semaphore) để các tác vụ khác có thể sử dụng tài nguyên hệ thống.

Ngược lại, nếu tài nguyên đang được sử dụng bởi tác vụ khác, tác vụ đó sẽ bị khóa cho đến khi semaphore được trả lại. Trong trường hợp cùng một lúc có nhiều tác vụ yêu cầu semaphore khi tài nguyên hệ thống đang được sử dụng thì tất cả tác vụ đó sẽ bị khóa lại. Các tác vụ bị khóa sẽ được xếp hàng theo thứ tự về mặt ưu tiên hay về mặt thời gian theo yêu cầu semaphore.

Hộp thư (mailbox):

Không những chỉ tạo các tin hiệu cho các sự kiện, các tác vụ thường xuyên cần chia sẻ dữ liệu. Việc chia sẻ này được thực hiện bằng cơ chế hộp thư (mailbox). Một tác vụ (là người gửi) sẽ gửi một bản tin (message) tới hộp thư trong khi tác vụ khác (người nhận) chờ bản tin tại hộp thư. Nếu không có bản tin trong hộp thư khi tác vụ nhận chờ ở hộp thư, tác vụ đó sẽ bị khóa đến khi bản tin được đưa tới.



Hình 3. 4: Cơ chế truyền tin trong mailbox

Hàng đợi (queue) hay đường ống (pipe):

Hàng đợi thường là hộp thư có khả năng chứa nhiều thư cùng một lúc. Khi tạo ra một hàng đợi, cần xác định kích thước của hàng, số bản tin hàng đợi có thể lưu. Trong hàng đợi ngoài các lệnh gửi và chờ như hộp thư, một hàng đợi cần có các dịch vụ để chuyển bản tin có ưu tiên cao hơn lên đầu hàng hoặc xóa các bản tin trong cả hàng.

Trong cơ chế đường ống, việc sắp xếp dữ liệu lại khác một chút. Đối với hộp thư và hàng đợi, việc dịch chuyển dữ liệu được thực hiện theo khối (bản tin). Còn trong cơ chế đường ống, một luồng dữ liệu (byte stream) liên tục được nối giữa hai tác vụ. Một tác vụ sẽ đọc đường ống, còn một tác vụ khác ghi lên đường ống.

3.2.5 HAL (Hardware Abstraction Layer)

Để việc triển khai hệ điều hành thời gian thực trên các phần cứng khác nhau được dễ dàng hơn, cần có việc đặt ra các giao diện chuẩn cho việc lập trình. Khi lập trình viên chuyển sang một nền tảng (platform) mới, các giao diện lập trình này được giữ nguyên mặc dù phần cứng phía dưới thay đổi.

VD: Hai nền tảng khác nhau có thể có các bộ đếm khác nhau. Mỗi bộ đếm cần một phiên bản mã mới để khởi tạo và thiết lập cấu hình thiết bị. Nếu sử dụng HAL, giao diện lập trình sẽ không thay đổi mặc dù cả phần cứng và phần mềm đều thay đổi.

HAL là một lớp phần mềm cung cấp một tập hợp các giao diện lập trình xác định, che đi cấu trúc phần cứng. HAL được thực thi bằng cách ảo hóa nền tảng phần cứng, làm cho các trình điều khiển có thể chuyển giữa các phần cứng khác nhau.

Phần mềm HAL giao tiếp với một thiết bị ngoại vi chuyên biệt được gọi là trình điều khiển thiết bị (device driver). Một trình điều khiển thiết bị cung cấp giao diện lập trình ứng dụng chuẩn (API) để đọc và ghi tới thiết bị ngoại vi đó.

Trên thực tế, khi phát triển các bo mạch nhúng, các hãng thường cung cấp gói phần mềm phát triển kèm theo (BSP). Các gói phần mềm cho việc phát triển này tương đương với HAL.

HAL thường hỗ trợ các thành phần phần cứng sau:

- CPU, cache, và MMU.
- Thiết lập bản đồ bộ nhớ.
- Hỗ trợ xử lý ngắt và các trường hợp đặc biệt.
- Truy cập bộ nhớ trực tiếp (DMA).

- Các bộ đếm (Timers).
- Console của hệ thống.
- Quản lý giao diện bus.
- Quản lý nguồn.

3.3 Giới thiệu các hệ điều hành thời gian thực

3.3.1 FreeRTOS:

FreeRTOS là một hệ điều hành thời gian thực miễn phí. Hệ điều hành này được Richard Barry công bố rộng rãi từ năm 2003, đến nay, hệ điều hành này đang được phát triển mạnh mẽ và được cộng đồng mạng mã nguồn mở ủng hộ. FreeRTOS có tính khả chuyển, mã nguồn mở, lõi có thể được tải miễn phí và nó có thể dùng cho các ứng dụng thương mại. Nó phù hợp với những hệ nhúng thời gian thực nhỏ. Hầu hết các code được viết bằng ngôn ngữ C nên nó có tính phù hợp cao với nhiều nền khác nhau.

Ưu điểm của nó là dung lượng nhỏ và có thể chạy trên những nền phần cứng mà nhiều hệ điều hành khác không chạy được. Có thể port cho nhiều kiến trúc vi điều khiển và những công cụ phát triển khác nhau.

FreeRTOS được cấp giấy phép bởi bản đã được chỉnh sửa bởi GPL (*General Public License*) và có thể sử dụng trong các ứng dụng thương mại với giấy phép này. Ngoài ra liên quan đến FreeRTOS có OpenRTOS và SafeRTOS. OpenRTOS là bản thương mại của FreeRTOS và không liên quan gì đến GPL. SafeRTOS là về cơ bản dựa trên FreeRTOS nhưng được phân tích, chứng minh bằng tài liệu và kiểm tra nghiêm ngặt với chuẩn IEC61508. Và chuẩn IEC61508 SIL3 đã được tạo ra và phát triển độc lập để hoàn thiện tài liệu cho SafeRTOS.

So sánh giữa FreeRTOS và OpenRTOS:

	FreeRTOS	OpenRTOS
Miễn phí	Có	Không
Có thể sử dụng trong ứng dụng thương mại?	Có	Có
Miễn phí trong ứng dụng thương mại?	Có	Có
Phải đưa ra mã nguồn của code ứng dụng?	Không	Không
Phải đưa ra thay đổi mã nguồn của lõi?	Có	Không
Phải đưa vào báo cáo nếu sử dụng FreeRTOS?	Có, đường dẫn	Không
Phải cung cấp mã FreeRTOS cho người sử dụng?	Có	Không
Có thể nhận hỗ trợ thương mại?	Không	Có

Bảng 3. 1: So sánh giữa FreeRTOS và OpenRTOS

Các đặc điểm chính của FreeRTOS:

- Lõi FreeRTOS hỗ trợ cả preemptive, cooperative hoặc kết hợp giữa 2 kiểu lập lịch này.

- SafeRTOS là sản phẩm dẫn xuất, cung cấp mã nguồn riêng ở mức độ cao.
- Được thiết kế nhỏ, đơn giản và dễ sử dụng.
- Cấu trúc mã nguồn rất linh động được viết bằng ngôn ngữ C.
- Hỗ trợ cả task và co-routine.
- Có lựa chọn nhận biết tràn ngăn xếp.
- Không giới hạn số task có thể tạo ra, phụ thuộc vào tài nguyên của hệ thống.
- Không giới hạn số mức ưu tiên được sử dụng.
- Không giới hạn số task cùng một mức ưu tiên.
- Hỗ trợ truyền thông và đồng bộ giữa các tác vụ hoặc giữa tác vụ và ngắt thông qua nhiều chiến lược khác nhau.
- Các công cụ phát triển miễn phí, port cho Cortex-M3, ARM7, PIC, MSP430, H8/S, AMD, AVR, x86 và 8051...
- Miễn phí mã nguồn phần mềm nhúng.
- Miễn phí trong ứng dụng thương mại.
- Tiền cấu hình cho các ứng dụng thử nghiệm, từ đó dễ dàng tìm hiểu và phát triển.

Các dòng vi điều khiển đã được thử nghiệm với FreeRTOS:

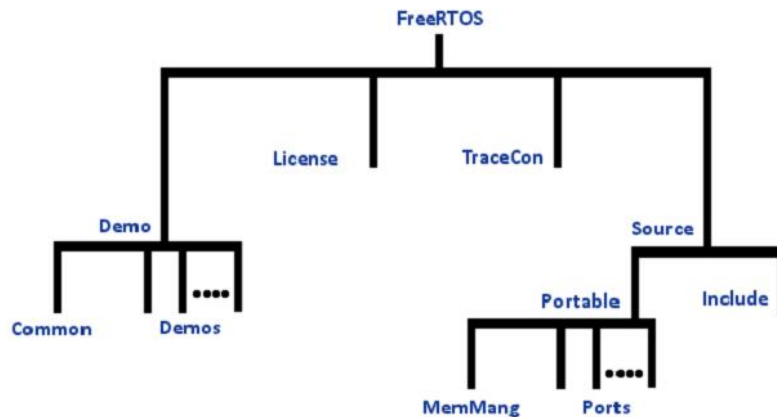
- Vi điều khiển ST STM32 Cortex-M3.
- ARM Cortex-M3 dựa trên vi điều khiển sử dụng ARM Keil (RVDS), IAR, Rowley và công cụ GCC.
- Atmel AVR32 AT32UC3A: vi điều khiển flash sử dụng GCC và IAR.
- Các vi điện tử ST: STR71x (ARM7), STR75x (ARM7), STR9 (ARM9) (STR711F, STR712F...).
- LPC2106, LPC2124 và LPC2129 (ARM7). Gồm mã nguồn cho I2C driver.
- H8S2329 (Hitachi H8/S) với EDK2329 demo.
- Atmel AT91SAM7 family (AT91SAM7X256, AT91SAM7X128, AT91SAM7S32, AT91SAM7S64, AT91SAM7S128, AT91SAM7S256). Bao

gồm mã nguồn USB driver cho IAR Kickstart, uIP và lwIP nhúng vào Ethernet TCP/IP.

- AT91FR40008 với Embest ATEB40X demo.
- MSP430 với demo cho LCD driver. MSPGCC and Rowley CrossWorks được hỗ trợ.
- HCS12 (MC9S12C32 loại bộ nhớ nhỏ và MC9S12DP256B kiểu bank nhớ)
- Fujitsu MB91460 series (32bit) and MB96340 series (16FX 16bit) sử dụng trình dịch Softune và Euroscope debugger.
- Cygnal 8051 / 8052
- Microchip PICMicro PIC18 (8 bit), PIC24 (16bit MCU) và dsPIC (16bit DSC) và PIC32 (32bit)
- Atmel AVR (MegaAVR) với STK500 demo.
- Vi điều khiển RDC8822 với demo cho Flashlite 186 SBC.
- PC (chạy ở FreeDOS hoặc DOS khác)
- ColdFire, chú ý rằng port nàyko được hỗ trợ.
- Zilog Z80, chú ý rằng port nàyko được hỗ trợ.
- Xilinx Microblaze chạy trên Virtex4 FPGA.
- Xilinx PowerPC (PPC405) chạy trên Virtex4 FPGA.

Ngoài ra các trình dịch đã hỗ trợ cho các bản thử nghiệm FreeRTOS trên các vi điều khiển: Rowley CrossWorks, Keil, CodeWarrior, IAR, GNU GCC (nhiều loại), MPLAB, SDCC, Open Watcom, Paradigm và Borland.

Cấu trúc thư mục các file của FreeRTOS:



Hình 3. 5: Cấu trúc thư mục FreeRTOS

Các bản download FreeRTOS trên www.freertos.org bao gồm mã nguồn cho các bản port trên các bộ vi xử lý và các ứng dụng demo. Cấu trúc thư mục khá đơn giản, và nhân FreeRTOS chỉ bao gồm đúng 3 files(hay 4 file với tính năng co-routines – đây là một dạng rút gọn của task thường được dùng cho các hệ thống với bộ nhớ rất giới hạn).

Từ thư mục gốc, bản download FreeRTOS gồm 2 thư mục con :

```

FreeRTOS
|
|--Demo Chứa các ứng dụng thử nghiệm.
|
|--Source Chứa mã nguồn của nhân.

```

Phần lớn nhân FreeRTOS (cụ thể hơn là mã nguồn của bộ lập lịch phân bổ - scheduler) chứa trong 3 file chung với mọi kiến trúc vi xử lý là **tasks.c**, **queue.c** và **list.c** (hoặc thêm file croutine.c thực thi chức năng co-routines) trong thư mục **Source**.

Mỗi kiến trúc vi xử lý sẽ yêu cầu 1 phần mã nhân nhỏ cho riêng kiến trúc đó. Mã riêng cho từng kiến trúc được nằm trong thư mục **/Source/Portable**.

```

FreeRTOS
|
|--Demo Chứa các ứng dụng thử nghiệm. .
| |
| | |--Common Các file ứng dụng chung cho tất cả các port.
| | |
| | |--ARM7_Atme1SAM7S64_IAR Ứng dụng mẫu cho Atme1SAM7S64 tool
IAR.
| | |
| | |--ARM7_AT91SAM7X256_Eclipse Ứng dụng mẫu cho AT91SAM7X256,
tool Eclipse

```

```

| |
| +-.....
|
+-Source Chứa các file mã nguồn của bộ lập lịch (sheduler)
|
| 3 file mã nguồn của bộ lập lịch chung cho tất cả các port
|           (4 với chức năng co-routines)
+-include Các file header.
|
+-portable Lớp port cho các bộ vi xử lý khác nhau.
|
+-MemMang Bộ cấp phát bộ nhớ mẫu.
|
+-GCC Lớp port cho các vi xử lý sử dụng tool GCC compiler
|
+-RVDS Lớp port cho các vi xử lý sử dụng tool RVDS
|
+....

```

5 file header trong thư mục **/Source/Include** chính chứa các khai báo các tham số, định nghĩa các kiểu dữ liệu và các khai báo prototype của các API của FreeRTOS đó là :

- FreeRTOS.h: chứa khai báo các file header và các định nghĩa được yêu cầu cho vi xử lý sẽ được sử dụng và kiểm tra các marco cần thiết phải viết cho riêng từng vi xử lý đã được định nghĩa chưa. Các marco này phải được định nghĩa trong FreeRTOSConfig.h trong thư mục demo của từng port.
- Task.h: chứa các khai báo về các hàm và các marco liên quan đến các thủ tục tạo, xóa, thiết lập và chỉnh sửa các tham số của các task(stack, trạng thái(running, ready, block), mức ưu tiên,...), các tác vụ liên quan đến bộ scheduler.
- List.h: chứa các khai báo về các hàm và các marco xây dựng cấu trúc danh sách để scheduler dùng để quản lý các task.
- Croutine.h: chứa các khai báo về các hàm và các marco xây dựng các co-routine.
- Portables.h: chứa khai báo các header phù hợp với từng port và cho phép tạo tính linh động đối với từng cấu hình port.

Ngoài ra còn một số các header chứa các khai báo về các API hỗ trợ các thuật toán cho phép truyền thông giữa các task như queues.h, semphr.h.

a) FreeRTOS.h:

Gồm 2 phần:

- Chứa các khai báo về các file header viết riêng cho từng port:

```
/* Chứa các định nghĩa cơ bản */

#include "projdefs.h"

/* Chứa các định nghĩa về các tham số cấu hình */

/* cho bộ scheduler */

#include "FreeRTOSConfig.h"

/* chứa các định nghĩa cho từng port */

#include "portable.h"
```

- Kiểm tra các khai báo cần thiết về các tham số cấu hình và các marco, thiết lập các giá trị mặc định với các tham số chưa được thiết lập giá trị cụ thể.

```
/*kiểm tra cấu hình cho chính sách sử dụng trong bộ scheduler */

#ifndef configUSE_PREEMPTION

    #error Missing definition: configUSE_PREEMPTION should
    be defined in FreeRTOSConfig.h as either 1 or 0. See the
    Configuration section of the FreeRTOS API documentation for
    details.

#endif

.....

/*kiểm tra cấu hình sử dụng co-routine hay không*/

#ifndef configUSE_CO_ROUTINES

    #error Missing definition: configUSE_CO_ROUTINES
    should be defined in FreeRTOSConfig.h as either 1 or 0. See
    the Configuration section of the FreeRTOS API documentation
    for details.

#endif

.....

/*kiểm tra cấu hình tham số cho semaphores và thiết lập*/

/*giá trị mặc định khi cần thiết*/

#ifndef configUSE_COUNTING_SEMAPHORES
```

```
#define configUSE_COUNTING_SEMAPHORES 0

#endif
```

b) Task.h:

Gồm các khai báo về các API liên quan đến task, chia theo 5 chức năng:

Các API liên quan đến tạo và xóa task:

Gồm 2 task chính thực hiện 2 nhiệm vụ qua trọng là tạo mới và xóa task.

- API tạo task : *xTaskCreate()*.
- API xóa task: *vTaskDelete()*.

Khai báo đầy đủ của *xTaskCreate()* là:

```
portBASE_TYPE xTaskCreate(
    pdTASK_CODE pvTaskCode,
    const char * const pcName,
    unsigned short usStackDepth,
    void *pvParameters,
    unsigned portBASE_TYPE uxPriority,
    xTaskHandle *pvCreatedTask
);
```

API này sẽ tạo ra 1 một task mới và thêm nó vào danh sách các task sẵn sàng thực thi. Task được tạo ra có quyền truy cập toàn bộ đối với toàn bộ bộ nhớ của vi xử lý. Đối với các hệ thống mà có thêm hỗ trợ MPU, có thể tạo ra một task với các tham số ràng buộc bởi MPU thông qua API *xTaskCreateRestricted()*.

Thực chất thì khai báo của *xTaskCreate()* là một marco gọi đến thủ tục *xTaskGenericCreate()* trong tasks.c. Đây là một hàm cho phép tạo ra các task với nhiều tùy chọn hơn, phù hợp với các cấu hình khác nhau của hệ thống, cụ thể trong trường hợp này, nó tạo ra một task trong cấu hình không sử dụng khối MPU.

```
/*định nghĩa API xTaskCreate()*/

#define xTaskCreate( pvTaskCode, pcName, usStackDepth,
    pvParameters, uxPriority, pxCreatedTask ) xTaskGenericCreate(
```

```
( pvTaskCode ), ( pcName ), ( usStackDepth ), ( pvParameters
), ( uxPriority ), ( pxCreatedTask ), ( NULL ), ( NULL ) )
```

Các API điều khiển task: tạo ra các hàm điều khiển API cụ thể là các nhiệm vụ như sau:

- Gây trễ task với các điều kiện khác nhau: *vTaskDelay()* và *vTaskDelayUntil()*. *vTaskDelay()* dùng để tạo trễ trong một khoảng thời gian tương đối bắt đầu từ thời điểm gọi API này, còn *vTaskDelayUntil()* tạo trễ trong một khoảng thời gian xác định.
- Các tác vụ liên quan đến ức ưu tiên: *xTaskPriorityGet()* và *vTaskPrioritySet()*. Hai API này làm nhiệm vụ xác định mức ưu tiên đang có và đặt mức ưu tiên cho task.
- Các API liên quan đến trạng thái task như chặn, khôi phục task: *vTaskSuspend()* nhằm để chặn bất kỳ task nào. *vTaskResume()* được gọi sau khi task bị dừng muốn quay về trạng thái sẵn sàng. Muốn gọi hàm *vTaskResume()* từ ngắt thì sử dụng *xTaskResumeFromISR()*.

Các API tiện ích cho task: chứa các API cung cấp các tiện ích hệ thống cho task như:

- *xTaskGetTickCount*: trả lại giá trị các tick đếm được từ khi *vTaskStartScheduler* bị hủy bỏ.
- *uxTaskGetNumberOfTasks(void)*: trả lại số lượng các task mà kernel đang quản lý. Hàm này còn bao gồm cả các task đã sẵn sàng, bị khóa hoặc bị treo. Task đã bị delete mà chưa được giải phóng bởi idle cũng được tính vào.
- *vTaskList()*: hàm này sẽ không cho phép ngắt trong khoảng thời gian nó làm việc. Nó không tạo ra để chạy các ứng dụng bình thường nhưng giúp cho việc debug.
- *vTaskStartTrace()*: đánh dấu việc bắt đầu hoạt động của kernel. Việc đánh dấu chia ra để nhận ra task nào đang chạy vào lúc nào. Đánh dấu file được lưu trữ ở dạng nhị phân. Sử dụng những tiện ích độc lập của DOS thì gọi convtrce.exe để chuyển chúng sang kiểu text file dạng mà có thể được xem và được vẽ.
- *ulTaskEndTrace()*: Dừng đánh dấu kernel hoạt động, trả lại số byte mà đã viết vào bộ đệm đánh dấu.
-

Các API điều khiển nhân: gồm các API cung cấp các giao diện đối với bộ scheduler như:

- *vTaskStartScheduler()* : khởi động bộ scheduler, cụ thể là kích hoạt hoạt động của freeRTOS.
- *vTaskEndScheduler()* : chấm dứt hoạt động của scheduler.
- *taskDISABLE_INTERRUPTS()/taskENABLE_INTERRUPTS()* :cấm/cho phép ngắt.
- *taskENTER_CRITICAL()/taskEXIT_CRITICAL()* : báo hiệu các đoạn mã quan trọng cần có những xử lý phù hợp.
-

Các API liên quan đến MPU: gồm các API mà ứng dụng khối MPU trong các cấu hình port có hỗ trợ MPU:

- *xTaskCreateRestricted()* : tạo 1 task với những ràng buộc về quyền truy cập đến tài nguyên bộ nhớ của vi xử lý. Nó cũng như *xTaskCreate()*, là một Macro tham chiếu đến *xTaskGenericCreate()* tuy nhiên nó có thêm các tham số về MPU.

```
#define xTaskCreateRestricted( x, pxCreatedTask )
xTaskGenericCreate( ((x)->pvTaskCode), ((x)->pcName), ((x)->usStackDepth), ((x)->pvParameters), ((x)->uxPriority),
(pxCreatedTask), ((x)->puxStackBuffer), ((x)->xRegions) )
```

- *vTaskAllocateMPURegions()* :thay đổi hoặc cấp phát lại các vùng nhớ ràng buộc của các task được tạo bởi *xTaskCreateRestricted()*.
-

c) List.h:

File này chứa các khai báo về các hàm và các marco liên quan đến các thủ tục tạo, xóa, thiết lập và chỉnh sửa các tham số của các task.

FreeRTOS xây dựng một cấu trúc danh sách các task, mỗi task là một phần tử có cấu trúc:

```
/*khai báo kiểu của các phần tử trong danh sách*/
struct xLIST_ITEM
{
    /*kết hợp với mức ưu tiên dùng trong sắp xếp danh sách */
    portTickType xItemValue;
```



```

/*Con trỏ đến phần tử trước nó trong danh sách */
volatile struct xLIST_ITEM * pxNext;

/* Con trỏ đến phần tử sau nó trong danh sách */
volatile struct xLIST_ITEM * pxPrevious;

/*con trỏ đến task, cụ thể là Task Control Block(TCB)*/
void * pvOwner;

/*con trỏ đến danh sách mà nó nằm trong đó*/
void * pvContainer;
};

/*Định nghĩa danh sách*/
typedef struct xLIST
{
    /*kích thước của danh sách*/
    volatile unsigned portBASE_TYPE uxNumberOfItems;

    /*pxIndex: dùng để chỉ đến phần tử hiện thời của danh
sách*/
    volatile xListItem * pxIndex;

    /*đánh dấu kết thúc danh sách*/
    volatile xMiniListItem xListEnd;
} xList;

```

Đi kèm cấu trúc danh sách được sử dụng trong bộ scheduler là các API thao tác trên các phần tử của danh sách và danh sách:

- Khởi tạo danh sách: *vListInitialise()*.
- Đặt đối tượng sở hữu các phần tử của danh sách.
- Đặt giá trị của phần tử danh sách. Trong hầu hết trường hợp giá trị đó được dùng để sắp xếp danh sách theo một thứ tự nhất định nào đó.
- Để lấy giá trị của phần tử danh sách. Giá trị này có thể biểu thị bất cứ cái gì, ví dụ như mức ưu tiên của tác vụ hoặc thời gian mà task có thể bị khoá.

- Xác định xem danh sách còn chứa phần tử nào không, chỉ có giá trị true nếu danh sách rỗng.
- Kiểm tra số phần tử trong danh sách.
- Xác định phần tử tiếp theo của danh sách.
- Tìm chương trình chủ của phần tử đầu tiên trong danh sách.
- Kiểm tra xem phần tử có nằm trong danh sách không.
- Thêm phần tử vào danh sách.
- Loại bỏ phần tử từ danh sách.

Ví dụ:

```

.....
/*API xác định độ dài hiện tại của danh sách*/
#define listCURRENT_LIST_LENGTH( pxList )      ( ( pxList
)->uxNumberOfItems )
.....
/*API khởi tạo danh sách*/
void vListInitialise( xList *pxList );

```

d) Croutine.h:

Chứa các khai báo về các hàm và các marco xây dựng các co-routine. Trong Croutine, cần chú ý một số các API quan trọng tương ứng với các thao tác trên task khai báo trong task.h:

- xCoroutineCreate(): tạo mới một co-routine và đưa nó vào danh sách các co-routine sẵn sàng chạy.
- crDELAY(): trễ một co-routine() trong một chu kì thời gian xác định.
- crSTART()/crEND(): khởi chạy/kết thúc một co-routine.
- ...

Ví dụ về cấu trúc chung của một co-routine:

```

void    vACoRoutine(    xCoRoutineHandle    xHandle,    unsigned
portBASE_TYPE uxIndex )

{

    // Khai báo các biến.

    static long ulAVariable;

    // bắt đầu co-routine;

    crSTART( xHandle );


    for( ;; )

    {

        // các câu lệnh của co-routine.

    }

    // kết thúc co-routine

    crEND();

}

```

e) Portables.h:

Khai báo file header tương ứng với port sẽ được sử dụng. File header được đưa vào sẽ tương ứng với marco về bản port được sử dụng:

```

.....

/*kiểm tra có phải là port cho avr, tool gcc*/

#ifdef GCC_MEGA_AVR

    #include "../portable/GCC/ATMega323/portmacro.h"

#endif

/*kiểm tra có phải là port cho avr, tool IAR*/

#ifdef IAR_MEGA_AVR

    #include "../portable/IAR/ATMega323/portmacro.h"

#endif

```

```

/*kiểm tra có phải là port cho PIC24, tool MPLAB*/

#ifdef MPLAB_PIC24_PORT

    #include
    "..\..\Source\portable\MPLAB\PIC24_dsPIC\portmacro.h"

#endif

....

```

Thêm nữa, file này cũng bao gồm các khai báo về các thủ tục quản lý bộ nhớ đối với từng port:

```

/*

 * Map to the memory management routines required for the
 port.

 */

void *pvPortMalloc( size_t xSize ) PRIVILEGED_FUNCTION;
void vPortFree( void *pv ) PRIVILEGED_FUNCTION;
void vPortInitialiseBlocks( void ) PRIVILEGED_FUNCTION;
size_t xPortGetFreeHeapSize( void ) PRIVILEGED_FUNCTION;
....

```

Thêm vào các file của nhân cần quan tâm 2 file *FreeRTOSConfig.h* và *portmacro.h*. Trong đó *FreeRTOSConfig.h* chứa các tham số cấu hình phù hợp với từng port cụ thể còn *portmacro.h* chứa các API riêng đối với từng port như cấu hình timer, các thủ tục sao lưu và khôi phục context.

3.3.2 Windows CE:

Windows CE là một hệ điều hành 32-bit nhỏ cấu thành từ các module được thiết kế nhằm sử dụng trong các thiết bị với các yêu cầu bộ nhớ nhỏ như các hệ thống nhúng được Microsoft phát triển. Windows CE là một hệ điều hành đa nhiệm giống như Windows NT. Nó bao gồm hầu hết các chức năng giao diện người sử dụng của Windows NT, các nhà phát triển phần mềm quen với các ứng dụng Windows trên PC có thể nhanh chóng phát triển các ứng dụng trên Windows CE.

Bộ nhớ trên các thiết bị sử dụng Windows CE gồm RAM và ROM. Ngoài ra còn có các thẻ bộ nhớ flash nhằm mục đích mở rộng không gian lưu trữ. Windows CE cũng hỗ trợ đầy đủ cho các thẻ PCMCIA. Nội dung của hệ thống file trong Windows CE được lưu trữ trên RAM. Hệ điều hành và tất cả các ứng dụng đi kèm với các thiết bị sử dụng Windows CE được lưu trữ trong ROM.

Windows CE gồm có 7 phân hệ con, mỗi phân hệ này được chia thành các thành phần nhỏ hơn. Các phân hệ gồm:

- Phân hệ Nhân.

Nhân của Windows CE tương tự với nhân của Windows NT. Nó sử dụng cùng mô hình tiến trình(process) và tiểu trình(thread) như Windows NT. Thêm nữa, Windows CE sử dụng mô hình bộ nhớ ảo cũng tương tự như Windows NT. Nhà phát triển có thể viết các ứng dụng trong Windows CE mà chia sẻ bộ nhớ trên các tiến trình sử dụng các file được ánh xạ bộ nhớ.

- Phân hệ GWES(Graphichs, Windowing, and Event Subsystem).

Windows CE đã tổng hợp các thành phần GDI và người sử dụng vào phân hệ này. Các tác vụ như tạo một cửa sổ, vẽ một cửa sổ hay tải một tài nguyên chuỗi kí tự được xử lý bên trong phân hệ này. Các thành phần điều khiển con của Windows CE như các button, list box,... được thực thi trong GWES.

Thêm nữa GWES cũng chứa trình quản lý sự kiện, thực hiện trao đổi thông điệp đối với các tiến trình.

- Phân hệ lưu trữ đối tượng.

RAM trong một thiết bị cài đặt Windows CE được chia làm 2 phần: bộ nhớ chương trình và phần lưu trữ đối tượng. Phần lưu trữ đối tượng chứa hệ thống file và phần đăng ký. Phần lưu trữ đối tượng cũng chứa các cơ sở dữ liệu của các ứng dụng.

- Phân hệ OAL (OEM Adaptation Layer).

Phân hệ này chứa các thành phần phần mềm mà một nhà sản xuất thiết bị gốc(gọi tắt là OEM – Original Equipment Manufacturer) phải tạo ra nhằm port Windows CE tới phần cứng mới. Nó là các thành phần mà bạn đặc chế cá giao diện phần cứng và các thường trình phục vụ ngắt cho phép phần cứng giao tiếp với Windows CE.

- Phân hệ lớp trình điều khiển thiết bị.

Là một phân hệ của hệ điều hành chứa các trình điều khiển thiết bị cho các ngoại vi của thiết bị. Nó gồm trình điều khiển bộ thẻ nhớ flash, video, bàn phím

- Phân hệ các API truyền thông.

Một chức năng khác cũng rất quan trọng của Windows CE đó là khả năng chia sẻ dữ liệu với một desktop PC. Windows CE vì vậy hỗ trợ kỹ thuật ActiveSync cho phép các nhà lập trình ứng dụng tới các nhà cung cấp dịch vụ đồng bộ dữ liệu của ứng dụng giữa các thiết bị cài đặt Windows CE và máy tính để bàn.

Windows CE chứa rất nhiều các API truyền thông tương tự như trong Windows NT như socket, nối tiếp, TAPO, WinINET

- Phân hệ các shell và Internet Explorer.

Các OEM có thể sử dụng thành phần shell của Windows CE để viết các shell đặc chế của họ cho thiết bị. Đồng thời Windows CE cũng hỗ trợ 1 phiên bản của Internet Explorer

Hiện tại Windows CE hỗ trợ các bộ vi xử lý tương thích Intel X86, MIPS, ARM và Hitachi SuperH.

3.3.3 Hệ điều hành Embedded Linux:

Thực chất của hệ điều hành embedded Linux là một phiên bản của hệ điều hành Linux được dùng trong các hệ thống nhúng như mobile phone, PDA, set-top box, các thiết bị điện tử gia dụng, các thiết bị mạng, các thiết bị điều khiển,... Theo thống kê hiện có, Linux hiện được sử dụng bởi khoảng 18% các kỹ sư nhúng.

Không giống như phiên bản dành cho desktop và server của Linux, phiên bản embedded Linux được thiết kế cho các thiết bị với tài nguyên giới hạn. Bởi những hạn chế về giá thành và kích thước, các thiết bị nhúng thường có ít RAM và bộ nhớ thứ cấp hơn so với desktop, và thường sử dụng bộ nhớ flash thay vì các ổ đĩa cứng. Bởi vì các thiết bị nhúng thực hiện các chức năng chuyên dụng nên các nhà phát triển đã tối ưu các phiên bản nhúng của Linux cho các cấu hình phần cứng của các hệ thống nhúng. Những tối ưu này giúp giảm thiểu các ứng dụng phần mềm và các trình điều khiển thiết bị, và cho phép thay đổi nhân Linux để nó trở thành 1 hệ điều hành thời gian thực.

Thiết kế của Linux phân biệt giữa các mã phụ thuộc và bộ xử lý cần thay đổi cho mỗi kiến trúc và mã có thể được khả chuyển đơn giản chỉ bằng việc biên dịch lại nó. Vì vậy, Linux đã được khả chuyển cho nhiều các kiến trúc bộ vi xử lý khác nhau như:

- Motorola 68k và các biến thể của nó.
- Alpha.
- Power PC.
- ARM.
- SPARC.
- MIPS.
- AVR32.
- Blackfin.
- ...

Việc sử dụng Linux trong các thiết bị nhúng có ưu điểm là bản quyền miễn phí, nhân ổn định, và được hỗ trợ lớn từ cộng đồng mã nguồn mở, nhà phát triển được tự do thay đổi và phân phối mã nguồn. Tuy nhiên nhược điểm đó là nó yêu cầu bộ nhớ tương đối lớn cho nhân và hệ thống file gốc, sự phức tạp trong việc truy nhập bộ nhớ ở chế độ nhân và chế độ người sử dụng cũng như nền tảng trình điều khiển thiết bị phức tạp.

Các đặc điểm quan trọng của Linux gồm:

- Đa nhiệm : bộ lập lịch phân bổ trong Linux sử dụng thuật toán lập lịch preemptive trong đó một tiến trình với mức ưu tiên cao hơn sẽ được ưu tiên là tiến trình tiếp theo được CPU thực hiện khi xảy ra một sự kiện không đồng bộ.
- Đa người sử dụng : Linux phát triển từ Unix là một hệ thống chia sẻ theo thời gian cho phép nhiều người sử dụng chia sẻ chung một máy tính. Vì vậy có rất nhiều các chức năng hỗ trợ việc bảo vệ dữ liệu và tính riêng tư.
- Đa xử lý: Linux cung cấp hỗ trợ mở rộng cho đa xử lý đối xứng SMP(Symmetric Multi-Processing).
- Bộ nhớ được bảo vệ: Mỗi tiến trình trong Linux hoạt động trong không gian nhớ riêng của nó và không được cho phép truy cập trực tiếp đến không gian nhớ của một tiến trình khác. Điều này tránh những con trỏ lỗi trong một tiến trình gây phá hoại không gian nhớ của tiến trình khác. Các truy cập sai được bẫy bởi phần cứng bảo vệ bộ nhớ của bộ xử lý và tiến trình sẽ bị ngắt với cảnh báo tương ứng.
- Hệ thống file phân cấp.

Hiện nay, với sự yêu cầu giảm thiểu thời gian phát triển, embedded Linux đang là một xu hướng phát triển mạnh mẽ trong thế giới nhúng.

3.3.4 Hệ điều hành uCLinux:

uCLinux hay micro-controller Linux là một phiên bản của Linux được thiết kế cho các bộ vi xử lý mà không có khối quản lý bộ nhớ MMU(Memory Management Unit). Việc không có MMU là một đặc điểm khá chung đối với các bộ vi xử lý giá thành thấp. uCLinux là một giải pháp mã nguồn mở và miễn phí bản quyền, mục đích là nhằm tương thích với Linux.

Dự án uCLinux được bắt đầu năm 1997 với mục đích cho ra một phiên bản của nhân Linux 2.0 dành cho các vi điều khiển giá thành thấp do Jeff Dionne, Kenneth Albanowski và một nhóm các nhà phát triển khác đề xuất trong quá trình họ thảo luận về việc nhúng Linux và các bộ điều khiển mạng không có MMU nhằm giải quyết bài toán truyền thông trong hệ thống truyền thông. Phiên bản đầu tiên của uCLinux là được phân bổ với bộ xử lý Motorola 68000, dựa trên bộ vi xử lý MC68328 được triển khai trong một bộ điều khiển SCADA.

Ngày nay, uCLinux đã được port cho rất nhiều dòng vi điều khiển như ColdFire, Axis ETRAX, ARM, Atari 68k,...

Câu hỏi ôn tập

1. Hệ điều hành thời gian thực là gì?
2. Nhiệm vụ của hệ điều hành thời gian thực?
3. Các đặc điểm của hệ điều hành thời gian thực.
4. Vẽ sơ đồ vị trí hệ điều hành thời gian thực trong model hệ thống nhúng.
5. Liệt kê một số hệ điều hành thời gian thực.
6. Kernel là gì?
7. Liệt kê và mô tả các tính năng chính của kernel.
8. Các model của hệ điều hành bao gồm những model nào?
9. Sự khác biệt giữa process và tác vụ?
10. Sự khác biệt giữa chiếm quyền và không chiếm quyền
11. Các trạng thái của tác vụ bao gồm các trạng thái nào?
12. Việc đồng bộ được thực hiện như thế nào?
13. Liệt kê các hàm chính trong FreeRTOS và mô tả các chức năng của các hàm?

CHƯƠNG 4: THIẾT KẾ VÀ CÀI ĐẶT CÁC HỆ THỐNG NHÚNG

4.1 Thiết kế hệ thống

4.1.1 Xác định yêu cầu

Một đặc điểm của hệ thống nhúng là cả phần cứng và phần mềm phải được tính toán, cân nhắc trong thời gian thiết kế chúng. Bởi vậy, loại thiết kế này còn được gọi là đồng thiết kế phần cứng/phần mềm (hardware/software codesign). Mục đích cuối cùng là tìm được sự kết hợp thích hợp của phần cứng và phần mềm để sản phẩm tạo ra có đầy đủ các đặc điểm kỹ thuật đã đề ra. Bởi vậy, các hệ thống nhúng không thể được thiết kế bởi một quá trình tổng hợp chỉ ghi chép lại các đáp ứng kỹ thuật vào bản kê khai. Tốt hơn, các phần tử cấu thành sẵn có để dùng phải được liệt kê cho hệ thống. Cũng có các lý do khác cho điều bắt buộc này: giảm thiểu việc tăng độ phức tạp của hệ thống nhúng, các yêu cầu nghiêm ngặt về thời gian đưa sản phẩm tới khách hàng, khả năng dùng lại của sản phẩm. Điều này dẫn đến thuật ngữ platform-based design (thiết kế trên nền):

Một platform (nền) là một họ kiến trúc thoả mãn một tập hợp các ràng buộc áp đặt để cho phép dùng lại các phần tử phần cứng và phần mềm. Tuy nhiên, một nền phần cứng là chưa đủ. Các thiết kế nhanh, tin cậy, có tính dẫn xuất cao thường yêu cầu sử dụng một nền giao diện lập trình ứng dụng (API) để từ đó phát triển, mở rộng để đạt tới phần mềm ứng dụng. Nhìn chung, một nền (platform) là một lớp khái niệm trừu tượng trong đó bao gồm nhiều sàng lọc khả dĩ để đưa tới một mức thấp hơn. Thiết kế trên cơ sở nền là một cách tiếp cận meet-in-the-middle: Theo dòng thiết kế từ đỉnh xuống, người thiết kế sẽ lập bản đồ các nền từ cao tới thấp, và công bố những ràng buộc thiết kế giữa chúng [Sangiovanni-Vincentelli, 2002].

Việc lập bản đồ là một quá trình lặp đi lặp lại, các công cụ đánh giá hoạt động trong quá trình này được hướng dẫn ở phần tiếp theo. Hình 4-1 thể hiện cách tiếp cận này.

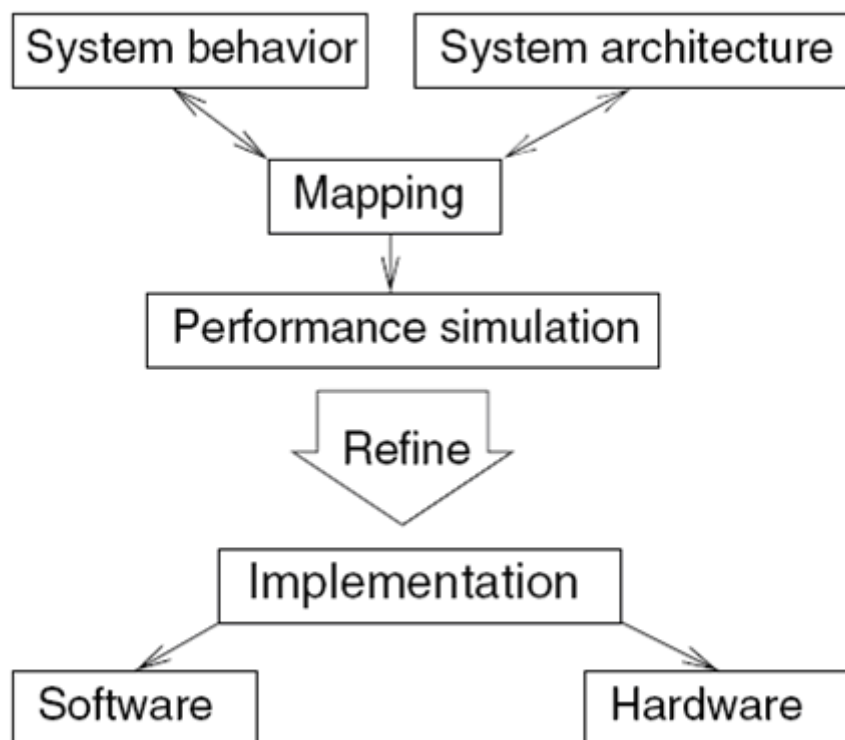
Hoạt động thiết kế phải tính đến sự tồn tại của những platform sẵn có và ghi chúng vào bảng kê khai. Trên thực tế có một lượng lớn các hoạt động thiết kế, nhưng chỉ một vài hoạt động trong số chúng được trình bày ở đây và việc tham chiếu đến các platform sẵn có không phải luôn luôn được thể hiện. Các hoạt động thiết kế bao gồm:

***Quản lý các nhiệm vụ cùng mức:** Hoạt động này có liên quan với việc xác định các nhiệm vụ phải được hiện diện trong hệ thống nhúng cuối cùng. Những nhiệm vụ này có thể khác với những nhiệm vụ đã bao gồm trong các đặc điểm kỹ thuật, từ đó có những lý do tốt để cho việc sáp nhập và chia tách các nhiệm vụ.

***Các biến đổi cấp cao:** Người ta đã nhận thấy rằng có nhiều biến đổi cấp cao tối ưu có thể được ứng dụng trong kỹ thuật. Ví dụ, các vòng lặp có thể được thay đổi để truy xuất đến các phần tử mảng ở nhiều vùng khác nhau. Ngoài ra, các phép toán số học dấu phẩy động có thể thường xuyên được thay thế bởi các phép toán số học dấu phẩy cố định mà không có bất kỳ tổn thất đáng kể trong chất lượng. Những biến đổi cấp cao thường vượt ra ngoài khả năng của trình biên dịch có sẵn và phải được áp dụng trước khi bắt đầu bất kỳ một quá trình biên dịch nào.

***Phân hoạch phần cứng/phần mềm:** Chúng ta thấy rằng trong trường hợp chung, một số chức năng phải được thực hiện bởi phần cứng đặc biệt để đáp ứng yêu cầu tăng tốc độ tính toán của hệ thống [De Man, 2002]. Phân hoạch phần cứng/phần mềm là hoạt động phân chia chức năng cần thực hiện cho phần cứng hoặc phần mềm.

***Biên dịch:** Những phần của đặc điểm kỹ thuật được ánh xạ tới phần mềm phải được biên dịch. Hiệu quả của mã tạo ra được cải thiện nếu trình biên dịch khai thác tốt kiến thức về bộ vi xử lý (và có thể bộ nhớ) nằm bên dưới phần cứng. Bởi vậy, có những chương trình biên dịch "nhận thức phần cứng" đặc biệt cho hệ thống nhúng.



Hình 4. 1: Thiết kế trên nền (platform)

***Lập lịch trình:** Lập lịch trình (sắp đặt thời gian bắt đầu các hoạt động) phải được thực hiện trong một vài bối cảnh. Lịch trình phải được ánh chừng (gần chính xác) trong thời

gian phân vùng phần cứng/phần mềm, trong thời gian quản lý các nhiệm vụ đồng mức và cũng có thể trong thời gian biên dịch. Lịch trình chính xác có thể nhận được cho mã chương trình cuối cùng.

***Khảo sát không gian thiết kế:** Trong hầu hết các trường hợp, sẽ có một vài thiết kế đáp ứng được các thông số kỹ thuật. Khảo sát không gian thiết kế là quá trình phân tích chi tiết tập các thiết kế khả dĩ (có thể thực hiện). Trong số các thiết kế đáp ứng được các thông số kỹ thuật, chỉ duy nhất một thiết kế được chọn.

Những luồng thiết kế riêng biệt có thể sử dụng các hoạt động này theo trình tự khác nhau. Không có một tập tiêu chuẩn của các hoạt động thiết kế.

4.1.2 Đặc tả

Có thể vẫn còn những trường hợp mà đặc tả kỹ thuật của các hệ thống nhúng được thu thập trong một ngôn ngữ tự nhiên, như tiếng Anh. Tuy nhiên, cách tiếp cận này là hoàn toàn không thích hợp vì nó thiếu yêu cầu quan trọng cho các kỹ thuật đặc tả: cần thiết để kiểm tra đặc điểm kỹ thuật đầy đủ, không có mâu thuẫn và nó phải được triển khai có thể xuất phát từ đặc điểm kỹ thuật theo một cách có hệ thống. Vì vậy, đặc tả kỹ thuật nên được thu thập trong các ngôn ngữ máy chính thức có thể đọc được. Ngôn ngữ đặc tả kỹ thuật cho các hệ thống nhúng phải có khả năng đại diện cho các tính năng sau đây:

Sự phân cấp: Con người thường không có khả năng thấu hiểu hệ thống có chứa nhiều đối tượng (các trạng thái, các thành phần) có quan hệ phức tạp với nhau. Việc mô tả tất cả các hệ thống thực tế cần nhiều hơn các đối tượng mà con người có thể hiểu được. Phân cấp là cơ chế duy nhất giúp giải quyết tình trạng khó xử này. Phân cấp có thể được đưa vào mà như vậy con người chỉ cần phải xử lý một số nhỏ các đối tượng tại bất kỳ thời điểm nào.

Có hai loại phân cấp:

Các phân cấp theo hành vi: phân cấp theo hành vi là các phân cấp chứa các đối tượng cần thiết để mô tả hành vi hệ thống. Các trạng thái, các sự kiện và các tín hiệu đầu ra là những ví dụ cho các đối tượng như vậy.

Các phân cấp cấu trúc: các phân cấp cấu trúc mô tả các hệ thống được bao gồm những thành phần vật lý như thế nào.

Ví dụ, các hệ thống nhúng có thể được bao gồm bộ vi xử lý, bộ nhớ, các cơ cấu chấp hành và các cảm biến. Các bộ xử lý, theo trình tự, lại bao gồm các thanh ghi, các bộ dồn kênh và các bộ cộng. Các bộ dồn kênh lại bao gồm trong nó là các cổng logic.

Thời gian-hành vi: một khi các yêu cầu tính toán thời gian rõ ràng (chính xác) là một trong những đặc trưng của hệ thống nhúng, các yêu cầu tính toán thời gian phải được thu thập trong đặc tả kỹ thuật.

Hành vi định hướng trạng thái: Nó đã được đề cập trong chương 1 mà các thiết bị tự động cung cấp một cơ chế tốt cho mô hình hóa các hệ thống phản ứng. Vì vậy, hành vi định hướng trạng thái cung cấp bởi các thiết bị tự động sẽ dễ mô tả. Tuy nhiên, các mô hình thiết bị tự động cổ điển là không đủ, vì chúng không thể mô hình hoá thời gian và vì sự phân cấp là không được hỗ trợ.

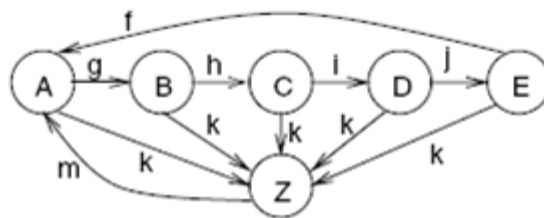
Xử lý-sự kiện: Do tính chất phản ứng tự nhiên của các hệ thống nhúng, các cơ chế cho việc mô tả các sự kiện phải tồn tại. Các sự kiện như vậy có thể là các sự kiện bên ngoài (gây ra bởi môi trường) hoặc các sự kiện nội bộ (gây ra bởi các thành phần của hệ thống).

Không có những trở ngại cho việc tạo ra những thực thi hiệu quả: vì các hệ thống nhúng phải hiệu quả, không có những trở ngại ngăn cản việc tạo ra những thực hiện hiệu quả cần phải thể hiện trong đặc tả.

Hỗ trợ cho việc thiết kế các hệ thống đáng tin cậy: Các kỹ thuật đặc tả cần phải cung cấp sự hỗ trợ cho việc thiết kế những hệ thống đáng tin cậy. Ví dụ, ngôn ngữ đặc tả kỹ thuật nên có ngữ nghĩa rõ ràng, tạo điều kiện thuận lợi cho sự xác minh hình thức và có khả năng mô tả bảo mật và những yêu cầu an toàn.

Hành vi hướng ngoại lệ: Trong nhiều trường hợp thực tế, những ngoại lệ hệ thống xuất hiện. Để thiết kế hệ thống đáng tin cậy, phải thật khả dĩ để mô tả những hoạt động để xử lý những ngoại lệ một cách dễ dàng. Không chấp nhận được rằng những ngoại lệ phải được chỉ thị cho mỗi và mọi trạng thái (giống như trong trường hợp những sơ đồ trạng thái cổ điển). Ví dụ: Trong Hình 4-2, đầu vào **k** có thể tương ứng tới một ngoại lệ.

Việc chỉ rõ ngoại lệ này tại mỗi trạng thái làm cho sơ đồ rất phức tạp. Tình hình sẽ tồi tệ hơn cho sơ đồ trạng thái lớn hơn với nhiều sự chuyển tiếp. Chúng ta sau đó sẽ biểu thị, làm thế nào tất cả các quá trình chuyển tiếp có thể được thay thế bằng một chuyển tiếp duy nhất.



Hình 4. 2: Sơ đồ trạng thái với ngoại lệ **k**

Truy cập đồng thời: những hệ thống thực là những hệ thống phân tán, tồn tại đồng thời. Do đó, cần thiết để có thể xác định đồng thời thuận tiện.

Đồng bộ hóa và truyền thông: các hoạt động đồng thời phải có khả năng liên lạc và nó phải khả dĩ phù hợp với việc sử dụng các tài nguyên. Chẳng hạn, cần thiết biểu thị sự loại trừ lẫn nhau.

Sự hiện diện của các yếu tố lập trình: các ngôn ngữ lập trình thông thường đã được chứng minh là một phương tiện thuận tiện để thể hiện các tính toán cần phải được thực hiện. Do đó, các yếu tố ngôn ngữ lập trình nên có sẵn trong kỹ thuật đặc tả được sử dụng. Những sơ đồ trạng thái cổ điển không đáp ứng yêu cầu này.

Có thể thực hiện được: Những đặc tả không tự động phù hợp với những ý tưởng trong đầu con người. Thực hiện các đặc tả kỹ thuật là một phương tiện kiểm tra đáng tin cậy. Các đặc tả kỹ thuật sử dụng ngôn ngữ lập trình có một lợi thế rõ ràng trong ngữ cảnh này.

Hỗ trợ cho việc thiết kế các hệ thống lớn: Có một xu thế hướng tới các chương trình phần mềm nhúng lớn và phức tạp. Công nghệ phần mềm đã tìm ra những cơ chế để thiết kế những hệ thống lớn như vậy. Chẳng hạn, hướng đối tượng là một cơ chế như vậy. Nó cần phải sẵn sàng trong phương pháp đặc tả kỹ thuật.

Hỗ trợ lĩnh vực chuyên biệt: Tất nhiên sẽ là tốt hơn nếu cùng một kỹ thuật đặc tả có thể được áp dụng cho tất cả các loại khác nhau của hệ thống nhúng, vì điều này sẽ giảm thiểu các nỗ lực để phát triển các kỹ thuật đặc tả kỹ thuật và công cụ hỗ trợ. Tuy nhiên, do phạm vi rộng các lĩnh vực ứng dụng, có rất ít hy vọng rằng một ngôn ngữ có thể được sử dụng để đại diện hiệu quả cho các đặc tả kỹ thuật trong mọi lĩnh vực. Chẳng hạn, những lĩnh vực ứng dụng tập trung và phân tán, thiên về điều khiển, thiên về xử lý dữ liệu đều có thể hưởng lợi từ các tính năng ngôn ngữ chuyên dụng đối với các lĩnh vực đó.

Có thể đọc được: Tất nhiên, những đặc tả kỹ thuật phải đọc được bởi con người. Tốt nhất là, chúng cũng có thể đọc được bằng máy trong trình tự để xử lý chúng trong một máy tính.

Tính khả chuyển và linh hoạt: Các đặc tả kỹ thuật phải được độc lập với các nền tảng phần cứng cụ thể để chúng có thể dễ dàng sử dụng cho một loạt các nền tảng mục tiêu. Chúng cần phải linh hoạt sao cho những thay đổi nhỏ của tính năng hệ thống cũng chỉ yêu cầu những thay đổi nhỏ trong đặc tả.

Điểm kết thúc: Nó phải có tính khả thi để xác định quá trình sẽ hoàn thành từ đặc tả kỹ thuật.

Hỗ trợ các thiết bị I/O không tiêu chuẩn: Nhiều hệ thống nhúng sử dụng các thiết bị I/O khác với các thiết bị thông thường được tìm thấy trên máy PC. Cần phải có khả năng mô tả những đầu vào và những đầu ra cho những thiết bị đó thuận tiện.

Những thuộc tính không hoạt động: các hệ thống thực tế phải thể hiện một số thuộc tính không hoạt động, chẳng hạn như sai hỏng cho phép, kích thước, tính co giãn, thời gian sống dự kiến, công suất tiêu thụ, trọng lượng, thân thiện với người sử dụng, tương thích điện từ (EMC) v.v. Không có hy vọng rằng tất cả những thuộc tính này có thể được định nghĩa một cách chính thức.

Mô hình tính toán thích hợp: Để mô tả tính toán, các mô hình tính toán là bắt buộc. Các mô hình như vậy sẽ được mô tả trong phần tiếp theo.

Từ danh sách các yêu cầu, đã thể hiện rõ ràng rằng sẽ không có bất kỳ ngôn ngữ chính thức nào có khả năng đáp ứng tất cả các yêu cầu này. Bởi vậy, trong thực tế, chúng ta phải sống với những thỏa hiệp. Việc lựa chọn ngôn ngữ được sử dụng cho một thiết kế thực tế sẽ phụ thuộc vào các miền ứng dụng và môi trường mà trong đó thiết kế sẽ được thực hiện. Trong phần sau, chúng ta sẽ trình bày một khảo sát các ngôn ngữ có thể được sử dụng cho các thiết kế thực tế.

a. Mô hình tính toán

Những ứng dụng công nghệ thông tin đã tồn tại cho đến nay rất nhiều dựa vào mô hình máy tính von Neumann của tính toán tuần tự. Mô hình này là không thích hợp cho các hệ thống nhúng, đặc biệt là những hệ thống có yêu cầu thời gian thực, vì không có khái niệm về thời gian trong máy tính von Neumann. Các mô hình tính toán khác đầy đủ hơn.

b. Biểu đồ trạng thái (StateCharts)

Ngôn ngữ thực tế đầu tiên sẽ được trình bày là StateCharts. StateCharts đã được giới thiệu vào năm 1987 bởi David Harel [Harel, 1987] và sau đó mô tả chính xác hơn [Drusinsky và Harel, 1989]. StateCharts mô tả việc truyền thông trong những máy trạng thái hữu hạn. Nó dựa trên khái niệm bộ nhớ chia sẻ truyền thông.

c. Những đặc trưng ngôn ngữ khái quát

Phần trước cung cấp cho chúng ta một số ví dụ đầu tiên về các thuộc tính của các ngôn ngữ đặc tả kỹ thuật. Những ví dụ này giúp chúng ta hiểu được một cuộc thảo luận tổng quát hơn các thuộc tính ngôn ngữ trong phần này trước khi chúng ta tiếp tục thảo luận về ngôn ngữ trong các phần tiếp theo. Có một số đặc điểm mà trên đó chúng ta có thể so sánh những thuộc tính của các ngôn ngữ. Thuộc tính đầu tiên là liên quan đến việc phân biệt giữa các mô hình xác định và không xác định đã được đề cập đến trong cuộc thảo luận của chúng ta về StateCharts.

4.1.3 Phân hoạch phần cứng - phần mềm

Trong quá trình thiết kế, chúng ta phải giải quyết vấn đề thực hiện các đặc tả kỹ thuật hoặc trong phần cứng hoặc ở dạng của các chương trình chạy trên bộ vi xử lý. Phần này mô tả một số kỹ thuật để lập bản đồ này. Áp dụng các kỹ thuật này, chúng ta sẽ có

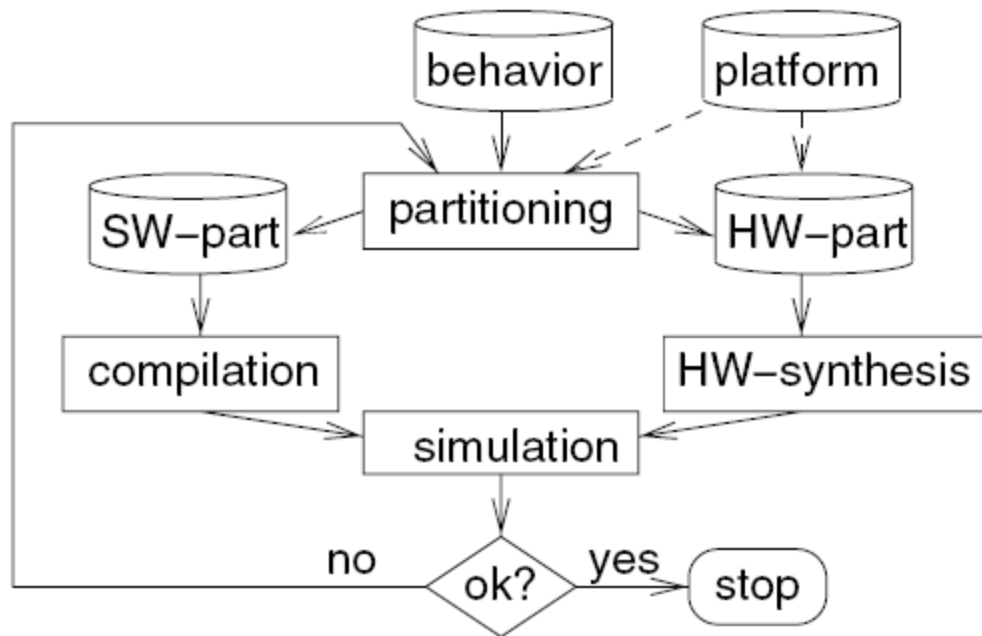
thể quyết định những phần phải được thực hiện trong phần cứng và những phần sẽ được thực bởi phần mềm.

Bởi phân hoạch phần cứng/phần mềm, có nghĩa là chúng ta ánh xạ các nút biểu đồ nhiệm vụ cho một trong hai phần cứng hoặc phần mềm. Một thủ tục tiêu chuẩn cho việc nhúng phân vùng phần cứng/phần mềm vào tiến trình thiết kế tổng thể được hiển thị trong Hình 4-3. Chúng ta bắt đầu từ một đại diện chung của đặc tả kỹ thuật, ví dụ ở dạng đồ thị nhiệm vụ và thông tin về nền (platform).

Đối với mỗi nút của đồ thị nhiệm vụ, chúng ta cần thông tin liên quan đến các nỗ lực cần thiết và những lợi ích nhận được từ việc lựa chọn một sự thực hiện nào đó các nút này. Ví dụ, thời gian thực hiện phải được dự đoán. Rất khó để dự đoán thời gian cần thiết cho truyền thông. Tuy nhiên, hai nhiệm vụ đòi hỏi một băng thông truyền thông rất cao tốt nhất nên được ánh xạ tới các thành phần giống nhau. Phương pháp lặp được sử dụng trong nhiều trường hợp. Một giải pháp ban đầu cho vấn đề phân vùng được tạo ra, phân tích và sau đó được cải thiện.

Một số phương pháp tiếp cận cho phân hoạch được giới hạn để ánh xạ các nút đồ thị nhiệm vụ tới hoặc phần cứng chức năng chuyên biệt hoặc phần mềm chạy trên một bộ xử lý đơn. Việc phân hoạch như vậy có thể được thực hiện với các thuật toán bipartitioning (phân đôi) cho các đồ thị nhiệm vụ [Kuchcinski, 2002].

Những thuật toán phân hoạch phức tạp hơn có khả năng lập bản đồ các nút đồ thị cho hệ thống đa xử lý và phần cứng. Trong phần sau, chúng ta sẽ mô tả cách làm việc của thuật toán này bằng cách sử dụng một kỹ thuật tối ưu hóa tiêu chuẩn từ các hoạt động nghiên cứu, lập trình số nguyên. Trình bày của chúng ta được dựa trên một phiên bản đơn giản hóa của các đề xuất tối ưu hóa cho công cụ thiết kế COOL (codesign tool) [Niemann, 1998].



Hình 4. 3: Tổng quan về phân hoạch phần cứng/phần mềm

a. COOL

Đối với COOL, đầu vào bao gồm ba phần:

* **Công nghệ mục tiêu:** Phần này của đầu vào cho COOL bao gồm những thông tin về các thành phần nền phần cứng sẵn có. COOL hỗ trợ hệ thống đa xử, nhưng yêu cầu tất cả các bộ xử lý là cùng loại, vì nó không bao gồm sự lựa chọn bộ vi xử lý tự động hay bằng tay. Tên của bộ xử lý được sử dụng (cũng như các thông tin về trình biên dịch tương ứng) phải được bao gồm trong phần này của đầu vào cho COOL. Đối với phần cứng ứng dụng chuyên biệt, các thông tin phải có đủ để bắt đầu tổng hợp phần cứng tự động với tất cả các thông số cần thiết. Đặc biệt, thông tin về các thư viện công nghệ phải được cung cấp.

* **Những ràng buộc thiết kế:** Phần thứ hai của đầu vào bao gồm những ràng buộc thiết kế như thông lượng yêu cầu, độ trễ, kích thước bộ nhớ tối đa, hoặc diện tích tối đa cho phần cứng ứng dụng chuyên biệt.

* **Hành vi:** Phần thứ ba của đầu vào mô tả hành vi tổng thể được yêu cầu. Đồ thị nhiệm vụ phân cấp được sử dụng cho việc này. Chúng ta có thể nghĩ ra, ví dụ bằng cách sử dụng đồ thị nhiệm vụ phân cấp cho việc này.

COOL sử dụng hai loại giới hạn (edge): giới hạn truyền thông và giới hạn về phân định thời gian. Giới hạn truyền thông có thể chứa thông tin về số lượng thông tin được trao đổi. Giới hạn về phân định thời gian cung cấp những sự ràng buộc về phân định thời gian. COOL đòi hỏi cách xử lý của mỗi nút trong các nút lá (leaf node) của phân cấp đồ thị đã biết. COOL cho là cách xử lý này đã được quy định trong VHDL.

Đối với phân vùng, COOL sử dụng các bước sau:

1. Chuyển đổi hành vi thành một mô hình đồ thị nội bộ.

2. Chuyển đổi hành vi của mỗi nút từ VHDL vào C.

3. Biên dịch tất cả các chương trình C cho bộ xử lý mục tiêu đã chọn, tính toán kích thước của chương trình kết quả, dự toán thời gian thực hiện kết quả. Nếu mô phỏng được sử dụng sau đó, thì dữ liệu đầu vào mô phỏng phải được sẵn sàng.

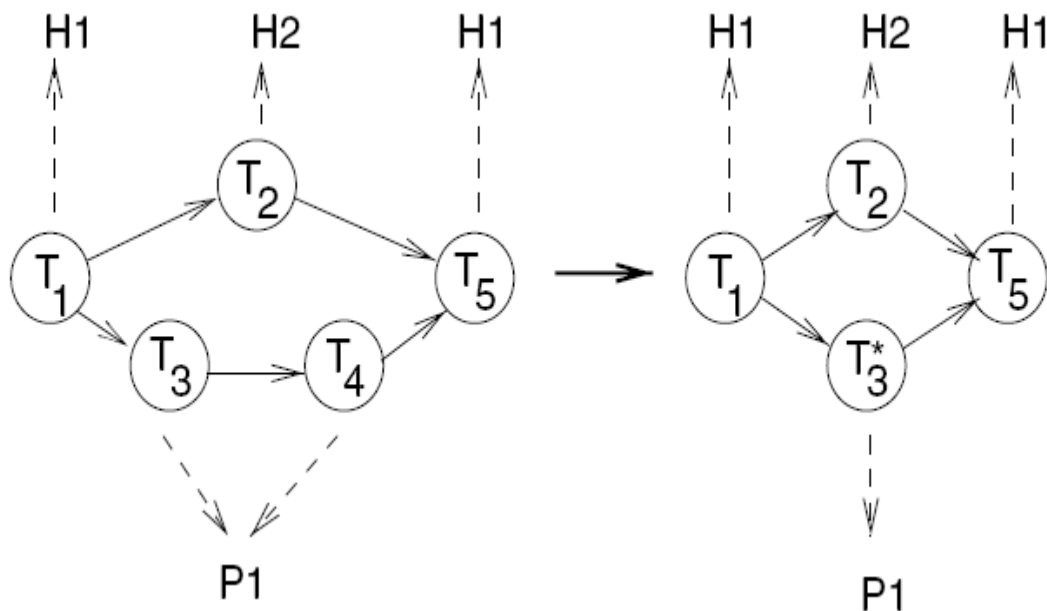
4. Tổng hợp các thành phần phần cứng: Đối với mỗi nút lá, phần cứng ứng dụng chuyên biệt phải được tổng hợp. Từ đó một số lớn thành phần phần cứng có thể phải được tổng hợp, tổng hợp phần cứng không nên quá chậm. Có thể nhận thấy rằng các công cụ tổng hợp thương mại tập trung tổng hợp ở mức cổng có thể là quá chậm để có ích cho COOL. Tuy nhiên, các công cụ tổng hợp cao cấp làm việc tại mức-chuyển đổi-thanh ghi (sử dụng các bộ cộng, các thanh ghi, và bộ dồn kênh như là các thành phần, thay vì các cổng) cung cấp tốc độ tổng hợp đầy đủ. Ngoài ra, các công cụ như vậy có thể cung cấp đầy đủ các giá trị chính xác cho những thời gian trì hoãn và diện tích silic yêu cầu. Trong thực hiện thực tế, công cụ tổng hợp cấp cao OSCAR [Landwehr and Marwedel, 1997] được sử dụng.

5. Trải phẳng sự phân cấp: Bước tiếp theo sẽ khai triển một đồ thị nhiệm vụ phẳng từ lưu đồ có phân cấp. Khi không có việc sáp nhập hoặc chia tách các nút được thực hiện, độ chi tiết được sử dụng bởi nhà thiết kế được duy trì. Chi phí và thông tin thực hiện thu được từ quá trình biên tập và tổng hợp phần cứng được bổ sung vào các nút. Đây thực sự là một trong những ý tưởng chính của COOL: các thông tin cần thiết cho phân vùng phần cứng/phần mềm được tính toán trước và nó được tính với độ chính xác tốt. Thông tin này thiết lập cơ sở để tạo ra các thiết kế chi phí tối thiểu thoả mãn các ràng buộc thiết kế.

6. Tạo ra và giải quyết một mô hình toán của vấn đề tối ưu hóa: COOL sử dụng lập trình số nguyên (IP) để giải quyết vấn đề tối ưu hóa. Một thiết bị giải IP thương mại được dùng để tìm những giá trị cho những biến quyết định đến việc tối thiểu hoá chi phí. Giải pháp là tối ưu đối với các hàm chi phí bắt nguồn từ những thông tin có sẵn. Tuy nhiên, chi phí này chỉ bao gồm một sự xấp xỉ thô của thời gian truyền thông. Thời gian truyền thông giữa hai nút bất kỳ của đồ thị nhiệm vụ phụ thuộc vào việc ánh xạ các nút này tương ứng tới các bộ xử lý và phần cứng. Nếu cả hai nút được ánh xạ tới cùng một bộ xử lý, truyền thông sẽ là cục bộ và do đó khá nhanh. Nếu các nút được ánh xạ tới các thành phần phần cứng khác nhau, truyền thông sẽ là không-cục bộ và có thể bị chậm hơn. Mô hình hóa các chi phí truyền thông cho tất cả các ánh xạ có thể có của các nút đồ thị nhiệm vụ sẽ làm cho mô hình rất phức tạp và do đó được thay thế bởi các cải tiến lập của giải pháp ban đầu. Chi tiết hơn về bước này sẽ được trình bày dưới đây.

7. Các cải tiến lặp: Để làm việc với các ước lượng tốt về thời gian truyền thông, các nút liên kế được ánh xạ tới cùng một thành phần phần cứng bây giờ được hợp nhất. Sự hợp nhất này được thể hiện trên Hình 4-4.

Chúng ta giả định rằng các nhiệm vụ T1, T2 và T5 được ánh xạ tới các thành phần phần cứng H1 và H2, trong khi đó T3 và T4 được ánh xạ tới bộ xử lý P1. Tương ứng, truyền thông giữa T3 và T4 là truyền thông cục bộ. Vì vậy, chúng ta hợp nhất T3 và T4, và thừa nhận rằng việc giao tiếp giữa hai nhiệm vụ không đòi hỏi một kênh truyền thông. Thời gian truyền thông bây giờ có thể được ước lượng với độ chính xác được cải thiện. Đồ thị kết quả sau đó được sử dụng như đầu vào mới cho việc tối ưu hóa toán học. Các bước trước đó và hiện tại được lặp lại cho đến khi không còn nút đồ thị nào là có thể được sáp nhập.



Hình 4. 4: Hợp nhất các nút nhiệm vụ được ánh xạ đến cùng một thành phần phần cứng

8. Tổng hợp giao diện: Sau khi phân vùng, theo trình tự logic đòi hỏi phải tạo ra giao diện giữa các bộ xử lý, phần cứng ứng dụng chuyên biệt và bộ nhớ.

Tiếp theo, chúng ta sẽ mô tả bước 6 chi tiết hơn. Các mô hình IP cung cấp một phương pháp tiếp cận chung để mô hình hóa những vấn đề tối ưu hóa. Các mô hình IP bao gồm hai phần: một hàm chi phí và một tập các ràng buộc. Cả hai phần cùng bao hàm các tham chiếu đến một tập $X = \{x_i\}$ của những biến giá trị nguyên. Những hàm chi phí phải là những hàm tuyến tính của những biến đó. Vì vậy, chúng phải có dạng chung:

$$C = \sum_{x_i \in X} a_i x_i, \text{ with } a_i \in \mathbb{Z}, x_i \in \mathbb{Z} \quad (4.1)$$

Tập J của các ràng buộc cũng phải chứa các hàm tuyến tính của các biến giá trị nguyên. Chúng phải có dạng:

$$\forall j \in J: \sum_{x_i \in X} b_{i,j} x_i \geq c_j, \text{ with } b_{i,j}, c_j \in \mathbb{Z} \quad (4.2)$$

Lưu ý rằng $\geq$ có thể được thay thế bởi $\leq$ trong phương trình (4.2) nếu các hằng số $b_{i,j}$ được sửa đổi phù hợp.

Def.: Vấn đề lập trình số nguyên (IP-) là vấn đề của việc tối thiểu hoá hàm chi phí (4.1) lệ thuộc vào những ràng buộc nhận được trong biểu thức (4.2). Nếu tất cả các biến bị cưỡng bức là 0 hoặc 1, thì mô hình tương ứng được gọi là một mô hình lập trình số nguyên 0/1. Trong trường hợp này, các biến cũng được biểu thị như những biến quyết định (nhị phân).

Ví dụ, giả định rằng x_1 , x_2 và x_3 không âm và phải là số nguyên, tập các phương trình sau đại diện cho một mô hình 0/1-IP:

$$C = 5x_1 + 6x_2 + 4x_3 \quad (4.3)$$

$$x_1 + x_2 + x_3 \geq 2 \quad (4.4)$$

$$x_1 \leq 1 \quad (4.5)$$

$$x_2 \leq 1 \quad (4.6)$$

$$x_3 \leq 1 \quad (4.7)$$

Vì những ràng buộc, tất cả các biến đều chỉ có thể là 0 hoặc 1 (số nguyên không âm). Có bốn giải pháp khả dĩ. Các giải pháp này được liệt kê trong Bảng 4-1. Giải pháp với một chi phí bằng 9 là tối ưu.

Các ứng dụng đòi hỏi phải tối đa hóa một vài lợi ích thì hàm C' có thể được thay vào dạng thức ở trên bằng cách đặt $C = -C'$.

x_1	x_2	x_3	C
0	1	1	10
1	0	1	9
1	1	0	11
1	1	1	15

Bảng 4. 1: Các giải pháp khả dĩ của vấn đề IP đang được trình bày

Các mô hình IP có thể được giải quyết tối ưu bằng cách sử dụng các kỹ thuật lập trình toán học. Thật không may, lập trình số nguyên là NP-đầy đủ và thời gian thực hiện có thể trở nên rất lớn. Tuy nhiên, nó rất hữu ích cho việc giải quyết các vấn đề tối ưu hóa miễn là kích thước mô hình không phải là vô cùng lớn. Thời gian thực hiện phụ thuộc vào số lượng các biến, cấu trúc và số lượng các ràng buộc. Các bộ giải IP tốt (như lp_solve [Berkelaar and et al., 2005] hay CPLEX) có thể giải quyết các vấn đề được cấu trúc tốt có chứa một vài nghìn biến trong thời gian tính toán chấp nhận được (ví dụ: vài phút). Để biết thêm thông tin về lập trình số nguyên và lập trình tuyến tính liên quan, hãy tham khảo các cuốn sách cùng chủ đề (ví dụ: to Wolsey [Wolsey, 1998]). Mô hình hoá những vấn đề tối ưu hóa như vấn đề lập trình số nguyên làm cho có cảm giác bất chấp sự phức tạp của vấn đề: nhiều vấn đề có thể được giải quyết trong thời gian thực hiện chấp nhận được và nếu chúng không thể, các mô hình IP cũng cung cấp một điểm khởi đầu tốt cho chẩn đoán.

Tiếp theo, chúng ta sẽ mô tả việc phân vùng có thể được mô hình bằng cách sử dụng một mô hình 0/1-IP như thế nào. Các tập chỉ số sau sẽ được sử dụng trong việc mô tả về mô hình IP:

- * Tập chỉ số I biểu thị những nút đồ thị nhiệm vụ. Mỗi $i \in I$ tương ứng với một nút biểu đồ nhiệm vụ.
- * Tập chỉ số L biểu thị những kiểu nút đồ thị nhiệm vụ. Mỗi $l \in L$ tương ứng với một kiểu nút đồ thị nhiệm vụ. Ví dụ, có thể có các nút mô tả căn bậc hai, các tính toán biến đổi Cosine rời rạc (DCT: Discrete Cosine Transform) hoặc biến đổi Fourier nhanh rời rạc (DFT: Discrete Fast Fourier Transform). Mỗi loại trong chúng được tính là một kiểu.
- * Tập chỉ số KH biểu thị các kiểu thành phần phần cứng. Mỗi $k \in KH$ tương ứng với một kiểu thành phần phần cứng. Ví dụ, có thể có các thành phần phần cứng đặc biệt cho DCT hoặc DFT. Có một giá trị chỉ số cho các thành phần phần cứng DCT và một cho các thành phần phần cứng FFT.
- * Đối với mỗi thành phần phần cứng, có thể có nhiều bản sao, hay những "trường hợp". Mỗi trường hợp được xác định bởi một chỉ số $j \in J$.
- * Tập chỉ số KP biểu thị những bộ xử lý. Mỗi $k \in KP$ xác định một trong những bộ xử lý (tất cả đều là cùng loại).

Các biến quyết định sau đây được yêu cầu bởi mô hình:

- * $X_{i,k}$: biến này sẽ là 1, nếu nút v_i được ánh xạ tới kiểu thành phần phần cứng $k \in KH$ và 0 nếu không.
- * $Y_{i,k}$: biến này sẽ là 1, nếu nút v_i được ánh xạ tới bộ xử lý $k \in KP$ và 0 nếu không.
- * $NY_{l,k}$: biến này sẽ là 1, nếu ít nhất một nút loại l được ánh xạ tới bộ xử lý $k \in KP$ và 0 nếu không.
- * T là một ánh xạ $I \rightarrow L$ từ các nút đồ thị nhiệm vụ tới các kiểu tương ứng của chúng.

Trong trường hợp cụ thể của chúng ta, hàm chi phí tích lũy tổng chi phí của tất cả các đơn vị phần cứng:

C = các chi phí cho bộ xử lý + các chi phí cho bộ nhớ + các chi phí của phần cứng ứng dụng chuyên biệt

Chúng ta rõ ràng sẽ giảm thiểu tổng chi phí nếu không có các bộ xử lý, bộ nhớ và phần cứng ứng dụng chuyên biệt đã được bao gồm trong "thiết kế". Do những ràng buộc, đây không phải là một giải pháp pháp lý. Bây giờ chúng ta có thể trình bày một mô tả ngắn gọn của một số các ràng buộc của mô hình IP:

* **Những ràng buộc ấn định hoạt động:** Những ràng buộc này bảo đảm rằng mỗi hoạt động được thực hiện hoặc trong phần cứng hoặc trong phần mềm. Những ràng buộc tương ứng có thể được công thức hóa như sau:

$$\forall i \in I : \sum_{k \in KH} X_{i,k} + \sum_{k \in KP} Y_{i,k} = 1$$

Trong văn bản gốc, điều này có nghĩa như sau: với mọi nút đồ thị nhiệm vụ i , ràng buộc sau đây phải được duy trì: i được thực hiện hoặc trong phần cứng (thiết lập một trong những biến $X_{i,k}$ tới 1, cho giá trị k nào đó) hoặc nó được thực hiện trong phần mềm (thiết lập một trong những biến $Y_{i,k}$ tới 1, cho giá trị k nào đó).

Tất cả các biến được giả định là các số nguyên không âm:

$$X_{i,k} \in \mathbb{N}_0, \quad (4.8)$$

$$Y_{i,k} \in \mathbb{N}_0 \quad (4.9)$$

Những sự ràng buộc bổ sung bảo đảm rằng những biến quyết định $X_{i,k}$ và $Y_{i,k}$ có 1 như một giới hạn trên và, do đó, là các biến thực tế có giá trị 0/1:

$$\forall i \in I : \forall k \in KH : X_{i,k} \leq 1$$

$$\forall i \in I : \forall k \in KP : Y_{i,k} \leq 1$$

Nếu chức năng của một nút nhất định của kiểu l được ánh xạ tới bộ xử lý k nào đó, thì bộ nhớ chương trình của bộ xử lý này phải bao gồm một bản sao của phần mềm cho chức năng này:

$$\forall l \in L, \forall i : T(v_i) = c_l, \forall k \in KP : NY_{l,k} \geq Y_{i,k}$$

Trong văn bản gốc, điều này có nghĩa là: với mọi kiểu l thuộc các nút đồ thị nhiệm vụ và với mọi nút i thuộc kiểu này, ràng buộc sau đây phải được duy trì: nếu i được ánh xạ tới bộ xử lý k nào đó (biểu thị bởi $Y_{i,k}$ là 1), thì phần mềm tương ứng với

chức năng l phải được cung cấp bởi bộ xử lý k , và phần mềm tương ứng phải tồn tại trên bộ xử lý đó (biểu thị bởi $NY_{l,k}$ là 1).

Những sự ràng buộc bổ sung bảo đảm rằng những biến quyết định $NY_{l,k}$ cũng là những biến giá trị 0/1:

$$\forall l \in L: \forall k \in KP : NY_{l,k} \leq 1$$

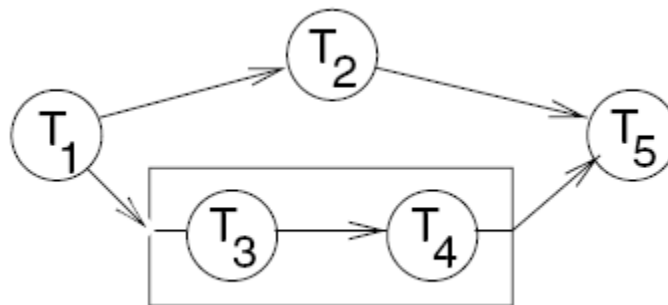
* **Những ràng buộc tài nguyên:** Tập tiếp theo của các ràng buộc đảm bảo rằng "không quá nhiều" các nút được ánh xạ tới cùng một thành phần phần cứng tại cùng một thời điểm. Chúng ta giả định rằng, cứ mỗi chu kỳ đồng hồ, nhiều nhất là một hoạt động có thể được thực hiện trên mỗi thành phần phần cứng. Thật không may, điều này có nghĩa rằng thuật toán phân vùng cũng phải tạo ra một lịch trình để thực hiện các nút đồ thị nhiệm vụ. Việc lập lịch tự nó đã là một vấn đề NP-đầy đủ cho hầu hết các trường hợp vấn đề có liên quan.

* **Những ràng buộc mức ưu tiên:** Các ràng buộc này đảm bảo rằng lịch trình để thực hiện các hoạt động phù hợp với các ràng buộc mức ưu tiên trong đồ thị nhiệm vụ.

* **Những ràng buộc thiết kế:** Những ràng buộc này đặt một giới hạn về chi phí của các thành phần phần cứng nào đó, chẳng hạn như bộ nhớ, các bộ xử lý hoặc khu vực dành cho phần cứng ứng dụng chuyên biệt.

* **Những ràng buộc về phân định thời gian:** Những ràng buộc về phân định thời gian, nếu hiện diện trong đầu vào cho COOL, thì được chuyển đổi thành các ràng buộc IP.

* Một số sự ràng buộc bổ sung, nhưng ít quan trọng hơn không được bao gồm trong danh sách này.



Hình 4. 5: Đồ thị nhiệm vụ

Ví dụ: Trong phần sau, chúng ta sẽ trình bày về việc các ràng buộc này có thể được tạo ra như thế nào cho đồ thị nhiệm vụ trong Hình 4-5.

Giả sử rằng chúng ta có một thư viện thành phần phần cứng có chứa ba kiểu thành phần là H1, H2 và H3 với chi phí tương ứng là 20, 25 và 30 đơn vị chi phí. Hơn nữa, giả sử rằng chúng ta cũng có thể sử dụng một bộ xử lý P với phí là 5. Ngoài ra, chúng ta giả

định rằng Bảng 4-2 mô tả thời gian thực hiện của các nhiệm vụ của chúng ta trên các thành phần này.

T	H1	H2	H3	P
1	20			100
2		20		100
3			12	10
4			12	10
5	20			100

Bảng 4. 2: Thời gian thực hiện của các nhiệm vụ từ T1 đến T5 trên các thành phần

Các nhiệm vụ T1 đến T5 chỉ có thể được thực hiện trên bộ xử lý hoặc trên một đơn vị phần cứng ứng dụng chuyên biệt. Rõ ràng, bộ xử lý được coi là rẻ nhưng chậm trong việc thực hiện các nhiệm vụ T1, T2, và T5.

Những sự ràng buộc ấn định hoạt động sau đây phải được tạo ra, giả thiết rằng tối đa một bộ xử lý (P1) sẽ được sử dụng:

$$X_{1,1} + Y_{1,1} = 1 \text{ (Nhiệm vụ 1 được ánh xạ hoặc tới H1 hoặc tới P1)}$$

$$X_{2,2} + Y_{2,1} = 1 \text{ (Nhiệm vụ 2 được ánh xạ hoặc tới H2 hoặc tới P1)}$$

$$X_{3,3} + Y_{3,1} = 1 \text{ (Nhiệm vụ 3 được ánh xạ hoặc tới H3 hoặc tới P1)}$$

$$X_{4,3} + Y_{4,1} = 1 \text{ (Nhiệm vụ 4 được ánh xạ hoặc tới H3 hoặc tới P1)}$$

$$X_{5,1} + Y_{5,1} = 1 \text{ (Nhiệm vụ 5 được ánh xạ hoặc tới H1 hoặc tới P1)}$$

Hơn nữa, giả định rằng các kiểu của các nhiệm vụ T1 đến T5 là $l = 1, 2, 3, 3$ và 1, tương ứng. Tiếp theo, những ràng buộc tài nguyên bổ sung sau đây được yêu cầu:

$$NY_{1,1} \geq Y_{1,1} \quad (4.10)$$

$$NY_{2,1} \geq Y_{2,1}$$

$$NY_{3,1} \geq Y_{3,1}$$

$$NY_{3,1} \geq Y_{4,1}$$

$$NY_{1,1} \geq Y_{5,1} \quad (4.11)$$

Phương trình (4.10) có nghĩa là: nếu nhiệm vụ 1 được ánh xạ tới bộ xử lý, thì chức năng $l = 1$ phải được thực hiện trên bộ xử lý đó. Chức năng tương tự cũng phải được thực hiện trên bộ xử lý này nếu nhiệm vụ 5 được ánh xạ tới bộ xử lý (Phương trình (4.11)).

Chúng ta không bao gồm những ràng buộc về phân định thời gian. Tuy nhiên, rõ ràng là bộ xử lý chậm trong thực hiện một số nhiệm vụ và phần cứng ứng dụng chuyên biệt là cần thiết cho những ràng buộc thời gian nhỏ hơn 100 đơn vị thời gian.

Hàm chi phí là:

$$C = 20 * \#(H1) + 25 * \#(H2) + 30 * \#(H3) + 5 * \#(P)$$

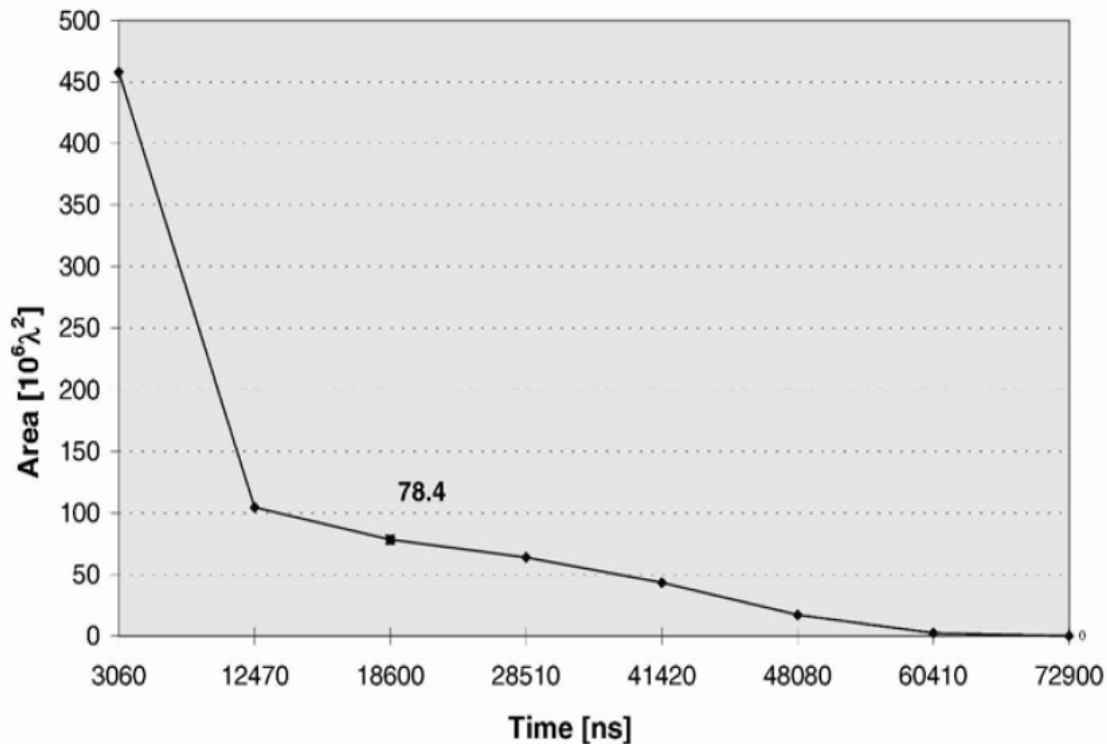
Trong đó $\#()$ biểu thị số lượng các trường hợp sử dụng của các thành phần phần cứng. Số này có thể được tính toán từ những biến được giới thiệu cho đến lúc này nếu lịch trình cũng được đưa vào bản kê khai. Với một ràng buộc thời gian của 100 đơn vị thời gian, thiết kế chi phí tối thiểu bao gồm các thành phần H1, H2 và P. Điều này có nghĩa rằng nhiệm vụ T3 và T4 được thực hiện trong phần mềm và tất cả những nhiệm vụ khác được thực hiện trong phần cứng.

Nói chung, do sự phức tạp của việc phân hoạch kết hợp và các vấn đề lập lịch, chỉ những trường hợp vấn đề nhỏ của vấn đề kết hợp có thể được giải quyết trong thời gian chạy có thể chấp nhận được. Vì vậy, vấn đề là sự suy nghiệm được tách thành vấn đề lập lịch và phân hoạch: một phân hoạch ban đầu được dựa trên thời gian thực hiện dự tính và lập lịch cuối cùng được thực hiện sau khi phân hoạch. Nếu nó chỉ ra rằng lịch trình đã quá lạc quan, thì toàn bộ quá trình phải được lặp lại với những sự ràng buộc về phân định thời gian chặt hơn. Thí nghiệm đối với các ví dụ nhỏ đã cho thấy rằng chi phí cho các giải pháp suy nghiệm chỉ là 1 hoặc 2% lớn hơn chi phí của các kết quả tối ưu.

Phân hoạch tự động có thể được sử dụng để phân tích không gian thiết kế. Trong phần sau, chúng ta sẽ trình bày các kết quả cho một phòng thí nghiệm âm thanh, bao gồm các khối mixer, fader, echo, equalizer và balance. Ví dụ này sử dụng công nghệ nhắm mục tiêu trước đó để chứng minh tác dụng của phân hoạch. Phần cứng mục tiêu gồm có một bộ xử lý SPARC (chậm), bộ nhớ ngoài, và phần cứng ứng dụng chuyên biệt sẽ được thiết kế từ một thư viện 1μ ASIC (Lỗi thời). Tổng thời gian trễ cho phép được thiết lập là 22675 ns, tương ứng với tốc độ lấy mẫu là 44,1 kHz, như được sử dụng trong đĩa CD. Hình 4-6 cho thấy những điểm thiết kế khác nhau mà có thể được tạo ra bằng cách thay đổi ràng buộc về thời gian trễ.

Đơn vị λ tham chiếu tới một đơn vị chiều dài phụ thuộc công nghệ. Nó về bản chất là một nửa của khoảng cách gần nhất giữa các tâm của hai dây kim loại trên chip (còn gọi là *half-pitch* [SEMATECH, 2003]). Điểm thiết kế ở bên trái tương ứng với một giải pháp thực hiện hoàn toàn trong phần cứng, điểm thiết kế ở bên phải là một giải pháp phần mềm. Những điểm thiết kế khác sử dụng một sự pha trộn của phần cứng và phần mềm. Điểm tương ứng với diện tích $78,4 \lambda^2$ là điểm rẻ nhất thoả mãn vạch cấm.

Rõ ràng, ngày nay công nghệ đã tiến bộ để cho phép một thiết kế phòng thí nghiệm âm thanh trên nền phần mềm 100%. Tuy nhiên, ví dụ này chứng tỏ phương pháp thiết kế tiềm ẩn trong đó cũng có thể được sử dụng cho nhiều ứng dụng đòi hỏi khắt khe hơn, đặc biệt là trong lĩnh vực đa phương tiện tốc độ cao, như MPEG-4.



Hình 4. 6: Không gian thiết kế cho phòng thí nghiệm âm thanh

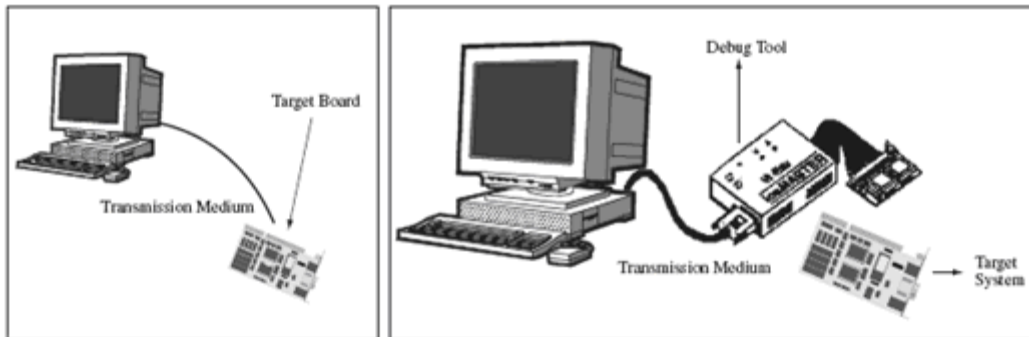
4.1.4 Thiết kế hệ thống

Việc có tài liệu kiến trúc rõ ràng sẽ giúp các kỹ sư và lập trình viên của đội phát triển thực hiện hệ thống nhúng phù hợp với các yêu cầu. Trong suốt tài liệu này, các đề xuất thực tế đã được làm để thực hiện các thành phần khác nhau của một thiết kế đáp ứng các yêu cầu này. Ngoài sự hiểu biết các thành phần và khuyến nghị này, điều quan trọng là hiểu những gì mà các công cụ phát triển sẵn sàng trợ giúp trong việc thực hiện một hệ thống nhúng. Việc phát triển và tích hợp các thành phần phần cứng và phần mềm khác nhau của một hệ thống nhúng được thực hiện có thể thông qua các công cụ phát triển cung cấp mọi thứ từ việc nạp phần mềm vào phần cứng đến việc cung cấp điều khiển hoàn toàn qua những thành phần hệ thống khác nhau.

Các hệ thống nhúng thường không được phát triển trên một hệ thống đơn lẻ (ví dụ, bo mạch phần cứng của hệ thống nhúng) nhưng thường cần ít nhất một hệ thống máy tính khác kết nối với nền tảng nhúng để quản lý sự phát triển của nền tảng đó. Ngắn gọn hơn, một môi trường phát triển thường hình thành một **đích** (hệ thống nhúng đang được thiết kế) và một máy chủ (một PC, Sparc Station, hoặc một số hệ thống máy tính khác nơi mà mã thực sự được phát triển). Đích và máy chủ được kết nối bởi phương tiện truyền dẫn nào đó như nối tiếp, Ethernet, hoặc phương pháp khác. Nhiều công cụ khác,

chẳng hạn như các công cụ tiện ích để ghi EPROM hoặc các công cụ gỡ lỗi, có thể được sử dụng trong môi trường phát triển cùng với máy chủ và đích. (Xem Hình 4-7)

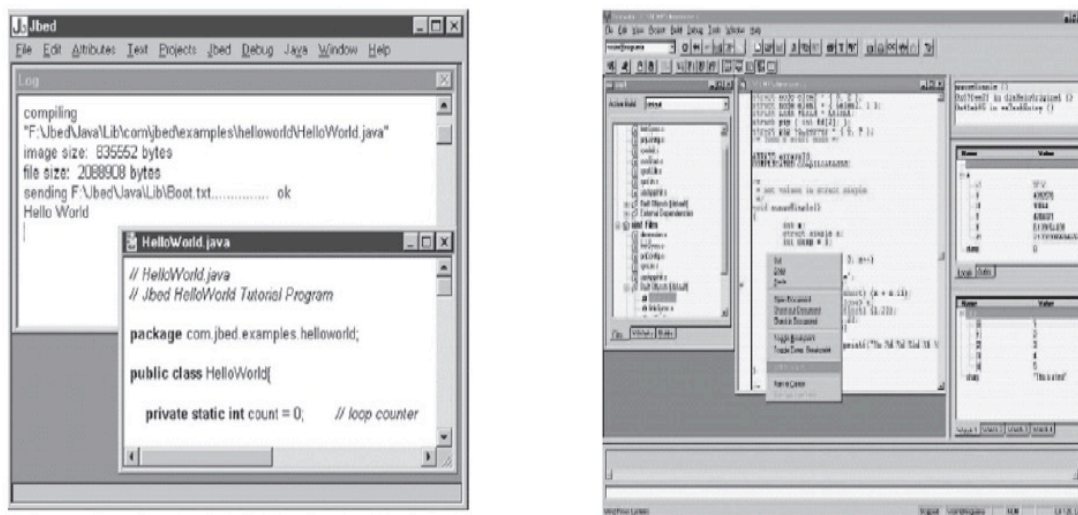
Các công cụ phát triển quan trọng trong thiết kế nhúng có thể được đặt trên máy chủ, trên đích, hoặc có thể tồn tại độc lập. Những công cụ này thường thuộc một trong ba loại: **tiện ích**, **dịch thuật**, và các công cụ **gỡ lỗi**. Các công cụ tiện ích là các công cụ chung mà hỗ trợ trong phát triển phần mềm hoặc phần cứng, chẳng hạn như trình soạn thảo (cho việc viết mã nguồn), VCS (điều khiển phiên bản phần mềm) các phần mềm quản lý tập tin, bộ ghi ROM cho phép phần mềm được đưa vào ROM... Các công cụ dịch thuật chuyển đổi mã phát triển dự định cho đích thành dạng mà đích có thể thực thi, và các công cụ gỡ lỗi có thể được sử dụng để theo dõi và sửa lỗi trong hệ thống. Tất cả các loại công cụ phát triển là quan trọng đối với một dự án như thiết kế kiến trúc, bởi vì nếu không có các công cụ đúng, việc thực hiện và gỡ lỗi hệ thống sẽ rất khó khăn, nếu không phải là không thể.



Hình 4. 7: Môi trường phát triển

*** Công cụ phần mềm tiện ích chính: viết mã trong một trình soạn thảo (Editor) hoặc môi trường phát triển tích hợp (IDE)**

Mã nguồn thường là được viết với một công cụ như một trình soạn thảo văn bản chuẩn ASCII, hoặc một **môi trường phát triển tích hợp (IDE)** nằm trên nền máy chủ (phát triển), như trong Hình 4-8. Một IDE là một tập hợp các công cụ, bao gồm một trình soạn thảo văn bản ASCII, tích hợp vào một ứng dụng giao diện người dùng. Trong khi trình soạn thảo văn bản ASCII bất kỳ có thể được sử dụng để viết bất kỳ loại mã nào, độc lập với ngôn ngữ và nền tảng, một IDE là cụ thể dành cho một nền tảng và thường được cung cấp bởi nhà cung cấp của IDE, một nhà sản xuất phần cứng (trong một bộ starter kit thường bao gồm bảng mạch phần cứng với các công cụ như một IDE hoặc trình soạn thảo văn bản), nhà cung cấp hệ điều hành, hoặc nhà cung cấp ngôn ngữ (Java, C, vv).

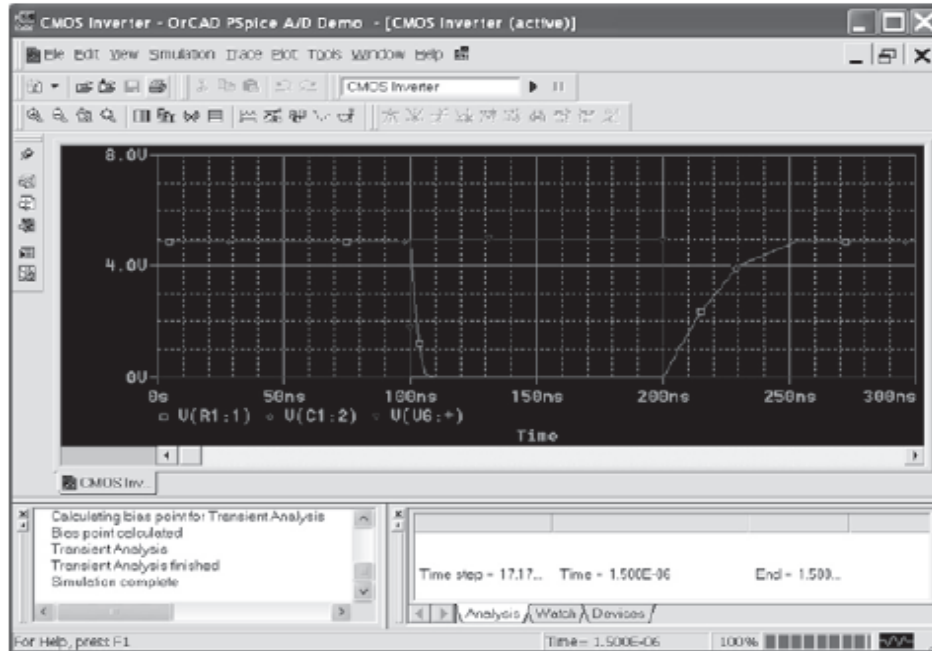


Hình 4. 8: IDE

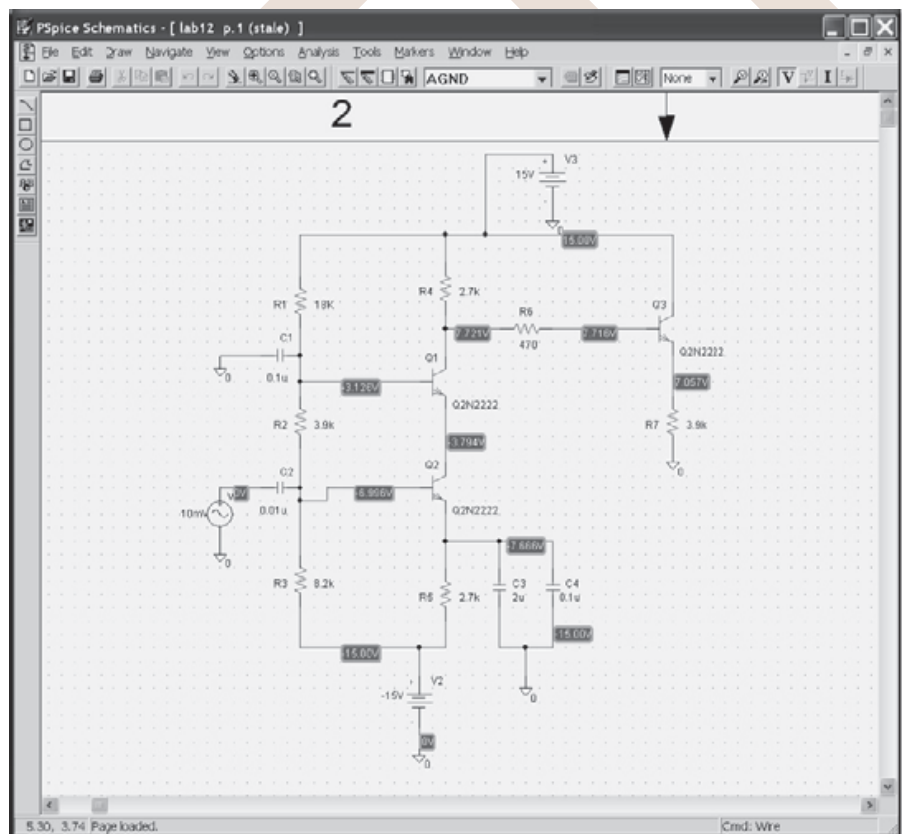
* Thiết kế trợ giúp máy tính (CAD) và phần cứng

Các công cụ Thiết kế trợ giúp máy tính (CAD) thường được sử dụng bởi các kỹ sư phần cứng để mô phỏng mạch điện ở cấp độ điện để nghiên cứu hành vi của một mạch trong những điều kiện khác nhau trước khi họ thực sự xây dựng các mạch.

Hình 4-9a là một bản chụp của một trình mô phỏng mạch phổ biến tiêu chuẩn, được gọi là PSpice. Phần mềm mô phỏng mạch này là một biến thể của một trình mô phỏng mạch khác mà đã được phát triển tại Đại học California, Berkeley gọi là SPICE (Simulation Program with Integrated Circuit Emphasis: Chương trình mô phỏng với Mạch Tích Hợp Quan Trọng). PSpice là phiên bản PC của SPICE, và là một ví dụ của một trình mô phỏng mà có thể làm một số loại phân tích mạch, chẳng hạn như phi tuyến ngắn hạn, DC phi tuyến, AC tuyến tính, nhiễu, và sự méo dạng. Như trong Hình 4-9b, các mạch được tạo ra trong trình mô phỏng này có thể được tạo thành từ một loạt các phần tử tích cực và/hoặc thụ động. Nhiều công cụ mô phỏng mạch điện thương mại sẵn có nói chung là tương tự như PSpice về mục đích tổng thể, và chỉ khác nhau chủ yếu là về những phân tích có thể được thực hiện, các thành phần mạch có thể được mô phỏng, hoặc vẽ bên ngoài của giao diện người dùng của công cụ.



Hình 4. 9: Ví dụ trình mô phỏng PSpice CAD



Hình 4. 10: Ví dụ mạch PSpice CAD

Do tầm quan trọng của thiết kế phần cứng và các chi phí liên quan, có nhiều ngành kỹ thuật công nghiệp mà trong đó các công cụ CAD được sử dụng để mô phỏng mạch. Cho một tập phức tạp các mạch trong một bộ xử lý hoặc trên một bảng mạch, rất là khó khăn, nếu không phải là không thể, để thực hiện một mô phỏng trên toàn bộ thiết kế, do đó một hệ thống phân cấp các mô phỏng và mô hình thường được sử dụng. Trong thực tế, việc sử dụng các mô hình là một trong những yếu tố quan trọng nhất trong thiết kế phần cứng, bất kể hiệu quả hoặc tính chính xác của mô phỏng này.

Ở cấp độ cao nhất, một mô hình hành vi của toàn bộ mạch được tạo ra cho cả hai mạch tương tự và số, và được sử dụng để nghiên cứu hành vi của toàn bộ mạch. Mô hình hành vi có thể được tạo ra với một công cụ CAD mà cung cấp tính năng này, hoặc có thể được viết bằng một ngôn ngữ lập trình tiêu chuẩn. Sau đó, phụ thuộc vào kiểu và cấu thành của mạch, các mô hình bổ sung được tạo ra xuống đến các thành phần tích cực và thụ động riêng lẻ của mạch, cũng như cho bất kỳ yếu tố phụ thuộc môi trường nào (ví dụ: nhiệt độ) mà mạch có thể có.

Ngoài việc sử dụng một số phương pháp cụ thể để viết các phương trình mạch cho một giả lập cụ thể, chẳng hạn như các phương pháp tiếp cận hoạt cảnh hoặc sửa đổi phương pháp nút, có các kỹ thuật mô phỏng để xử lý các mạch phức tạp bao gồm một hoặc một sự kết hợp nào đó của:

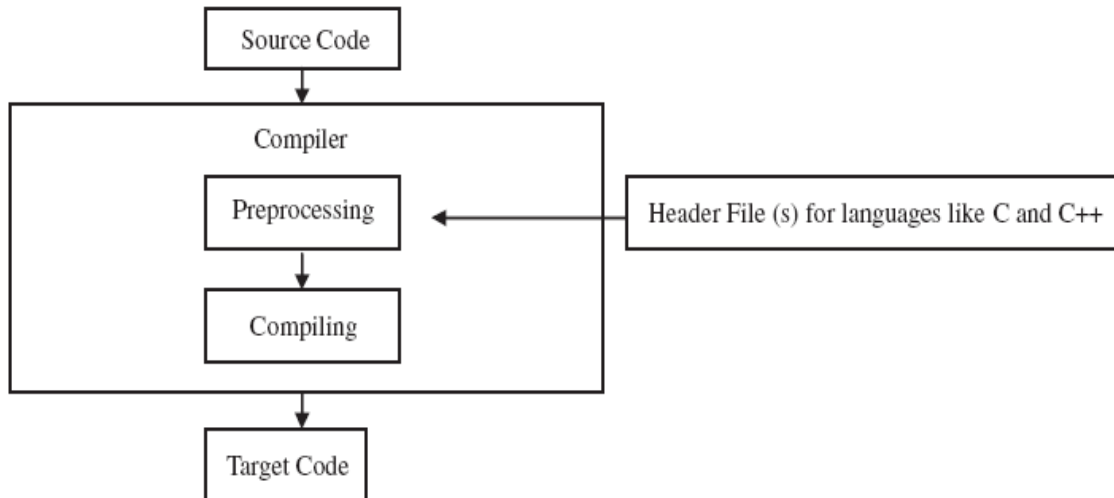
- phân chia các mạch phức tạp hơn thành các mạch nhỏ hơn, và sau đó kết hợp các kết quả.
- sử dụng các đặc tính đặc biệt của một số loại mạch nhất định.
- sử dụng các máy tính vector và/hoặc máy tính song song tốc độ cao.

*** Những công cụ phiên dịch: các bộ tiền xử lý, các trình thông dịch, các trình biên dịch, và các trình liên kết.**

Việc dịch mã đã được giới thiệu đầu tiên trong Chương 2, cùng với một lời giới thiệu ngắn gọn tới một số những công cụ được dùng trong việc Dịch mã, bao gồm các bộ tiền xử lý, các trình biên dịch, các trình biên dịch, và các trình kết nối. Như một tổng quát, sau khi mã nguồn đã được viết, nó cần phải được dịch sang mã máy, vì mã máy là ngôn ngữ duy nhất mà phần cứng có thể trực tiếp thực hiện. Tất cả các ngôn ngữ khác cần công cụ phát triển để tạo ra mã máy tương ứng mà phần cứng có thể hiểu. Cơ chế này thường bao gồm một hoặc sự kết hợp nào đó của các kỹ thuật phát mã máy như tiền xử lý, biên dịch, và/hoặc thông dịch. Các cơ chế này được thực hiện trong một loạt các công cụ phát triển phục vụ cho việc dịch.

Tiền xử lý là một bước tùy chọn có thể xuất hiện trước khi dịch hoặc thông dịch mã nguồn, và chức năng này thường được thực hiện bởi một bộ tiền xử lý. Vai trò của bộ tiền xử lý là tổ chức và cấu trúc lại mã nguồn để thực hiện dịch hoặc thông dịch mã này dễ dàng hơn. Bộ tiền xử lý có thể là một thực thể riêng biệt, hoặc có thể được tích hợp bên trong khối biên dịch, hoặc thông dịch.

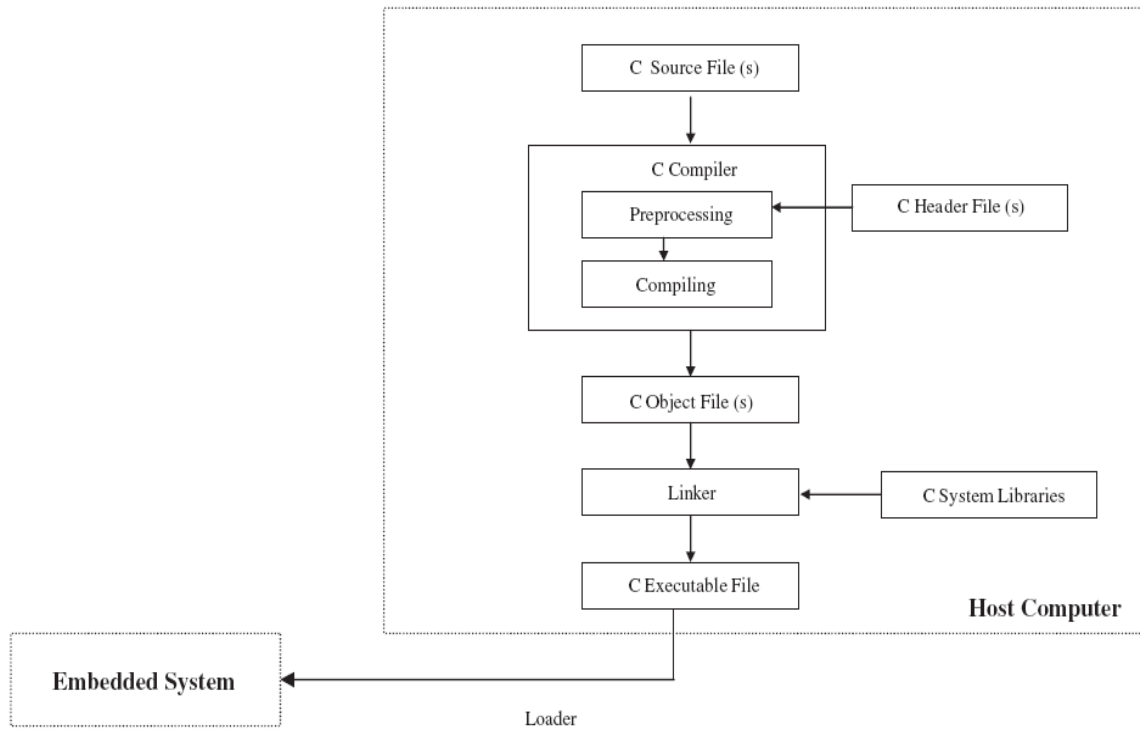
Nhiều ngôn ngữ chuyển đổi mã nguồn, hoặc trực tiếp hoặc sau khi đã được tiền xử lý, tới mã đích (mã máy) thông qua việc sử dụng một trình biên dịch, một chương trình tạo ra một số ngôn ngữ đích, chẳng hạn như mã máy, mã Java byte, v.v., từ ngôn ngữ nguồn, chẳng hạn như hợp ngữ, C, Java, v.v. (xem Hình 4-10).



Hình 4. 11: Sơ đồ biên dịch

Một trình biên dịch thông thường dịch tất cả các mã nguồn tới mã đích trong một lần. Như thường là trường hợp trong các hệ thống nhúng, hầu hết các trình biên dịch được đặt trên máy chủ của lập trình viên và tạo ra các mã đích cho các nền tảng phần cứng khác biệt với nền tảng mà trình biên dịch thực sự đang chạy trên đó. Các trình biên dịch này thường được gọi là trình biên dịch chéo. Trong trường hợp chương trình hợp ngữ, một trình biên dịch hợp ngữ là một trình biên dịch chéo đặc biệt gọi là **trình dịch hợp ngữ (assembler)**, và nó sẽ luôn luôn tạo ra mã máy. Các trình biên dịch ngôn ngữ bậc cao khác thường được gọi bằng tên ngôn ngữ cộng với "trình biên dịch" (ví dụ: trình biên dịch Java(Java compiler), trình biên dịch C (C compiler)). Các trình biên dịch ngôn ngữ bậc cao có thể thay đổi rộng về những gì được tạo ra. Một số tạo ra mã máy, trong khi những cái khác tạo ra các ngôn ngữ cấp cao khác, mà sau đó yêu cầu những gì được tạo ra phải được chạy thông qua ít nhất một trình biên dịch nữa. Vẫn còn các trình biên dịch khác tạo ra mã hợp ngữ, mà mã sau đó phải được chạy thông qua một trình dịch hợp ngữ (assembler).

Sau khi tất cả việc biên dịch trên máy tính chủ của lập trình viên được hoàn thành, các tập tin mã đích còn lại thường được gọi là một tập tin đối tượng (object file), và có thể chứa bất cứ điều gì từ mã máy đến mã byte Java, tùy thuộc vào ngôn ngữ lập trình được sử dụng. Như trong Hình 4-12, một **trình liên kết(linker)** kết hợp tập tin đối tượng này với bất kỳ thư viện hệ thống cần thiết khác, để tạo ra tập tin thường được gọi là một tập tin nhị phân có thể thực thi, hoặc trực tiếp đưa vào bộ nhớ của bảng mạch hoặc sẵn sàng để được chuyển tới bộ nhớ của hệ thống nhúng đích bởi một bộ nạp (loader).

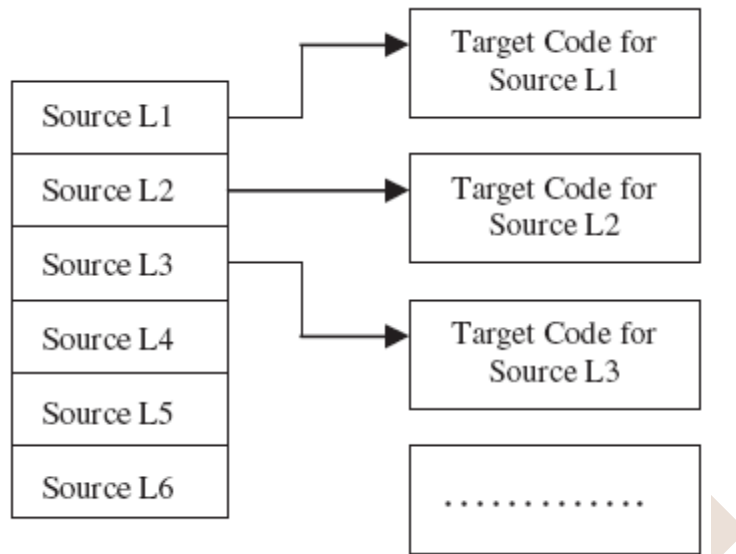


Hình 4. 12: Các bước biên dịch/liên kết và tập tin đối tượng kết quả, thực hiện trong C

Một trong những điểm mạnh cơ bản của một quy trình dịch là dựa trên khái niệm về sự sắp đặt phần mềm (còn gọi là sự sắp đặt đối tượng), khả năng phân chia phần mềm thành các module và tái định vị các module mã và dữ liệu này ở bất cứ nơi nào trong bộ nhớ. Đây là một tính năng đặc biệt hữu ích trong các hệ thống nhúng, bởi vì: (1) các thiết kế nhúng có thể chứa một vài loại khác nhau của bộ nhớ vật lý, (2) chúng thường có một số lượng hạn chế bộ nhớ so với các loại hệ thống máy tính khác; (3) bộ nhớ có thể thường trở nên rất phân mảnh và chức năng chống phân mảnh là không có sẵn out-of-the-box hoặc quá đắt tiền, và (4) một số loại phần mềm nhúng có thể cần phải được thực hiện từ một vị trí bộ nhớ cụ thể(xác định).

Khả năng sắp đặt phần mềm theo vị trí này có thể được hỗ trợ bởi bộ xử lý chủ (master), trong đó cung cấp các chỉ dẫn chuyên dụng mà có thể được sử dụng để tạo ra "mã độc lập vị trí", hoặc nó có thể được chia cách bởi các công cụ dịch phần mềm riêng. Trong cả hai trường hợp, khả năng này phụ thuộc vào liệu có trình hợp dịch/trình biên dịch chỉ có thể xử lý các địa chỉ tuyệt đối, nơi mà các địa chỉ bắt đầu được cố định bởi phần mềm trước khi sự hợp dịch xử lý mã, hoặc liệu có sự hỗ trợ một sơ đồ định địa chỉ tương đối mà trong đó địa chỉ bắt đầu của mã có thể được chỉ rõ sau và nơi module mã được xử lý liên quan đến sự bắt đầu của module. Trường hợp một trình biên dịch/trình hợp dịch tạo ra các mô-đun tái định vị, các định dạng chỉ dẫn quá trình, và có thể thực hiện một số biên dịch liên quan đến các địa chỉ vật lý (tuyệt đối), ví dụ, dịch các địa chỉ tương đối còn lại thành địa chỉ vật lý, về cơ bản việc sắp đặt phần mềm, được thực hiện bằng trình liên kết.

Trong khi các IDE, các trình tiền xử lý, các trình biên dịch, các trình liên kết vận hành nằm trên hệ thống phát triển chủ, một số ngôn ngữ, chẳng hạn như Java và các ngôn ngữ kịch bản, có trình biên dịch hoặc trình thông dịch nằm trên đích. Một trình thông dịch tạo ra (phiên dịch ra) mã máy từ một dòng mã nguồn tại một thời điểm từ mã nguồn hoặc mã đích được tạo ra bởi một trình biên dịch trung gian trên hệ thống máy chủ (xem Hình 4-12 dưới đây).



Hình 4. 13: Sơ đồ sự phiên dịch

Một người phát triển nhúng có thể làm một tác động lớn trong việc lựa chọn các công cụ dịch cho một dự án bởi sự hiểu biết làm thế nào trình biên dịch làm việc và, nếu có những tùy chọn, bằng việc chọn trình biên dịch mạnh nhất có thể. Điều này là do trình biên dịch, trong phần lớn, xác định kích thước của các mã thực thi bởi cách thức nó dịch mã như thế nào.

Điều này không chỉ có nghĩa là lựa chọn một trình biên dịch dựa trên sự hỗ trợ của bộ vi xử lý chủ, phần mềm hệ thống đặc biệt, và các công cụ còn lại (một trình biên dịch có thể được mua riêng rẽ, như một phần của một starter kit từ một nhà cung cấp phần cứng, và/hoặc tích hợp trong một IDE). Nó cũng có nghĩa là lựa chọn một trình biên dịch dựa trên một tập hợp tính năng tối ưu hóa tính đơn giản, tốc độ, và kích cỡ của mã. Những tính năng này có thể, tất nhiên, khác nhau giữa các trình biên dịch của các ngôn ngữ khác nhau, hoặc thậm chí các trình biên dịch khác nhau của cùng một ngôn ngữ, nhưng như là một ví dụ sẽ bao gồm việc cho phép đặt các dòng lệnh hợp ngữ (in-line assembly) bên trong mã nguồn và các hàm thư viện chuẩn mà làm cho việc lập trình mã nhúng dễ dàng hơn một chút. Việc tối ưu hóa mã cho hiệu suất có nghĩa là trình biên dịch hiểu và sử dụng các tính năng khác nhau của một ISA cụ thể, chẳng hạn như hoạt động toán học, tập thanh ghi, nhận biết được các loại ROM và RAM tích hợp trên chip (on-chip), số lượng các chu kỳ đồng hồ cho các loại truy cập, v.v. Bởi việc hiểu làm thế nào trình biên dịch dịch mã, một người phát triển có thể nhận ra những gì hỗ trợ được cung

cấp bởi trình biên dịch và tìm hiểu, ví dụ, làm thế nào để chương trình trong ngôn ngữ bậc cao được hỗ trợ bởi trình biên dịch một cách hiệu quả ("mã thân thiện trình biên dịch"), và khi để mã trong một ngôn ngữ bậc thấp hơn, nhanh hơn, chẳng hạn như hợp ngữ.

Lời khuyên cho thế giới-thực

Trình biên dịch những lý tưởng

Các hệ thống những có các yêu cầu khác thường và những ràng buộc không điển hình của thế giới không-những của các máy tính và các hệ thống lớn hơn. Trong nhiều cách, những tính năng và kỹ thuật thực hiện trong nhiều thiết kế trình biên dịch những phát triển từ các thiết kế của trình biên dịch không những. Các trình biên dịch này làm việc tốt cho sự phát triển hệ thống không những, nhưng không giải quyết các yêu cầu khác biệt của sự phát triển các hệ thống những, chẳng hạn như giới hạn về tốc độ và không gian. Một trong những lý do chính mà hợp ngữ vẫn còn rất thịnh hành trong các thiết bị những sử dụng các ngôn ngữ bậc cao hơn là các nhà phát triển không thể nhìn thấy được những gì các trình biên dịch làm với mã bậc cao hơn. Nhiều trình biên dịch những không cung cấp thông tin về cách mã được tạo ra. Vì vậy, các nhà phát triển đã không có cơ sở để đưa ra quyết định lập trình khi sử dụng một ngôn ngữ ở bậc cao hơn để cải thiện về kích thước và hiệu suất. Các tính năng trình biên dịch mà sẽ giải quyết một số yêu cầu, chẳng hạn như các yêu cầu về kích thước và tốc độ liên quan đến thiết kế hệ thống những, bao gồm:

- Một tập tin danh sách trình biên dịch mà đính mỗi dòng mã với những dự toán về số lần thực hiện dự kiến, phạm vi dự kiến của thời gian thực hiện, hoặc một số loại công thức được dùng để tính toán (thu thập từ các thông tin mục tiêu cụ thể của các công cụ khác được tích hợp với trình biên dịch).*

- Một công cụ biên dịch cho phép nhà phát triển xem một dòng mã dưới hình thức biên dịch của nó, và đánh dấu bất kỳ vùng vấn đề tiềm tàng nào.*

- Cung cấp thông tin về kích thước của mã thông qua một bản đồ kích thước chính xác, cùng với một trình duyệt cho phép lập trình viên xem bao nhiêu bộ nhớ đang được sử dụng bởi các thủ tục con riêng biệt.*

Nhớ đến các đặc tính hữu ích này khi thiết kế hoặc khi mua một trình biên dịch những.

*** Công cụ gỡ lỗi**

Bên cạnh việc tạo ra kiến trúc, việc gỡ lỗi mã có lẽ là nhiệm vụ khó khăn nhất của chu kỳ phát triển. Gỡ lỗi chủ yếu là nhiệm vụ định vị và cố định lỗi trong hệ thống. Công việc này được thực hiện đơn giản khi lập trình viên là quen thuộc với các loại công cụ gỡ lỗi sẵn có và cách thức họ có thể sử dụng (loại thông tin được hiển thị trong Bảng 4-3).

Như đã thấy từ một số các mô tả trong Bảng 4-3, các công cụ gỡ lỗi cư trú và kết nối trong một vài sự kết hợp của những thiết bị độc lập, trên máy chủ, và/hoặc trên bảng mạch đích.

Một nhận xét nhanh trên việc đo lường hiệu suất hệ thống với các chuẩn đánh giá (Benchmark)

Ngoài các công cụ gỡ lỗi, mỗi khi bảng mạch được khởi động và chạy, chuẩn đánh giá là các chương trình phần mềm thường được sử dụng để đo lường hiệu quả hoạt động (độ trễ, hiệu quả, vv) của các tính năng riêng biệt trong một hệ thống nhúng, như bộ vi xử lý chủ, hệ điều hành, hoặc JVM. Trong trường hợp của một hệ điều hành, ví dụ, hiệu suất được đo bằng hiệu quả của việc bộ vi xử lý chủ được sử dụng bởi các chương trình lập lịch của hệ điều hành. Bộ lập lịch cần ấn định định lượng thời gian thích hợp - thời gian một quá trình được tiếp cận với CPU- cho một quá trình, vì nếu định lượng thời gian là quá nhỏ, sự xung đột sẽ xuất hiện.

Mục tiêu chính của một ứng dụng chuẩn đánh giá là đại diện cho một khối lượng công việc thực tế cho hệ thống. Có rất nhiều ứng dụng chuẩn đánh giá có sẵn. Chúng bao gồm các chuẩn đánh giá EEMBC (Embedded Microprocessor Benchmark Consortium), tiêu chuẩn công nghiệp để đánh giá khả năng của các bộ xử lý nhúng, các trình biên dịch, và Java; Whetstone, trong đó mô phỏng các ứng dụng khoa học số học chuyên sâu; và Dhrystone, trong đó mô phỏng các ứng dụng lập trình hệ thống, thường bắt nguồn từ MIPS được giới thiệu trong Phần II. Các hạn chế của các chuẩn đánh giá là chúng có thể không được thực tế hoặc có thể tái sản xuất trong một thiết kế thế giới thực có liên quan đến nhiều hơn một tính năng của một hệ thống. Vì vậy, thường là tốt hơn khi sử dụng các chương trình nhúng thực sự mà sẽ được triển khai trên hệ thống để xác định không chỉ hiệu suất của phần mềm, mà toàn bộ hiệu suất hệ thống.

Nói tóm lại, khi làm sáng tỏ các chuẩn đánh giá, đảm bảo bạn hiểu chính xác những gì phần mềm đã chạy và những gì các chuẩn đánh giá đã đo hoặc đã không đo.

Loại công cụ	Công cụ gỡ lỗi	Mô tả	Ví dụ về sử dụng và những hạn chế
Phần cứng	Bộ mô phỏng trên mạch(In-Circuit Emulator: ICE)	Thiết bị hoạt động thay thế bộ vi xử lý trong hệ thống	* giải pháp gỡ lỗi đắt nhất điển hình, nhưng có nhiều khả năng gỡ lỗi * có thể hoạt động ở tốc độ đầy đủ của bộ xử lý (phụ thuộc vào ICE) và tới phần còn lại của hệ thống đó là bộ vi xử lý * cho phép có thể xem và có thể thay đổi được nội dung bộ nhớ trong , các thanh ghi, các biến, v.v..trong thời gian thực * tương tự như những trình gỡ rối, cho phép đặt những điểm dừng, thực hiện từng bước,...

			* thường có lớp che bộ nhớ để mô phỏng bộ nhớ ROM * bộ xử lý phụ thuộc
	Bộ mô phỏng ROM	Công cụ hoạt động thay thế ROM với các cáp kết nối với RAM cổng kép bên trong bộ mô phỏng ROM, mô phỏng ROM. Nó là một thiết bị phần cứng trung gian kết nối với các đích thông qua một số cáp (tức là BDM), và kết nối với máy chủ thông qua cổng khác	* cho phép sửa đổi nội dung trong ROM (không giống như một trình gỡ lỗi) * có thể đặt breakpoint trong mã ROM, và xem mã ROM thời gian thực * thường không hỗ trợ ROM on-chip, ASIC tùy chỉnh, v.v.. * có thể tích hợp với những trình gỡ lỗi
	Chế độ gỡ lỗi nền (Background Debug Mode: BDM)	Phần cứng BDM trên bảng mạch (cổng và bộ giám sát gỡ lỗi tích hợp vào CPU chủ), và trình gỡ lỗi trên máy chủ, kết nối thông qua một cáp nối tiếp đến cổng BDM. Đầu nối trên cáp đến cổng BDM, thường được gọi là wiggler. Gỡ lỗi BDM đôi khi được gọi là gỡ lỗi On-Chip (On-Chip Debugging: OCD)	* thường rẻ hơn so với ICE, nhưng không linh động như ICE * quan sát sự thực hiện phần mềm một cách kín đáo trong thời gian thực * có thể đặt breakpoint để dừng sự thực hiện phần mềm * cho phép đọc và ghi tới các thanh ghi, RAM, các cổng I/O, v.v.. * phụ thuộc bộ xử lý/đích, giao diện gỡ lỗi độc quyền của Motorola
	IEEE 1149.1 Joint Test Action Group (JTAG)	phần cứng tương thích JTAG trên bảng mạch	* tương tự như BDM, nhưng không độc quyền cho kiến trúc cụ thể (là một tiêu chuẩn mở)
	IEEE-ISTO Nexus 5001	Các tùy chọn của cổng JTAG, cổng	* cung cấp khả năng mở rộng chức năng gỡ lỗi tùy theo mức

		tương thích Nexus, hoặc cả hai, một số lớp phù hợp (phụ thuộc độ phức tạp của bộ xử lý chủ, lựa chọn kỹ thuật, v.v..)	độ tương thích của phần cứng
	Oscilloscope (máy hiện sóng)	Thiết bị tương tự thụ động vẽ đồ thị điện áp (trên trục thẳng đứng) so với thời gian (trên trục ngang), cho phép tìm ra điện áp chính xác tại một thời điểm nhất định	* giám sát tới 2 tín hiệu đồng thời * có thể đặt một điện áp kích hoạt để bắt giữ điện áp trong những điều kiện cụ thể * sử dụng như là von mét (mặc dù đắt hơn nhiều) * có thể kiểm tra mạch đang làm việc bằng cách xem tín hiệu trên bus hoặc các cổng I/O * bắt giữ những thay đổi trong một tín hiệu trên cổng I/O để kiểm tra các đoạn của phần mềm đang chạy, tính toán thời gian từ mỗi thay đổi tín hiệu tới thay đổi tiếp theo, v.v.. * độc lập với bộ xử lý
	Logic Analyzer (bộ phân tích logic)	Thiết bị thụ động này có thể chụp và theo dõi đồng thời nhiều tín hiệu và có thể vẽ đồ thị chúng	* có thể đắt * thường chỉ có thể theo dõi 2 mức điện áp (VCC và GND); các tín hiệu nằm ở khoảng giữa được vẽ như VCC hay GND * có thể lưu trữ dữ liệu * 2 chế độ hoạt động chính (thời gian, trạng thái) cho phép kích hoạt trên những thay đổi trạng thái của tín hiệu (tức là cao-xuống-thấp hoặc thấp-lên-cao) * bắt giữ những thay đổi trong một tín hiệu trên cổng I/O để

			kiểm tra các đoạn của phần mềm đang chạy, tính toán thời gian từ một thay đổi tín hiệu đến một thay đổi tín hiệu tiếp theo, v.v.. (chế độ thời gian) * có thể được kích hoạt để bắt giữ dữ liệu từ một sự kiện xung nhịp bên trong đích hoặc một xung nhịp bên trong bộ phân tích logic * có thể kích hoạt nếu bộ xử lý truy cập vào vùng cấm của bộ nhớ, ghi dữ liệu không hợp lệ vào bộ nhớ, hoặc truy cập một loại lệnh đặc biệt (chế độ trạng thái) * một số sẽ hiển thị mã hợp ngữ, nhưng thường không có thể đặt breakpoint và chạy bước đơn thông qua mã sử dụng bộ phân tích * bộ phân tích logic chỉ có thể truy cập dữ liệu được truyền từ bên ngoài đến và từ bộ xử lý, chứ không phải bộ nhớ trong, các thanh ghi, v.v. * bộ xử lý độc lập và cho phép xem hệ thống thực hiện trong thời gian thực với rất ít sự xâm nhập
	Voltmeter	Đo điện áp chênh lệch giữa 2 điểm trên mạch	* để đo các giá trị điện áp đặc biệt * để xác định sự hiện diện của nguồn tại tất cả các điểm trên mạch * rẻ hơn những công cụ phân cứng khác
	Ohmmeter	Đo điện trở giữa 2	* rẻ hơn so với các công cụ

		điểm trên mạch	phần cứng khác * để đo những thay đổi về dòng điện/điện áp trong mối quan hệ với điện trở (dùng định luật Ôm $V=IR$) *....
	Multimeter	Đo cả điện áp lẫn điện trở	* giống như volt và ohm meter *.....
Phần mềm	Debugger (trình gỡ rối)	Công cụ gỡ lỗi chức năng	Phụ thuộc vào trình gỡ rối - nói chung: * nạp/chạy từng bước/giám sát mã trên đích * thực hiện những điểm breakpoint để dừng sự thực hiện phần mềm * thực hiện những điểm breakpoint có điều kiện để dừng nếu điều kiện đặc biệt xuất hiện trong thời gian thực hiện * có thể sửa đổi nội dung của RAM, thường không thể sửa đổi nội dung của ROM
	Profiler	Thu thập giá trị theo thời gian của các biến, các thanh ghi... được lựa chọn	* thời gian bắt giữ phụ thuộc (khi) hành vi của phần mềm đang thực hiện * để bắt mẫu thực hiện (ở đâu) của phần mềm đang thực hiện
	Monitor	Giao diện gỡ lỗi tương tự như ICE, với phần mềm gỡ lỗi chạy trên đích và máy chủ. Một phần của monitor nằm trong ROM của bảng mạch đích (thường được gọi là tác nhân gỡ lỗi hoặc tác nhân đích),	* tương tự như in phát biểu nhưng nhanh hơn, ít xâm nhập, làm việc tốt hơn cho giới hạn thời gian thực mềm, nhưng không tốt cho thời gian thực cứng * chức năng tương tự tới trình gỡ rối * Hệ điều hành nhúng có thể bao gồm monitor cho những

		và một hạt nhân gỗ lỗi trên máy chủ. Phần mềm trên máy chủ và đích thường giao tiếp nối tiếp hoặc thông qua Ethernet (phụ thuộc vào những gì có sẵn trên đích).	kiến trúc đặc biệt
	Trình mô phỏng tập lệnh	Chạy trên máy chủ và mô phỏng bộ xử lý chủ và bộ nhớ (chương trình nhị phân có thể thực thi được nạp vào trình mô phỏng như là nó sẽ được nạp vào đích) và bắt chước phần cứng	* thường không chạy ở cùng một tốc độ chính xác như đích thực tế, nhưng có thể đánh giá phản ứng và thông qua thời gian bằng cách xem xét sự khác biệt giữa tốc độ máy chủ và đích * thường không mô phỏng phần cứng khác mà có thể tồn tại trên đích, nhưng có thể cho phép thử nghiệm các thành phần được xây dựng trong bộ xử lý * có thể mô phỏng hành vi ngắt * bắt giữ giá trị của biến, bộ nhớ và các thanh ghi * sẽ không mô phỏng chính xác hành vi của các phần cứng thực tế trong thời gian thực * thường phù hợp hơn để thử nghiệm các thuật toán chứ không phải là phản ứng đối với các sự kiện bên ngoài của một kiến trúc hay bảng mạch (dạng sóng và như vậy cần được mô phỏng thông qua phần mềm) * thường rẻ hơn so với đầu tư vào phần cứng và các công cụ thực

			
Manual (thủ công)	Luôn sẵn có, miễn phí hoặc rẻ hơn so với các giải pháp khác, hiệu quả, đơn giản để sử dụng nhưng thường xâm nhập cao hơn các loại công cụ khác, không đủ kiểm soát đối với sự lựa chọn sự kiện, cách ly, hoặc lặp lại. Khó khăn để gỡ lỗi hệ thống thời gian thực nếu phương pháp thủ công phải mất quá lâu để thực thi.		
	In các báo cáo	Công cụ gỡ lỗi chức năng, việc in các báo cáo được đưa vào bên trong mã để in thông tin thay đổi, vị trí trong mã thông tin, v.v.	* để xem đầu ra của các biến, giá trị các thanh ghi, v.v.. trong khi các mã đang chạy * để kiểm tra đoạn mã đang được thực hiện * có thể làm chậm đáng kể thời gian thực hiện chương trình * có thể dẫn đến mất thời hạn trong hệ thống thời gian thực.
	Dumps	Công cụ gỡ lỗi chức năng kết xuất dữ liệu vào một số loại cấu trúc lưu trữ trong thời gian chạy	* giống như in báo cáo nhưng cho phép thời gian thực hiện nhanh hơn trong việc thay thế một số báo cáo in ấn (đặc biệt là nếu có một bộ lọc xác định những loại cụ thể của thông tin để kết xuất hoặc những điều kiện cần phải được đáp ứng để kết xuất dữ liệu vào cấu trúc) * xem nội dung của bộ nhớ tại thời gian chạy để xác định nếu có bất kỳ sự tràn stack/heap
	Counters/Timers	Công cụ gỡ lỗi hiệu suất và hiệu quả mà trong đó các bộ đếm hoặc các bộ định thời thiết được thiết lập lại và tăng lên tại các điểm khác nhau của mã	* thu thập thông tin thời gian thực hiện chung bằng cách giảm bớt xung nhịp hệ thống hoặc đếm các chu kỳ bus, v.v.. * một số xâm nhập
	Fast Display	Công cụ gỡ lỗi chức năng trong đó các đèn	* tương tự như in báo cáo nhưng nhanh hơn, ít xâm

		LED được bật/tắt hoặc các màn hình LCD đơn giản được dùng để biểu diễn một số dữ liệu	nhập, làm việc tốt cho các giới hạn thời gian thực * cho phép xác nhận các phần đặc biệt của mã đang chạy
	Ouput ports	Công cụ gỡ lỗi chức năng hiệu suất, hiệu quả rong đó các cổng đầu ra bật/tắt tại các điểm khác nhau trong phần mềm	* với một máy hiện sóng hoặc máy phân tích logic, có thể đo khi cổng bị bật/tắt và nhận được thời gian thực hiện giữa các lần bật/tắt của cổng * giống như trên nhưng có thể thấy trên máy hiện sóng mã đang được thực hiện tại vị trí đầu tiên * trong hệ thống đa nhiệm/đa luồng gán những cổng khác nhau tới mỗi luồng/nhiệm vụ để nghiên cứu hành vi

Bảng 4. 3: Những công cụ gỡ lỗi

Một số trong những công cụ này là các công cụ gỡ lỗi tích cực và xâm nhập được vào các hoạt động của hệ thống nhúng, trong khi các công cụ gỡ lỗi khác thụ động nắm bắt các hoạt động của hệ thống không có sự xâm nhập khi hệ thống đang chạy. Gỡ lỗi một hệ thống nhúng thường đòi hỏi một sự kết hợp của những công cụ này để xác định tất cả các loại khác nhau của các vấn đề có thể phát sinh trong quá trình phát triển.

Lời khuyên cho thế giới- thực

Cách rẻ nhất để Gỡ Lỗi

Thậm chí với tất cả các công cụ có sẵn, các nhà phát triển vẫn còn phải cố gắng để giảm thời gian gỡ lỗi và chi phí, bởi vì 1) chi phí lỗi tăng đến gần hơn với thời gian sản xuất và tiến độ triển khai được, và 2) chi phí của một lỗi là logarit (nó có thể tăng mười lần khi được phát hiện bởi một khách hàng so với nếu nó đã được tìm thấy trong quá trình phát triển của thiết bị). Một số các phương tiện hiệu quả nhất của việc giảm thời gian gỡ lỗi và chi phí bao gồm:

- Không phát triển quá nhanh và cầu thả. Cách rẻ nhất và nhanh nhất để gỡ lỗi là không chèn bất kỳ lỗi nào ở vị trí đầu tiên. Phát triển nhanh và cầu thả thực sự làm chậm trễ tiến độ với phần lớn thời gian tiêu tốn cho việc gỡ lỗi những sai lầm.

- Những kiểm tra Hệ thống. Điều này bao gồm các kiểm tra phần cứng và phần mềm trong suốt quá trình phát triển mà đảm bảo rằng các nhà phát triển đang thiết kế theo các đặc tả kỹ thuật kiến trúc, và bất kỳ tiêu chuẩn nào khác được yêu cầu của các kỹ sư. Mã hay phần cứng không đáp ứng các tiêu chuẩn sẽ phải được "sửa lỗi" sau đó nếu

các kiểm tra hệ thống không được sử dụng để đưa ra chúng ra nhanh chóng và rẻ (liên quan đến thời gian tiêu tốn cho việc gỡ lỗi và định vị tất cả lỗi mà với phần cứng và code nhiều hơn sau đó).

- Không sử dụng phần cứng bị lỗi hoặc mã được viết tồi. Một thành phần thường là sẵn sàng để được thiết kế lại khi các kỹ sư chịu trách nhiệm lo sợ việc thực hiện bất kỳ thay đổi nào đối với thành phần lỗi đó.

- Theo dõi các lỗi trong một tập tin văn bản chung hoặc sử dụng một trong nhiều các công cụ phần mềm theo dõi lỗi có sẵn. Nếu các thành phần (phần cứng hoặc phần mềm) đang tiếp tục gây ra những vấn đề, thì có thể dành thời gian để thiết kế lại thành phần đó.

- Đừng tiết kiệm với các công cụ gỡ lỗi. Một công cụ gỡ lỗi tốt (mặc dù đắt hơn) sẽ cắt giảm thời gian gỡ lỗi thì có trị giá hơn một tá các công cụ rẻ hơn mà, với không tốn nhiều thời gian và nhức đầu, chỉ có thể theo dõi các loại lỗi gặp phải trong quá trình thiết kế một hệ thống nhúng.

Và cuối cùng, một trong những phương pháp tốt nhất để giảm bớt thời gian và chi phí gỡ lỗi là đọc tài liệu được cung cấp bởi các nhà cung cấp và/hoặc các kỹ sư chịu trách nhiệm đầu tiên, trước khi cố gắng chạy hoặc sửa đổi bất cứ điều gì. Tôi đã nghe nói nhiều, nhiều lời bào chữa trong những năm qua, từ "Tôi không biết đọc cái gì" đến "Có tài liệu hướng dẫn không?" - Là tại sao một kỹ sư đã không đọc những tài liệu hướng dẫn. Các kỹ sư này đã dành nhiều giờ, nếu không phải nhiều ngày, về các vấn đề riêng lẻ với việc cấu hình phần cứng hay việc nhận được một phần của phần mềm chạy đúng. Tôi biết rằng nếu các kỹ sư này đọc tài liệu ở ngay thời điểm đầu tiên, vấn đề sẽ được giải quyết trong vài giây hoặc vài phút - hoặc có thể chẳng có vấn đề khó nào xuất hiện.

Nếu bạn đang quá tải với tài liệu và không biết những gì để đọc đầu tiên, bất cứ tiêu đề nào dọc theo dòng như "Getting Started...", "Bootting up the system...", hoặc "README" là các chỉ thị tốt của một nơi để bắt đầu. Hơn nữa, dành thời gian để đọc tất cả các tài liệu được cung cấp với bất kỳ phần cứng hay phần mềm để trở nên quen thuộc với những loại thông tin này, sẽ giúp ích trong trường hợp cần thiết sau này.

---Dựa trên bài viết "Firmware Basics for the Boss" của Jack Ganssle, Embedded Systems Programming, tháng 2 năm 2004

*** Khởi động (Boot-Up) hệ thống**

Với những công cụ phát triển sẵn sàng để thử, và hoặc một bảng mạch tham chiếu hoặc một bảng mạch phát triển kết nối với máy chủ phát triển, đó là thời điểm để khởi động hệ thống và xem những gì sẽ xảy ra. Khởi động hệ thống có nghĩa là một vài loại cấp nguồn hoặc khởi động lại nguồn, chẳng hạn như một khởi động lại cứng bên trong/bên ngoài (tức là, tạo ra bởi một lỗi kiểm tra dừng, các cơ quan giám sát phần mềm, mất khóa của PLL, bộ gỡ rối, v.v.), hoặc một khởi động lại mềm bên trong/bên ngoài (tức là, tạo ra bởi một trình sửa lỗi, mã ứng dụng, v.v.), xuất hiện. Khi nguồn được cấp tới một bảng mạch nhúng (bởi một khởi động lại), mã khởi động (start-up code), cũng

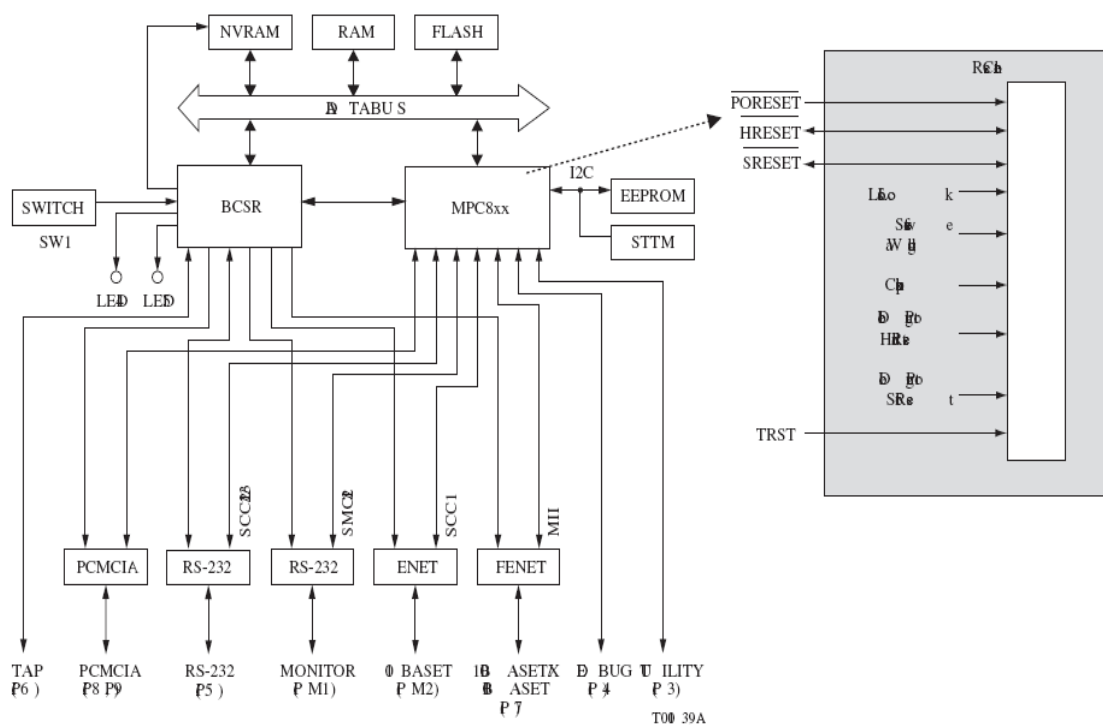
được gọi là **boot code**, **bootloader**, mã **bootstrap**, hoặc **BIOS** (hệ thống vào ra cơ sở) tùy thuộc vào kiến trúc, trong ROM của hệ thống được nạp và thực hiện bởi bộ xử lý chủ. Một số kiến trúc nhúng (master) có một bộ đếm chương trình nội bộ được cấu hình tự động với một địa chỉ trong ROM mà trong đó sự bắt đầu của mã boot-up (hoặc bảng) được định vị, trong khi những cái khác được nối cứng để bắt đầu thực hiện tại một địa điểm cụ thể trong bộ nhớ.

Mã Boot khác ở chiều dài và chức năng tùy thuộc vào thời điểm trong chu kỳ phát triển bảng mạch, cũng như các thành phần của nền tảng thực tế cần sự khởi tạo. Các chức năng chung (tối thiểu) được thực hiện bởi mã khởi động trên các nền tảng khác nhau, những cái khởi tạo chức năng cơ bản phần cứng, bao gồm vô hiệu hóa ngắt, khởi tạo các bus, thiết lập các bộ xử lý chủ và tở trong một trạng thái cụ thể, và khởi tạo bộ nhớ. Phần khởi tạo phần cứng đầu tiên này của mã boot-up về cơ bản là thực hiện các trình điều khiển thiết bị khởi tạo, như được thảo luận trong Chương 2. Làm thế nào khởi tạo là thực sự được thực hiện, đó là-thứ tự mà các trình điều khiển được thực hiện- thường được phác thảo bởi tài liệu kiến trúc tổng thể hay trong tài liệu được cung cấp bởi các nhà sản xuất bảng mạch. Sau chuỗi khởi tạo phần cứng, thực hiện thông qua khởi tạo các trình điều khiển thiết bị, phần mềm hệ thống còn lại, nếu có, sau đó được khởi tạo. Mã bổ sung này có thể tồn tại trong ROM, cho một hệ thống đang được vận chuyển ra khỏi nhà máy, hoặc nạp từ một nền tảng máy chủ bên ngoài (xem hộp lời thoại với bootcodeExample).

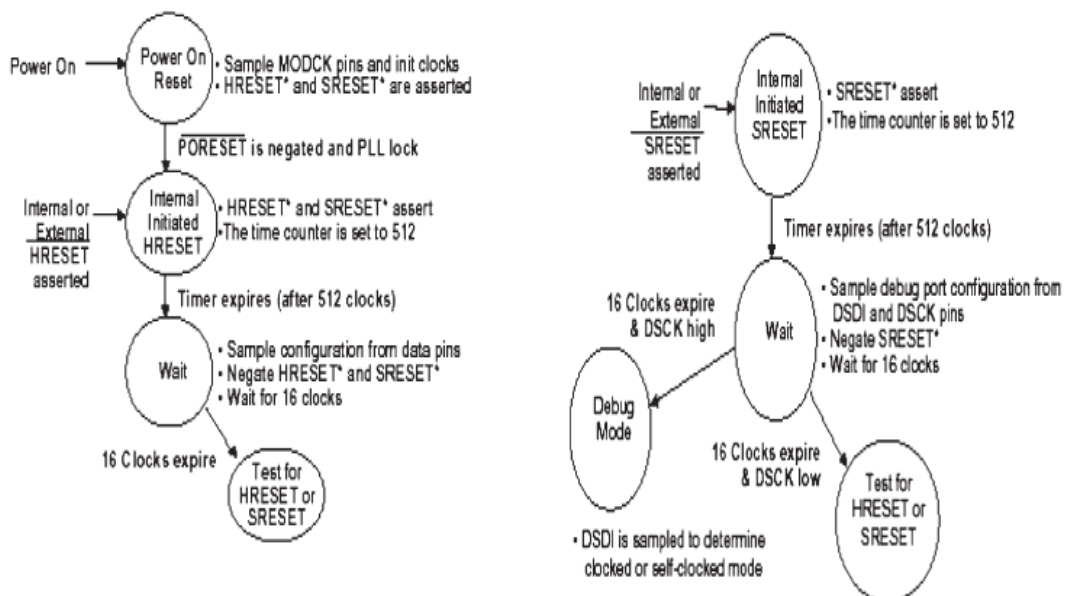
```
bootcodeExample ()
{
...
// Serial Port Initialization Device Driver
initializeRS232(UART,BAUDRATE,DATA_BITS,STOP_BITS,PARITY);
// Initialize Networking Device Driver
initializeEthernet(IPAddress,Subnet, GatewayIP, ServerIP);
//check for host development system for down loaded file of
rest of code to RAM
// through ethernet
// start executing rest of code(i.e. define memory map,
load OS, etc.)
...
}
```

Ví dụ khởi động bảng mạch dùng MPC823

Bộ xử lý MPC823 chứa một bộ điều khiển khởi động lại mà có trách nhiệm đáp ứng lại tới mọi nguồn khởi động lại. Những hành động được thực hiện bởi bộ điều khiển khởi động lại khác nhau tùy theo nguồn gốc của một sự kiện khởi động lại, nhưng nhìn chung quá trình bao gồm Cấu hình lại phần cứng, và sau đó lấy mẫu các chân dữ liệu hoặc sử dụng một hằng số mặc định nội bộ để xác định các giá trị khởi động lại ban đầu của các thành phần hệ thống.



Hình 4. 14: Sơ đồ giải thích



Hình 4. 15: Sơ đồ giải thích

4.2 Cài đặt và thử nghiệm hệ thống nhúng

Trong số các mục tiêu của việc thử nghiệm và đảm bảo chất lượng của một hệ thống là tìm lỗi trong thiết kế và theo dõi xem các lỗi được cố định. Bảo đảm chất lượng và thử nghiệm tương tự như gỡ lỗi, đã thảo luận ở trên, ngoại trừ các mục tiêu của gỡ lỗi là thực sự để sửa lỗi được phát hiện. Một khác biệt chính giữa gỡ lỗi và thử nghiệm hệ thống là gỡ lỗi thường xảy ra khi nhà phát triển gặp một vấn đề trong cố gắng để hoàn thành một phần của thiết kế, và sau đó thường thử nghiệm để thông qua những sửa chữa lỗi (có nghĩa là các thử nghiệm chỉ để đảm bảo hệ thống tối thiểu làm việc dưới trường hợp thông thường). Với thử nghiệm, mặt khác, lỗi được phát hiện như là kết quả của sự cố gắng để phá vỡ hệ thống, bao gồm cả *testing-to-pass* và *testing-to-fail*, nơi mà những yếu kém trong hệ thống được thăm dò.

Dưới thử nghiệm, các lỗi thường xuất phát từ hoặc hệ thống không trung thành với đặc tả kiến trúc hoặc không có khả năng kiểm tra hệ thống. Các loại lỗi gặp phải trong thử nghiệm phụ thuộc vào loại thử nghiệm đang được thực hiện. Nhìn chung, các kỹ thuật thử nghiệm thuộc một trong bốn mô hình: thử nghiệm *hộp đen tĩnh*, thử nghiệm *hộp trắng tĩnh*, thử nghiệm *hộp đen động*, hoặc thử nghiệm *hộp trắng động* (xem ma trận trong Hình 4-14). Thử nghiệm hộp đen xảy ra với một bộ thử nghiệm mà không có khả năng hiển thị các hoạt động nội bộ bên trong của hệ thống (không có sơ đồ nguyên lý, không có mã nguồn, v.v.). Thử nghiệm hộp đen được dựa trên tài liệu các yêu cầu sản phẩm nói chung, trái ngược với thử nghiệm hộp trắng (còn gọi là thử nghiệm hộp trong suốt hoặc thử nghiệm hộp thủy tinh) trong đó bộ thử nghiệm có thể truy cập vào mã nguồn, sơ đồ nguyên lý v.v. Thử nghiệm tĩnh được thực hiện trong khi hệ thống không hoạt động, trong khi thử nghiệm động được thực hiện khi hệ thống đang chạy.

	Thử nghiệm hộp đen (Black Box Testing)	Thử nghiệm hộp trắng (White Box Testing)
Thử nghiệm tĩnh	Thử nghiệm các đặc tả kỹ thuật của sản phẩm bởi: 1. tìm kiếm các vấn đề cơ bản cấp cao, các trường hợp sơ suất, bỏ sót (ví dụ, giả vờ là khách hàng, nghiên cứu các hướng dẫn/tiêu chuẩn hiện hành, xem xét và thử nghiệm phần mềm tương tự, vv.) 2. thử nghiệm đặc tả kỹ thuật cấp thấp bởi đảm bảo đầy đủ, chính xác, sự tinh tế, nhất quán, phù hợp, khả thi, v.v.	Quy trình xem xét lại cẩn thận phần cứng và mã cho các lỗi mà không thực hiện nó.

Thử nghiệm động	Yêu cầu định nghĩa những gì phần mềm và phần cứng thực hiện, bao gồm: * <i>thử nghiệm dữ liệu</i>, đó là việc kiểm tra thông tin các đầu vào và đầu ra người sử dụng * <i>thử nghiệm điều kiện biên</i>, đó là thử nghiệm các trạng thái tại cạnh của các giới hạn hoạt động dự kiến của phần mềm * <i>thử nghiệm đầu vào</i>, đó là thử nghiệm với dữ liệu vô giá trị, không hợp lệ. * <i>thử nghiệm trạng thái</i>, đó là thử nghiệm các phương thức và quá trình chuyển đổi giữa các chế độ phần mềm với các biến trạng thái tức là, các điều kiện chạy đua, thử nghiệm sự lặp lại (lý do chính là để phát hiện rò rỉ bộ nhớ), ứng suất (phần mềm đói = bộ nhớ thấp, cpu chậm = mạng chậm), tải (nguồn cấp dữ liệu phần mềm = kết nối nhiều thiết bị ngoại vi, xử lý một lượng lớn dữ liệu, web server có nhiều khách hàng truy cập vào nó, v.v.), ...	Thử nghiệm hệ thống đang chạy trong khi theo dõi mã, các sơ đồ nguyên lý, v.v. Trực tiếp thử nghiệm ở mức độ thấp và mức độ cao dựa trên sự hiểu biết hoạt động chi tiết, truy cập vào các biến và các kết xuất bộ nhớ. Tìm kiếm các lỗi tham chiếu dữ liệu, các lỗi khai báo dữ liệu, lỗi tính toán, các lỗi so sánh, lỗi lưu đồ điều khiển, các lỗi tham số cho chương trình con, các lỗi I/O, v.v.
-----------------	---	---

Hình 4. 16: Ma trận mô hình thử nghiệm

Bên trong mỗi mô hình (như trong Hình 4-16), thử nghiệm có thể được tiếp tục chia nhỏ để bao gồm các *thử nghiệm unit/module* (thử nghiệm gia tăng các yếu tố riêng lẻ trong hệ thống), *thử nghiệm tính tương thích* (thử nghiệm rằng các phần tử không gây ra các vấn đề với các phần tử khác trong hệ thống), *thử nghiệm sự tích hợp* (thử nghiệm gia tăng các yếu tố tích hợp), *thử nghiệm hệ thống* (thử nghiệm toàn bộ hệ thống nhúng với tất cả các yếu tố tích hợp), *thử nghiệm hồi quy* (quay lại các thử nghiệm đã được thông qua trước đó sau khi sửa đổi hệ thống), và *thử nghiệm sản xuất* (thử nghiệm để đảm bảo rằng việc sản xuất hệ thống không đưa ra lỗi).

Từ các loại thử nghiệm này, một tập các hiệu quả của các trường hợp thử nghiệm có thể nhận được từ việc kiểm tra rằng một yếu tố và/hoặc hệ thống đáp ứng các đặc tả kỹ thuật kiến trúc, cũng như xác nhận rằng các yếu tố và/hoặc hệ thống đáp ứng các yêu cầu thực tế, có thể hoặc có thể không được phản ánh một cách chính xác hoặc ở tất cả trong

tài liệu. Một khi các trường hợp thử nghiệm đã được hoàn thành và các thử nghiệm được chạy, các kết quả được xử lý có thể thay đổi tùy thuộc vào sự tổ chức như thế nào, nhưng thường khác nhau giữa những cái không chính thức, nơi thông tin được trao đổi mà không cần bất kỳ quy trình cụ thể nào theo sau, và sự xem xét lại thiết kế chính thức, hoặc sự xem xét ngang hàng nơi mà các nhà phát triển thành viên trao đổi các yếu tố để thử nghiệm, walkthroughs nơi các kỹ sư chịu trách nhiệm chính thức duyệt các sơ đồ nguyên lý và mã nguồn, kiểm tra nơi mà ai đó khác các kỹ sư chịu trách nhiệm sẽ thực hiện duyệt thiết kế... Các phương pháp thử nghiệm cụ thể và các mẫu cho các trường hợp thử nghiệm, cũng như toàn bộ quá trình thử nghiệm, đã được định nghĩa trong một số các tiêu chuẩn thử nghiệm và đảm bảo chất lượng công nghiệp thông dụng, bao gồm các tiêu chuẩn đảm bảo chất lượng ISO9000, Capability Maturity Model (CMM), và ANSI / IEEE 829.

Cuối cùng, như với gỡ lỗi, có nhiều loại tự động hóa, công cụ thử nghiệm và kỹ thuật mà có thể trợ giúp trong tốc độ, tính hiệu quả, và tính chính xác của việc thử nghiệm các yếu tố khác nhau. Chúng bao gồm các công cụ tải, công cụ ứng suất, máy phun nhiễu, máy phát tiếng ồn, các công cụ phân tích, ghi và phát lại macro, và macro được lập trình, bao gồm các công cụ được liệt kê trong Bảng 4-3.

Câu hỏi ôn tập

1. Liệt kê các bước trong quá trình thiết kế hệ thống nhúng
2. Mô tả các bước trong quá trình thiết kế hệ thống nhúng
3. Liệt kê các bước trong quá trình cài đặt và thử nghiệm hệ thống nhúng
4. Mô tả các bước trong quá trình cài đặt và thử nghiệm hệ thống nhúng

PDF

CHƯƠNG 5: PHÁT TRIỂN HỆ THỐNG NHÚNG DỰA TRÊN HỆ VI XỬ LÝ NHÚNG

5.1 Giới thiệu chung

Trong các hệ thống nhúng hiện nay, vi xử lý lõi ARM được sử dụng rộng rãi nhất. Trong chương này, các kiến thức căn bản về kiến trúc vi xử lý lõi ARM và tập lệnh ARM được giới thiệu. Sau đó, các kiến thức căn bản về việc thiết kế các thành phần căn bản của hệ thống nhúng được đề cập. Phần cuối sẽ tập trung vào thiết lập hệ điều hành nhúng trên nền ARM.

5.2 Kiến trúc của hệ vi xử lý nhúng ARM

Lõi ARM

Kiến trúc của ARM được thiết kế chuyên dụng cho các ứng dụng nhúng. Do đó, hiện thực hóa chip ARM được thiết kế để cho các ứng dụng nhỏ nhưng có hiệu năng cao, tiêu thụ ít năng lượng.

Lõi ARM được thiết kế theo kiến trúc RISC, nó chứa các kiến trúc RISC chung

- Các thanh ghi đồng dạng.
- Kiến trúc dạng Load-Store. Các địa chỉ Load/Store chỉ được xác định từ nội dung thanh ghi và các chỉ lệnh
- Các kiểu đánh địa chỉ đơn giản.
- Các chỉ lệnh có độ dài cố định và đồng dạng, do đó đơn giản hóa việc giải mã các câu lệnh
- Thay vì chỉ dùng 1 chu kỳ xung nhịp cho tất cả các chỉ lệnh, ARM thiết kế để sao cho tối giản số chu kỳ xung nhịp cho một chỉ lệnh, do đó tăng được sự phức tạp cho các chỉ lệnh đơn lẻ.

Ngoài ra, kiến trúc ARM có thể cung cấp:

- Điều khiển cả khối logic số học (ALU) và bộ dịch chuyển (shifter) trong các lệnh xử lý dữ liệu để tối đa hóa việc sử dụng ALU và bộ dịch chuyển.
- Các chế độ địa chỉ tự tăng hoặc tự giảm để tối ưu hóa các lệnh vòng lặp
- Các lệnh nhân Load/Store để tối đa dữ liệu truyền qua.

Nhờ các tối ưu trên nền kiến trúc RISC căn bản, lõi ARM có thể đạt được một sự cân bằng giữa hiệu năng cao, kích thước mã nguồn ít, công suất tiêu thụ thấp.

Thanh ghi và các chế độ hoạt động

Lõi ARM có 37 thanh ghi trong đó có 31 thanh ghi đa dụng. Tuy nhiên tại một thời điểm chỉ có 16 thanh ghi đa dụng và 2 thanh ghi trạng thái hiển thị. Các thanh ghi khác ở dạng ẩn, chỉ hiển thị ở một số chế độ hoạt động riêng.

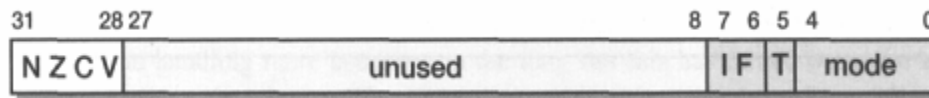
Các thanh ghi đa dụng có thể dùng để lưu dữ liệu hoặc địa chỉ. Các thanh ghi này được đánh dấu bằng ký hiệu r. Tất cả các thanh ghi đều là 32 bit.

Trong các thanh ghi đa dụng trên, có 3 thanh ghi còn được dùng để các chức năng hoặc nhiệm vụ đặc biệt riêng: r13, r14, r15.

- Thanh ghi r13 được dùng làm stack pointer (SP)
- Thanh ghi r14 được gọi là thanh ghi kết nối (LR) chứa địa chỉ quay lại của chương trình khi chương trình chạy một hàm con.
- Thanh ghi r15 là bộ đếm chương trình (pc) và chứa địa chỉ của lệnh tiếp theo.

Hai thanh ghi trạng thái bao gồm thanh ghi trạng thái chương trình hiện tại (CPSR) dùng để giám sát các trạng thái hoạt động hiện tại và thanh ghi trạng thái chương trình lưu (SPSR) dùng để lưu trữ giá trị của CPSR khi có một trường hợp ngoại lệ xảy ra.

Thanh ghi trạng thái chương trình hiện tại (CPSR) : CPSR có 4 trường, mỗi trường có 8 bit: cờ, trạng thái, mở rộng và điều khiển. Hiện tại phần trạng thái và mở rộng được dự trữ cho các thiết kế tương lai.



Hình 5. 1: Cấu trúc của thanh ghi trạng thái chương trình hiện tại

Các cờ của CPSR như sau:

- N: Negative- cờ này được bật khi bit cao nhất của kết quả xử lý ALU bằng 1.
- Z: Zero- cờ này được bật khi kết quả cuối cùng trong ALU bằng 0.
- C: Carry- cờ này được bật khi kết quả cuối cùng trong ALU lớn hơn giá trị 32bit và tràn.
- V: Overflow-cờ báo tràn sang bit dấu.

Các thanh ghi hiện

Abort Mode	r0
	r1
	r2
	r3
	r4
	r5
	r6
	r7
	r8
	r9
	r10
	r11
	r12
	r13
	r14
	r15
	cpsr
	spsr

Các thanh ghi ẩn

User	FIQ	IRQ	SVC	Undef
	r8			
	r9			
	r10			
	r11			
	r12			
r13	r13	r13	r13	r13
r14	r14	r14	r14	r14
	sdsr	sdsr	sdsr	spsr

Hình 5. 2: Các thanh ghi của lõi ARM

Chế độ hoạt động của bộ VXL sẽ xác định thanh ghi nào hoạt động và quyền truy cập tới thanh ghi *cpsr*. Mỗi chế độ hoạt động của bộ VXL sẽ là chế độ đặc quyền và không đặc quyền: chế độ đặc quyền cho phép đọc và ghi tới thanh ghi *cpsr*. Người lại chế độ không đặc quyền chỉ cho phép đọc trường điều khiển của *cpsr* nhưng vẫn cho phép đọc và ghi tới các cờ điều kiện.

Có bảy (7) chế độ hoạt động của bộ VXL: sáu chế độ đặc quyền (abort, fast interrupt request, interrupt request, supervisor, system, and undefined) và một chế độ không đặc quyền (user). Sơ đồ các thanh ghi các chế độ như hình dưới.

User	FIQ	IRQ	SVC	Undef	Abort
r0	User mode r0-r7, r15, và cpsr	User mode r0-r12, r15, và cpsr	User mode r0-r12, r15, và cpsr	User mode r0-r12, r15, và cpsr	User mode r0-r12, r15, và cpsr
r1					
r2					
r3					
r4					
r5					
r6					
r7					
r8	r8				
r9	r9				
r10	r10				
r11	r11				
r12	r12				
r13	r13	r13	r13	r13	r13
r14	r14	r14	r14	r14	r14
r15					
cpsr					
	spsr	spsr	spsr	spsr	spsr

Hình 5. 3: Các chế độ hoạt động và các thanh ghi

Hoạt động của các chế độ như sau:

- Bộ VXL hoạt động ở chế độ Abort khi bộ VXL không thể truy cập bộ nhớ.
- Bộ VXL hoạt động ở chế độ *interrupt request* (IRQ) và *fast interrupt request* (FIQ) tương ứng với hai mức ngắt của chip ARM.
- Bộ VXL hoạt động ở chế độ Supervisor khi sau khi hệ thống khởi động (reset) và khi nhân của hệ điều hành hoạt động.
- Bộ VXL hoạt động ở chế độ System khi hệ thống có thể truy cập và đọc, ghi toàn bộ thanh ghi *cpsr*. Đây là một chế độ đặc biệt của chế độ User.
- Bộ VXL chuyển sang chế độ Undefined khi bộ VXL gặp một lệnh không xác định hoặc không được hỗ trợ.
- Bộ VXL hoạt động ở chế độ User là để chạy các chương trình và các ứng dụng thông thường.

Đối với từng chế độ, có thể có các thanh ghi riêng cho từng chế độ đó.

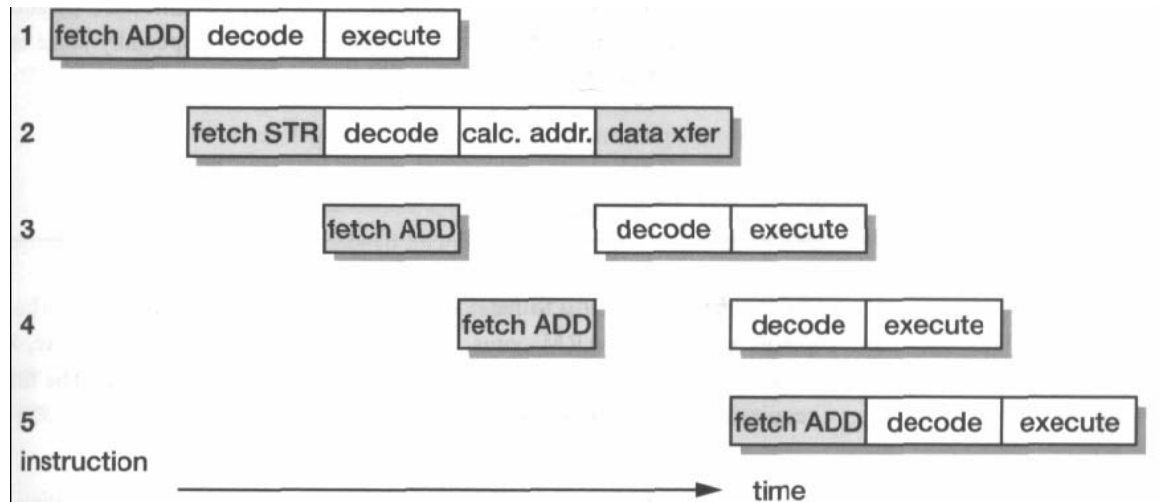
Pipeline

Cách tổ chức của nhân ARM không thay đổi nhiều trong khoảng 1983-1995: đến ARM7-sử dụng dòng chảy lệnh sử dụng 3 tác vụ. Từ 1995 trở về sau, đã xuất hiện một vài nhân ARM mới được giới thiệu có dòng chảy lệnh sử dụng 5 tác vụ. Các dòng ARM

sau này có thể có dòng chảy lệnh 6 tác vụ (ARM10), 9 tác vụ (ARM11) hoặc 13 tác vụ (ARM Cortex).

Dòng chảy lệnh 3 tác vụ:

Bao gồm các tác vụ sau: Fetch-decode-Execute (nhận lệnh, giải mã, thực thi)



Hình 5. 4: Dòng chảy lệnh 3 tác vụ áp dụng trong trường hợp 1 lệnh có nhiều chu kỳ máy

Dòng chảy lệnh 5 tác vụ:

Bao gồm các tác vụ : Fetch-decode-execute-buffer/data-write back

Dòng chảy lệnh 6 tác vụ:

- Fetch và đoán nhánh
- Issue
- Decode và đọc thanh ghi
- Execute shift và ALU, hoặc tính địa chỉ, hoặc nhân
- Truy cập bộ nhớ, hoặc nhân
- Ghi vào thanh ghi

Dòng chảy lệnh 9 tác vụ:

- Ba tác vụ Fetch.
- Một tác vụ Decode
- Một tác vụ Issue
- Bốn tác vụ integer execution pipeline

Cấu trúc bus:

Khi muốn thiết kế riêng một vi điều khiển, cũng như thiết kế một hệ thống nhúng, ngoài quan tâm đến các chức năng các khối, bus kết nối các khối lại với nhau cũng là một vấn đề nên được xem xét kỹ lưỡng. Các hệ thống nhúng sử dụng các kỹ thuật bus khác với thiết kế trên các PC dựa trên họ x86. Một dạng bus PC thông dụng nhất đó là bus PCI kết nối các thiết bị như card đồ họa, bộ điều khiển đĩa cứng,... đến bus bộ vi xử lý x86. Đây là loại bus ngoài (off-chip), nghĩa là nó kết nối các thiết bị bên ngoài tới chip. Ngược lại, các thiết bị nhúng sử dụng bus on-chip nằm ở bên trong chip và cho phép các thiết bị ngoại vi được kết nối với lõi ARM.

Có hai loại thiết bị khác nhau gắn với bus: là master và slave. Trong đó lõi ARM là master – có khả năng điều khiển quá trình truyền dữ liệu với thiết bị khác trên cùng bus, và các thiết bị ngoại vi – các slave chỉ có thể đáp ứng với sự điều khiển một thiết bị master.

Kiến trúc bus vi điều khiển tiên tiến gọi tắt là AMBA (Advanced Microcontroller Bus Architecture) được giới thiệu năm 1996 và đã được sử dụng làm kiến trúc bus on-chip dành cho các bộ xử lý ARM. Các bus AMBA đầu tiên được giới thiệu là ARM System Bus (ASB) và ARM Peripheral Bus (APB). Một thiết kế bus khác của ARM được giới thiệu sau đó là ARM High Performance Bus (AHB). Với AMBA, các nhà thiết kế ngoại vi có thể sử dụng lại cùng thiết kế trên nhiều project khác nhau.

Bởi vì có nhiều các ngoại vi được thiết kế với giao diện AMBA nên các nhà thiết kế phần cứng có nhiều lựa chọn đối với các ngoại vi đã được kiểm thử nhằm sử dụng trong thiết kế của họ.

AHB cung cấp băng thông dữ liệu cao hơn so với ASB bởi nó được thiết kế dựa trên lược đồ bus ghép kênh tập trung hơn là thiết kế bus hai chiều ASB. Sự thay đổi này cho phép bus AHB chạy ở tốc độ clock cao hơn và là bus ARM đầu tiên hỗ trợ độ rộng bus lên tới 64 và 128bit. ARM cũng đã giới thiệu 2 biến thể của bus AHB là Multi-layer AHB và AHB-Lite. Ngược lại với bus AHB ban đầu chỉ cho phép một master tích cực trên bus tại mọi thời điểm, Multi-layer AHB cho phép nhiều master cùng tích cực một lúc. AHB-Lite là một tập con của bus AHB, nó bị giới hạn chỉ có một master trên bus. Bus này được phát triển cho các thiết kế không yêu cầu đầy đủ chức năng của bus AHB chuẩn.

Tập lệnh ARM

VXL ARM sử dụng cấu trúc load-store. Điều đó có nghĩa là: tất cả các lệnh đều được thực hiện trên thanh ghi. Các lệnh ARM thường có 2 đến 3 toán tử.

Mặc dù các phiên bản kiến trúc ARM khác nhau hỗ trợ các tập lệnh khác nhau, các phiên bản mới thường tương thích ngược với các tập lệnh cũ.

Danh sách các lệnh của ARM

Các lệnh xử lý dữ liệu

Các lệnh xử lý dữ liệu thực thi các phép tính đối với dữ liệu trên các thanh ghi. Các lệnh đó bao gồm chuyển dữ liệu, các phép tính số học, logic, các phép so sánh và phép nhân. Nếu các lệnh này có thêm S ở cuối, nó sẽ cập nhật cờ trên thanh ghi CPRS. Các lệnh dịch chuyển và phép toán logic cập nhật cờ Carry C, cờ Negative N, và cờ Zero Z.

Các lệnh dịch chuyển

MOVE là lệnh đơn giản nhất trong các lệnh của ARM. Lệnh thay copy giá trị N đến thanh ghi đích Rd .

Cú pháp : <instruction> {<cond>} {S} Rd, N

Có 2 lệnh dịch chuyển

MOV: Chuyển một giá trị 32 bit đến thanh ghi ($Rd=N$)

MVN: Chuyển giá trị đảo 32 bit đến thanh ghi ($Rd=\sim N$)

Ví dụ:

Trước: $r5=5$
 $r7=8$
MOV $r7, r5$; let $r7 = r5$

Sau $r5=5$
 $r7=5$

MOVCS $R0, R1$; carry SET thì $R0:=R1$

MOVS $R0, \#0$; $R0:=0$ $Z=1, N=0$
;C, V không thay đổi

Các lệnh số học

Các lệnh số học thực hiện cộng và trừ các giá trị 32 bit có dấu và không có dấu. Kết quả là 32 bit và được đặt trong một thanh ghi. Cấu trúc của lệnh có 3 địa chỉ.

Cú pháp : <instruction> {<cond>} {S} Rd, Rn, N

ADD $R0, R1, R2$	; $R0 = R1+R2$
ADC $R0, R1, R2$	; $R0 = R1+R2+C$
SUB $R0, R1, R2$	; $R0 = R1-R2$
SBC $R0, R1, R2$	; $R0 = R1-R2+C-1$
RSB $R0, R1, R2$	; $R0 = R2-R1$
RSC $R0, R1, R2$	; $R0 = R2-R1+C-1$

Ví dụ:

Trước:

```
r0 = 0x00000000  
r1 = 0x00000002  
r2 = 0x00000001  
SUB r0, r1, r2
```

Sau:

```
r0 = 0x00000001
```

Trước:

```
r0 = 0x00000000  
r1 = 0x00000077  
RSB r0, r1, #0 ; Rd = 0x0 - r1 (giá trị âm của r1)
```

Sau:

```
r0 = -r1 = 0xfffff8
```

Các lệnh logic

Các lệnh logic thực hiện các phép toán logic theo bit trên các thanh ghi

Cú pháp: <instruction>{<cond>}{S} Rd, Rn, N

```
AND R0, R1, R2    ;R0 = R1 and R2  
ORR R0, R1, R2    ; R0 = R1 or R2  
EOR R0, R1, R2    ; R0 = R1 xor R2  
BIC R0, R1, R2    ; R0 = R1 and (~R2)
```

Ví dụ:

Trước;

```
r0 = 0x00000000  
r1 = 0x02040608  
r2 = 0x1030507056  
ORR r0, r1, r2
```

Sau:

```
r0 = 0x12345678
```

Trước:

```
r1 = 0b1111  
r2 = 0b0101  
BIC r0, r1, r2
```

Sau:

```
r0 = 0b1010
```


Trước:

r1=0x11111111

r2=0x01100101

BIC r0, r1, r2

Sau:

R0=0x10011010

Các lệnh so sánh

Các lệnh so sánh dùng để so sánh hoặc kiểm tra một thanh ghi với một giá trị 32 bit. Các lệnh này không ảnh hưởng đến các thanh ghi, chỉ thay đổi cập nhật các bit cờ trên thanh ghi CPRS. Các lệnh này không cần thêm S

Cấu trúc: <instruction>{<cond>} Rn, N

CMP R1, R2 ; set cờ cho kết quả R1-R2

CMN R1, R2 ; set cờ cho kết quả R1+R2

TST R1, R2 ; set cờ cho kết quả R1 and R2

TEQ R1, R2 ; set cờ cho kết quả R1 xor R2

Các lệnh nhân

Các lệnh nhân thực hiện phép nhân giá trị trên hai thanh ghi và có thể thực hiện cộng dồn với một thanh ghi khác. Kết quả cuối cùng được ghi vào thanh ghi đích hoặc hai thanh ghi nếu kết quả là 64 bits.

Cấu trúc:

MLA{<cond>}{S} Rd, Rm, Rs, Rn

MUL{<cond>}{S} Rd, Rm, Rs

MUL R0, R1, R2 ; R0 = (R1xR2)[31:0] (32 bits)

MLA R4, R3, R2, R1 ; R4 = R3xR2+R1

Các lệnh rẽ nhánh

Các lệnh rẽ nhánh thay đổi chu trình chạy của chương trình, hoặc dùng để gọi một hàm con khác. Việc này làm thay đổi giá trị thanh ghi chương trình PC làm thanh ghi PC trở về một địa chỉ mới

Cấu trúc:

B{<cond>} label

BL{<cond>} label

BX{<cond>} Rm

BLX{<cond>} label | Rm

B label ;chuyển đến địa chỉ label, pc=label

BL label: rẽ nhánh có liên kết, pc=label, địa chỉ quay lại được lưu vào thanh ghi lr (R14)

BX Rm; rẽ nhánh trao đổi, pc = Rm & 0xffffffe, T = Rm & 1 (T là Thumb bit trên CPSR)

BLX

Ví dụ:

B label

...

label: ... ; chương trình sẽ chuyển đến địa chỉ label

MOV R0, #0

loop: ...

ADD R0, R0, #1

CMP R0, #10

BNE loop ; vòng lặp lại loop nếu R0 khác 10

Ví dụ:

BL sub ; gọi sub

CMP R1, #5 ; quay lại địa chỉ này

MOVEQ R1, #0

...

sub: ... ;

...

MOV PC, LR ;quay lại

Các lệnh chuyển dữ liệu Load- Store

Các lệnh chuyển dữ liệu load-Store chuyển dữ liệu giữa bộ nhớ và các thanh ghi trên CPU. Có ba loại lệnh load-store : chỉ dùng một thanh ghi, dùng nhiều thanh ghi hoặc trao đổi giữa thanh ghi và bộ nhớ

Load-store dùng một thanh ghi

Các lệnh load-store dùng một thanh ghi dùng để chuyển dữ liệu vào và ra khỏi thanh ghi. Các lệnh này hỗ trợ các kiểu dữ liệu có dấu và không có dấu, kích cỡ có thể là 32 bit, 16 bit hoặc 8 bit (byte)

Cấu trúc:

<LDR|STR>{<cond>} {B} Rd,addressing

LDR{<cond>} SB|H|SH Rd, addressing

STR{<cond>}H Rd, addressing

LDR R0, [R1] ;R0 := mem32[R1]
STR R0, [R1] ;mem32[R1] := R0

LDR, LDRH, LDRB lần lượt cho 32, 16, 8 bit
STR, STRH, STRB lần lượt cho 32, 16, 8 bit

Ví dụ:

; Nạp thanh ghi r0 với nội dung ở địa chỉ do r1 trỏ đến
LDR r0, [r1] ; = LDR r0, [r1, #0]
; Lưu nội dung của thanh ghi r0 vào địa chỉ do r1 trỏ đến
STR r0, [r1] ; = STR r0, [r1, #0]

Các chế độ đánh địa chỉ

VXL ARM hỗ trợ ba chế độ đánh địa chỉ:

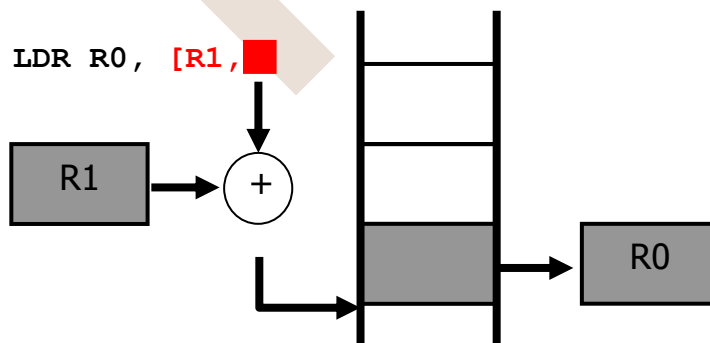
- Pre-indexed addressing (**LDR R0, [R1, #4]**) without a writeback

Ví dụ:

Trước

r0 = 0x00000000
r1 = 0x00090000
mem32[0x00009000] = 0x01010101
mem32[0x00009004] = 0x02020202

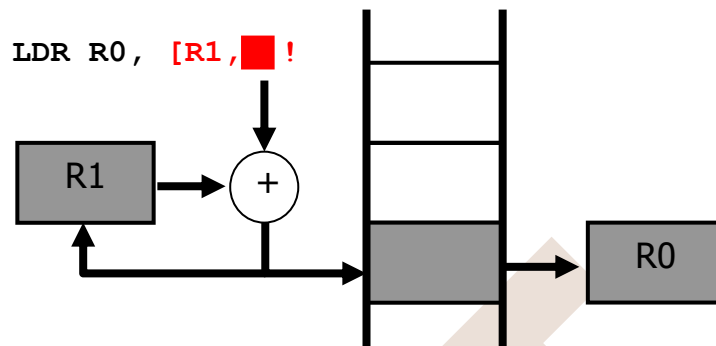
LDR R0, [R1, #4] ; R0=mem[R1+4]
; R1 unchanged
r0 = 0x02020202
r1 = 0x00090000



- Auto-indexing addressing (**LDR R0, [R1, #4]!**) calculation before accessing with a writeback

Ví dụ:

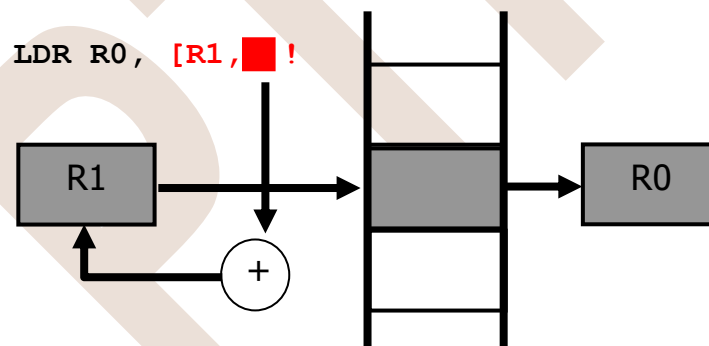
```
LDR R0, [R1, #4]! ; R0=mem[R1+4]
; R1=R1+4
r0 = 0x02020202
r1 = 0x00009004
```



- Post-indexed addressing (**LDR R0, [R1], #4**) calculation after accessing with a writeback

Ví dụ :

```
LDR R0, R1, #4 ; R0=mem[R1]
; R1=R1+4
r0 = 0x01010101
r1 = 0x00009004
```



Load-store dùng nhiều thanh ghi

Khi cần chuyển dữ liệu giữa nhiều thanh ghi và bộ nhớ hoặc CPU, các lệnh load-store cho nhiều thanh ghi có thể được dùng. Quá trình chuyển dữ liệu từ địa chỉ gốc do thanh ghi R_n chỉ đến. Việc sử dụng các lệnh này sẽ có hiệu quả hơn khi cần chuyển một khối dữ liệu giữa bộ nhớ và thanh ghi hoặc giữa các vị trí trong bộ nhớ

Cấu trúc:

$\langle \text{LDM|STM} \rangle \{ \langle \text{cond} \rangle \} \langle \text{addressing mode} \rangle R_n \{ ! \}, \langle \text{registers} \rangle \{ ^ \}$

LDM Nạp nhiều thanh ghi
STM Lưu nhiều thanh ghi

Chế độ đánh địa chỉ:

Đuôi ý nghĩa

IA tăng địa chỉ sau khi thực hiện

IB tăng địa chỉ trước khi thực hiện

DA giảm địa chỉ sau khi thực hiện

DB giảm địa chỉ trước khi thực hiện

IA	increment after	Rn	$Rn + 4 * N - 4$	$Rn + 4 * N$
IB	increment before	$Rn + 4$	$Rn + 4 * N$	$Rn + 4 * N$
DA	decrement after	$Rn - 4 * N + 4$	Rn	$Rn - 4 * N$
DB	decrement before	$Rn - 4 * N$	$Rn - 4$	$Rn - 4 * N$

Ví dụ :

Trước:

$\text{mem32}[0x80018] = 0x03$

$\text{mem32}[0x80014] = 0x02$

$\text{mem32}[0x80010] = 0x01$

$r0 = 0x00080010$

$r1 = 0x00000000$

$r2 = 0x00000000$

$r3 = 0x00000000$

Address pointer	Memory address	Data	
	0x80020	0x00000005	
	0x8001c	0x00000004	
	0x80018	0x00000003	$r3 = 0x00000000$
	0x80014	0x00000002	$r2 = 0x00000000$
$r0 = 0x80010 \rightarrow$	0x80010	0x00000001	$r1 = 0x00000000$
	0x8000c	0x00000000	

LDMIA $r0!$, $\{r1-r3\}$

Sau:

$r0 = 0x0008001c$

$r1 = 0x00000001$
 $r2 = 0x00000002$
 $r3 = 0x00000003$

Address pointer	Memory address	Data	
	0x80020	0x00000005	
$r0 = 0x8001c \rightarrow$	0x8001c	0x00000004	
	0x80018	0x00000003	$r3 = 0x00000003$
	0x80014	0x00000002	$r2 = 0x00000002$
	0x80010	0x00000001	$r1 = 0x00000001$
	0x8000c	0x00000000	

Nếu dùng lệnh LDMIB

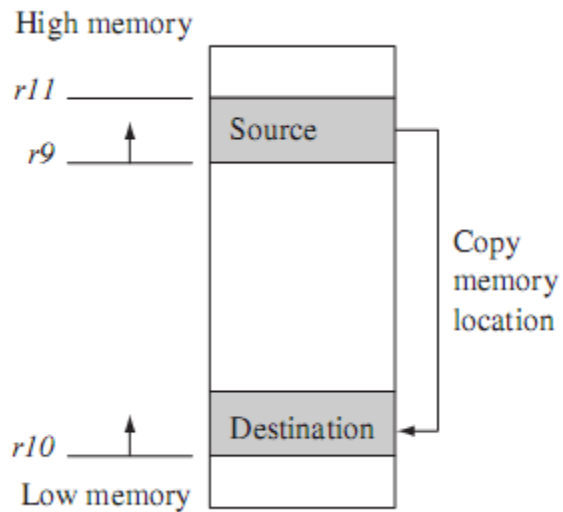
Address pointer	Memory address	Data	
	0x80020	0x00000005	
$r0 = 0x8001c \rightarrow$	0x8001c	0x00000004	
	0x80018	0x00000003	$r3 = 0x00000003$
	0x80014	0x00000002	$r2 = 0x00000002$
	0x80010	0x00000001	$r1 = 0x00000001$
	0x8000c	0x00000000	

Ví dụ : Ứng dụng các lệnh trên để copy một khối dữ liệu

R9: địa chỉ nguồn

R10: địa chỉ đích

R11: địa chỉ cuối của nguồn



```

loop:  LDMIA R9!, {R0-R7} ; nạp 32 byte từ nguồn và cập nhật con trỏ R9
        STMIA R10!, {R0-R7}; lưu 32 byte từ nguồn và cập nhật con trỏ R10
        CMP R9, R11; so sánh địa chỉ copy và địa chỉ cuối cùng
        BNE loop

```

Các lệnh này có thể ứng dụng trong stack

Trao đổi dữ liệu của thanh ghi với bộ nhớ (swap)

Lệnh swap đổi chỗ nội dung của bộ nhớ và thanh ghi.

Cấu trúc:

```

SWP{B}{<cond>} Rd,Rm,[Rn]
SWP đổi nội dung 32 bits giữa thanh ghi và bộ nhớ
    tmp =mem32[Rn]
    mem32[Rn] = Rm
    Rd = tmp
SWPB đổi nội dung 8 bits giữa thanh ghi và bộ nhớ
    tmp =mem8[Rn]
    mem8[Rn] = Rm
    Rd = tmp

```

Ví dụ

Trước:

```

mem32[0x9000] = 0x12345678
r0 = 0x00000000
r1 = 0x11112222
r2 = 0x00009000
SWP r0, r1, [r2]

```

Sau:

```

mem32[0x9000] = 0x11112222
r0 = 0x12345678
r1 = 0x11112222
r2 = 0x00009000

```

Tập lệnh Thumb

Một tính năng quan trọng của ARM là hỗ trợ tập lệnh Thumb. Tập lệnh Thumb bao gồm các lệnh 16 bit. Do đó chúng chiếm ít bộ nhớ hơn và có hiệu năng cao hơn đối với các dữ liệu 16 bit. Đối với các ứng dụng nhúng cần tối ưu bộ nhớ, mật độ lệnh rất quan trọng.

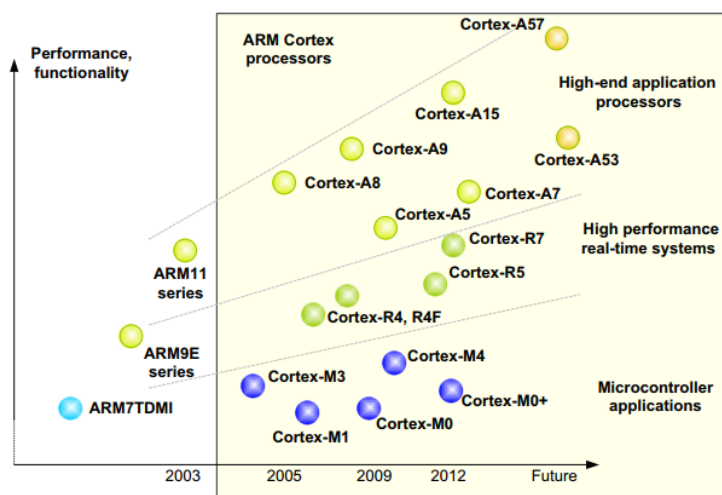
5.3 Giới thiệu về dòng vi xử lý ARM Cortex và ARM Cortex M3

Các dòng ARM Cortex

ARM Cortex là một dòng vi xử lý hoạt động với cấu trúc lệnh 32/64 bit và là một trong những lõi phổ biến trong thế giới hệ thống nhúng. Vi điều khiển ARM Cortex có thể được chia làm ba dòng chính như sau:

- **ARM Cortex-A (A- Application):** Là thế hệ các dòng vi xử lý được ứng dụng trong các sản phẩm yêu cầu thực hiện các tính toán phức tạp như xử lý các hệ điều hành (Linux, Android, Tizen, IOS,...). Lõi ARM Cortex-A được tìm thấy trong hầu hết các thiết bị di động hiện nay như: Smartphone, Tablet,...
- **ARM Cortex-M (M- eMbedded):** Là một lõi vi xử lý có thể mở rộng, tương thích nhiều ứng dụng, tiết kiệm năng lượng và dễ sử dụng cho các ứng dụng về thiết kế hệ thống nhúng với chi phí thấp. Lõi ARM Cortex-M đang được thiết kế tối ưu về giá, năng lượng nhằm mục đích phù hợp với các ứng dụng như IoT (Internet of Things), kết nối, điều khiển động cơ, giám sát, các thiết bị tương tác người dùng, điều khiển tự động, thiết bị y tế,... Ở phần sau của giáo trình sẽ tập trung vào hướng dẫn lập trình một dòng vi điều khiển dựa trên lõi ARM Cortex-M đó là STM32F1 (Lõi ARM Cortex-M3).
- **ARM Cortex-R (Real Time):** Là lõi vi xử lý được thiết kế cho các ứng dụng đòi hỏi hiệu năng tính toán cao đáp ứng trong các ứng dụng hệ thống nhúng yêu cầu về thời gian thực như: Độ tin cậy cao, khả năng đáp ứng tốt, khả năng chịu lỗi, bảo trì và đáp ứng thời gian thực

ARM Cortex-M3



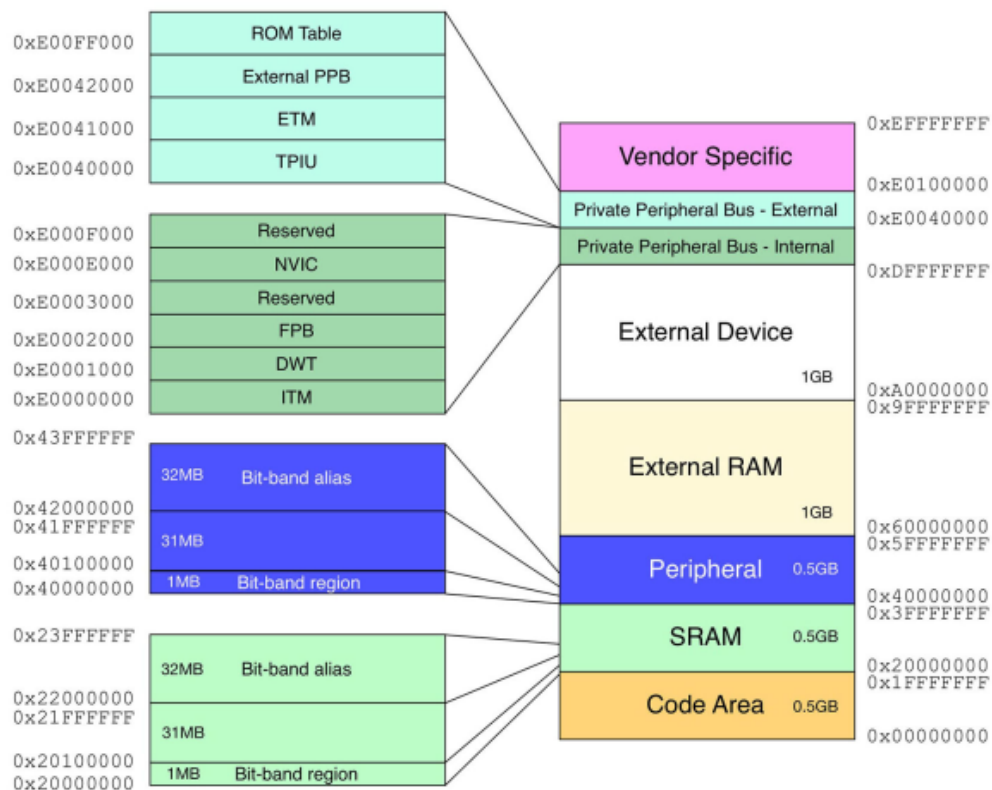
ARM Cortex M3 là một dòng vi xử lý được thiết kế bởi ARM. ARM Cortex M3 được thiết kế dựa trên nền tảng ARMv7-M. Thế hệ ARM Cortex-M3 đầu tiên được công bố

vào năm 2005 và dòng IC đầu tiên dựa trên lõi ARM Cortex-M3 được đưa ra thị trường năm 2006. Lõi ARM-M3 hoạt động trên kiến trúc 32 bit và hỗ trợ chế độ hoạt động của tập lệnh Thumb và Thumb-2.

ARM Cortex M3 có một số đặc tính sau:

- Cơ chế đường lệnh (pipeline) 3 giai đoạn.
- Kiến trúc Harvard: Bộ nhớ và dữ liệu sử dụng chung không gian địa chỉ nhớ.
- 32-bit đánh địa chỉ: Hỗ trợ tối đa 4GB bộ nhớ.
- Bus nội được thiết kế dựa trên ARM AMBA (Advanced Microcontroller Bus Architecture) Technology.
- Hỗ trợ hoạt động của nhiều hệ điều hành: System tick timer, Ngăn xếp ản.
- Hỗ trợ hoạt động ở chế độ ngủ để tiết kiệm năng lượng.
- Hỗ trợ chế độ hoạt động bảo vệ bộ nhớ bởi MPU (Memory Protection Unit).
- Hỗ trợ truy vấn đến từng bit theo dạng bit-band.

ARM Cortex-M3 và một số dòng vi điều khiển khác như M0, M0+, M4 đang được sử dụng rộng rãi trên thị trường các vi điều khiển thông dụng.



Hình 5. 5: Sơ đồ bộ nhớ của ARM M3

Vi điều khiển STM32F1

STM32 là một trong những dòng chip phổ biến của ST với nhiều họ thông dụng như F0,F1,F2,F3,F4..... STM32 có đang được thiết kế dựa trên 3 dòng sản phẩm chính như sau: Hiệu năng cao (*High-performance*), Phổ thông (*Mainstream*) và Tiết kiệm năng lượng (*Ultra Low-Power*).

Hiệu năng cao (*High-performance*): là các dòng vi điều khiển có hiệu năng tính toán cao, tốc độ hoạt động lớn và thường được sử dụng trong các ứng dụng đa phương tiện. Dòng này bao gồm các dòng vi điều khiển Cortex-M3/4F/7 với tần số hoạt động từ khoảng 120MHz đến 268MHz. Các dòng này điều hỗ trợ công nghệ ART Accelerator.

Phổ thông (*Mainstream*): Là dòng vi điều khiển được thiết kế với mục tiêu tối ưu về giá. Các sản phẩm này thường có giảm thấp hơn 1\$/pcs với tần số hoạt động trong khoảng từ 48MHz đến 72MHz.

Tiết kiệm năng lượng (*Ultra Low-Power*): Là nhóm các vi điều khiển hoạt động với tiêu chí tiết kiệm năng lượng tiêu thụ. Các nhóm vi điều khiển này đều có khả năng hoạt động với pin trong thời gian dài và hỗ trợ nhiều chế độ ngủ khác nhau. Với các sản phẩm của hãng STMicrochips, điển hình của nhóm này là dòng ARM Cortex M0+.

Dưới đây là bảng mô tả hiệu năng, chức năng, hoạt động của các dòng vi điều khiển trên:

Common core peripherals and architecture:	High-performance									
Communication peripherals: USART, SPI, I ² C	STM32F7 series – Very high performance with DSP and FPU (STM32F7x6)									
Multiple general-purpose timers	200 MHz Cortex-M7 CPU	Up to 1-Mbyte Flash	Up to 336-Kbyte SRAM	2x USB 2.0 OTG FS/HS	3x 16-bit advanced MC timer	2x CAN CEC FMC	SDIO 2x I ² S audio Camera IF	Crypto Ethernet IEEE 1588 2x SAI	LCD-TFT SDRAM I/F Quad SPI SPDIF input	STM32 F7
Integrated reset and brown-out warning	STM32F4 series – High performance with DSP and FPU (STM32F401/411/405-415/407-417/427-437/429-439 and STM32F446)									
Multiple DMA	Up to 180 MHz Cortex-M4 DSP/FPU	Up to 2-Mbyte Flash	Up to 256-Kbyte SRAM	2x USB 2.0 OTG FS/HS	3x 16-bit advanced MC timer	2x CAN CEC F(S)MC	SDIO 3x I ² S audio Camera IF	Crypto Ethernet IEEE 1588 2x SAI	LCD-TFT SDRAM I/F Quad SPI SDIF input	STM32 F4
2x watchdogs Real-time clock	STM32F2 series – High performance (STM32F2x5 and 2x7)									
Integrated regulator PLL and clock circuit	120 MHz Cortex-M3 CPU	Up to 1-Mbyte Flash	Up to 128-Kbyte SRAM	2x USB 2.0 OTG FS/HS	3x 16-bit advanced MC timer	2x CAN 2.0B FSMC	SDIO 2x I ² S audio Camera IF	Crypto Ethernet IEEE 1588		STM32 F2
Up to 3x 12-bit DAC	Mainstream									
Up to 4x 12-bit ADC (Up to 5 MSPS)	STM32F3 series – Mixed-signal with DSP (STM32F301/302/303/334/373/3x8)									
Main oscillator and 32 kHz oscillator	72 MHz Cortex-M4 with DSP/FPU	Up to 512-Kbyte Flash	Up to 80-Kbyte SRAM CCM-RAM	USB 2.0 FS	3x 16-bit advanced MC timer	CAN CEC FSMC	7x comparator 4x PGA	HR-Timer	3x 16-bit $\Sigma\Delta$ ADC	STM32 F3
Low-speed and high-speed internal RC oscillator	STM32F1 series – Mainstream (STM32F100/101/102/103 and 105-107)									
-40 to +85 °C and up to 125 °C operating temperature range	Up to 72 MHz Cortex-M3 CPU	Up to 1-Mbyte Flash	Up to 96-Kbyte SRAM	USB 2.0 OTG FS	2x 16-bit advanced MC timer	2x CAN CEC FSMC	SDIO 2x I ² S audio	Ethernet IEEE 1588		STM32 F1
Low voltage 2.0 to 3.6 V or 1.65/1.7 to 3.6 V (depending on series)	STM32F0 series – Entry-level (STM32F0x0/0x1/0x2 and 0x8)									
Temperature sensor	48 MHz Cortex-M0 CPU	Up to 256-Kbyte Flash	Up to 32-Kbyte SRAM 20-byte backup data	USB 2.0 FS device Crystal less	CAN CEC	DAC Comparator				STM32 F0
	Ultra-Low-Power									
	STM32L4 series – Ultra-Low-Power (STM32L4x6)									
	80 MHz Cortex-M4 CPU	Up to 1-Mbyte Flash	Up to 128-Kbyte SRAM	USB 2.0 OTG FS	2x 16-bit advanced MC timer	LCD up to 8x40	Op-amps comparator	FSMC SDIO CAN DFSDM	AES 256-bit T-RNG 2 x SAI	STM32 L4
	STM32L1 series – Ultra-Low-Power (STM32L100/151-152/162)									
	32 MHz Cortex-M3 CPU	Up to 512-Kbyte Flash	Up to 80-Kbyte SRAM	Up to 16-Kbyte EEPROM	USB 2.0 FS Device	LCD up to 8x40	Op-amps comparator	FSMC SDIO	AES 128-bit	STM32 L1
	STM32L0 series – Ultra-Low-Power (STM32L0x1/0x2/0x3)									
	32 MHz Cortex-M0+ CPU	Up to 192-Kbyte SRAM	Up to 20-Kbyte SRAM	Up to 6-Kbyte EEPROM	USB 2.0 FS device Crystal less	LCD 8x40 4x52	T-RNG comparator	LP Timer LP UART LP 12-bit ADC	AES 128-bit	STM32 L0

Hình 5. 6: hiệu năng, chức năng, hoạt động của các dòng vi điều khiển STM32 Cortex M

STM32F103 thuộc họ F1 với lõi là ARM Cortex M3. STM32F103 là vi điều khiển 32 bit, tốc độ tối đa là 72Mhz. Giá thành cũng khá rẻ so với các loại vi điều khiển có chức năng tương tự. Mạch nạp cũng như công cụ lập trình khá đa dạng và dễ sử dụng.

Một số ứng dụng chính: dùng cho driver để điều khiển ứng dụng, điều khiển ứng dụng thông thường, thiết bị cầm tay và thuốc, máy tính và thiết bị ngoại vi chơi game, GPS cơ bản, các ứng dụng trong công nghiệp, thiết bị lập trình PLC, biến tần, máy in, máy quét, hệ thống cảnh báo, thiết bị liên lạc nội bộ...

Phần mềm lập trình: có khá nhiều trình biên dịch cho STM32 như IAR Embedded Workbench, Keil C...

Thư viện lập trình: có nhiều loại thư viện lập trình cho STM32 như: STM32snippets, STM32Cube LL, STM32Cube HAL, Standard Peripheral Libraries, Mbed core. Mỗi thư viện đều có ưu và khuyết điểm riêng, ở đây mình xin phép sử dụng Standard Peripheral Libraries vì nó ra đời khá lâu và khá thông dụng, hỗ trợ nhiều ngoại vi và cũng dễ hiểu rõ bản chất của lập trình.

Cấu hình chi tiết của STM32F103 như sau:

ARM 32-bit Cortex M3 với clock max là 72Mhz.

Bộ nhớ:

- 64 kbytes bộ nhớ Flash(bộ nhớ lập trình).
- 20kbytes SRAM.

Clock, reset và quản lý nguồn.

- Điện áp hoạt động 2.0V -> 3.6V.
- Power on reset(POR), Power down reset(PDR) và programmable voltage detector (PVD).
- Sử dụng thạch anh ngoài từ 4Mhz -> 20Mhz.
- Thạch anh nội dùng dao động RC ở mode 8Mhz hoặc 40khz.
- Sử dụng thạch anh ngoài 32.768khz được sử dụng cho RTC.

Trong trường hợp điện áp thấp:

- Có các mode :ngủ, ngừng hoạt động hoặc hoạt động ở chế độ chờ.
- Cấp nguồn ở chân Vbat bằng pin để hoạt động bộ RTC và sử dụng lưu trữ data khi mất nguồn cấp chính.

2 bộ ADC 12 bit với 9 kênh cho mỗi bộ.

- Khoảng giá trị chuyển đổi từ 0 – 3.6V.
- Lấy mẫu nhiều kênh hoặc 1 kênh.
- Có cảm biến nhiệt độ nội.

DMA: bộ chuyển đổi này giúp tăng tốc độ xử lý do không có sự can thiệp quá sâu của CPU.

- 7 kênh DMA.
- Hỗ trợ DMA cho ADC, I2C, SPI, UART.

7 timer.

- 3 timer 16 bit hỗ trợ các mode IC/OC/PWM.
- 1 timer 16 bit hỗ trợ để điều khiển động cơ với các mode bảo vệ như ngắt input, dead-time..

- 2 watchdog timer dùng để bảo vệ và kiểm tra lỗi.
- 1 sysTick timer 24 bit đếm xuống dùng cho các ứng dụng như hàm Delay....

Hỗ trợ 9 kênh giao tiếp bao gồm:

- 2 bộ I2C(SMBus/PMBus).
- 3 bộ USART(ISO 7816 interface, LIN, IrDA capability, modem control).
- 2 SPIs (18 Mbit/s).
- 1 bộ CAN interface (2.0B Active)
- USB 2.0 full-speed interface

Kiểm tra lỗi CRC và 96-bit ID.

Lập trình các thanh ghi

Đối với mỗi ngoại vi sẽ có một bộ điều khiển, bộ điều khiển này giao diện với chương trình phần mềm qua 1 hoặc nhiều thanh ghi. Bộ xử lý giao tiếp với các bộ điều khiển bằng việc đọc và ghi các bit lên các thanh ghi này thông qua các lệnh truyền 1 byte hay 1 từ tới 1 địa chỉ cổng I/O. Các lệnh I/O này sẽ kích khởi các đường bus để chọn đúng thiết bị và chuyển các bit tới hoặc đọc các bit từ một thanh ghi thiết bị. Các thanh ghi này được ánh xạ vào không gian địa chỉ của bộ vi xử lý. Kỹ thuật này được gọi là kỹ thuật I/O ánh xạ bộ nhớ - **memory-mapped I/O**.

Một thiết bị điển hình gồm 4 thanh ghi được gọi là thanh ghi trạng thái, thanh ghi điều khiển, thanh ghi dữ liệu vào (**data-in**), thanh ghi dữ liệu ra (**data-out**):

- **Thanh ghi dữ liệu vào – data-in register** : được đọc bởi CPU để đọc đầu vào
- **Thanh ghi dữ liệu ra – data-out register** : được ghi bởi CPU để gửi dữ liệu tới thiết bị
- **Thanh ghi trạng thái – status register** : chứa các bit có thể được đọc bởi CPU. Những bit này chỉ trạng thái của thiết bị như yêu cầu hiện thời đã được thực thi xong chưa, một byte đã sẵn để đọc trong thanh ghi data-in chưa, có lỗi xảy ra không,...
- **Thanh ghi điều khiển – control register** : có thể được ghi bởi CPU để bắt đầu 1 yêu cầu hoặc để thay đổi mode hoạt động của thiết bị.

Chuẩn CMSIS

Các vi điều khiển dựa trên nền tảng ARM Cortex-M3 đang trở nên rất phổ biến trong các thiết bị công nghiệp. Gần đây là sự giới thiệu của dòng Cortex-M0 với các đặc tính kỹ thuật nổi trội cùng với chi phí thấp hơn. Cùng với đó là sự ra đời của chuẩn Cortex

Microcontroller Software Interface Standard (**CMSIS**) làm cho việc tương thích phần mềm cũng như sử dụng lại mã nguồn trở nên đơn giản và dễ dàng.

Một trong các hạn chế lớn nhất của kiến trúc 8 bit và 16 bit hiện nay là khả năng sửa lỗi chương trình. Trong nhiều trường hợp chúng ta chỉ có thể sử dụng điểm dừng (breakpoint) để sửa lỗi chương trình còn theo dõi dữ liệu động khi chương trình đang thực thi thì không thể. Với các chương trình đơn giản, các hạn chế trên có thể không ảnh hưởng nhiều đến việc sửa lỗi, nhưng với các ứng dụng phức tạp, lượng mã nguồn nhiều và có sự tích hợp với nhiều thành phần cứng và mềm thì hạn chế trên làm phức tạp quá trình sửa lỗi ảnh hưởng đến tiến độ, thậm chí là sự thành công của dự án. Một ví dụ thực tế trên nền tảng PC, khi chuẩn phần cứng dành cho PC ra đời (Ethernet, USB, Firewire...), cộng thêm sự hỗ trợ đặc lực từ phần cứng đối với việc theo dõi lỗi, rất nhiều phần mềm hữu ích đã được phát triển và triển khai thành công.

Bộ xử lý Cortex-M3 tích hợp nhân CPU 32-bit, bộ điều khiển đánh thức khi có ngắt giúp cho các hoạt động tiêu tốn ít năng lượng hơn, và hệ thống điều khiển ngắt lồng nhau (nested vector interrupt controller-NVIC) cho phép rút ngắn thời gian trì hoãn đáp ứng ngắt (tức hệ thống đáp ứng ngắt nhanh hơn) với nhiều mức ưu tiên khác nhau. ETM là viết tắt của cụm từ “embedded trace macrocell”, đây là hệ thống chuẩn cung cấp đầy đủ chức năng sửa lỗi và theo dõi biến của chương trình. Nó cho phép lấy thông tin từ CPU trước và sau khi thực thi lệnh. ETM có thể được cấu hình bởi phần mềm và thông qua cổng giao tiếp đặc biệt(serial wire viewer) từ đó chương trình ở ngoài có thể giao tiếp, cấu hình và lấy các thông tin cần thiết cho việc sửa lỗi mà không ảnh hưởng gì đến chương trình đang thực thi.

Để chuẩn bị cho việc mở rộng và phổ biến nhân xử lý Cortex trong tương lai, chuẩn CMSIS đã ra đời nhằm giải quyết các vấn đề tương thích giữa phần mềm và phần cứng. CMSIS là chuẩn đặt ra bởi các nhà sản xuất phần cứng và phần mềm nhằm tạo nên một chuẩn phần mềm được chấp thuận rộng rãi trong công nghiệp.

Dễ dàng sử dụng và dễ dàng học, cung cấp các chuẩn giao tiếp cho các thiết bị ngoại vi, hệ điều hành thời gian thực và phần mềm hỗ trợ là mục tiêu chính của CMSIS. Ngoài ra CMSIS cũng tương thích với các trình biên dịch phổ biến hiện nay như GCC, IAR, Keil...

CMSIS gồm 2 lớp:

- Peripheral Access Layer (CMSIS-PAL): lớp này xác định tên, địa chỉ, và các hàm cần thiết để truy cập đến thanh ghi của lõi và thiết bị ngoại vi chuẩn. Ngoài ra CMSIS-PAL còn giới thiệu cách truy cập phù hợp với nhiều loại thiết bị ngoại vi bên trong lõi, và các vector xử lý ngoại lệ (exception) và ngắt. Cùng với đó là các hàm khởi động các thiết lập ban đầu cho hệ thống, hệ thống hàm giao diện chuẩn để giao tiếp với nhân của hệ điều hành và hệ thống hỗ trợ sửa lỗi.

- **Middleware Access Layer (CMSIS-MAL):** cung cấp các phương thức để truy cập vào thiết bị ngoại vi từ các lớp ứng dụng phía trên. Lớp này được quy định bởi hiệp hội các nhà sản xuất chip, do đó giúp tái sử dụng lại các thư viện cấp cao và phức tạp. Hiện nay, CMSIS-MAL vẫn đang trong quá trình phát triển và được hỗ trợ bởi IAR, Keil, Micrium, Segger và nhiều nhà sản xuất phần mềm khác.

Sự xuất hiện của CMSIS hoàn toàn không ảnh hưởng đến các thiết bị phần cứng hiện thời cũng như các hệ thống sẵn có. Chúng ta hoàn toàn có thể dùng các cách thông thường để truy cập trực tiếp đến thanh ghi để điều khiển thiết bị mà không cần dùng CMSIS. Tài nguyên hệ thống dành cho CMSIS-PAL cũng không đáng kể. CMSIS-MAL chỉ đòi hỏi giao diện mềm và chuẩn cho các lớp bên trên, do vậy cũng không làm tiêu tốn tài nguyên. Xây dựng hệ thống dựa trên chuẩn CMSIS sẽ tạo nên một bộ khung lập trình rõ ràng, dễ dàng theo dõi, mở rộng và tái sử dụng. Thêm vào đó mã nguồn chuẩn CMSIS được hỗ trợ bởi nhiều trình biên dịch. Nhằm tạo sự dễ dàng cho các nhà phát triển, ARM đã biên soạn hệ thống tài liệu tham khảo dành cho Cortex-M3 cũng như các thiết bị ngoại vi đi kèm tương thích với CMSIS.

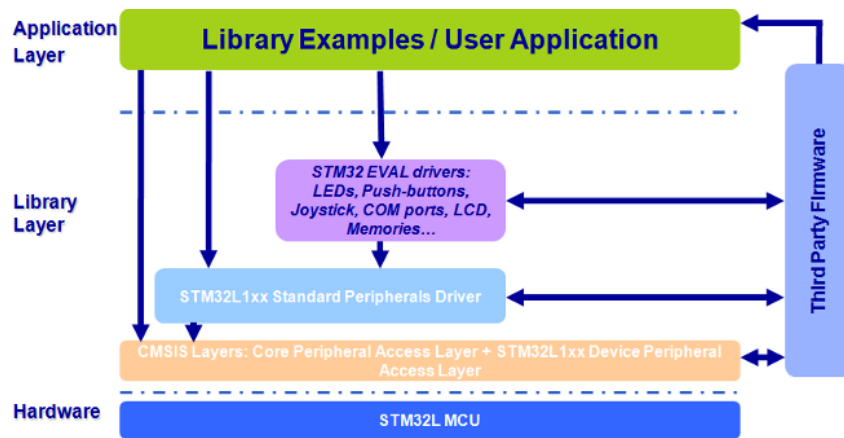
CMSIS được thiết kế độc lập với trình biên dịch. Hệ thống hàm chuẩn được hỗ trợ bởi phần cứng và phần mềm cung cấp khung phần mềm chuẩn do đó giảm thiểu rủi ro trong quá trình phát triển. Một khi chuẩn được sử dụng rộng rãi, mã nguồn sẽ trở nên dễ hiểu, dễ sử dụng lại, cũng như dễ dàng chỉnh sửa lỗi.

Trong công nghiệp, hệ thống tiêu chuẩn kỹ thuật rất quan trọng trong việc cải tiến chất lượng sản phẩm, phát triển các thành phần phụ cũng như giảm chi phí phát triển. Tuy nhiên sự phát triển của bộ vi xử lý lại không tuân theo một chuẩn nhất định, mỗi nhà sản xuất lại sử dụng những hệ thống riêng biệt không tương thích lẫn nhau. Do đó với sự ra đời của vi xử lý chuẩn Cortex-M3 không chỉ cho phép sử dụng lại bộ công cụ phát triển mà cùng với CMSIS giúp làm giảm chi phí, thời gian triển khai cùng với rủi ro kỹ thuật. Các nhà phát triển phần cứng có thể tập trung nguồn lực phát triển tính năng nổi trội của thiết bị và hỗ trợ ngoại vi.

Thư viện Standard Peripheral Library (SPL)

Thư viện Standard Peripheral Library (SPL) là một trong các bộ thư viện phổ biến được sử dụng cho các dòng ARM M3 của STM32 như: STM32Snippets, STM32Cube, HAL API, LL API,...

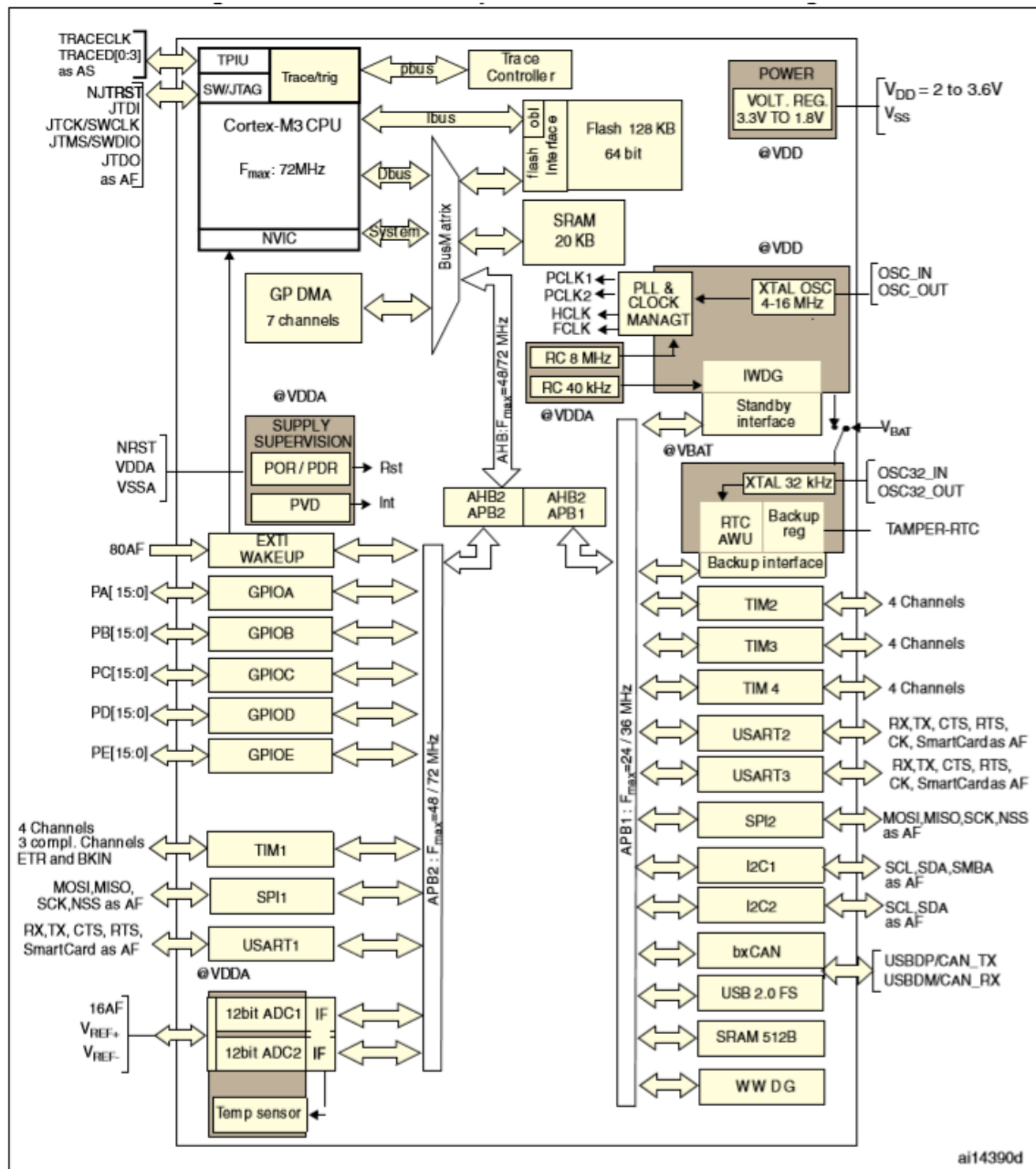
Ở giáo trình này dừng lại ở phân tích và hướng dẫn sử dụng với bộ thư viện SPL, với các bộ thư viện khác người dùng hoàn toàn có thể sử dụng với cách sử dụng tương đương.



Hình 5. 7: Vai trò của CMSIS và SPL trong phát triển phần mềm cho vi điều khiển ARM

Với sự phức tạp của kiến trúc STM32F1, việc sử dụng các thanh ghi trở nên khó khăn do vậy thư viện SPL được thiết kế nhằm mục đích cho người lập trình nhanh chóng tiếp cận và lập trình điều khiển được các ngoại vi của các dòng IC này. Việc sử dụng thư viện SPL giúp cho người lập trình có thể nhanh chóng thiết kế ứng dụng mà không cần tập trung quá nhiều vào cấu trúc cũng như các thanh ghi cấu hình của lõi vi điều khiển đang sử dụng.

Quy trình lập trình chương trình STM32F1



Hình 5. 8: Kiến trúc các ngoại vi của STM32

Theo kiến trúc các ngoại vi của dòng vi điều khiển STM32, để lập trình một chương trình điều khiển cho dòng vi điều khiển STM32F1 cần tuân thủ các bước sau:

- Cấu hình Clock chung của vi điều khiển.
- Cấp phát Clock cho ngoại vi cần sử dụng.
- Thiết lập cấu hình cho ngoại vi cần sử dụng.

Việc cấp clock chung cho hệ thống của vi điều khiển STM32F1 có thể được thực hiện nhanh chóng qua các tập lệnh đã được hỗ trợ bởi CMSIS như sau:

```
SystemInit();
SystemCoreClockUpdate();
```

Với dòng vi điều khiển Cortex M3 của ST chúng ta hoàn toàn có thể cấu hình để thay đổi tần số hoạt động của vi điều khiển. Người đọc muốn tìm hiểu sâu hơn có thể tìm hiểu trong User Manual của hãng. Trong giáo trình này dừng lại ở thiết kế hệ thống với vi điều khiển được cấu hình hoạt động của tần số hoạt động mặc định.

Các việc cấu hình clock cho ngoại vi và cấu hình cho ngoại vi được trình bày chi tiết ở từng phần ngoại vi cần lập trình tiếp theo.

Bộ thanh ghi RCC (Register Clock Control)

Bộ thanh ghi RCC là bộ thanh ghi dùng để điều khiển Clock của IC và tất cả các ngoại vi, tài nguyên của IC. Để một bộ ngoại vi như: GPIO, Timer, UART,... hoạt động ngoài Clock của hệ thống được kích hoạt chương trình phần mềm cần thực hiện các lệnh cấu hình để cấp clock cho bộ ngoại vi tương ứng.

Bộ RCC có số lượng thanh ghi rất lớn. Ở giáo trình này tập trung vào 2 thanh ghi điều khiển cấp Clock cho các bộ ngoại vi của vi điều khiển.

Thanh ghi RCC_AHBENR

Thanh ghi này có nhiệm vụ kích hoạt clock cho các ngoại vi được nối với bus điều khiển ngoại vi AHB như: SDIO, CRC, DMA1, DMA2,...

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved					SDIO EN	Res.	FSMC EN	Res.	CRCE N	Res.	FLITF EN	Res.	SRAM EN	DMA2 EN	DMA1 EN
					rw		rw		rw		rw		rw	rw	rw

Hình 5. 9: Thanh ghi RCC_AHBENR

Thanh ghi RCC_APB2ENR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved										TIM11 EN	TIM10 EN	TIM9 EN	Reserved		
										rw	rw	rw			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADC3 EN	USART 1EN	TIM8 EN	SPI1 EN	TIM1 EN	ADC2 EN	ADC1 EN	IOPG EN	IOPF EN	IOPE EN	IOPD EN	IOPC EN	IOPB EN	IOPA EN	Res.	AFIO EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw

Hình 5. 10: RCC_APB2ENR

Thanh ghi này có nhiệm vụ kích hoạt clock cho các ngoại vi được nối với bus điều khiển ngoại vi APB2 như: Timer, ADC, USART, SPI, IO, AFIO,...

Các bộ GPIO của vi điều khiển đều được kích hoạt thông qua thanh ghi này.

Thanh ghi RCC_APB1ENR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		DAC EN	PWR EN	BKP EN	Res.	CAN EN	Res.	USB EN	I2C2 EN	I2C1 EN	UART5 EN	UART4 EN	USART3 EN	USART2 EN	Res.
		rw	rw	rw		rw		rw	rw	rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 EN	SPI2 EN	Reserved		WWD GEN	Reserved		TIM14 EN	TIM13 EN	TIM12 EN	TIM7 EN	TIM6 EN	TIM5 EN	TIM4 EN	TIM3 EN	TIM2 EN
rw	rw			rw			rw	rw	rw	rw	rw	rw	rw	rw	rw

Hình 5. 11: Thanh ghi RCC_APB1ENR

Thanh ghi này có nhiệm vụ kích hoạt clock cho các ngoại vi được nối với bus điều khiển ngoại vi APB1 như: DAC, CAN, USB, Timer,...

Ví dụ để kích hoạt clock cho GPIOA của vi điều khiển chương trình phần mềm cần thực hiện lệnh sau.

```
RCC->APB2ENR |= 1<<2;
```

Hoặc thực hiện dưới dạng macro theo thư viện CMSIS như sau:

```
RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;
```

Có thể thực hiện được cách viết như trên là do thư viện CMSIS đã định nghĩa giá trị của RCC_APB2ENR_IOPAEN tương ứng với 0x00000004 (Có thể tham khảo file stm32f10x.h của bộ thư viện CMSIS).

Lưu ý: Việc cấp clock cho các ngoại vi nên được thực hiện trước khi thiết lập cấu hình và điều khiển các ngoại vi đó.

Lập trình điều khiển IO với STM32F1 sử dụng thanh ghi

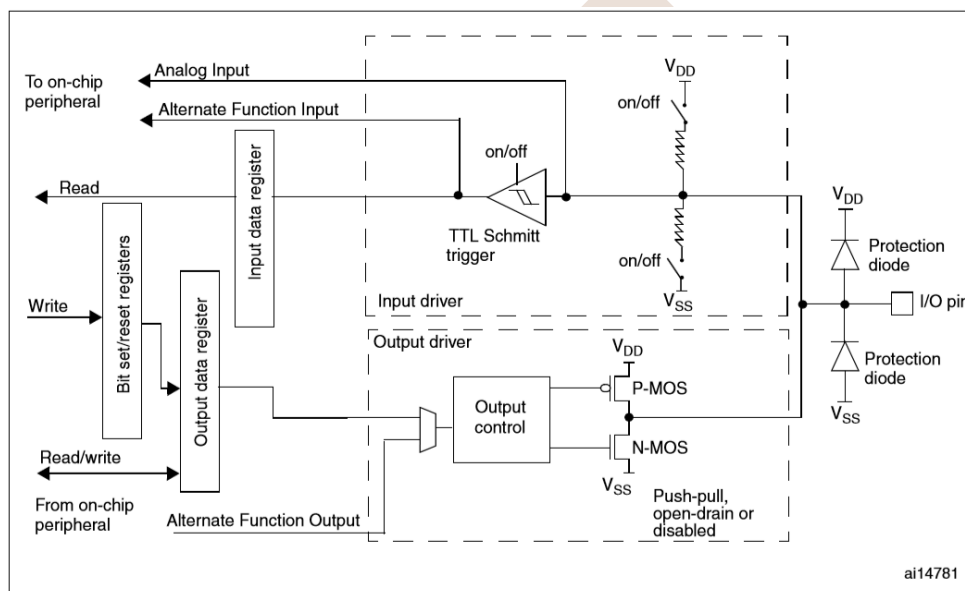
Dòng STM32F1 (Cortex M3) của STM32 cung cấp tối đa 7 Port IO (PortA, PortB, PortC, PortD, PortE, PortF, PortG) với các chân vào ra được cấu hình có thể có tối đa đến 4 chức năng. Các Port này có tên là GPIO (General-Purpose I/O).

Mỗi bộ GPIO của STM32F1 có 2 thanh ghi cấu hình (GPIOx_CRL, GPIOx_CRH), 2 thanh ghi dữ liệu (GPIOx_IDR, GPIOx_ODR), một thanh ghi Lập/Xóa ((GPIOx_BSRR), một thanh ghi (GPIOx_BRR) và một thanh ghi khóa (GPIOx_LCKR).

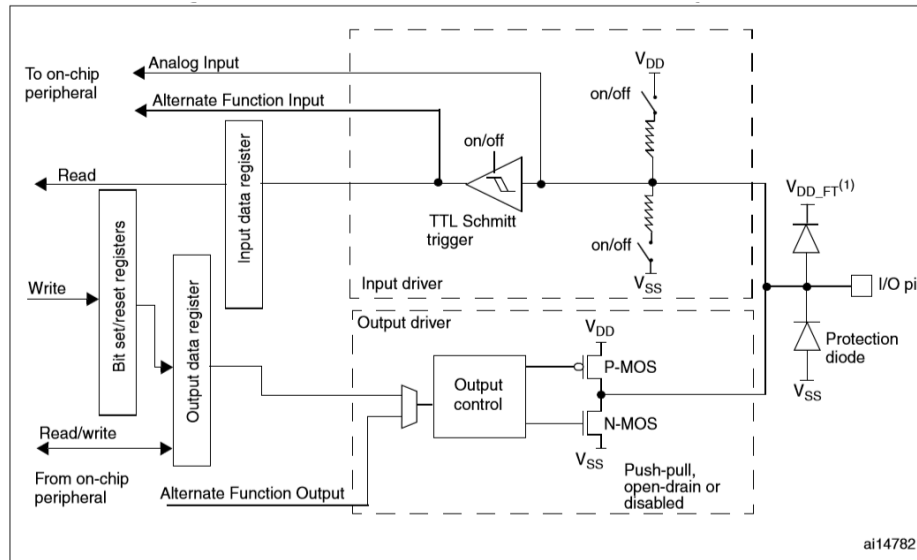
Mỗi bộ GPIO hỗ trợ điều khiển tối đa 16 chân GPIO. Số lượng chân của mỗi GPIO trên mỗi IC phụ thuộc vào số lượng cổng mà IC đó hỗ trợ. Các chân này có thể cấu hình độc lập với các chức năng hoạt động khác nhau. Danh sách các chế độ hoạt động của mỗi cổng như sau:

- Đầu vào thả nổi số
- Đầu vào kéo lên số
- Đầu vào kéo xuống số
- Tương tự
- Đầu ra thả nổi
- Đầu ra kéo lên
- Chức năng luân phiên kéo lên
- Chức năng luân phiên thả nổi

Mỗi GPIO có thể cấu hình độc lập nhưng mỗi thanh ghi cấu hình phải được truy cập cấu hình thông qua giao diện 32 bit.



Hình 5. 12: Cấu trúc của các chân điều khiển thông dụng



Hình 5. 13: Cấu trúc của các chân IO hỗ trợ giao tiếp 5V

Bảng cấu hình của các chân IO được giải thích ở bảng sau:

Configuration mode		CNF1	CNF0	MODE1	MODE0	PxODR register
General purpose output	Push-pull	0	0	01	0 or 1	
	Open-drain		1			0 or 1
Alternate Function output	Push-pull	1	0	11 see Table 21	Don't care	
	Open-drain		1		Don't care	
Input	Analog	0	0	00	Don't care	
	Input floating		1		Don't care	
	Input pull-down	1	0		0	
	Input pull-up				1	

Hình 5. 14: Thông tin cấu hình IO

Với chế độ hoạt động dạng đầu ra số còn có thêm cấu hình về tốc độ phụ thuộc vào bảng sau:

MODE[1:0]	Meaning
00	Reserved
01	Maximum output speed 10 MHz
10	Maximum output speed 2 MHz
11	Maximum output speed 50 MHz

Hình 5. 15: Các chế độ tốc độ đầu ra số

Ở phần này, giáo trình sẽ phân tích chi tiết về các thanh ghi của bộ GPIO, các phần sau chỉ dừng lại ở thông tin gợi mở, người đọc muốn tìm hiểu chi tiết có thể tham khảo datasheet và hướng dẫn sử dụng IC của hãng.

Mỗi GPIO của STM32F1 hỗ trợ điều khiển 16 cổng (0-15) thông qua giao diện thanh ghi 32 bit. Việc cấu hình hoạt động cho mỗi cổng của IC cần thông qua 4 bit điều khiển CNF[1:0] và MODE [1:0]. Do vậy thanh ghi cấu hình hoạt động cho cả bộ GPIO cần chia làm 2 thanh ghi: Một thanh ghi điều khiển chế độ của các cổng từ 0 đến 7 và một thanh ghi điều khiển chế độ hoạt động của các cổng từ 8 đến 15.

Thanh ghi GPIOx_CRL

Address offset: 0x00

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Hình 5. 16: Thanh ghi GPIOx_CRL

Thanh ghi GPIOx_CRH

Address offset: 0x04

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Hình 5. 17: Thanh ghi GPIOx_CRH

Trong đó chi tiết về các cách cấu hình cho mỗi cổng như sau:

Chế độ Input (MODE[1:0] = 00):

CNF[1:0]: 00: Vào tương tự
 01: Vào số thả nổi
 10: Vào số kéo lên hoặc kéo xuống (Phụ thuộc vào thanh ghi ODR)
 11: Dự trữ

Chế độ Output (MODE[1:0] > 00):

CNF[1:0]: 00: Đầu ra đa dụng kéo lên
 01: Đầu ra đa dụng thả nổi
 10: Đầu ra luân phiên kéo lên
 11: Đầu ra luân phiên thả nổi

Với MODE[1:0] > 00 tức là cổng được cấu hình đầu ra, giá trị của MODE[1:0] quyết định tốc độ hoạt động tối đa của cổng như sau:

MODE[1:0]: 00: Đầu vào
 01: Đầu ra, tần số tối đa 10MHz
 10: Đầu ra, tần số tối đa 2MHz
 11: Đầu ra, tần số tối đa 50MHz

Thanh ghi GPIOx_IDR

Address offset: 0x08h

Reset value: 0x0000 XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Hình 5. 18: Thanh ghi GPIOx_IDR

Thanh ghi này là thanh ghi chỉ đọc, dùng để đọc dữ liệu đầu vào từ các GPIO tương ứng. Mỗi bit IDR tương ứng đọc giá trị đầu vào của chân tương ứng. Thanh ghi này thường được sử dụng khi cần đọc dữ liệu đầu vào số của một cổng nào đó.

Thanh ghi GPIOx_ODR

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Hình 5. 19: Thanh ghi GPIOx_ODR

Thanh ghi này có thể truy cập đọc và ghi có chức năng dùng để điều khiển đầu ra số tại các chân đầu ra tương ứng. Khi tác động đến thanh ghi này các chân được cài đặt ở chế độ đầu ra tương ứng sẽ bị ảnh hưởng.

Thanh ghi Lập/Xóa GPIOx_BSRR

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Hình 5. 20: Thanh ghi Lập/Xóa GPIOx_BSRR

Thanh ghi này là thanh ghi chỉ ghi và không thể đọc về. Trong đó thanh ghi được chia làm 2 nửa từ BS0-BS15 và BR0-BR15.

BS0-BS15: Nếu ghi giá trị 0 vào các bit tương ứng này thì đầu ra của chân tương ứng không thay đổi, nếu ghi giá trị 1 vào các bit tương ứng thì bit tương ứng của thanh ghi ODR được lập lên 1 (Tức là điều khiển cổng tương ứng lên mức đầu ra là 1 – 3,3V).

BR0-BR15: Nếu ghi giá trị 0 vào các bit tương ứng này thì đầu ra của chân tương ứng không thay đổi, nếu ghi giá trị 1 vào các bit tương ứng thì bit tương ứng của thanh ghi ODR được xóa về 0 (Tức là điều khiển cổng tương ứng với mức đầu ra là 0 – 0V).

Thanh ghi Xóa GPIOx_BRR

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Hình 5. 21: Thanh ghi Xóa GPIOx_BRR

Thanh ghi này chứa 16 bit có thể truy cập dưới dạng chỉ ghi có chức năng tương đương như các bit từ BR0-BR15 của thanh ghi BSRR.

Thanh ghi khóa cấu hình GPIOx_LCKR

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															LCKK
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Hình 5. 22: Thanh ghi khóa cấu hình GPIOx_LCKR

Thanh ghi này dùng để khóa cấu hình của chân GPIO tương ứng cho mỗi cổng. Ngoài ra, thanh ghi còn có thêm 1 bit LCKK là bit Lock Key.

Chức năng của các bit cụ thể như sau:

LCKK: 0: Không cho phép chế độ khóa cấu hình cổng của Port tương ứng hoạt động.

1: Kích hoạt chế độ khóa cấu hình cổng của Port tương ứng.

LCKy: 0: Cấu hình của cổng tương ứng được mở.

1: Cấu hình của cổng tương ứng bị khóa.

Khi cấu hình của cổng tương ứng bị khóa chương trình phần mềm không được phép thay đổi chức năng của cổng đó cho đến khi khóa cấu hình được mở.

Điều khiển đầu ra: Với tiến trình và các thanh ghi đã tìm hiểu chương trình điều khiển thực hiện nhấp nháy đèn LED sử dụng STM32F1 có thể được viết trực tiếp bằng các thanh ghi theo một số cách như sau:

Cách 1: Sử dụng thanh ghi ODR

```
#include "stm32f10x.h"

void delay(int _Time){// Hàm delay
    unsigned int Count;
    while(_Time--){
        for (Count = 0; Count < 1000; Count++);
    }
}

int main (void){
    SystemInit();
    SystemCoreClockUpdate();
    RCC->APB2ENR |= RCC_APB2ENR_IOPCEN; // Cap Clock cho GPIOC
    // Khoi tao cau hinh nguyen thuy cho Port C
    GPIOA->CRH = 0x00000000;
```

```

    GPIOA->CRL = 0x00000000;

    GPIOA->CRH |= 0x00300000; // Cau hinh chan C13 o che do Output
    Push-pull CNF[1:0] = 00, MODE[1:0] = 11

    while(1){

        GPIOC->ODR |= (1<<13); // Bat LED - Xuat 3.3V ra C13
        delay(1000);           // Tam dung
        GPIOC->ODR &= ~(1<<13); // Tat LED - Xuat 0V ra C13
        delay(1000);

    }
}

```

Cách 2: Sử dụng thanh ghi BSRR và BRR

```

#include "stm32f10x.h"

void delay(int _Time){// Hàm delay
    unsigned int Count;
    while(_Time--){
        for (Count = 0; Count < 1000; Count++);
    }
}

int main (void){
    SystemInit();
    SystemCoreClockUpdate();
    RCC->APB2ENR |= RCC_APB2ENR_IOPCEN; // Cap Clock cho GPIOC
    // Khoi tao cau hinh nguyen thuy cho Port C
    GPIOA->CRH = 0x00000000;
    GPIOA->CRL = 0x00000000;
    GPIOA->CRH |= 0x00300000; // Cau hinh chan C13 o che do Output
    Push-pull CNF[1:0] = 00, MODE[1:0] = 11
    while(1){
        GPIOC->BSRR |= (1<<13); // Bat LED - Xuat 3.3V ra C13
    }
}

```

```

        delay(1000);           // Tam dung
        GPIOC->BRR |= (1<<13); // Tat LED - Xuat 0V ra C13
        delay(1000);

    }
}

```

Như vậy tiến trình để điều khiển đầu ra của một cổng như sau:

- Điều khiển clock hệ thống
- Cấp clock cho GPIO tương ứng chứa cổng đó
- Cấu hình cho cổng tương ứng ở chế độ Output
- Điều khiển dữ liệu của cổng tương ứng thông qua thanh ghi ODR hoặc BSRR hoặc BRR.

Đọc dữ liệu đầu vào: Xây dựng chương trình đọc dữ liệu từ 8 bit thấp của cổng A và xuất ra 8 bit cao của cổng A.

Để làm được điều này cần thực hiện cấu hình GPIOA hoạt động với 8 cổng thấp ở chế độ vào số và 8 cổng cao ở chế độ đầu ra số. Sau đó, chương trình thực hiện đọc dữ liệu từ 8 bit thấp của thanh ghi IDR và xuất ra 8 bit cao của thanh ghi IDR. Chương trình cụ thể như sau:

```

#include "stm32f10x.h"

void delay(int _Time){// Hàm delay
    unsigned int Count;
    while(_Time--){
        for (Count = 0; Count < 1000; Count++);
    }
}

int main (void){
    unsigned int TempRead;

    SystemInit();
    SystemCoreClockUpdate();
}

```

```

RCC->APB2ENR |= RCC_APB2ENR_IOPAEN; // Cap Clock cho GPIOC

GPIOA->CRH = 0x33333333; // Cau hinh 8 chan cao o che do Output
Push-pull CNF[1:0] = 00, MODE[1:0] = 11

// Cau hinh 8 chan thap o che do Input Pull-up/Pull-down CNF[1:0]
= 10, MODE[1:0] = 00
GPIOA->CRL = 0x88888888;
GPIOA->ODR = 0x00FF; // Chuyen 8 bit thap ve che do Pull-Up
while(1){
    //Doc du lieu dau vao
    TempRead = GPIOA->IDR & 0x00FF; // Chi doc du lieu 8 bit
    thap tu thanh ghi IDR
    GPIOA->ODR = TempRead<<8; // Dich 8 bit thap thanh 8 bit
    cao va xuất ra thanh ghi ODR
    delay(1);
}
}

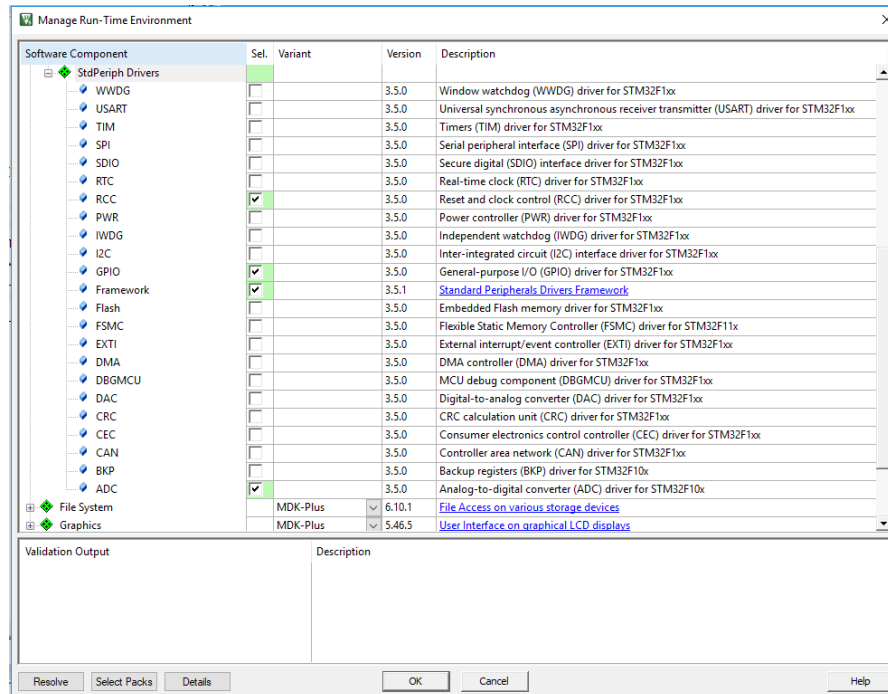
```

Lập trình điều khiển IO với STM32F1 sử dụng SPL

Như đã trình bày ở trên thư viện SPL là một trong những thư viện phổ biến sử dụng để lập trình cho STM32F1. Thư viện này có vai trò giúp người lập trình nhanh chóng tiếp cận được lập trình các dòng vi điều khiển của ARM mà không cần nghiên cứu sâu về kiến trúc các thanh ghi của vi điều khiển. Sau khi nêu ra một ví dụ sử dụng thanh ghi giáo trình chuyển sang hướng dẫn trên nền tảng sử dụng thư viện SPL để sinh viên, người đọc có thể dễ dàng tiếp cận các dòng vi điều khiển. Tuy nhiên, cũng khuyến nghị các sinh viên, người đọc sau khi có thể điều khiển bằng SPL có thể đọc lại các thanh ghi để nắm được hoạt động cụ thể của các vi điều khiển này.

Sau đây, để thực hiện lập trình điều khiển IO sử dụng SPL, giáo trình xin giới thiệu về cấu trúc thư viện SPL cho STM32F1.

Với KeilC 5 người dùng có thể dễ dàng tùy chọn gói thư viện SPL bằng cách chọn: Manage Run → Device -> StdPeriph Drivers sau đó chọn các gói thư viện cần sử dụng. Sau đó, KeilC sẽ tự động thêm các gói này vào dự án của người lập trình.



Hình 5. 23: Cấu hình các package sử dụng cho Project với KeilC 5

Với chương trình thực hiện điều khiển IO cần thực hiện chọn các gói sau:

- Framework: Cho phép sử dụng nền tảng SPL trong Project
- RCC: Điều khiển clock cho các ngoại vi, ở đây sử dụng để điều khiển clock cho các bộ GPIO.
- GPIO: Cấu hình và điều khiển các bộ GPIO tương ứng

Gói RCC có các hàm chính như sau:

```
void RCC_LSEConfig(uint8_t RCC_LSE);
void RCC_LSICmd(FunctionalState NewState);
void RCC_RTCCLKConfig(uint32_t RCC_RTCCLKSource);
void RCC_RTCCLKCmd(FunctionalState NewState);
void RCC_GetClocksFreq(RCC_ClocksTypeDef* RCC_Clocks);
void RCC_AHBPeriphClockCmd(uint32_t RCC_AHBPeriph, FunctionalState NewState);
void RCC_APB2PeriphClockCmd(uint32_t RCC_APB2Periph, FunctionalState NewState);
void RCC_APB1PeriphClockCmd(uint32_t RCC_APB1Periph, FunctionalState NewState);
void RCC_APB2PeriphResetCmd(uint32_t RCC_APB2Periph, FunctionalState NewState);
```

```

NewState);
void RCC_APB1PeriphResetCmd(uint32_t RCC_APB1Periph, FunctionalState
NewState);
void RCC_BackupResetCmd(FunctionalState NewState);
void RCC_ClockSecuritySystemCmd(FunctionalState NewState);
void RCC_MCOConfig(uint8_t RCC_MCO);
FlagStatus RCC_GetFlagStatus(uint8_t RCC_FLAG);
void RCC_ClearFlag(void);
ITStatus RCC_GetITStatus(uint8_t RCC_IT);
void RCC_ClearITPendingBit(uint8_t RCC_IT);

```

Gói GPIO có các hàm như sau:

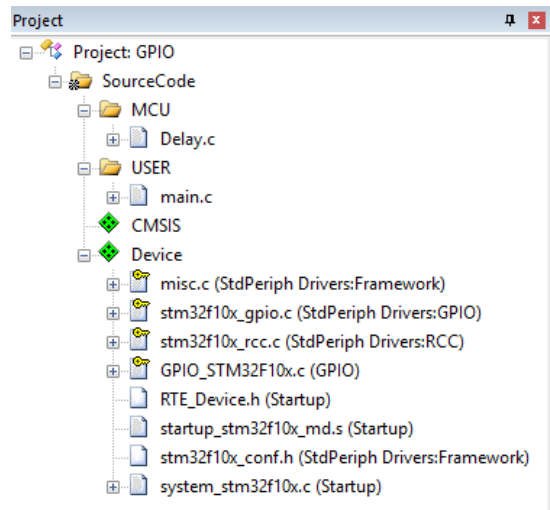
```

void GPIO_DeInit(GPIO_TypeDef* GPIOx);
void GPIO_AFIODeInit(void);
void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct);
void GPIO_StructInit(GPIO_InitTypeDef* GPIO_InitStruct);
uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
uint16_t GPIO_ReadInputData(GPIO_TypeDef* GPIOx);
uint8_t GPIO_ReadOutputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
uint16_t GPIO_ReadOutputData(GPIO_TypeDef* GPIOx);
void GPIO_SetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
void GPIO_ResetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
void GPIO_WriteBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, BitAction
BitVal);
void GPIO_Write(GPIO_TypeDef* GPIOx, uint16_t PortVal);
void GPIO_PinLockConfig(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
void GPIO_EventOutputConfig(uint8_t GPIO_PortSource, uint8_t
GPIO_PinSource);
void GPIO_EventOutputCmd(FunctionalState NewState);
void GPIO_PinRemapConfig(uint32_t GPIO_Remap, FunctionalState
NewState);
void GPIO_EXTILineConfig(uint8_t GPIO_PortSource, uint8_t
GPIO_PinSource);

```

```
void GPIO_ETH_MediaInterfaceConfig(uint32_t GPIO_ETH_MediaInterface);
```

Sau khi lựa chọn mục Device của KeilC sẽ có giao diện như sau:



Hình 5. 24: Thông tin các gói sau khi cấu hình

Cá file ở mục Device là các file đã được phần mềm tự động thêm vào theo lựa chọn của người lập trình. Sau khi đã cấu hình có thể lập trình chương trình main cho project thực hiện nhấp nháy LED tại chân A0 như sau:

```
#include "stm32f10x.h" // Device header
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO

void delay(int _Time){// Hàm delay
    unsigned int Count;
    while(_Time--){
        for (Count = 0; Count < 1000; Count++);
    }
}

void Fn_GPIO_Init (void);
int main (void){
    SystemInit();
    SystemCoreClockUpdate();

    Fn_GPIO_Init(); // Goi ham khai tao GPIO
    while(1){
```

```

        GPIO_WriteBit(GPIOA, GPIO_Pin_0, Bit_RESET); // Xuất du
        lieu 0V (0) ra chan A0
        delay(1000);
        GPIO_WriteBit(GPIOA, GPIO_Pin_0, Bit_SET);      // Xuất du
        lieu 3.3V(1) ra chan A0
        delay(1000);
    }
}

void Fn_GPIO_Init (void){
    GPIO_InitTypeDef GPIO_InitStructure; // Bien chua thong tin cau
    hinh GPIO

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE); // Cap
    Clock cho GPIOA

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0; // Chan cau hinh la
    chan 0

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; // Che do hoat
    dong cua chan cau hinh la Ouput

    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz; // Tan so
    hoat dong toi da cua chan cau hinh la 50MHz

    GPIO_Init(GPIOA, &GPIO_InitStructure); // Nap cau hinh tren vao
    GPIOA
}

```

Chương trình điều khiển dữ liệu cho cả GPIOA có thể thực hiện như sau:

```

#include "stm32f10x.h" // Device header
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO

void delay(int _Time){// Hàm delay
    unsigned int Count;
    while(_Time--){
        for (Count = 0; Count < 1000; Count++);
    }
}

```



```

}

void Fn_GPIO_Init (void);

int main (void){
    unsigned int Count = 0;
    SystemInit();
    SystemCoreClockUpdate();

    Fn_GPIO_Init(); // Goi ham khoi tao GPIO
    while(1){
        GPIO_Write(GPIOA, Count); // Xuat du lieu tu bien Count ra
16 cong cua GPIOA
        Count++;
        delay(1000);
    }
}

void Fn_GPIO_Init (void){
    GPIO_InitTypeDef GPIO_InitStructure; // Bien chua thong tin cau
hinh GPIO

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA);
    // Cau hinh toan bo GPIOA la Output
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_All;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
}

```

Chương trình đọc dữ liệu tại GPIOB và xuất ra GPIOA được thực hiện như sau:

```

#include "stm32f10x.h" // Device header
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO

void delay(int _Time){// Hàm delay
    unsigned int Count;
    while(_Time--){
        for (Count = 0; Count < 1000; Count++);
    }
}

void Fn_GPIO_Init (void);

int main (void){
    unsigned int ReadValue = 0;
    SystemInit();
    SystemCoreClockUpdate();

    Fn_GPIO_Init(); // Goi ham khoi tao GPIO
    while(1){
        ReadValue = GPIO_ReadInputData(GPIOB); // Doc du lieu GPIOB
        vao bien ReadValue
        GPIO_Write(GPIOA, ReadValue); // Xuat gia tri tu bien
        ReadValue ra GPIOA
        Fn_DELAY_ms(10);
    }
}

void Fn_GPIO_Init (void){
    GPIO_InitTypeDef GPIO_InitStructure; // Bien chua thong tin cau
    hinh GPIO

```

```

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA
RCC_APB2Periph_GPIOB, ENABLE);

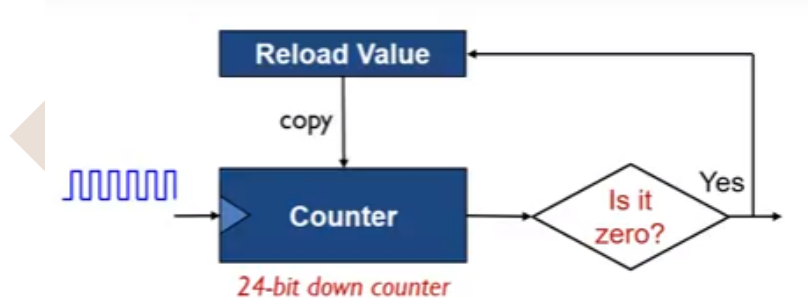
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_All;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_All;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
    GPIO_Init(GPIOB, &GPIO_InitStructure);
}

```

Lập trình SysTick

Systick là một bộ đếm xuống 24 bit và có khả năng tự động nạp lại giá trị (auto reload).



Hình 5. 25: Hoạt động của SysTick

Systick được ví như một cái đồng hồ đếm ngược, nó được tạo ra để cung cấp một bộ thời gian chuẩn cho hệ thống. Đồng hồ SysTick được sử dụng để cung cấp một nhịp đập hệ thống cho hệ điều hành thời gian thực RTOS hoặc để tạo một ngắt có tính chu kỳ hay đơn giản để tạo một khoảng delay có độ chính xác cao.

Ví dụ xây dựng chương trình sử dụng SysTick để tạo một ngắt định kỳ với chu kỳ là 1ms. Để định kỳ hệ thống ngắt sau 1ms cần cấu hình SysTick hoạt động với thời gian 1ms theo lệnh như sau:

```

SysTick_Config(SystemCoreClock/1000);

```

SystemCoreClock là tần số hoạt động thực của vi điều khiển tương ứng với thời gian là 1 giây. Vậy để định kỳ 1ms chương trình thực hiện ngắt một lần cần cấu hình SysTick với thời gian là thời gian hệ thống đã chia cho 1000. Chương trình cụ thể như sau:

```
#include "stm32f10x.h" // Device header
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO

volatile char    vruc_SYSTICK_Flag = 0;

GPIO_InitTypeDef GPIO_InitStructure;
void Fn_GPIO_Init (void);

int main (void){
    SystemInit();
    SystemCoreClockUpdate();

    SysTick_Config(SystemCoreClock/1000);
    Fn_GPIO_Init();
    while(1){
        if(vruc_SYSTICK_Flag == 1){
            vruc_SYSTICK_Flag = 0;
            if      (GPIO_ReadOutputDataBit(GPIOA,GPIO_Pin_0) ==
Bit_RESET){
                GPIO_WriteBit(GPIOA,GPIO_Pin_0, Bit_SET);
            }
            else{
                GPIO_WriteBit(GPIOA,GPIO_Pin_0, Bit_RESET);
            }
        }
    }
}

void Fn_GPIO_Init (void){
```

```

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
}

void SysTick_Handler(void){
    vruc_SYSTICK_Flag = 1;
}

```

Hàm ngắt của SysTick có tên là SysTick_Handler đã được quy định sẵn trong file stm32f10x.h và gắn với bảng vector ngắt của hệ thống. Mỗi khi đủ 1ms hàm ngắt được gọi 1 lần và thực hiện gán giá trị của biến vruc_SYSTICK_Flag bằng 1. Ở hàm main nếu kiểm tra biến vruc_SYSTICK_Flag bằng 1 thì chương trình thực hiện đảo trạng thái của đèn LED tại cổng A0. Do vậy chương trình thực hiện liên tục đổi trạng thái của chân A0 sau định kỳ 1ms.

Ở ví dụ này giáo trình sử dụng SysTick để tạo một thư viện Delay với độ chính xác cao và sử dụng xuyên suốt giáo trình. Cụ thể nội dung của thư viện như sau:

```

#include "stm32f10x.h"
#include "Delay.h"

static u8 fac_us=0;
static u16 fac_ms=0;

void Fn_DELAY_Init (unsigned char _CLK)
{
    SysTick->CTRL&=0xffffffffb;
    fac_us=_CLK/8;
    fac_ms=(u16)fac_us*1000;
}

```

```

void Fn_DELAY_ms (unsigned int _vrui_Time)
{
    u32 temp;
    SysTick->LOAD=(u32)_vrui_Time*fac_ms;
    SysTick->VAL =0x00;
    SysTick->CTRL=0x01;
    do
    {
        temp=SysTick->CTRL;
    }
    while((temp&0x01)&&(!(temp&(1<<16))));
    SysTick->CTRL=0x00;
    SysTick->VAL =0x00;
}

```

```

void Fn_DELAY_us (unsigned long _vrui_Time)
{
    u32 temp;
    SysTick->LOAD=_vrui_Time*fac_us;
    SysTick->VAL=0x00;
    SysTick->CTRL=0x01 ;
    do
    {
        temp=SysTick->CTRL;
    }
    while((temp&0x01)&&(!(temp&(1<<16))));
    SysTick->CTRL=0x00;
    SysTick->VAL =0x00;
}

```

Trong đó hàm Fn_DELAY_Init khởi tạo sử dụng delay sử dụng bộ SysTick với thông số đầu vào là tần số hoạt động của vi điều khiển.

Với thư viện này chúng ta có thể tạo được chương trình nhấp nháy đèn LED định kỳ 1 giây như sau:

```
#include "stm32f10x.h" // Device header
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph
Drivers:GPIO
#include "Delay.h"

GPIO_InitTypeDef GPIO_InitStructure;
void Fn_GPIO_Init (void);

int main (void){
    SystemInit();
    SystemCoreClockUpdate();
    Fn_DELAY_Init(72);
    Fn_GPIO_Init();
    while(1){
        if( GPIO_ReadOutputDataBit(GPIOA, GPIO_Pin_0) ==
        Bit_SET){
            GPIO_WriteBit(GPIOA, GPIO_Pin_0, Bit_RESET);
        }
        else{
            GPIO_WriteBit(GPIOA, GPIO_Pin_0, Bit_SET);
        }
        Fn_DELAY_ms(1000);
    }
}

void Fn_GPIO_Init (void){
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);
```

```

GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;

GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;

GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;

GPIO_Init(GPIOA, &GPIO_InitStructure);

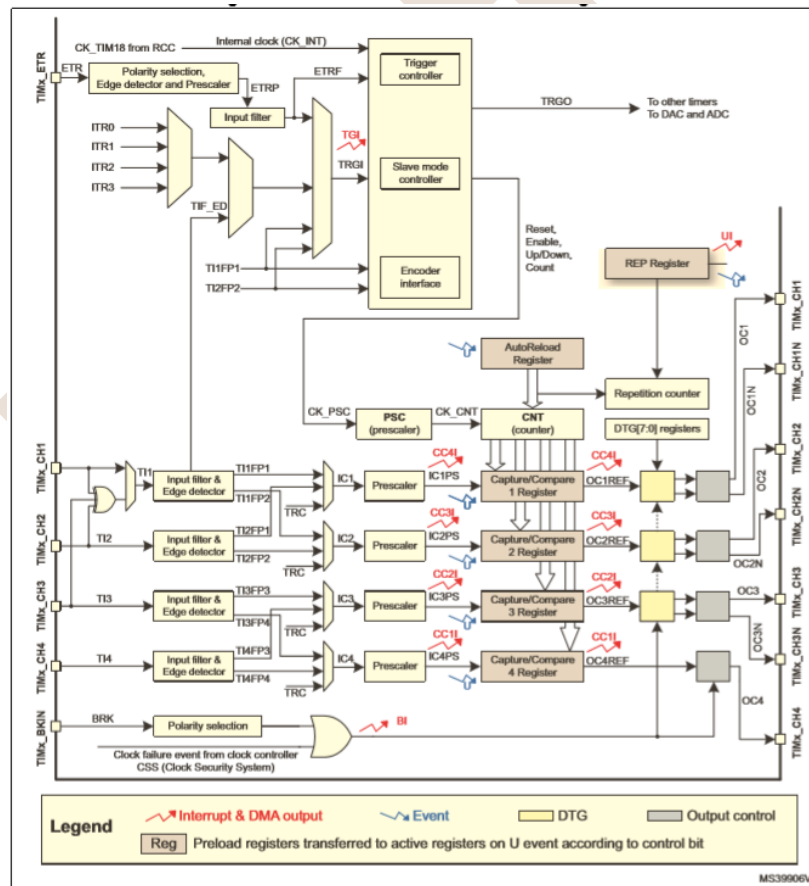
}

```

Sau khi cấu hình xong thời gian các hàm Delay sẽ thực hiện đúng với thời gian yêu cầu do SysTick là một hệ thống thời gian chính xác của vi điều khiển luôn chạy song song với chương trình phần mềm.

Lập trình điều khiển Timer

Cấu trúc các bộ Timer

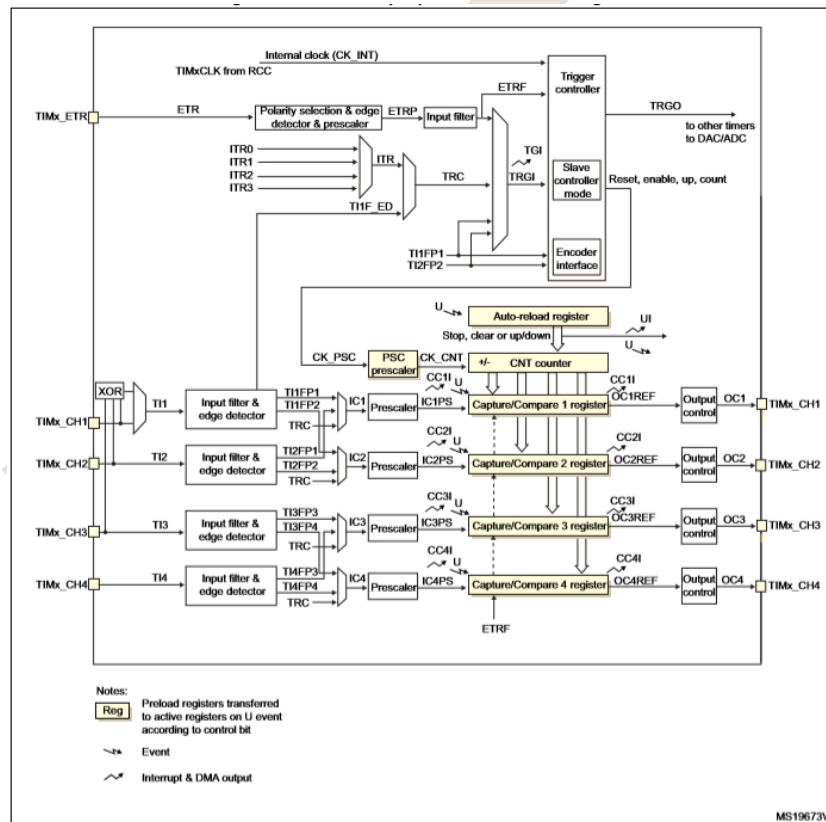


Hình 5. 26: Cấu trúc bộ Timer Advanced-Control

Bộ Timer Advanced-Control bao gồm TIM1 và TIM8 đối với dòng vi điều khiển STM32F1. Các bộ timer này có một số đặc điểm như sau:

- Bộ đếm 16 bit có thể đếm tiến, lùi, tiến/lùi tự động nạp lại

- Hỗ trợ bộ chia tần 16 bit có thể lập trình được.
- Hỗ trợ 4 kênh hoạt động độc lập cho các chức năng:
 - Caputre đầu vào
 - So sánh đầu ra
 - Điều chế độ rộng xung (PWM)
 - Chế độ đầu ra một xung
- Hỗ trợ kích hoạt các sự kiện DMA
- Hỗ trợ kích hoạt ngắt
- Hỗ trợ trigger cho các sự kiện

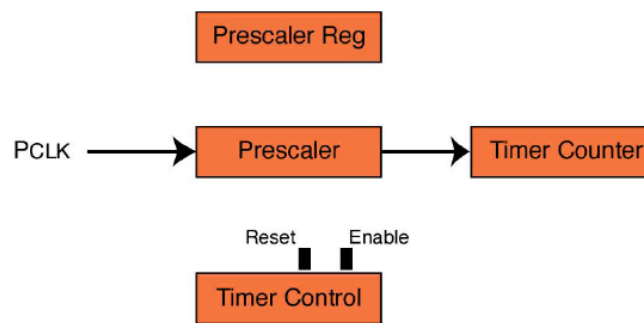


Hình 5. 27: Cấu trúc bộ Timer General-purpose

Bộ Timer Advanced-Control bao gồm TIM2 đến TIM5 đối với dòng vi điều khiển STM32F1. Các bộ timer này có một số đặc điểm như sau:

- Bộ đếm 16 bit có thể đếm tiến, lùi, tiến/lùi tự động nạp lại
- Hỗ trợ bộ chia tần 16 bit có thể lập trình được.
- Hỗ trợ 4 kênh hoạt động độc lập cho các chức năng:

- Caputre đầu vào
 - So sánh đầu ra
 - Điều chế độ rộng xung (PWM)
 - Chế độ đầu ra một xung
- Hỗ trợ kích hoạt các sự kiện DMA
 - Hỗ trợ kích hoạt ngắt
 - Hỗ trợ trigger cho các sự kiện
 - Hỗ trợ đọc cảm biến hall 4 đầu vào



Hình 5. 28: Hoạt động của Timer

Một Timer cơ bản gồm các thành phần như sau:

TIM_CLK : clock cung cấp cho timer.

PSC (prescaler): là thanh ghi 16bits làm bộ chia cho timer, có thể chia từ 1 tới 65535

ARR (auto-reload register): là giá trị đếm của timer (16bits hoặc 32bits).

RCR (repetition counter register): giá trị đếm lặp lại 16bits

Timer của STM32 là timer 16 bits có thể tạo ra các sự kiện trong khoảng thời gian từ nano giây tới vài phút gọi là UEV(update event).

Giá trị của UEV được tính theo công thức sau:

$$UEV = TIM_CLK / ((PSC + 1) * (ARR + 1) * (RCR + 1))$$

Ở giáo trình này dừng lại ở đưa ra một ví dụ sử dụng Timer ở tính năng chính là đếm thời gian sử dụng ngắt và không sử dụng ngắt. Chương trình được xây dựng như sau:

```

#include "stm32f10x.h" // Device header
#include "stm32f10x_rcc.h" // Keil::Device:StdPeriph Drivers:RCC
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO
#include "stm32f10x_tim.h" // Keil::Device:StdPeriph Drivers:TIM
  
```

```

#include "Delay.h"

GPIO_InitTypeDef GPIO_InitStructure;
NVIC_InitTypeDef NVIC_InitStructure;
TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;

void Fn_GPIO_Init (void);
void Fn_NVIC_Init (void);
void Fn_TIM4_Init (void);
void Fn_TIM2_Init (void);

int main (void){
    SystemInit();
    SystemCoreClockUpdate();
    Fn_DELAY_Init(72);

    Fn_GPIO_Init();
    Fn_TIM4_Init();
    Fn_TIM2_Init();
    Fn_NVIC_Init();
    while(1){
        if(TIM_GetITStatus(TIM2,TIM_IT_Update) != RESET){
            if( GPIO_ReadOutputDataBit(GPIOB, GPIO_Pin_1) ==
Bit_SET){
                GPIO_WriteBit(GPIOB, GPIO_Pin_1, Bit_RESET);
            }
            else{
                GPIO_WriteBit(GPIOB, GPIO_Pin_1, Bit_SET);
            }
            TIM_ClearITPendingBit(TIM2, TIM_IT_Update);
        }
    }
}

```

```

    }
}

void Fn_GPIO_Init (void){
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0 | GPIO_Pin_1;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOB, &GPIO_InitStructure);
}

void Fn_NVIC_Init (void){
    NVIC_InitStructure.NVIC_IRQChannel = TIM4_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStructure);
}

void Fn_TIM2_Init (void){
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
    TIM_TimeBaseStructure.TIM_Prescaler = 20-1;
    TIM_TimeBaseStructure.TIM_Period = 50000;           //
    TIM_TimeBaseStructure.TIM_ClockDivision = 0;
    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM2, &TIM_TimeBaseStructure);
    TIM_ITConfig(TIM2, TIM_IT_Update, ENABLE);
    TIM_Cmd(TIM2, ENABLE);
}

void Fn_TIM4_Init (void){

```

```

        RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM4, ENABLE);

        TIM_TimeBaseStructure.TIM_Prescaler
=
        ((SystemCoreClock/2)/1000000)-1; // TIM2,3,4 Chia tan 2, 1 chia tan 1
        TIM_TimeBaseStructure.TIM_Period = 1000 - 1;
        TIM_TimeBaseStructure.TIM_ClockDivision = 0;
        TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
        TIM_TimeBaseInit(TIM4, &TIM_TimeBaseStructure);
        TIM_ITConfig(TIM4, TIM_IT_Update, ENABLE);
        TIM_Cmd(TIM4, ENABLE);
    }

void TIM4_IRQHandler(void){
    if (TIM_GetITStatus(TIM4, TIM_IT_Update) != RESET){
        if( GPIO_ReadOutputDataBit(GPIOB, GPIO_Pin_0) == Bit_SET){
            GPIO_WriteBit(GPIOB, GPIO_Pin_0, Bit_RESET);
        }
        else{
            GPIO_WriteBit(GPIOB, GPIO_Pin_0, Bit_SET);
        }
        TIM_ClearITPendingBit(TIM4, TIM_IT_Update);
    }
}

```

Chương trình sử dụng 2 Timer là TIM2 và TIM4 trong đó TIM4 hoạt động với ngắt và TIM2 hoạt động không sử dụng ngắt.

Với TIM4 mỗi khi giá trị đếm tràn so với giá trị mong muốn chương trình thực hiện gọi một ngắt thực hiện đảo trạng thái của đèn LED tại chân B0.

Với TIM2 do không sử dụng ngắt nên chương trình cần liên tục kiểm tra trạng thái của cờ tràn của bộ đếm. Khi cờ tràn xảy ra chương trình phần mềm sẽ thực hiện đảo trạng thái tại chân B1.

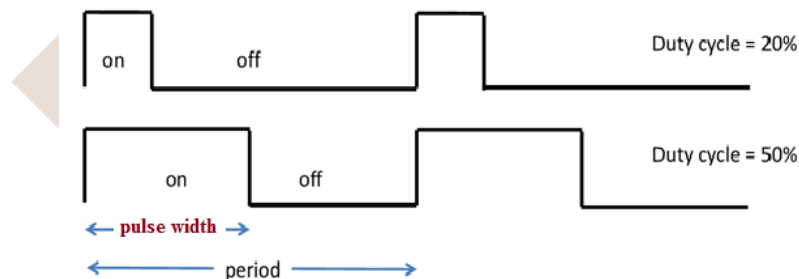
Để khởi tạo và sử dụng Timer sử dụng thư viện SPL cần khai báo thư viện `stm32f10x_tim.h` trong đó có một struct quan trọng để khởi tạo cấu hình Timer có tên là: `TIM_TimeBaseInitTypeDef`.

Để khởi tạo cho một Timer cần nạp giá trị đầy đủ cho một biến dữ liệu có kiểu là TIM_TimeBaseInitTypeDef sau đó nạp giá trị đó vào cho bộ Timer cần sử dụng. Cụ thể các giá trị cần quan tâm được liệt kê ở bảng dưới đây. Tùy thuộc vào mục đích sử dụng, thời gian đếm cho mỗi chu kỳ mà cần có những tính toán phù hợp để cấu hình cho Timer hoạt động.

TIM_Prescaler	Hệ số chia tần của Timer. Ví dụ muốn chia tần 1000 thì cần nạp vào giá trị là 199
TIM_Period	Chu kỳ đếm
TIM_ClockDivision	Giá trị chia tần nguồn clock cung cấp cho Timer.
TIM_CounterMode	Chế độ đếm: Up hoặc Down

Lập trình điều chế độ rộng xung - PWM (Pulse-width modulation)

Bộ điều chế xung, hay còn gọi là bộ “*băm xung*” là bộ xử lý và điều khiển tạo ra dạng xung vuông chu kỳ thay đổi theo cấu hình. Đây là phương pháp điều chỉnh điện áp ra tải dựa vào trung bình tín hiệu điều chế. Khi độ rộng xung tăng, trung bình điện áp ra tăng. Các module PWM thường sử dụng tần số điều chế không đổi, và điều chỉnh dựa trên sự thay đổi của chu kỳ nhiệm vụ (Duty Cycle).



Hình 5. 29: Điều chế độ rộng xung PWM

- Duty cycle là tỷ lệ phần trăm mức cao.
- Period là chu kỳ xung.
- Pulse width là giá trị mức cao so với period.

PWM được ứng dụng nhiều trong điều khiển như điều khiển động cơ và các bộ xung áp, điều áp... Sử dụng PWM điều khiển độ nhanh chậm của động cơ và sườn định tốc độ động cơ. Ngoài lĩnh vực điều khiển hay ổn định tải thì PWM còn tham gia và điều chế các mạch nguồn như : boot, buck, nghịch lưu 1 pha và 3 pha... PWM chuyên dùng để điều khiển các phần tử điện tử công suất có đường đặc tính là tuyến tính khi có sẵn 1 nguồn 1 chiều cố định.

Như hình vẽ bên trên và theo công thức $Duty\ cycle = pulse\ width \times 100 / period$. Ta thấy pwm có 2 thành phần chính để tạo ra duty cycle của pwm là : độ rộng của 1 chu kỳ xung (period) và giá trị của xung cao là bao nhiêu so với period (pulse width). Do đó để cấu hình sử dụng pwm ta có thể chia làm 2 phần như sau :

- Cấu hình timer cho pwm: quyết định độ rộng của 1 chu kỳ xung pwm là bao nhiêu (period)
- Cấu hình pwm: quyết định phần trăm của xung mức cao là bao nhiêu phần trăm (pulse width)

PWM là một chức năng tích hợp của bộ Timer vì vậy để sử dụng PWM cần khai báo thư viện timer. Cụ thể là thư viện: `stm32f10x_tim.h`.

Để sử dụng PWM trong Timer cần tìm hiểu 2 kiểu dữ liệu dạng struct sau:

`TIM_TimeBaseInitTypeDef`: Khai báo thông tin hoạt động cơ bản của Timer

`TIM_OCInitTypeDef`: Khai báo thông tin hoạt động của bộ PWM

Trong đó đoạn chương trình nạp thông tin cấu hình cho bộ PWM bao gồm 2 đoạn: Cấu hình cho Timer điều khiển PWM và cấu hình cho hoạt động của bộ PWM. Mỗi Timer hỗ trợ 4 bộ PWM vì vậy tùy thuộc vào nhu cầu sử dụng mà người lập trình thiết lập và cấu hình số chân cần điều khiển phù hợp. Chương trình sau viết cho Timer 1 và thực hiện cấu hình cho cả 4 kênh PWM.

Chương trình cụ thể như sau:

```
TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
TIM_OCInitTypeDef TIM_OCInitStructure;

RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM1, ENABLE);
TIM_TimeBaseStructure.TIM_Prescaler = 0;
TIM_TimeBaseStructure.TIM_Period = PWM_DUTY;
TIM_TimeBaseStructure.TIM_ClockDivision = 0;
TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
TIM_TimeBaseInit(TIM1, &TIM_TimeBaseStructure);

TIM_OCStructInit(&TIM_OCInitStructure);
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
```

```

TIM_OCInitStructure.TIM_OutputNState = TIM_OutputNState_Enable;
TIM_OCInitStructure.TIM_Pulse = 0;

TIM_OC1Init(TIM1, &TIM_OCInitStructure);
TIM_OC1PreloadConfig(TIM1, TIM_OCPreload_Enable);

TIM_OC2Init(TIM1, &TIM_OCInitStructure);
TIM_OC2PreloadConfig(TIM1, TIM_OCPreload_Enable);

TIM_OC3Init(TIM1, &TIM_OCInitStructure);
TIM_OC3PreloadConfig(TIM1, TIM_OCPreload_Enable);

TIM_OC4Init(TIM1, &TIM_OCInitStructure);
TIM_OC4PreloadConfig(TIM1, TIM_OCPreload_Enable);

TIM_ARRPreloadConfig(TIM1, ENABLE);
TIM_Cmd(TIM1, ENABLE);
TIM_CtrlPWMOutputs(TIM1, ENABLE);

```

Trong quá trình sử dụng chương trình có thể thay đổi độ rộng xung của từng kênh một cách độc lập sử dụng thanh ghi CCR1, CCR2, CCR3, CCR4 tương ứng của bộ Timer. Khi nạp lại giá trị vào thanh ghi này giá trị độ rộng xung sẽ tự động thay đổi.

Lưu ý: Giá trị của các thanh ghi CCRx không được lớn hơn giá trị Period đã nạp trước đó.

Dưới đây là một đoạn chương trình dùng để cập nhật độ rộng xung cho cả 4 kênh PWM của một bộ Timer:

```

#define PWM_DUTY (50000)

TIM1->CCR1 = 10 * (PWM_DUTY / 100); //10% Duty cycle
TIM1->CCR2 = 30 * (PWM_DUTY / 100); //30% Duty cycle
TIM1->CCR3 = 60 * (PWM_DUTY / 100); //60% Duty cycle
TIM1->CCR4 = 90 * (PWM_DUTY / 100); //90% Duty cycle

```


Hơn nữa, việc điều chế độ rộng xung cần thông qua các cổng của vi điều khiển và vi điều khiển STM32 có những yêu cầu đặc biệt về chế độ của cổng khi sử dụng với các bộ ngoại vi bên trong. Cụ thể với Timer vi điều khiển yêu cầu thông tin cấu hình của các cổng tương ứng như sau:

Table 22. Advanced timers TIM1 and TIM8

TIM1/8 pinout	Configuration	GPIO configuration
TIM1/8_CHx	Input capture channel x	Input floating
	Output compare channel x	Alternate function push-pull
TIM1/8_CHxN	Complementary output channel x	Alternate function push-pull
TIM1/8_BKIN	Break input	Input floating
TIM1/8_ETR	External trigger timer input	Input floating

Table 23. General-purpose timers TIM2/3/4/5

TIM2/3/4/5 pinout	Configuration	GPIO configuration
TIM2/3/4/5_CHx	Input capture channel x	Input floating
	Output compare channel x	Alternate function push-pull
TIM2/3/4/5_ETR	External trigger timer input	Input floating

Hình 5. 30: Thông tin cấu hình PWM cho các Timer

PWM hoạt động dựa trên chế độ Output Compare Channel do vậy để các chân tương ứng hoạt động ở chế độ PWM cần cấu hình các chân đó ở chế độ hoạt động Alternate function push-pull. Có thể tham khảo phần lập trình điều khiển IO để nắm lại phần kiến thức này.

Chương trình đầy đủ điều khiển 4 bộ PWM cụ thể như sau:

```
#include "stm32f10x.h" // Device header
#include "stm32f10x_rcc.h" // Keil::Device:StdPeriph Drivers:RCC
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO
#include "stm32f10x_tim.h" // Keil::Device:StdPeriph Drivers:TIM

GPIO_InitTypeDef GPIO_InitStructure;
TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
TIM_OCInitTypeDef TIM_OCInitStructure;

#define PWM_DUTY (50000)

void TIM_PWM_Configuration(void);
```

```

int main (void){
    SystemInit();
    SystemCoreClockUpdate();

    TIM_PWM_Configuration();
    TIM1->CCR1 = 10 * (PWM_DUTY / 100); //10% Duty cycle
    TIM1->CCR2 = 30 * (PWM_DUTY / 100); //30% Duty cycle
    TIM1->CCR3 = 60 * (PWM_DUTY / 100); //60% Duty cycle
    TIM1->CCR4 = 90 * (PWM_DUTY / 100); //90% Duty cycle
    while(1);
}

void TIM_PWM_Configuration(void){
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM1, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_8 | GPIO_Pin_9 |
GPIO_Pin_10 | GPIO_Pin_11;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    TIM_TimeBaseStructure.TIM_Prescaler = 0;
    TIM_TimeBaseStructure.TIM_Period = PWM_DUTY;
    TIM_TimeBaseStructure.TIM_ClockDivision = 0;
    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM1, &TIM_TimeBaseStructure);

    TIM_OCStructInit(&TIM_OCInitStructure);
    TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;

```

```

    TIM_OCInitStructure.TIM_OCpolarity = TIM_OCpolarity_High;
    TIM_OCInitStructure.TIM_OutputNState = TIM_OutputNState_Enable;
    TIM_OCInitStructure.TIM_Pulse = 0;

    TIM_OC1Init(TIM1, &TIM_OCInitStructure);
    TIM_OC1PreloadConfig(TIM1, TIM_OCPreload_Enable);

    TIM_OC2Init(TIM1, &TIM_OCInitStructure);
    TIM_OC2PreloadConfig(TIM1, TIM_OCPreload_Enable);

    TIM_OC3Init(TIM1, &TIM_OCInitStructure);
    TIM_OC3PreloadConfig(TIM1, TIM_OCPreload_Enable);

    TIM_OC4Init(TIM1, &TIM_OCInitStructure);
    TIM_OC4PreloadConfig(TIM1, TIM_OCPreload_Enable);

    TIM_ARRPreloadConfig(TIM1, ENABLE);
    TIM_Cmd(TIM1, ENABLE);
    TIM_CtrlPWMOutputs(TIM1, ENABLE);
}

```

Lập trình điều khiển bộ UART

Máy tính truyền dữ liệu theo hai phương pháp: song song hoặc nối tiếp. Truyền dữ liệu song song thường sử dụng nhiều đường dây dẫn để truyền dữ liệu đến một khoảng cách xa vài mét ví dụ như trong giao tiếp với máy in và ổ đĩa cứng. Phương pháp này cho phép truyền với tốc độ cao nhưng khoảng cách truyền bị hạn chế. Để truyền dữ liệu đi xa thì cần phương pháp truyền nối tiếp. Theo phương pháp này, dữ liệu được truyền đi theo từng bit.

Để tổ chức truyền tin nối tiếp, trước hết byte dữ liệu được chuyển thành các bit nối tiếp nhờ thanh ghi dịch vào song song –ra nối tiếp. Tiếp đó dữ liệu được truyền qua một đường dữ liệu đơn, ở đầu thu, nhờ một thanh ghi dịch vào nối tiếp-ra song song, dữ liệu

được nhận nối tiếp và được gói thành từng byte một. Khi tín hiệu được truyền đi xa, yêu cầu có một modem để điều chế tín hiệu và sau đó giải điều chế ở bộ thu.

Truyền tin nối tiếp có hai phương pháp: đồng bộ và bất đồng bộ

- Đồng bộ: truyền mỗi lần một khối dữ liệu
- Bất đồng bộ: chỉ truyền từng byte một.

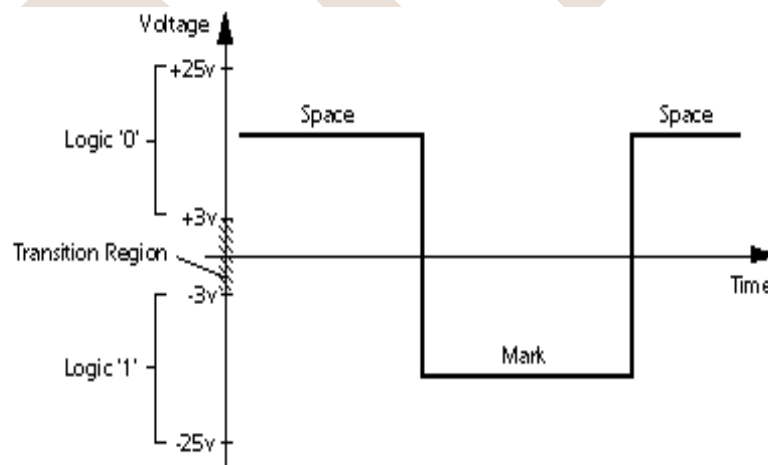
Trong 8051 có bộ thu phát bất đồng bộ tổng hợp UART(Universal Asynchronous Receiver Transmitter).

Trong quá trình truyền, dữ liệu vừa có thể phát vừa có thể thu được gọi là truyền song công, trái lại là truyền đơn công (ví dụ máy in: máy tính chỉ phát dữ liệu).

Chuẩn RS-232

Trong truyền tin nối tiếp, RS-232 là một chuẩn cho kết nối tín hiệu dữ liệu nhị phân nối tiếp giữa một DTE(data terminal equipment) và một DCE(data circuit-terminating equipment) được xây dựng năm 1960 nhằm tương thích dữ các thiết bị truyền dữ liệu nối tiếp giữa các hãng khác nhau. Ngày nay nó là một chuẩn được sử dụng khá rộng rãi. Tuy nhiên do chuẩn này ra đời khá lâu trước khi có họ mạch điện tử TTL vì vậy mức điện áp của nó không tương thích với TTL.

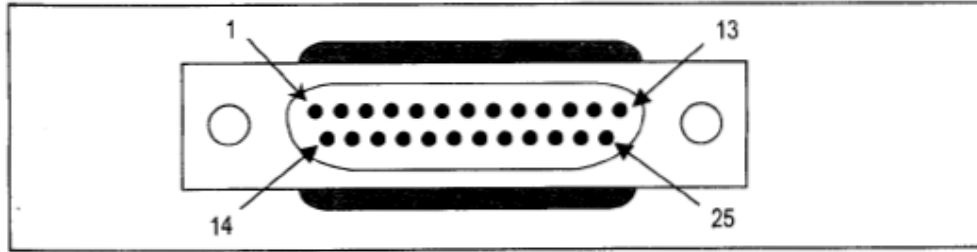
RS-232 định nghĩa mức điện thế mà tương ứng với các mức logic 0 và 1. Tín hiệu có mức điện thế như sau : mức logic 0: từ +3V đến +25V, mức logic 1 : từ -25V đến -3V.



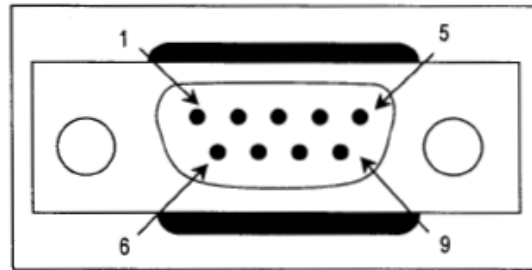
Hình 5. 31: Mức điện thế biểu diễn tín hiệu nhị phân

Về bố trí chân của RS232

RS232 có 2 phiên bản giắc đầu nối là DB-25 và DB-9



Hình 5. 32: Cổng DB25



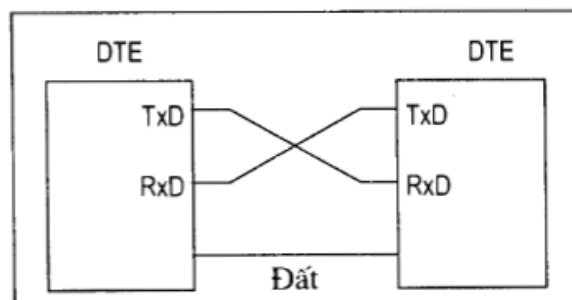
Hình 5. 33: Cổng DB-9

Hiện tại DB-25 ít được sử dụng nên ta chỉ xem xét cách bố trí chân đầu nối của DB-9 thông qua bảng dưới:

Chân	Mô tả	Ý nghĩa
1	Data carrier detect(DCD)	Tách tín hiệu mang dữ liệu
2	Received Data(RxD)	Dữ liệu nhận được
3	Transmitted Data(TxD)	Dữ liệu được gửi
4	Data terminal ready(DTR)	Đầu cuối dữ liệu sẵn sàng
5	Signal Ground(GND)	Đất của tín hiệu
6	Data set ready(DSR)	Dữ liệu sẵn sàng
7	Request to send(RTS)	Yêu cầu gửi
8	Clear to send(CTS)	Xóa để gửi
9	Ring indicator(RL)	Báo chuông

Bảng 5. 1: Bảng các chân của cổng DB-9

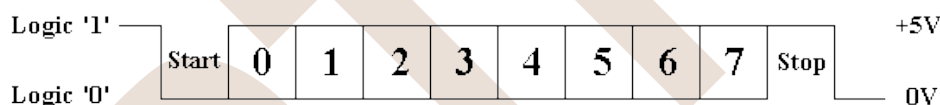
Một kết nối đơn giản nhất giữa PC và bộ vi điều khiển yêu cầu tối thiểu 3 chân TxD, RxD, GND như trong hình vẽ dưới đây :



Hình 5. 34: Kết nối tối thiểu giữa PC với vi điều khiển trên giao tiếp UART

Định dạng bản tin:

Trong truyền bất đồng bộ, đường liên kết không bao gồm một đường clock, vì trong mỗi đầu cuối thiết bị đều có một clock. Vì vậy hai clock phải có chung tần số, cho phép sai khác nhỏ. Mỗi một byte được truyền gồm một bit Start để đồng bộ các clock, và một hoặc nhiều bit Stop để báo hiệu kết thúc một từ truyền. Một đường truyền bất đồng bộ sử dụng nhiều định dạng, nhưng định dạng phổ biến nhất là 8-N-1 ở đó bộ truyền sẽ gửi mỗi byte dữ liệu với 1 bit Start, theo sau 8 bit dữ liệu, bắt đầu với bit 0 (LSB) và kết thúc với 1 bit Stop như mô tả hình dưới



Hình 5. 35: Định dạng 8-N-1

Mô tả các chân UART:

Pin	Type	Mô tả
RXD0/2/3	Input	Serial Input : dữ liệu nhận nối tiếp
TXD0/2/3	Output	Serial Output: dữ liệu truyền nối tiếp

Hình 5. 36: Thông tin các chân giao tiếp UART

STM32F1 có 3 bộ UART với nhiều mode hoạt động, với nhiều bộ UART ta có thể sử dụng được nhiều ứng dụng. Một số tính năng nổi bật như sau:

- Đầy đủ các tính năng của bộ giao tiếp không đồng bộ.
- Điều chỉnh baud rate bằng lập trình và tốc độ tối đa lên đến 4.5Mb/s.
- Độ dài được lập trình là 8 hoặc 9 bit.
- Cấu hình bit stop hỗ trợ là 1 hoặc 2.
- Có chân clock nếu muốn chuyển giao tiếp thành đồng bộ.

- Cấu hình sử dụng 1 dây hoặc 2 dây.
- Có bộ DMA nếu muốn đẩy cao thời gian truyền nhận.
- Bit cho phép truyền nhận riêng biệt.

Để lập trình UART với thư viện SPL chúng ta cần tiến hành thêm thư viện `stm32f10x_usart.h` vào dự án bằng cách sử dụng Manage Run như hướng dẫn ở trên. Thư viện này cung cấp một kiểu dữ liệu nhằm đơn giản hóa việc khai báo, cấu hình và sử dụng bộ USART của STM32F1. Cụ thể:

```
typedef struct
{
    uint32_t USART_BaudRate; /*!< This member configures the USART
communication baud rate.

The baud rate is computed using the following formula:
- IntegerDivider = ((PCLKx) / (16 * (USART_InitStruct-
>USART_BaudRate)))
- FractionalDivider = ((IntegerDivider - ((u32) IntegerDivider)) * 16)
+ 0.5 */

    uint16_t USART_WordLength; /*!< Specifies the number of data bits
transmitted or received in a frame.

This parameter can be a value of @ref USART_Word_Length */

    uint16_t USART_StopBits; /*!< Specifies the number of stop bits
transmitted.

This parameter can be a value of @ref USART_Stop_Bits */

    uint16_t USART_Parity; /*!< Specifies the parity mode.

This parameter can be a value of @ref USART_Parity
@note When parity is enabled, the computed parity is inserted
at the MSB position of the transmitted data (9th bit when
the word length is set to 9 data bits; 8th bit when the
word length is set to 8 data bits). */

    uint16_t USART_Mode; /*!< Specifies whether the Receive or Transmit
```

mode is enabled or disabled.

This parameter can be a value of @ref USART_Mode */

```
uint16_t USART_HardwareFlowControl; /*!< Specifies whether the hardware
flow control mode is enabled
```

or disabled.

This parameter can be a value of @ref USART_Hardware_Flow_Control */

```
} USART_InitTypeDef;
```

Để khởi động bộ USART của STM32F1 cần thực hiện nạp đầy đủ các thông số của biến có kiểu dữ liệu USART_InitTypeDef sau đó nạp cấu hình đó vào bộ USART cần sử dụng. Cụ thể các giá trị của kiểu dữ liệu này được giải thích như sau:

USART_BaudRate	Tốc độ Baudrate cần sử dụng
USART_WordLength	Độ dài dữ liệu mỗi khung truyền
USART_StopBits	Chế độ stop bit cần sử dụng
USART_Parity	Sử dụng bộ chẵn lẻ để phát hiện sai
USART_HardwareFlowControl	Chế độ sử dụng phản cứng theo dõi hay không?
USART_Mode	Chế độ sử dụng với USART: Truyền, nhận

Tương tự như bộ PWM, bộ USART cũng yêu cầu các cổng giao tiếp cần hoạt động ở các chế độ đặc biệt. Thông tin yêu cầu của bộ USART về cổng kết nối cụ thể như sau:

Table 24. USARTs

USART pinout	Configuration	GPIO configuration
USARTx_TX ⁽¹⁾	Full duplex	Alternate function push-pull
	Half duplex synchronous mode	Alternate function push-pull
USARTx_RX	Full duplex	Input floating / Input pull-up
	Half duplex synchronous mode	Not used. Can be used as a general IO
USARTx_CK	Synchronous mode	Alternate function push-pull
USARTx_RTS	Hardware flow control	Alternate function push-pull
USARTx_CTS	Hardware flow control	Input floating/ Input pull-up

Hình 5. 37: Cấu hình thông tin các chân của giao tiếp UART

Với 2 chân sử dụng chính trong USART là RX và TX vì điều khiển yêu cầu được cấu hình ở 2 chế độ sau: Alternate function push-pull và Input floating/Input pull-up.

Chương trình sau được viết để sử dụng bộ USART1 hoạt động ở cả 2 chế độ RX và TX. Trong đó vì điều khiển thực hiện định kỳ truyền sau mỗi 100ms và thực hiện dữ liệu và lưu trữ vào biến `vrsc_UART_ReceiveTemp`.

Cụ thể chương trình như sau:

```
#include "stm32f10x.h" // Device header
#include "stm32f10x_rcc.h" // Keil::Device:StdPeriph Drivers:RCC
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO
#include "stm32f10x_usart.h" // Keil::Device:StdPeriph Drivers:USART
#include "Delay.h"

USART_InitTypeDef USART_InitStructure;
GPIO_InitTypeDef GPIO_InitStructure;

volatile char vrsc_UART_ReceiveTemp;
volatile char vr_UART_Buffer[1024];
volatile char vr_UART_Count;
volatile char vr_UART_RXDone;

void Fn_UART_Init (unsigned int _vrui_BaudRate);
void Fn_UART_SendChar (char _vrsc_Char);
void Fn_UART_PutStr (char *_vrsc_String);
char Fn_UART_Recieve (void);

int main (void){
    SystemInit();
    SystemCoreClockUpdate();
    Fn_DELAY_Init(72);
    Fn_UART_Init(115200);
    while(1){
```

```

        Fn_UART_PutStr("Hello!\n");
        Fn_DELAY_ms(100);
    }
}

void Fn_UART_Init (unsigned int _vrui_BaudRate){
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1, ENABLE);

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA | RCC_APB2ENR_AFIOEN,
    ENABLE);

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    USART_InitStructure.USART_BaudRate = _vrui_BaudRate;
    USART_InitStructure.USART_WordLength = USART_WordLength_8b;
    USART_InitStructure.USART_StopBits = USART_StopBits_1;
    USART_InitStructure.USART_Parity = USART_Parity_No;
    USART_InitStructure.USART_HardwareFlowControl =
    USART_HardwareFlowControl_None;
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx;
    USART_Init(USART1, &USART_InitStructure);
    USART_ITConfig(USART1, USART_IT_RXNE ,ENABLE);// Cho phep ngat
    USART_Cmd(USART1, ENABLE);
}

```

```

void Fn_UART_SendChar (char _vrsc_Char){
    USART_SendData(USART1, _vrsc_Char);
    while (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET); //Cho
    den khi gui di xong
}

void Fn_UART_PutStr (char *_vrsc_String){
    while(*_vrsc_String){
        Fn_UART_SendChar(*_vrsc_String);
        _vrsc_String++;
    }
}

char Fn_UART_Recieve (void){
    while (USART_GetFlagStatus(USART1, USART_IT_RXNE) == RESET); //
    Cho co nhan Reset
    return (USART_ReceiveData(USART1));
}

void USART1_IRQHandler (void){
    if(USART_GetFlagStatus(USART1, USART_IT_RXNE) != RESET){
        vrsc_UART_ReceiveTemp = USART_ReceiveData(USART1);
    }
    USART_ClearITPendingBit(USART1,USART_IT_RXNE);
}

```

Lập trình điều khiển SPI

SPI (Serial Peripheral Bus) là một chuẩn truyền thông nối tiếp tốc độ cao do hãng Motorola đề xuất. Đây là kiểu truyền thông Master-Slave, trong đó có 1 chip Master điều phối quá trình truyền thông và các chip Slaves được điều khiển bởi Master vì thế truyền thông chỉ xảy ra giữa Master và Slave. SPI là một cách truyền song công (full duplex)

nghĩa là tại cùng một thời điểm quá trình truyền và nhận có thể xảy ra đồng thời. SPI đôi khi được gọi là chuẩn truyền thông “4 dây” vì có 4 đường giao tiếp trong chuẩn này đó là SCK (Serial Clock), MISO (Master Input Slave Output), MOSI (Master Output Slave Input) và SS (Slave Select). Hình 1 thể hiện một kết SPI giữa một chip Master và 3 chip Slave thông qua 4 đường.

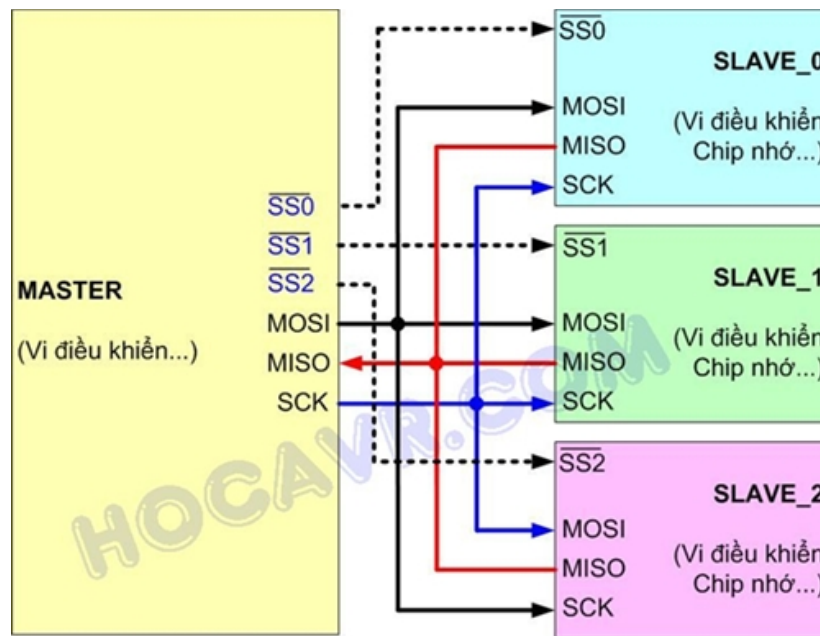
SCK: Xung giữ nhịp cho giao tiếp SPI, vì SPI là chuẩn truyền đồng bộ nên cần 1 đường giữ nhịp, mỗi nhịp trên chân SCK báo 1 bit dữ liệu đến hoặc đi. Đây là điểm khác biệt với truyền thông không đồng bộ mà chúng ta đã biết trong chuẩn UART. Sự tồn tại của chân SCK giúp quá trình truyền ít bị lỗi và vì thế tốc độ truyền của SPI có thể đạt rất cao. Xung nhịp chỉ được tạo ra bởi chip Master.

MISO – Master Input / Slave Output: nếu là chip Master thì đây là đường Input còn nếu là chip Slave thì MISO lại là Output. MISO của Master và các Slaves được nối trực tiếp với nhau.

MOSI – Master Output / Slave Input: nếu là chip Master thì đây là đường Output còn nếu là chip Slave thì MOSI là Input. MOSI của Master và các Slaves được nối trực tiếp với nhau.

SS – Slave Select: SS là đường chọn Slave cần giao tiếp, trên các chip Slave đường SS sẽ ở mức cao khi không làm việc. Nếu chip Master kéo đường SS của một Slave nào đó xuống mức thấp thì việc giao tiếp sẽ xảy ra giữa Master và Slave đó. Chỉ có 1 đường SS trên mỗi Slave nhưng có thể có nhiều đường điều khiển SS trên Master, tùy thuộc vào thiết kế của người dùng.

Hoạt động: mỗi chip Master hay Slave có một thanh ghi dữ liệu 8 bits. Cứ mỗi xung nhịp do Master tạo ra trên đường giữ nhịp SCK, một bit trong thanh ghi dữ liệu của Master được truyền qua Slave trên đường MOSI, đồng thời một bit trong thanh ghi dữ liệu của chip Slave cũng được truyền qua Master trên đường MISO. Do 2 gói dữ liệu trên 2 chip được gửi qua lại đồng thời nên quá trình truyền dữ liệu này được gọi là “song công”.



Hình 5. 38: Kết nối trong giao tiếp SPI

Cực của xung giữ nhịp, phase và các chế độ hoạt động: cực của xung giữ nhịp (Clock Polarity) được gọi tắt là CPOL là khái niệm dùng chỉ trạng thái của chân SCK ở trạng thái nghỉ. Ở trạng thái nghỉ (Idle), chân SCK có thể được giữ ở mức cao (CPOL=1) hoặc thấp (CPOL=0). Phase (CPHA) dùng để chỉ cách mà dữ liệu được lấy mẫu (sample) theo xung giữ nhịp. Dữ liệu có thể được lấy mẫu ở cạnh lên của SCK (CPHA=0) hoặc cạnh xuống (CPHA=1). Sự kết hợp của SPOL và CPHA làm nên 4 chế độ hoạt động của SPI. Nhìn chung việc chọn 1 trong 4 chế độ này không ảnh hưởng đến chất lượng truyền thông mà chỉ cốt sao cho có sự tương thích giữa Master và Slave.

Kiểu dữ liệu cấu trúc dùng để cấu hình SPI của STM32F1 có dạng như sau:

```
typedef struct
{
    uint16_t SPI_Direction; /*!< Specifies the SPI unidirectional or
    bidirectional data mode.
    This parameter can be a value of @ref SPI_data_direction */

    uint16_t SPI_Mode; /*!< Specifies the SPI operating mode.
    This parameter can be a value of @ref SPI_mode */

    uint16_t SPI_DataSize; /*!< Specifies the SPI data size.
    This parameter can be a value of @ref SPI_data_size */
```

```

uint16_t SPI_CPOL; /*!< Specifies the serial clock steady state.
This parameter can be a value of @ref SPI_Clock_Polarity */

uint16_t SPI_CPHA; /*!< Specifies the clock active edge for the bit
capture.
This parameter can be a value of @ref SPI_Clock_Phase */

uint16_t SPI_NSS; /*!< Specifies whether the NSS signal is managed by
hardware (NSS pin) or by software using the SSI bit.
This parameter can be a value of @ref SPI_Slave_Select_management */

uint16_t SPI_BaudRatePrescaler; /*!< Specifies the Baud Rate prescaler
value which will be
used to configure the transmit and receive SCK clock.
This parameter can be a value of @ref SPI_BaudRate_Prescaler.
@note The communication clock is derived from the master
clock. The slave clock does not need to be set. */

uint16_t SPI_FirstBit; /*!< Specifies whether data transfers start
from MSB or LSB bit.
This parameter can be a value of @ref SPI_MSB_LSB_transmission */

uint16_t SPI_CRCPolynomial; /*!< Specifies the polynomial used for the
CRC calculation. */
}SPI_InitTypeDef;

```

Trong đó mỗi trường quy định thông tin cụ thể theo bảng sau:

SPI_Mode	Chế độ hoạt động Master hoặc Slave
SPI_DataSize	Kích thước dữ liệu 8 bit hoặc 16 bit
SPI_CPOL	Cấu hình CPOL
SPI_CPHA	Cấu hình CPHA

SPI_NSS	Chế độ chip Select (Cứng hoặc mềm)
SPI_BaudRatePrescaler	Tốc độ truyền dữ liệu
SPI_FirstBit	Bit đầu tiên truyền (MSB/LSB)
SPI_CRCPolynomial	Chế độ kiểm tra lỗi CRC

Bảng 5. 2: Các thông tin cấu hình giao tiếp SPI

Với các thông tin trên chúng ta thiết lập một đoạn chương trình cấu hình SPI như sau:

```

RCC_APB2PeriphClockCmd(RCC_APB2Periph_SPI1, ENABLE);
SPI_InitStruct.SPI_Mode = SPI_Mode_Master;
SPI_InitStruct.SPI_DataSize = SPI_DataSize_8b;
SPI_InitStruct.SPI_CPOL = SPI_CPOL_High;
SPI_InitStruct.SPI_CPHA = SPI_CPHA_2Edge;
SPI_InitStruct.SPI_NSS = SPI_NSS_Soft;
SPI_InitStruct.SPI_BaudRatePrescaler = SPI_BaudRatePrescaler_64;
SPI_InitStruct.SPI_FirstBit = SPI_FirstBit_MSB;
SPI_InitStruct.SPI_CRCPolynomial = 7;
SPI_Init(SPI1, &SPI_InitStruct);
SPI_Cmd(SPI1, ENABLE);
SPI_EnableSlave();

```

Tương tự như các bộ ngoại vi khác SPI yêu cầu các cổng kết nối với ngoại vi SPI cần có cấu hình cổng như sau:

Table 25. SPI

SPI pinout	Configuration	GPIO configuration
SPIx_SCK	Master	Alternate function push-pull
	Slave	Input floating
SPIx_MOSI	Full duplex / master	Alternate function push-pull
	Full duplex / slave	Input floating / Input pull-up
	Simplex bidirectional data wire / master	Alternate function push-pull
	Simplex bidirectional data wire/ slave	Not used. Can be used as a GPIO
SPIx_MISO	Full duplex / master	Input floating / Input pull-up
	Full duplex / slave (point to point)	Alternate function push-pull
	Full duplex / slave (multi-slave)	Alternate function open drain
	Simplex bidirectional data wire / master	Not used. Can be used as a GPIO
	Simplex bidirectional data wire/ slave (point to point)	Alternate function push-pull
	Simplex bidirectional data wire/ slave (multi-slave)	Alternate function open drain
SPIx_NSS	Hardware master /slave	Input floating/ Input pull-up / Input pull-down
	Hardware master/ NSS output enabled	Alternate function push-pull
	Software	Not used. Can be used as a GPIO

Bảng 5. 3: Thông tin cấu hình các chân giao tiếp SPI

Chương trình khởi tạo SPI đầy đủ được viết như sau:

<SPI.h>

```

#ifndef _SPI_
#define _SPI_

#include "stm32f10x.h" // Device header
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO
#include "stm32f10x_rcc.h" // Keil::Device:StdPeriph Drivers:RCC
#include "stm32f10x_spi.h" // Keil::Device:StdPeriph Drivers:SPI

#define SPI_PORT      GPIOA
#define SPI_PIN_MOSI   GPIO_Pin_7
#define SPI_PIN_MISO   GPIO_Pin_6
#define SPI_PIN_SCK     GPIO_Pin_5
#define SPI_PIN_SS     GPIO_Pin_4
#define SPIx           SPI1

#ifdef __cplusplus

```



```

extern "C"{
#ifdef

void Fn_SPI_Init (void);
uint8_t Fn_SPIx_Transfer(uint8_t data);

#ifdef __cplusplus
}
#endif

#endif

```

<SPI.c>

```

#include "stm32f10x.h" // Device header
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO
#include "stm32f10x_rcc.h" // Keil::Device:StdPeriph Drivers:RCC
#include "stm32f10x_spi.h"
#include "SPI.h"

void SPI_EnableSlave (void){
    // Set slave SS pin low
    //SPI_PORT->BRR = SPI_PIN_SS;
    GPIO_ResetBits(SPI_PORT, SPI_PIN_SS);
}

static void SPI_DisableSlave (void){
    // Set slave SS pin high
    //SPI_PORT->BSRR = SPI_PIN_SS;
    GPIO_SetBits(SPI_PORT, SPI_PIN_SS);
}

```

```

void Fn_SPI_Init (void){
    SPI_InitTypeDef SPI_InitStruct;
    GPIO_InitTypeDef GPIO_InitStruct;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO
RCC_APB2Periph_GPIOA, ENABLE);

    // GPIO pins for MOSI, MISO, and SCK
    GPIO_InitStruct.GPIO_Pin = SPI_PIN_MOSI | SPI_PIN_MISO | SPI_PIN_SCK;
    GPIO_InitStruct.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_InitStruct.GPIO_Speed = GPIO_Speed_2MHz;
    GPIO_Init(GPIOA, &GPIO_InitStruct);
    // GPIO pin for SS
    GPIO_InitStruct.GPIO_Pin = SPI_PIN_SS;
    GPIO_InitStruct.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStruct.GPIO_Speed = GPIO_Speed_10MHz;
    GPIO_Init(GPIOA, &GPIO_InitStruct);

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_SPI1, ENABLE);
    SPI_InitStruct.SPI_Mode = SPI_Mode_Master;
    SPI_InitStruct.SPI_DataSize = SPI_DataSize_8b;
    SPI_InitStruct.SPI_CPOL = SPI_CPOL_High;
    SPI_InitStruct.SPI_CPHA = SPI_CPHA_2Edge;
    SPI_InitStruct.SPI_NSS = SPI_NSS_Soft;
    SPI_InitStruct.SPI_BaudRatePrescaler = SPI_BaudRatePrescaler_64;
    SPI_InitStruct.SPI_FirstBit = SPI_FirstBit_MSB;
    SPI_InitStruct.SPI_CRCPolynomial = 7;
    SPI_Init(SPI1, &SPI_InitStruct);
    SPI_Cmd(SPI1, ENABLE);
    SPI_EnableSlave();
}

```

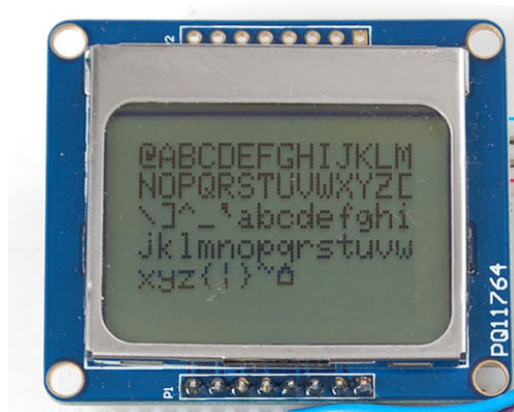
```

uint8_t Fn_SPIx_Transfer(uint8_t data)
{
    // Wait until transmit complete
    while ((SPIx->SR & (SPI_I2S_FLAG_TXE)) == RESET);
    // Write data to be transmitted to the SPI data register
    SPI_I2S_SendData(SPIx,data);
    // Wait until transmit complete
    while ((SPIx->SR & (SPI_I2S_FLAG_TXE)) == RESET);
    // Wait until receive complete
    while (!(SPIx->SR & (SPI_I2S_FLAG_RXNE)));
    // Wait until SPI is not busy anymore
    while (SPIx->SR & (SPI_I2S_FLAG_BSY));
    // Return received data from SPI data register
    return SPIx->DR;
}

```

Như vậy với thư viện SPI bao gồm SPI.h và SPI.c chương trình có thể nhanh chóng thiết lập cấu hình SPI thông qua hàm Fn_SPI_Init và gửi dữ liệu, nhận dữ liệu thông qua hàm Fn_SPIx_Transfer.

Phần này của giáo trình ứng dụng SPI để điều khiển màn hình Graphic LCD có tên là PCD8544. Màn hình này được sử dụng cho điện thoại Nokia 5110. (Chương trình đầy đủ được đính kèm ở phụ lục của giáo trình).



Hình 5. 39: Màn hình PCD8544 sử dụng giao tiếp SPI

Thư viện điều khiển màn hình PCD8544 có các hàm chính như sau:

```
#ifndef LCD5110_Graph_h
#define LCD5110_Graph_h

#include <stdio.h>
#include <string.h>
#include "stm32f10x.h" // Device header
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO
#include "stm32f10x_rcc.h" // Keil::Device:StdPeriph Drivers:RCC
#include "stm32f10x_spi.h" // Keil::Device:StdPeriph Drivers:SPI
#include "Delay.h"
#include "SPI.h"

#define LEFT 0
#define RIGHT 9999
#define CENTER 9998

#define LCD_COMMAND 0
#define LCD_DATA 1
// PCD8544 Commandset
// -----
// General commands
#define PCD8544_POWERDOWN 0x04
#define PCD8544_ENTRYMODE 0x02
#define PCD8544_EXTENDEDINSTRUCTION 0x01
#define PCD8544_DISPLAYBLANK 0x00
#define PCD8544_DISPLAYNORMAL 0x04
#define PCD8544_DISPLAYALLON 0x01
#define PCD8544_DISPLAYINVERTED 0x05
// Normal instruction set
#define PCD8544_FUNCTIONSET 0x20
```

```

#define PCD8544_DISPLAYCONTROL      0x08
#define PCD8544_SETYADDR            0x40
#define PCD8544_SETXADDR            0x80
// Extended instruction set
#define PCD8544_SETTEMP              0x04
#define PCD8544_SETBIAS              0x10
#define PCD8544_SETVOP              0x80
// Display presets
#define LCD_BIAS                     0x03
#define LCD_TEMP                     0x02
#define LCD_CONTRAST                 0x46
#define fontbyte(x) cfont.font[x]
#define bitmapbyte(x) bitmap[x]

#define byte      char

struct _current_font
{
    uint8_t* font;
    uint8_t x_size;
    uint8_t y_size;
    uint8_t offset;
    uint8_t numchars;
    uint8_t inverted;
};

class LCD5110
{
public:
    LCD5110 (GPIO_TypeDef *_SPI_Port,int SCK, int MOSI,
GPIO_TypeDef *_LCD_Port, int DC, int RST, int CS);

```

```

void InitLCD(int contrast=LCD_CONTRAST);
void setContrast(int contrast);
void enableSleep();
void disableSleep();
void update();
void clrScr();
void fillScr();
void invert(bool mode);
void setPixel(uint16_t x, uint16_t y);
void clrPixel(uint16_t x, uint16_t y);
void invPixel(uint16_t x, uint16_t y);
void invertText(bool mode);
void print(char *st, int x, int y);
void print(const char *st, int x, int y);
void printNumI(long num, int x, int y, int length=0, char
filler=' ');
void printNumF(double num, byte dec, int x, int y, char
divider='.', int length=0, char filler=' ');
void setFont(uint8_t* font);
void drawBitmap(int x, int y, uint8_t* bitmap, int sx, int
sy);

void drawLine(int x1, int y1, int x2, int y2);
void clrLine(int x1, int y1, int x2, int y2);
void drawRect(int x1, int y1, int x2, int y2);
void clrRect(int x1, int y1, int x2, int y2);
void drawRoundRect(int x1, int y1, int x2, int y2);
void clrRoundRect(int x1, int y1, int x2, int y2);
void drawCircle(int x, int y, int radius);
void clrCircle(int x, int y, int radius);

```

protected:

```

        GPIO_TypeDef      *SPI_Port, *LCD_Port;
        int                Pin_MOSI, Pin_SCK, Pin_CD, Pin_RST,
Pin_CS;

        _current_font     cfont;
        uint8_t            scrbuf[504];
        bool               _sleep;
        int                 _contrast;

        int abs (int x);
        void _LCD_Write(unsigned char data, unsigned char mode);
        void _print_char(unsigned char c, int x, int row);
        void _convert_float(char *buf, double num, int width, byte
prec);

        void drawHLine(int x, int y, int l);
        void clrHLine(int x, int y, int l);
        void drawVLine(int x, int y, int l);
        void clrVLine(int x, int y, int l);
};

#endif

```

Lập trình ADC

Các bộ ADC như tên gọi đã chỉ rõ là các thiết bị thực hiện chuyển đổi 1 tín hiệu liên tục thành các xung rời rạc. Cụ thể trong điện tử chúng chuyển đổi 1 đầu vào tương tự thành 1 giá trị số hóa tương đương với mức của biên độ của tín hiệu đầu vào hoặc ở dạng dòng điện hoặc ở dạng điện thế trên một dải giá trị biên độ của tín hiệu đầu vào.

Khi xét đến bộ ADC cần quan tâm đến 2 tham số là độ chính xác và tốc độ chuyển đổi.

Độ chính xác mô tả số các mức lượng tử rời rạc mà bộ ADC có thể tạo ra trên dải các giá trị tín hiệu đầu vào tương tự. Các giá trị thông thường được lưu trong các thiết bị số ở dạng nhị phân nên độ chính xác cũng được biểu diễn dưới dạng số bit. Ví dụ một ADC với độ chính xác là 8 bit có khả năng mã hóa một đầu vào tương tự sang 1 trong 256 mức khác nhau. Giá trị có thể là từ 0 đến 256 (số nguyên không dấu) hoặc từ -128 đến 127 (số nguyên có dấu) tùy thuộc vào ứng dụng.

Tốc độ chuyển đổi là thời gian đáp ứng từ khi bộ ADC bắt đầu chuyển đổi cho đến khi bộ ADC đưa ra được mức lượng tử của đầu vào.

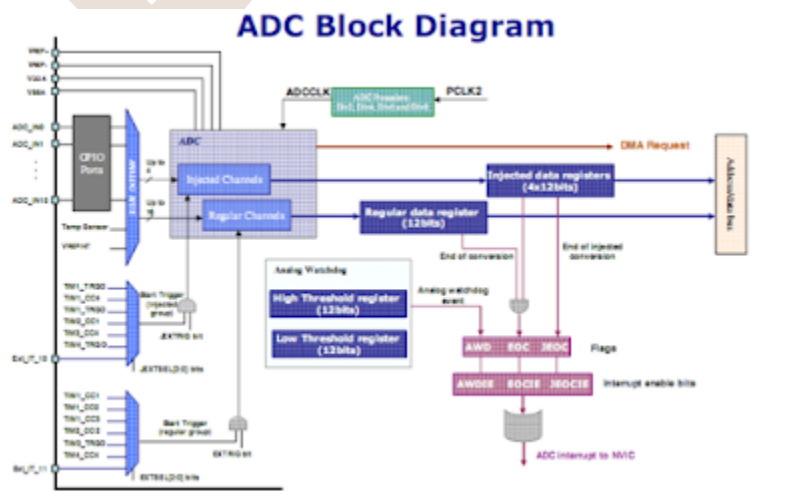
Độ chính xác càng cao và thời gian chuyển đổi càng nhanh thì bộ ADC có chất lượng càng tốt.

ADC trong STM32F1 là bộ ADC có 12 bit tức là giá trị đọc về nằm trong khoảng $0 \rightarrow 2^{12} = 4096$. Giá trị điện áp đầu vào bộ ADC được cung cấp trên chân VDDA và thường lấy bằng giá trị cấp nguồn cho vi điều khiển VDD(+3V3). STM32F103C8 có 2 kênh ADC đó là ADC1 và ADC2, mỗi kênh có tối đa là 9 channel với nhiều mode hoạt động như: single, continuous, scan hoặc discontinuous. Kết quả chuyển đổi được lưu trữ trong thanh ghi 16 bit.

Một số đặc tính chính được liệt kê như sau:

- Độ phân giải 12 bit tương ứng với giá trị maximum là 4095.
- Có các ngắt hỗ trợ như End conversion, End of Injected Conversion and Analog Watchdog Event.
- Single mode hay Continuous mode.
- Tự động calib và có thể điều khiển hoạt động ADC bằng xung Trigger.
- Thời gian chuyển đổi nhanh : 1us tại tần số 65Mhz.
- Điện áp cung cấp cho bộ ADC là 2.4V $\rightarrow$ 3.6V. Nên điện áp Input của thiết bị có ADC $2.4V \leq V_{IN} \leq 3.6V$.
- Có bộ DMA giúp tăng tốc độ xử lý.

Sơ đồ khối của bộ ADC trong STM32F1 được mô tả trong hình sau:



Hình 5. 40: Sơ đồ khối bộ ADC

Cấu trúc biến khởi tạo ADC cho STM32F1 có dạng như sau:


```

typedef struct
{
    uint32_t ADC_Mode; /*!< Configures the ADC to operate in independent
or
    dual mode.
    This parameter can be a value of @ref ADC_mode */

    FunctionalState ADC_ScanConvMode; /*!< Specifies whether the
conversion is performed in
    Scan (multichannels) or Single (one channel) mode.
    This parameter can be set to ENABLE or DISABLE */

    FunctionalState ADC_ContinuousConvMode; /*!< Specifies whether the
conversion is performed in
    Continuous or Single mode.
    This parameter can be set to ENABLE or DISABLE. */

    uint32_t ADC_ExternalTrigConv; /*!< Defines the external trigger used
to start the analog
    to digital conversion of regular channels. This parameter
    can be a value of @ref
ADC_external_trigger_sources_for_regular_channels_conversion */

    uint32_t ADC_DataAlign; /*!< Specifies whether the ADC data alignment
is left or right.
    This parameter can be a value of @ref ADC_data_align */

    uint8_t ADC_NbrOfChannel; /*!< Specifies the number of ADC channels
that will be converted
    using the sequencer for regular channel group.
    This parameter must range from 1 to 16. */
}ADC_InitTypeDef;

```

Trong đó các trường của kiểu dữ liệu cấu trúc này có ý nghĩa cụ thể như sau:

ADC_Mode	Chế độ hoạt động của bộ ADC
ADC_ScanConvMode	Chế độ đọc nhiều kênh
ADC_ContinuousConvMode	Chế độ đọc liên tục, rời rạc
ADC_ExternalTrigConv	Kích hoạt chuyển đổi bởi nguồn kích thích ngoài
ADC_DataAlign	Chế độ căn chỉnh dữ liệu
ADC_NbrOfChannel	Số lượng kênh dữ liệu đọc

Bảng 5. 4: Các thông tin cấu hình ADC

Chương trình dưới đây trình bày một ví dụ đơn giản về đọc 1 kênh dữ liệu ADC ở chế độ không liên tục. Để bộ ADC hoạt động thì cổng ADC cần đọc cần được cấu hình ở trạng thái Analog Input. Chương trình cụ thể như sau:

```
#include "stm32f10x.h" // Device header
#include "stm32f10x_rcc.h" // Keil::Device:StdPeriph Drivers:RCC
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO
#include "stm32f10x_adc.h" // Keil::Device:StdPeriph Drivers:ADC
#include "Delay.h"

ADC_InitTypeDef ADC_InitStructure;
GPIO_InitTypeDef GPIO_InitStructure;
unsigned int vrui_ADC_Value;

void Fn_ADC_Init (void);
unsigned int Fn_ADC_Read (void);

int main (void){
    SystemInit();
    SystemCoreClockUpdate();

    Fn_ADC_Init();
    while(1){
        vrui_ADC_Value = Fn_ADC_Read();
    }
}
```

```

        Fn_DELAY_ms(100);
    }
}

void Fn_ADC_Init (void){
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1 ;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    RCC_ADCCLKConfig (RCC_PCLK2_Div8);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);

    ADC_InitStructure.ADC_Mode = ADC_Mode_Independent;
    ADC_InitStructure.ADC_ScanConvMode = DISABLE;
    ADC_InitStructure.ADC_ContinuousConvMode = ENABLE;
    ADC_InitStructure.ADC_ExternalTrigConv
ADC_ExternalTrigConv_None;
    ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
    ADC_InitStructure.ADC_NbrOfChannel = 1;

    ADC-RegularChannelConfig(ADC1,ADC_Channel_1,
1,ADC_SampleTime_71Cycles5); // Cau hinh thoi gian chuyen doi
    ADC_Init( ADC1, &ADC_InitStructure);

    ADC_Cmd (ADC1,ENABLE);

    ADC_ResetCalibration(ADC1); // Reset hieu Chuan ADC
    while(ADC_GetResetCalibrationStatus(ADC1));
    ADC_StartCalibration(ADC1); // Hieu chuan lai

```

```

while(ADC_GetCalibrationStatus(ADC1));

ADC_Cmd (ADC1,ENABLE);

ADC_SoftwareStartConvCmd(ADC1, ENABLE);

}

unsigned int Fn_ADC_Read (void){
    return(ADC_GetConversionValue(ADC1));
}

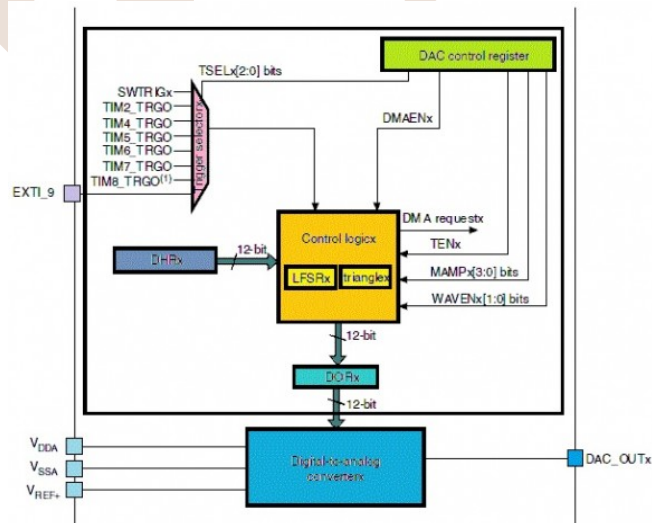
```

Chương trình thực hiện đọc dữ liệu ADC ở chân A1 và lưu vào biến vuii _ADC_Value. Người học có thể sử dụng biến này cho các ứng dụng khác hoặc đơn giản gửi dữ liệu đó thông qua kênh UART lên máy tính để theo dõi.

Lập trình DAC

Bộ chuyển đổi số sang tương tự DAC là thiết bị thực hiện chuyển đổi tín hiệu rời rạc dạng số sang tín hiệu liên tục tương tự. Như vậy nó thực hiện nhiệm vụ ngược với bộ ADC.

Sơ đồ khối bộ DAC của STM32F1 có dạng như sau:



Hình 5. 41: Sơ đồ khối bộ DAC

DAC của STM32F1 có một số đặc tính như sau:

- 2 bộ DAC
- Độ phân dải 12 bit dữ liệu
- Có thể căn chỉnh dữ liệu trái hoặc phải
- Hỗ trợ tạo sóng có nhiều
- Hỗ trợ 2 kênh DAC chuyển đổi độc lập
- Hỗ trợ truyền dữ liệu vào từ DMA
- Hỗ trợ kích khởi chuyển đổi từ các nguồn kích ngoài

Cấu trúc biến khởi tạo cấu hình cho bộ DAC cho STM32F1 trên thư viện SPL có dạng như sau:

```
typedef struct
{
    uint32_t DAC_Trigger; /*!< Specifies the external trigger for the
selected DAC channel.
This parameter can be a value of @ref DAC_trigger_selection */

    uint32_t DAC_WaveGeneration; /*!< Specifies whether DAC channel noise
waves or triangle waves
are generated, or whether no wave is generated.
This parameter can be a value of @ref DAC_wave_generation */

    uint32_t DAC_LFSRUnmask_TriangleAmplitude; /*!< Specifies the LFSR
mask for noise wave generation or
the maximum amplitude triangle generation for the DAC channel.
This parameter can be a value of @ref DAC_lfsrunmask_triangleamplitude
*/

    uint32_t DAC_OutputBuffer; /*!< Specifies whether the DAC channel
output buffer is enabled or disabled.
This parameter can be a value of @ref DAC_output_buffer */
}DAC_InitTypeDef;
```

Ví dụ về một chương trình khởi tạo:

```

#include "stm32f10x.h" // Device header
#include "stm32f10x_rcc.h" // Keil::Device:StdPeriph Drivers:RCC
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO
#include "stm32f10x_dac.h" // Keil::Device:StdPeriph Drivers:DAC
#include "Delay.h"

DAC_InitTypeDef DAC_InitStructure;
GPIO_InitTypeDef GPIO_InitStructure;

void Fn_DAC_Init (void);
void Fn_DAC_Write(unsigned int _vrui_Value);

int main (void){
    unsigned int Count;
    SystemInit();
    SystemCoreClockUpdate();
    Fn_DELAY_Init(72);

    Fn_DAC_Init();
    while(1){
        for (Count = 0; Count < 4096; Count++){
            Fn_DAC_Write(Count);
            Fn_DELAY_ms(5);
        }
    }
}

void Fn_DAC_Init (void){
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_DAC, ENABLE);

```

```

    GPIO_InitStruct.GPIO_Pin = GPIO_Pin_4 | GPIO_Pin_5;
    GPIO_InitStruct.GPIO_Mode = GPIO_Mode_AIN;
    GPIO_Init(GPIOA, &GPIO_InitStruct);

    DAC_InitStructure.DAC_Trigger = DAC_Trigger_Software;
    DAC_InitStructure.DAC_WaveGeneration = DAC_WaveGeneration_Noise;
    DAC_InitStructure.DAC_LFSRUnmask_TriangleAmplitude =
DAC_LFSRUnmask_Bits8_0;
    DAC_InitStructure.DAC_OutputBuffer = DAC_OutputBuffer_Enable;
    DAC_Init(DAC_Channel_1, &DAC_InitStructure);

    DAC_Cmd(DAC_Channel_1, ENABLE);

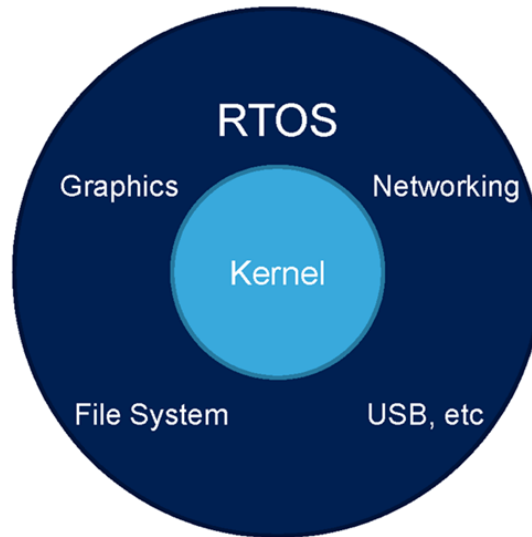
    Fn_DAC_Write(0);
}

void Fn_DAC_Write(unsigned int _vrui_Value){
    DAC_SetChannel1Data(DAC_Align_12b_R, _vrui_Value & 0xFFFF);
    DAC_SoftwareTriggerCmd(DAC_Channel_1, ENABLE);
}

```

Chương trình thực hiện lần lượt xuất dữ liệu từ 0 đến 4095 (12 bit) lần lượt ra kênh 1 của bộ DAC số 1 với chế độ căn chỉnh là căn phải. Chúng ta có thể dùng DAC để điều khiển dữ liệu tương tự như xuất âm thanh, tạo dạng sóng,...

Lập trình với FreeRTOS

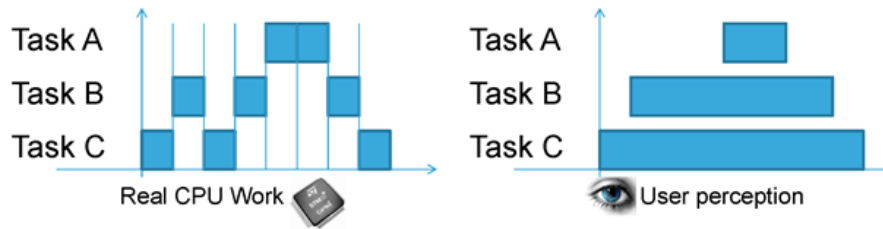


Hình 5. 42: Kiến trúc RTOS

FreeRTOS là một hệ điều hành nhúng thời gian thực (Real Time Operating System) mã nguồn mở được phát triển bởi Real Time Engineers Ltd, sáng lập và sở hữu bởi Richard Barry. FreeRTOS được thiết kế phù hợp cho nhiều hệ nhúng nhỏ gọn vì nó chỉ triển khai rất ít các chức năng như: cơ chế quản lý bộ nhớ và tác vụ cơ bản, các hàm API quan trọng cho cơ chế đồng bộ. Nó không cung cấp sẵn các giao tiếp mạng, drivers, hay hệ thống quản lý tệp (file system) như những hệ điều hành nhúng cao cấp khác. Tuy vậy, FreeRTOS có nhiều ưu điểm, hỗ trợ nhiều kiến trúc vi điều khiển khác nhau, kích thước nhỏ gọn (4.3 Kbytes sau khi biên dịch trên ARM7), được viết bằng ngôn ngữ C và có thể sử dụng, phát triển với nhiều trình biên dịch C khác nhau (GCC, OpenWatcom, Keil, IAR, Eclipse, ...), cho phép không giới hạn các tác vụ chạy đồng thời, không hạn chế quyền ưu tiên thực thi, khả năng khai thác phần cứng. Ngoài ra, nó cũng cho phép triển khai các cơ chế điều độ giữa các tiến trình như: queues, counting semaphore, mutexes.

FreeRTOS là một hệ điều hành nhúng rất phù hợp cho nghiên cứu, học tập về các kỹ thuật, công nghệ trong viết hệ điều hành nói chung và hệ điều hành nhúng thời gian thực nói riêng, cũng như việc phát triển mở rộng tiếp các thành phần cho hệ điều hành hành (bổ sung modules, driver, thực hiện porting).

Kernel



Hình 5. 43: Hoạt động của CPU và quan sát của người dùng với Multi Thread

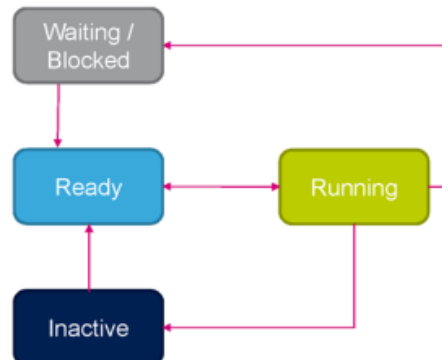
Kernel sẽ có nhiệm vụ quản lý nhiều task cùng chạy 1 lúc, mỗi task thường chạy mất vài ms. Tại lúc kết thúc task cần thực hiện các công việc cơ bản sau:

- Lưu trạng thái task
- Thanh ghi CPU sẽ load trạng thái của task tiếp theo
- Task tiếp theo cần khoảng vài ms để thực hiện

Vì CPU thực hiện tác vụ rất nhanh nên dưới góc nhìn người dùng thì hầu như các task là được thực hiện 1 cách liên tục.

Task state

Một task trong RTOS thường có các trạng thái như sau:



Hình 5. 44: Các trạng thái của Task trong RTOS

RUNNING: đang thực thi

READY: sẵn sàng để thực hiện

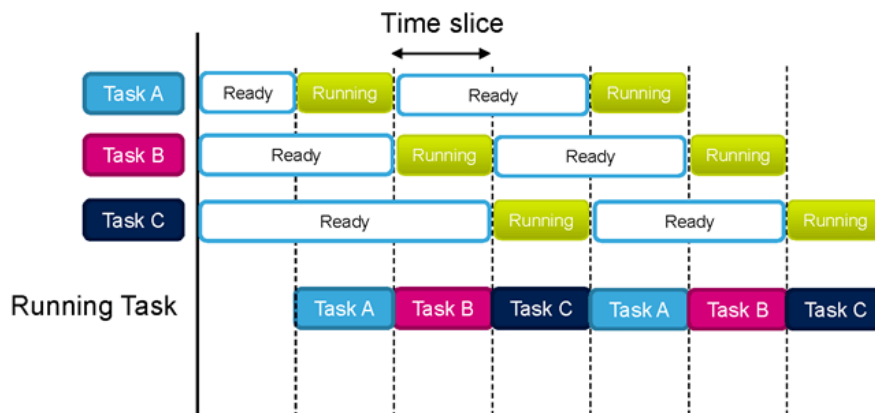
WAITING: chờ sự kiện

INACTIVE: không được kích hoạt

Scheduler

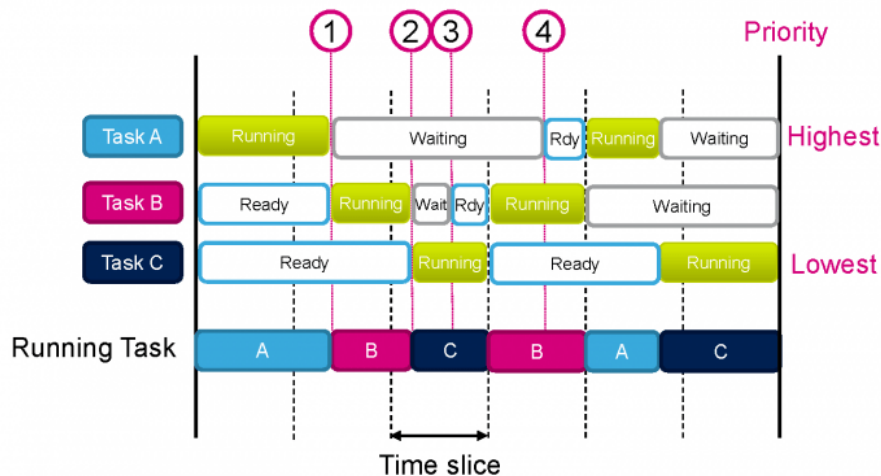
Đây là 1 thành phần của kernel quyết định task nào được thực thi. Có một số luật cho scheduling như:

- Cooperative: Giống với lập trình thông thường, mỗi task chỉ có thể thực thi khi task đang chạy dừng lại, nhược điểm của cơ chế này là task này có thể dùng hết tất cả tài nguyên của CPU
- Round-robin: Mỗi task được thực hiện trong thời gian định trước (time slice) và không có ưu tiên.



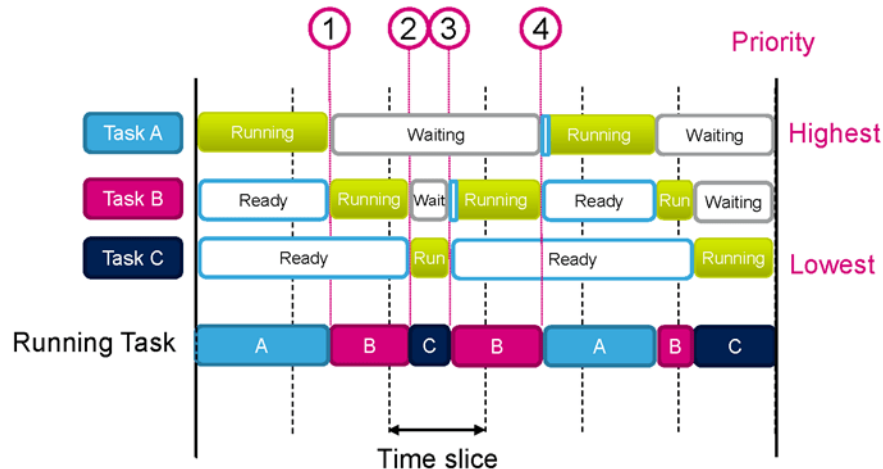
Hình 5. 45: Cơ chế Round-Robin

- Priority base: Task được phân quyền cao nhất sẽ được thực hiện trước, nếu các task có cùng quyền như nhau thì sẽ giống với round-robin, các task có mức ưu tiên thấp hơn sẽ được thực hiện cho đến cuối time slice



Hình 5. 46: Cơ chế Priority Base

- Priority-based pre-emptive: Các task có mức ưu tiên cao nhất luôn nhường các task có mức ưu tiên thấp hơn thực thi trước.



Hình 5. 47: Cơ chế Priority-based pre-emptive

Task

Một task là một chương trình, chương trình này chạy liên tục trong vòng lặp vô tận và không bao giờ dừng lại. Một chương trình thường sẽ có nhiều task khác nhau. Ví dụ như máy bán đồ uống tự động sẽ có các thành task sau:

- Task quản lý việc lựa chọn của người dùng
- Task để kiểm tra đúng số tiền người dùng đã trả
- Task để điều khiển động cơ/ cơ cấu cung cấp nước uống.

Kernel quản lý việc chuyển đổi giữa các task, nó lưu lại ngữ cảnh của task sắp bị hủy và khôi phục lại ngữ cảnh của task tiếp theo bằng cách:

- Kiểm tra thời gian thực thi đã được định nghĩa trước (time slice được tạo ra bởi ngắt systick)
- Khi có các sự kiện unblocking một task có quyền cao hơn xảy ra (signal, queue, semaphore,...)
- Khi task gọi hàm Yield() để ép Kernel chuyển sang các task khác mà không phải chờ cho hết time slice
- Khi khởi động thì kernel sẽ tạo ra một task mặc định gọi là Idle Task.

Để tạo một task thì cần phải khai báo hàm định nghĩa task, sau đó tạo task và cấp phát bộ nhớ.

Kết nối Inter-task & Chia sẻ tài nguyên

Các task cần phải kết nối và trao đổi dữ liệu với nhau để có thể chia sẻ tài nguyên, có một số khái niệm cần lưu ý như sau:

Với Inter-task Communication:

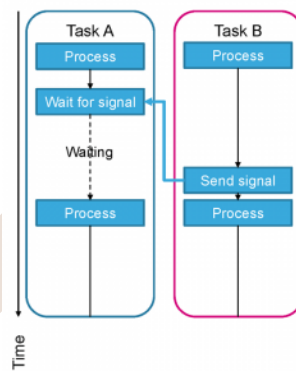
- Signal Events – Đồng bộ các task
- Message queue – Trao đổi tin nhắn giữa các task trong hoạt động giống như FIFO
- Mail queue – Trao đổi dữ liệu giữa các task sử dụng hằng đợi của khối bộ nhớ

Với Resource Sharing

- Semaphores – Truy xuất tài nguyên liên tục từ các task khác nhau
- Mutex – Đồng bộ hóa truy cập tài nguyên sử dụng Mutual Exclusion

Signal event

Signal event được dùng để đồng bộ các task, ví dụ như bắt task phải thực thi tại một sự kiện nào đó được định sẵn.



Hình 5. 48: Sử dụng Signal Event trong đồng bộ thông tin các Task

Ví dụ: Một cái máy giặt có 2 task là Task A điều khiển động cơ, Task B đọc mức nước từ cảm biến nước đầu vào:

- Task A cần phải chờ nước đầy trước khi khởi động động cơ. Việc này có thể thực hiện được bằng cách sử dụng signal event
- Task A phải chờ signal event từ Task B trước khi khởi động động cơ
- Khi phát hiện nước đã đạt tới mức yêu cầu thì Task B sẽ gửi tín hiệu tới Task A
- Với trường hợp này thì task sẽ đợi tín hiệu trước khi thực thi, nó sẽ nằm trong trạng thái là WAITING cho đến khi signal được cài đặt.

Mỗi task có thể được gán tối đa là 32 signal event. Ưu điểm của Event là thực hiện nhanh, sử dụng ít RAM hơn so với semaphore và message queue nhưng có nhược điểm lại chỉ được dùng khi một task nhận được signal.

Message queue

Message queue là cơ chế cho phép các task có thể kết nối với nhau, nó là một FIFO buffer được định nghĩa bởi độ dài (số phần tử mà buffer có thể lưu trữ) và kích thước dữ

liệu (kích thước của các thành phần trong buffer). Một ứng dụng tiêu biểu là buffer cho Serial I/O, buffer cho lệnh được gửi tới task.



Hình 5. 49: Trao đổi dữ liệu giữa các Task sử dụng Queue

Task có thể ghi vào hàng đợi (queue)

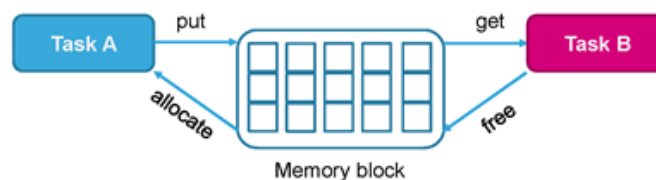
- Task sẽ bị khóa (block) khi gửi dữ liệu tới một message queue đầy đủ
- Task sẽ hết bị khóa (unblock) khi bộ nhớ trong message queue trống
- Trường hợp nhiều task mà bị block thì task với mức ưu tiên cao nhất sẽ được unblock trước

Task có thể đọc từ hàng đợi (queue)

- Task sẽ bị block nếu message queue trống
- Task sẽ được unblock nếu có dữ liệu trong message queue.
- Tương tự ghi thì task được unblock dựa trên mức độ ưu tiên

Mail queue

Giống như message queue nhưng dữ liệu sẽ được truyền dưới dạng khối (memory block) thay vì dạng đơn. Mỗi memory block thì cần phải cấp phát trước khi đưa dữ liệu vào và giải phóng sau khi đưa dữ liệu ra.



Hình 5. 50: Trao đổi dữ liệu giữa các Task sử dụng Mail Queue

Để gửi dữ liệu với mail queue cần thực hiện các bước sau:

- Cấp phát bộ nhớ từ mail queue cho dữ liệu được đặt trong mail queue
- Lưu dữ liệu cần gửi vào bộ nhớ đã được cấp phát
- Đưa dữ liệu vào mail queue

Nhận dữ liệu trong mail queue bởi task khác:

- Lấy dữ liệu từ mail queue, sẽ có một hàm để trả lại cấu trúc/ đối tượng

- Lấy con trỏ chứa dữ liệu
- Giải phóng bộ nhớ sau khi sử dụng dữ liệu

Semaphore

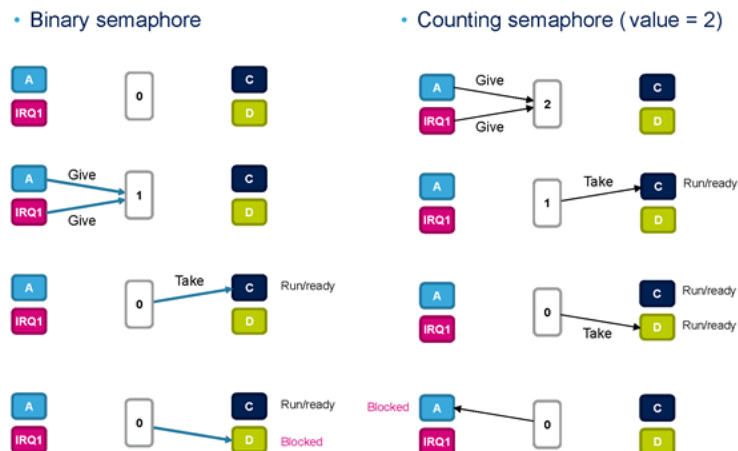
Được sử dụng để đồng bộ task với các sự kiện khác trong hệ thống. Có 2 loại là: Binary và Counting.

Binary semaphore:

- Trường hợp đặc biệt của counting semaphore
- Có duy nhất 1 token
- Chỉ có 1 hoạt động đồng bộ

Counting semaphore

- Có nhiều token
- Có nhiều hoạt động đồng bộ



Hình 5. 51: Binary semaphore và Counting semaphore

Mutex

Sử dụng cho việc loại trừ (mutual exclusion), hoạt động như là một token để bảo vệ tài nguyên được chia sẻ. Một task nếu muốn truy cập vào tài nguyên chia sẻ cần thực hiện các việc sau:

- Yêu cầu mutex trước khi truy cập vào tài nguyên chia sẻ.
- Đưa ra token khi kết thúc với tài nguyên.

Tại mỗi một thời điểm thì chỉ có 1 task có được mutex. Những task khác muốn cùng mutex thì phải block cho đến khi task cũ thả mutex ra.



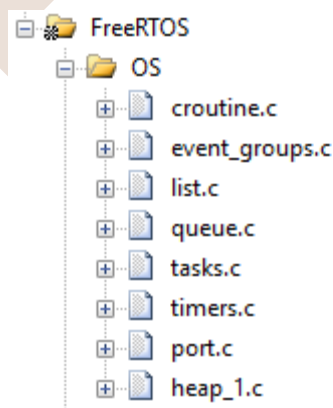
Hình 5. 52: Sử dụng chung tài nguyên hệ thống với Mutex

Về cơ bản thì Mutex giống như binary semaphore nhưng được sử dụng cho việc loại trừ chứ không phải đồng bộ. Ngoài ra thì nó bao gồm cơ chế thừa kế mức độ ưu tiên(Priority inheritance mechanism) để giảm thiểu vấn đề đảo ngược ưu tiên, cơ chế này có thể hiểu đơn giản qua ví dụ sau:

- Task A (low priority) yêu cầu mutex
- Task B (high priority) muốn yêu cầu cùng mutex trên.
- Mức độ ưu tiên của Task A sẽ được đưa tạm về Task B để cho phép Task A được thực thi
- Task A sẽ thả mutex ra, mức độ ưu tiên sẽ được khôi phục lại và cho phép Task B tiếp tục thực thi.

Người dùng có thể tải về bộ chương trình FreeRTOS từ địa chỉ sau: <https://www.freertos.org/>

Để tích hợp hệ điều hành FreeRTOS người dùng cần thêm vào Project các file sau:



Hình 5. 53: Thêm FreeRTOS vào Project

Trong đó heap có thể được lựa chọn giữa các file heap_1, heap_2, heap_3, heap_4 tùy thuộc vào nhu cầu sử dụng của người dùng. Ở giáo trình này do chỉ dùng ở các ứng dụng đơn giản nên sử dụng heap_1.

Lập trình MultiTask với FreeRTOS

Chương trình sau xây dựng một ví dụ về lập trình 2 task chạy song song với nhau bằng cách sử dụng cơ chế tạo Task của FreeRTOS. Chương trình cụ thể như sau:

```
#include "stm32f10x.h" // Device header
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO
#include "Delay.h"
#include "FreeRTOS.h"
#include "task.h"

GPIO_InitTypeDef GPIO_InitStructure;
void Fn_GPIO_Init (void);
void Fn_RTOS_TaskLed1(void *p);
void Fn_RTOS_TaskLed2(void *p);

int main (void){
    SystemInit();
    SystemCoreClockUpdate();

    Fn_GPIO_Init();
    xTaskCreate(Fn_RTOS_TaskLed1, (const char*) "Red LED Blink", 128,
    NULL, 1, NULL);
    xTaskCreate(Fn_RTOS_TaskLed2, (const char*) "Green LED Blink",
    128, NULL, 1, NULL);
    vTaskStartScheduler();
    return 0;
}

void Fn_GPIO_Init (void){
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOE, ENABLE);

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_13 | GPIO_Pin_15;
```



```

        GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
        GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
        GPIO_Init(GPIOE, &GPIO_InitStructure);
    }

void Fn_RTOS_TaskLed1(void *p){
    while(1){
        GPIOE->ODR ^= GPIO_Pin_15;
        vTaskDelay(100/portTICK_RATE_MS);
    }
}

void Fn_RTOS_TaskLed2(void *p){
    while(1){
        GPIOE->ODR ^= GPIO_Pin_13;
        vTaskDelay(500/portTICK_RATE_MS);
    }
}
}

```

Chương trình xây dựng 2 task điều khiển 2 đèn LED nhấp nháy với 2 tần số khác nhau. Nếu như không sử dụng hệ điều hành người lập trình cần lập trình xen kẽ phù hợp và linh động nếu không sẽ rất dễ xảy ra xung đột. Với hệ điều hành việc thực hiện các tác vụ định kỳ và song song sẽ trở nên dễ dàng hơn rất nhiều.

Với chương trình trên chúng ta cần xem xét các hàm sau:

xTaskCreate	Tạo một Task vụ gắn với một hàm đã được xây dựng trước.
vTaskDelay	Hàm trễ của hệ điều hành
vTaskStartScheduler	Thực hiện lập lịch cho các task

Tùy thuộc vào số tiến trình cần quản lý người dùng có thể xây dựng một chương trình với nhiều Task hơn nữa. Các Task này tùy thuộc vào độ ưu tiên, thời gian

chạy định kỳ mà sẽ được hệ thống lập lịch để hoạt động vào các thời điểm phù hợp với tài nguyên của hệ thống.

Lập trình FreeRTOS và Mutex

Sử dụng cho việc loại trừ lẫn nhau(mutual exclusion), nghĩa là nó được dùng để hạn chế quyền truy cập tới một số resource, mutex hoạt động như là một token để bảo vệ tài nguyên chia sẻ.

Một task nếu muốn truy cập vào tài nguyên chia sẻ thì:

- Cần đợi mutex trước khi truy cập vào tài nguyên chia sẻ.
- Giải phóng token khi kết thúc với tài nguyên.

Tại mỗi một thời điểm thì chỉ có 1 task duy nhất có được mutex. Khi task này có mutex thì những task khác cũng muốn mutex này đều sẽ bị chặn cho tới khi task hiện tại giải phóng mutex ra.

Chương trình sau xây dựng một ví dụ sử dụng chung tài nguyên là màn hình GLCD đã được xây dựng ở trên. Từ đó đồng bộ chia sẻ tài nguyên để các task có thể lần lượt truy cập và điều khiển màn hình khi cần thiết.

Chương trình cụ thể như sau:

```
#include "stm32f10x.h" // Device header
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO
#include "Delay.h"
#include "FreeRTOS.h"
#include "task.h"

#include "N5110.h"
#include "semphr.h"

xSemaphoreHandle xMutex;

char TextTemp[200];
int Number1;
int Number2;
```

```

void Fn_GLCD_Init (void);
void Fn_RTOS_N5110_Text1 (void *p);
void Fn_RTOS_N5110_Text2 (void *p);

int main (void){
    xMutex = xSemaphoreCreateMutex();

    SystemInit();
    SystemCoreClockUpdate();

    Fn_GPIO_Init();
    Fn_N5110_Init();
    sprintf(TextTemp,"Task1:%4d",1234);
    Fn_N5110_GotoXY(0,0);
    Fn_N5110_Puts(TextTemp);
    xTaskCreate(Fn_RTOS_N5110_Text1, (const char*) "Task1 LCD", 128,
NULL, 1, NULL);
    xTaskCreate(Fn_RTOS_N5110_Text2, (const char*) "Task2 LCD", 128,
NULL, 1, NULL);
    vTaskStartScheduler();
    return 0;
}

void Fn_RTOS_N5110_Text1 (void *p){
    while(1){
        xSemaphoreTake(xMutex, portMAX_DELAY);
        Number1++;
        sprintf(TextTemp,"Task1:%4d",Number1);
        Fn_N5110_GotoXY(0,0);
        Fn_N5110_Puts(TextTemp);
        xSemaphoreGive( xMutex );
    }
}

```

```

        vTaskDelay(100/portTICK_RATE_MS);
    }
}

void Fn_RTOS_N5110_Text2 (void *p){
    while(1){
        xSemaphoreTake(xMutex, portMAX_DELAY);
        Number2++;
        sprintf(TextTemp, "Task2:%4d", Number2);
        Fn_N5110_GotoXY(0,3);
        Fn_N5110_Puts(TextTemp);
        xSemaphoreGive( xMutex );
        vTaskDelay(500/portTICK_RATE_MS);
    }
}

```

Hai task thực hiện điều khiển 2 dòng của màn hình hiển thị 2 số khác nhau. Nếu như không có sự phân chia tài nguyên sử dụng mutex thì có thể sẽ xảy ra hiện tượng khi Task1 đang thực hiện ghi ra màn hình thì Task2 chạy thực hiện ghi tiếp vào vị trí mà Task 1 đang ghi gây ra tình trạng ghi nhầm lẫn dữ liệu.

Mutex sẽ giúp hệ thống quản lý tài nguyên giúp cho khi một Task sử dụng thì đảm bảo Task đó sử dụng xong thì Task sau mới được quyền can thiệp vào tài nguyên chia sẻ.

Chương trình cần chú ý vào các hàm sau:

xSemaphoreCreateMutex	Tạo một Mutex làm cờ cho một tài nguyên nào đó
xSemaphoreTake	Thực hiện lấy cờ tài nguyên về Task đang chạy. Nếu cờ tài nguyên đang được sử dụng thì chương trình tạm dừng ở lệnh này.
xSemaphoreGive	Giải phóng Mutex sau khi sử dụng xong tài nguyên.

Chú ý: Một Task vụ muốn truy cập tài nguyên thì cần thực hiện đúng quy trình lấy chờ tài nguyên trước rồi mới thực hiện điều khiển nếu không sẽ gây ra hiện tượng tài nguyên bị dùng một cách xung đột. Ngay sau khi Task sử dụng xong tài nguyên cần trả lại chờ tài nguyên để các ứng dụng khác có thể truy cập. Nếu không giải phóng thì các ứng dụng khác cũng sử dụng tài nguyên đó sẽ không bao giờ có thể chạy được.

Lập trình FreeRTOS và Queues

Queue có 2 dạng là message queue và mail queue, đây là 2 cơ chế được dùng để các task có thể trao đổi với nhau, sự khác biệt lớn nhất giữa 2 dạng này là message queue thì truyền dữ liệu dưới dạng đơn, còn mail queue sẽ truyền dưới dạng khối.

Sau đây là một ứng dụng sử dụng queues để trao đổi dữ liệu giữa 2 Task. Ví dụ này dùng lại ở sử dụng message queue. Người đọc có thể tìm hiểu thêm về mail queue ở các tài liệu khác.

Ý tưởng chính của chương trình là tạo một Task có trách nhiệm điều khiển màn hình và một Task khác cung cấp dữ liệu cho Task điều khiển màn hình thông qua queues.

Chương trình cụ thể như sau:

```
#include "stm32f10x.h" // Device header
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO
#include "Delay.h"
#include "N5110.h"
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

xQueueHandle xQueue;

char TextTemp[200];

void Fn_GLCD_Init (void);
void Fn_RTOS_N5110_Send (void *p);
```

```

void Fn_RTOS_N5110_Disp (void *p);

int main (void){
    xQueue = xQueueCreate( 10, sizeof( uint32_t ) );

    SystemInit();
    SystemCoreClockUpdate();

    Fn_GPIO_Init();
    Fn_N5110_Init();
    Fn_N5110_GotoXY(0,0);
    Fn_N5110_Puts(TextTemp);
    xTaskCreate(Fn_RTOS_N5110_Send, (const char*) "Send Data", 128,
    NULL, 1, NULL);
    xTaskCreate(Fn_RTOS_N5110_Disp, (const char*) "Display Data",
    128, NULL, 1, NULL);
    vTaskStartScheduler();
    return 0;
}

void Fn_GPIO_Init (void){
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOE, ENABLE);

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_13 | GPIO_Pin_15;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOE, &GPIO_InitStructure);
}

void Fn_RTOS_N5110_Send (void *p){
    uint32_t DataSend = 0;

```

```

        while(1){
            xQueueSend(xQueue, (void*)&DataSend, (portTickType)0);
            DataSend++;
            vTaskDelay(1000);
            xQueueSend(xQueue, (void*)&DataSend, (portTickType)0);
            DataSend++;
            vTaskDelay(5000);
        }
    }

void Fn_RTOS_N5110_Dispatch (void *p){
    uint32_t DataReceive;
    while(1){
        if(xQueueReceive(xQueue, (void*)&DataReceive,
        (portTickType)0xFFFFFFFF))
        {
            sprintf(TextTemp, "Number:%5d", DataReceive);
            Fn_N5110_GotoXY(0,0);
            Fn_N5110_Puts(TextTemp);
        }
    }
}
}

```

Task Fn_RTOS_N5110_Dispatch thực hiện nhiệm vụ nếu có dữ liệu gửi đến thì thực hiện in số đó lên màn hình GLCD. Dữ liệu nhận được thông qua hàm xQueueReceive nhằm lấy dữ liệu từ trong queues có tên là xQueue nếu có bản tin được gửi đến Task đó.

Task Fn_RTOS_N5110_Send thực hiện tạo một dữ liệu số tăng dần sau đó gửi dữ liệu đó đến queues xQueue để các Task khác sử dụng con số này cho mục đích phù hợp.

Các hàm liên quan đến queues cơ bản nằm trong bảng sau:

xQueueCreate	Tạo một queues dữ liệu theo kích thước mong muốn.
xQueueSend	Truyền một dữ liệu vào queues tương ứng.
xQueueReceive	Kiểm tra queues có dữ liệu mới không, nếu có thì lấy dữ liệu đó ra khỏi queues.

Lập trình FreeRTOS và Events

Signal event được dùng để đồng bộ các task, ví dụ như bắt task phải thực thi tại một sự kiện nào đó được định sẵn. Khác với queues, Event dùng để định nghĩa các sự kiện từ đó điều khiển các Task hoạt động đồng bộ với nhau.

Ví dụ như một Task thực hiện hiện việc A và một Task thực hiện việc B với điều kiện việc B được thực hiện sau khi A đã thực hiện xong. Vì vậy trong suốt quá trình A được thực hiện thì Task thực hiện việc B cần hoạt động ở chế độ khóa. Khi Task thực hiện A xong sẽ báo với Task thực hiện B rằng đã sẵn sàng đến Task B hoạt động. Lúc đó B sẽ ở trạng thái sẵn sàng và chờ thời điểm phù hợp để hoạt động.

Chương trình sau nêu một ví dụ về thực hiện 2 Task đồng bộ thông qua Event. Cụ thể TaskMain thực hiện lần lượt điều khiển TaskA và TaskB theo định kỳ. Các TaskA và TaskB là các tác độc lập hoạt động dưới sự điều khiển của TaskMain một cách không trực tiếp thông qua các Event mà hệ thống cài đặt. Cụ thể chương trình như sau:

```
#include "stm32f10x.h" // Device header
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO
#include "Delay.h"
#include "N5110.h"
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include "event_groups.h"

EventGroupHandle_t event_hdl;
```



```

char TextTemp[200];

#define EVENT0      (1<<0)
#define EVENT1      (1<<1)
#define EVENT2      (1<<2)
#define EVENT_ALLS  ((EVENT2<<1) - 1)

void Fn_GPIO_Init (void);

void TaskMain(void *pvParameters);
void TaskA(void *pvParameters);
void TaskB(void *pvParameters);

int main (void){

    SystemInit();
    SystemCoreClockUpdate();

    Fn_GPIO_Init();
    Fn_N5110_Init();

    xTaskCreate(TaskMain, "TaskMain", 128, NULL, tskIDLE_PRIORITY,
NULL);
    xTaskCreate(TaskA, "TaskA", 128, NULL, tskIDLE_PRIORITY, NULL);
    xTaskCreate(TaskB, "TaskB", 128, NULL, tskIDLE_PRIORITY, NULL);
    vTaskStartScheduler();
    return 0;
}

void Fn_GPIO_Init (void){
    GPIO_InitTypeDef GPIO_InitStructure;

```

```

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOE, ENABLE);

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_All;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOE, &GPIO_InitStructure);
}

void TaskMain(void *pvParameters){
    event_hdl = xEventGroupCreate();
    while(1){
        xEventGroupSetBits(event_hdl, EVENT0);
        vTaskDelay(2000);
        xEventGroupSetBits(event_hdl, EVENT1);
        vTaskDelay(2000);
        xEventGroupSetBits(event_hdl, EVENT_ALLS);
        vTaskDelay(2000);
    }
}

void TaskA(void *pvParameters){
    EventBits_t event;
    while(1){
        event = xEventGroupWaitBits(event_hdl, EVENT_ALLS, pdTRUE,
pdFALSE, portMAX_DELAY);
        if(event & EVENT0){
            GPIOE->ODR ^= 0x000FF00;
        }
    }
}
}

```

```

void TaskB(void *pvParameters){
    int Number = 0;
    EventBits_t event;
    while(1){
        event = xEventGroupWaitBits(event_hdl, EVENT_ALLS, pdTRUE,
pdFALSE, portMAX_DELAY);
        if(event & EVENT1){
            Number++;
            sprintf(TextTemp, "Number:%5d", Number);
            Fn_N5110_GotoXY(0,0);
            Fn_N5110_Puts(TextTemp);
        }
    }
}

```

Tiến trình cụ thể như sau:

TaskMain điều khiển TaskA chạy sau đó 2 giây sau thực hiện cho Task B chạy và sau 2 giây thì cho phép TaskA và TaskB cùng chạy. Ở khoảng đầu chỉ TaskA chạy còn TaskB ở trạng thái khóa. Sau 2 giây TaskA ở trạng thái khóa và TaskB ở trạng thái chạy. Tiếp sau 2 giây nữa TaskA và TaskB đều ở trạng thái sẵn sàng và chia sẻ tài nguyên hệ thống để có thể chạy song song.

Ở ví dụ trên TaskA thực hiện điều khiển nhấp nháy đèn LED được nối với các chân từ E8 đến E15 và TaskB thực hiện hiển thị số tăng dần lên màn hình GLCD dưới sự điều khiển định kỳ của TaskMain.

Khi lập trình Events với FreeRTOS cần chú ý các sự kiện sau:

xEventGroupCreate	Tạo một cụm Events
xEventGroupWaitBits	Chờ sự kiện cập nhật events nào đó
event & EVENT0	Kiểm tra một Event nào đó trong cụm Event xem xét có được kích hoạt không

5.4 Thiết lập hệ điều hành nhúng trên nền ARM

Firmware và Bootloader

Firmware thường là phần mã nguồn được chạy đầu tiên trong một hệ thống nhúng khi khởi động. Do đó, nó là một trong những thành phần quan trọng nhất. Firmware rất đa dạng, có thể chỉ là một đoạn code khởi động hoặc cả một phần mềm nhúng.

Có nhiều định nghĩa về firmware, chúng ta dùng định nghĩa sau:

Firmware là phần mềm nhúng cấp thấp cung cấp giao diện giữa phần cứng và các phần mềm ứng dụng hoặc hệ điều hành. Firmware thường được lưu trên ROM và chạy khi hệ thống nhúng được khởi động.

Đối với một hệ thống có hệ điều hành, khi khởi động lên, hệ thống cần chạy một chương trình từ ROM để nạp hệ điều hành và dữ liệu để sau đó có thể chạy trên RAM. Do đó, người ta thường dùng định nghĩa sau:

Bootloader là một ứng dụng để nạp hệ điều hành hoặc ứng dụng của một hệ thống.

Bootloader thường chỉ tồn tại tới thời điểm hệ điều hành hoặc ứng dụng chạy. Bootloader thường được tích hợp cùng với firmware.

Chu trình chạy thường gặp của một hệ thống nhúng như sau

- Bước đầu tiên là thiết lập nền tảng của hệ thống (hay chuẩn bị môi trường để khởi động một hệ điều hành).
 - Bước này bao gồm đảm bảo hệ thống được khởi tạo đúng (VD như thay đổi bản đồ bộ nhớ hoặc thiết lập các giá trị thanh ghi..).
 - Sau đó, firmware sẽ xác định chính xác nhân chip và hệ thống. Thường thì thanh ghi số 0 của bộ đồng xử lý lưu loại VXL và tên nhà sx.
 - Tiếp theo, phần mềm chuẩn đoán hệ thống sẽ xác định xem có vấn đề gì với các phần cứng cơ bản không.
 - Thiết lập giao diện sửa lỗi: khả năng sửa lỗi hỗ trợ cho việc sửa lỗi phần mềm chạy trên phần cứng. Các sự hỗ trợ này bao gồm
 - Thiết lập breakpoint trong RAM.
 - Liệt kê và sửa đổi bộ nhớ.
 - Hiển thị nội dung của các thanh ghi.
 - Disassemble nội dung bộ nhớ thành các lệnh ARM và Thumb.
 - Thông dịch dòng lệnh (Command Line Interpreter hoặc CLI) : Tính năng thông dịch dòng lệnh cho phép người sử dụng thay đổi hệ điều hành được khởi động bằng cách thay đổi cấu hình mặc định thông qua các lệnh tại command prompt. Đối với hệ điều hành nhúng, CLI được điều khiển từ

một ứng dụng chạy trên máy host. Việc liên lạc giữa host và target thường qua cáp nối tiếp hoặc kết nối mạng

- Bước thứ hai là trừu tượng hóa phần cứng. Lớp trừu tượng hóa phần cứng (HAL) là một lớp phần mềm giấu chi tiết về phần cứng phía dưới bằng cách cung cấp một tập hợp giao diện lập trình đã được định sẵn trước. Khi chuyển sang một nền tảng khác, các giao diện lập trình này được giữ nguyên nhưng phần cứng phía dưới có thể thay đổi. Đối với mỗi một phần cứng cụ thể, phần mềm HAL để giao tiếp với phần cứng đó gọi là trình điều khiển thiết bị (device driver). Trình điều khiển thiết bị cung cấp giao diện lập trình ứng dụng (API) chuẩn để đọc và ghi tới một thiết bị riêng.
- Bước thứ ba là nạp một ảnh khởi động được (bootable image). File này có thể lưu ở trong ROM, network hoặc thiết bị lưu trữ khác. Đối với chip ARM, format phổ biến nhất cho image file là Executable and Linking Format (ELF).
- Bước thứ tư là từ bỏ quyền điều khiển. Đến đây khi hệ điều hành đã bắt đầu được nạp lên và chạy, firmware sẽ chuyển quyền điều khiển hệ thống cho hệ điều hành. Trong một số trường hợp đặc biệt, firmware vẫn có thể giữ quyền điều khiển hệ thống.
 - Trong hệ thống sử dụng chip ARM, trao quyền điều khiển có nghĩa là cập nhật bảng vector và thay đổi thanh ghi PC. Thay đổi bảng vector sẽ làm chúng chỉ đến các hàm điều khiển các ngắt và các ngoại lệ. Thanh ghi PC được cập nhật để chỉ đến địa chỉ ban đầu của hệ điều hành.

Hệ thống file (Filesystem)

Hệ thống file là cách thức để lưu trữ và tổ chức các file và dữ liệu trên máy tính.

Khác với các hệ thống lưu trữ trên máy tính hay máy chủ, các hệ thống nhúng thường sử dụng các thiết bị lưu trữ thể rắn như flash memory, flash disk. Các thiết bị này phải được thiết lập cấu hình hệ thống file hoàn chỉnh cho hệ thống.

Có nhiều loại hệ thống file khác nhau, sau đây là một số hệ thống file phổ biến cho hệ thống nhúng:

ROMFS (ROM file system)

ROMFS là hệ thống file đơn giản nhất, lưu các dữ liệu có kích thước nhỏ, chỉ đọc được. Thường các dữ liệu này phục vụ quá trình khởi tạo RAM disk.

RAMdisk

Ramdisk (còn gọi là ổ nhớ RAM ảo hoặc ổ nhớ RAM mềm) là một ổ đĩa ảo được thiết lập từ một khối RAM. Hệ thống máy tính sẽ làm việc với khối RAM này như một ổ đĩa.

Hiệu năng của RAMdisk thường cao hơn các dạng lưu trữ khác nhiều, tuy nhiên các dữ liệu lưu trên RAM disk sẽ bị mất khi mất nguồn.

CRAMFS (Compressed RAM file system)

CRAMFS là một hệ thống file tiện dụng cho các hệ thống lưu trữ thể rắn. Đây là một hệ thống file chỉ đọc ra, và có khả năng lưu dữ liệu lưu trên hệ thống có dạng nén. CRAMFS dùng thư viện zlib để nén dữ liệu.

Công cụ làm việc với hệ thống file này là `mkcramfs`.

Journaling Flash File System (JFFS và JFFS2)

JFFS là hệ thống file cổ điển cho hệ thống nhúng. JFFS hỗ trợ bộ nhớ flash NOR.

Phiên bản cập nhật của JFFS là JFFS2 có thêm nhiều tính năng cải tiến, hỗ trợ bộ nhớ flash NAND. Đồng thời, JFFS2 cũng hỗ trợ nén với một trong ba thuật toán : zlib, rubin và rtime.

Thiết lập nhân (kernel)

Nhân của hệ điều hành là thành phần phần mềm trung tâm của hệ thống nhúng. Khả năng của nhân cũng là chỉ số của khả năng của cả hệ thống.

Kiến trúc, cấu trúc của nhân hệ điều hành nhúng tùy theo từng hệ điều hành có thể từ đơn giản đến phức tạp. Các tài liệu về kiến trúc, cấu trúc, lập trình, sử dụng nhân rất phong phú và phổ biến. Do phạm vi vượt quá nội dung liên quan, ở đây chúng ta chỉ đề cập đến vấn đề chuẩn bị một nhân cho hệ điều hành nhúng. Các ví dụ được lấy là nhân Linux.

Cụ thể, chúng ta sẽ đề cập đến việc lựa chọn nhân, cấu hình, dịch và cài đặt.

Lựa chọn nhân

Hiện nay có nhiều hệ điều hành nhúng khác nhau, mỗi hệ điều hành nhúng có nhiều phiên bản khác nhau. Trên thực tế, nhiều phiên bản chỉ dành riêng cho một số bo mạch nhúng, một số phiên bản có thể hoạt động trên nhiều bo mạch nhúng, nhưng cần được sửa đổi phù hợp lại với cấu hình.

Đối với hệ điều hành Linux, các phiên bản nhân cho chip ARM được lưu ở trang web <http://www.arm.linux.org.uk/developer/>. Ngoài ra, các phiên bản có thể được lưu ở các trang mã nguồn khác, hoặc đi kèm theo kit phát triển.

Sau khi download hoặc copy phiên bản nhân dự định sử dụng. Ta có thể cần phải dùng các bản vá cho nhân (patch). Các bản vá này có thể sửa chữa các lỗi hoặc thay đổi một số tính năng trong nhân.

Cấu hình nhân

Thiết lập cấu hình nhân là bước đầu tiên để xây dựng nhân cho bo mạch. Có nhiều cách để thiết lập cấu hình nhân, trong quá trình thiết lập cấu hình cũng có nhiều lựa chọn khác nhau. Tuy nhiên, không phụ thuộc vào cách dùng hoặc cách lựa chọn các options, nhân sẽ tạo ra một file *.config* và tạo ra các liên kết symbolic và các file headers sẽ được dùng trong quá trình còn lại của việc xây dựng nhân.

Các lựa chọn: Có rất nhiều lựa chọn khác nhau, các lựa chọn này sẽ được dùng để thiết lập nên nhân. Tùy theo yêu cầu của bo mạch và chip mà ta lựa chọn cấu hình thích hợp. Ở đây ta chỉ liệt kê một số lựa chọn chính:

- Code maturity level options
- Loadable module support
- General setup
- Memory technology devices
- Block devices
- Networking options
- ATA/IDE/MFM/RLL support
- SCSI support
- Network device support
- Input core support
- Character devices
- Filesystems
- Console drivers
- Sound
- Kernel hacking

Các cách thiết lập cấu hình: có 4 cách chính

make config

Sử dụng giao diện dòng lệnh (command-line interface) cho lần lượt từng lựa chọn

make oldconfig

Sử dụng cấu hình trong một file *.config* sẵn có và chỉ hỏi những lựa chọn chưa được thiết lập.

make menuconfig

Thiết lập cấu hình dùng giao diện thực đơn trên terminal.

make xconfig

Thiết lập cấu hình dùng giao diện X Window

Để xem thực đơn cấu hình nhân, có thể dùng dòng lệnh. VD đối với dòng chip ARM:

```
$ make ARCH=arm CROSS_COMPILE=arm-linux- menuconfig
```

Sau đó, ta có thể chọn các tùy chọn cấu hình tương ứng cho bo mạch. Nhiều tính năng và trình điều khiển thường có sẵn theo dạng các module và có thể được chọn để tích hợp trong nhân hoặc kèm theo dạng module.

Sau khi hoàn thành việc cấu hình nhân, thoát và lưu cấu hình nhân. File cấu hình sẽ được lưu lại trong một file *.config*.

Biên dịch nhân

Biên dịch nhân bao gồm ba bước chính sau: Xây dựng các file phụ thuộc, xây dựng ảnh nhân (kernel image), xây dựng các module nhân. Mỗi bước đều dùng các lệnh *make* và được giải thích dưới đây. Tuy nhiên, các bước này có thể được làm gộp bằng cách dùng một lệnh duy nhất.

Xây dựng các file phụ thuộc:

Phần lớn các files trong mã nguồn của nhân phụ thuộc vào nhiều file header. Để xây dựng nhân, file Makefiles cần biết tất cả các phụ thuộc này. Đối với mỗi thư mục trong cây thư mục nhân, một file *.depend* được tạo trong quá trình xây dựng các phụ thuộc. File này chứa danh mục các header file mà các file trong thư mục phụ thuộc vào.

Từ thư mục gốc (root directory) của mã nguồn nhân, để xây dựng các phụ thuộc của nhân dùng lệnh sau:

```
$ make ARCH=arm CROSS_COMPILE=arm-linux- clean dep
```

Ở đây ARCH (kiến trúc) là nhân ARM. CROSS_COMPILE là biên dịch chéo, dùng trong việc biên dịch trên kiến trúc khác với ARM (VD kiến trúc x86). CROSS_COMPILE sẽ được dùng để lựa chọn công cụ xây dựng kernel. Trong trường

hợp chạy trên Linux, trình biên dịch là *gcc*, do đó nếu bo mạch đích sử dụng chip ARM, trình biên dịch sẽ là *arm-linux-gcc*.

Xây dựng file ảnh nhân (kernel image):

File ảnh của nhân được tạo ra bằng lệnh sau:

```
$ make ARCH=arm CROSS_COMPILE=arm-linux- zImage
```

Ở đây, file đích là *zImage*, có nghĩa là ảnh của nhân được nén dùng thuật toán *gzip*. Ngoài thuật toán này ra có thể dùng *vmlinux* để tạo một ảnh không nén.

Xây dựng các module nhân:

Sau khi ảnh của nhân đã được xây dựng xong, các module nhân có thể được xây dựng bằng dòng lệnh sau:

```
$ make ARCH=arm CROSS_COMPILE=arm-linux- modules
```

Tùy theo các tùy chọn nhân đã được lựa chọn xây dựng theo dạng module trong quá trình thiết lập cấu hình nhân (thay vì tích hợp trong nhân), thời gian cho quá trình này có thể dài ngắn khác nhau.

Sau khi ảnh của nhân và các module của nhân đã được xây dựng, ta có thể cài vào trong bo mạch đích. Trước khi cài, chúng ta cần backup file cấu hình nhân và dọn sạch mã nguồn của nhân bằng câu lệnh sau:

```
$ make ARCH=arm CROSS_COMPILE=arm-linux- distclean
```

Cài đặt nhân

Bước cuối cùng là cài đặt nhân vào bo mạch đích. Đối với mỗi cấu hình nhân, ta cần copy bốn file: file ảnh nhân đã nén, file ảnh nhân không nén, bản đồ symbol nhân và file cấu hình. File ảnh nhân đã nén ở trong thư mục *arch/arm/boot/zImage*, ba file còn lại lần lượt là *vmlinux*, *System.map*, và *.config*.

Để dễ dàng nhận dạng các file, chúng ta cần đặt tên file theo phiên bản mã nhân. Ví dụ nếu nhân được tạo từ mã nguồn 2.4.18-rmk5, chúng ta copy các file như sau:

```
$ cp arch/arm/boot/zImage ${PRJROOT}/images/zImage-2.4.18-rmk5
$ cp System.map ${PRJROOT}/images/System.map-2.4.18-rmk5
$ cp vmlinux ${PRJROOT}/images/vmlinux-2.4.18-rmk5
$ cp .config ${PRJROOT}/images/2.4.18-rmk5.config
```

File Makefile của nhân kèm theo một file *modules_install* cho việc cài đặt các module. Theo mặc định, các module được cài trong thư mục */lib/modules*. Bởi vì các module nhân được dùng với ảnh của nhân tương ứng, các module này nên được cài trong thư mục với tên tương tự như ảnh của nhân. Trong ví dụ trên nhân sử dụng là 2.4.18-rmk5 được cài trong thư mục

\${PRJROOT}/images/modules-2.4.18-rmk5. Toàn bộ nội dung của thư mục này sẽ được copy vào hệ thống file gốc của bo mạch đích để dùng với nhân tương ứng, do đó để cài các module ta dùng lệnh:

```
$ make ARCH=arm CROSS_COMPILE=arm-linux- \  
> INSTALL_MOD_PATH=${PRJROOT}/images/modules-2.4.18-rmk5 \  
> modules_install
```

Sau khi đã hoàn thành việc copy các modules, nhân sẽ thực hiện việc xây dựng sự phụ thuộc của các module cần cho các tiện ích trong quá trình chạy thực. Để làm việc này, chúng ta cần công cụ xây dựng đi kèm với gói phần mềm BusyBox. Download BusyBox về, copy vào thư mục *\${PRJROOT}/sysapps* và giải nén tại đây. Từ thư mục BusyBox, copy file Perl script *scripts/depmod.pl* sang thư mục *\${PREFIX}/bin*.

Ta dùng lệnh sau để xây dựng các phụ thuộc module cho bảng mạch đích:

```
$ depmod.pl \  
> -k ./vmlinux -F ./System.map \  
> -b ${PRJROOT}/images/modules-2.4.18-rmk5/lib/modules > \  
>      ${PRJROOT}/images/modules-2.4.18-rmk5/lib/modules/2.4.18-  
rmk5/modules.dep
```

Ở trên đây tùy chọn *-k* để xác định ảnh của nhân là không nén, *-F* dùng để xác định bản đồ hệ thống, *-b* dùng để xác định thư mục căn bản.

PHỤ LỤC

Thư viện GLCD PCD8544

LCD5110_Graph.h

```
#ifndef LCD5110_Graph_h
#define LCD5110_Graph_h

#include <stdio.h>
#include <string.h>
#include "stm32f10x.h" // Device header
#include "stm32f10x_gpio.h" // Keil::Device:StdPeriph Drivers:GPIO
#include "stm32f10x_rcc.h" // Keil::Device:StdPeriph Drivers:RCC
#include "stm32f10x_spi.h" // Keil::Device:StdPeriph Drivers:SPI
#include "Delay.h"
#include "SPI.h"

#define LEFT 0
#define RIGHT 9999
#define CENTER 9998

#define LCD_COMMAND 0
#define LCD_DATA 1

// PCD8544 Commandset
// -----
// General commands
#define PCD8544_POWERDOWN 0x04
#define PCD8544_ENTRYMODE 0x02
#define PCD8544_EXTENDEDINSTRUCTION 0x01
#define PCD8544_DISPLAYBLANK 0x00
#define PCD8544_DISPLAYNORMAL 0x04
#define PCD8544_DISPLAYALLON 0x01
#define PCD8544_DISPLAYINVERTED 0x05
// Normal instruction set
#define PCD8544_FUNCTIONSET 0x20
#define PCD8544_DISPLAYCONTROL 0x08
#define PCD8544_SETYADDR 0x40
#define PCD8544_SETXADDR 0x80
// Extended instruction set
#define PCD8544_SETTEMP 0x04
#define PCD8544_SETBIAS 0x10
#define PCD8544_SETVOP 0x80
```

```

// Display presets
#define LCD_BIAS                                0x03 // Range: 0-7 (0x00-0x07)
#define LCD_TEMP                                0x02 // Range: 0-3 (0x00-0x03)
#define LCD_CONTRAST                            0x46 // Range: 0-127 (0x00-0x7F)

#define fontbyte(x) cfont.font[x]
#define bitmapbyte(x) bitmap[x]

extern unsigned char TinyFont[];
extern unsigned char SmallFont[];

#define byte      char

struct _current_font
{
    uint8_t* font;
    uint8_t x_size;
    uint8_t y_size;
    uint8_t offset;
    uint8_t numchars;
    uint8_t inverted;
};

class LCD5110
{
public:
    LCD5110 (GPIO_TypeDef *_SPI_Port,int SCK, int MOSI,
GPIO_TypeDef *_LCD_Port, int DC, int RST, int CS);
    void InitLCD(int contrast=LCD_CONTRAST);
    void setContrast(int contrast);
    void enableSleep();
    void disableSleep();
    void update();
    void clrScr();
    void fillScr();
    void invert(bool mode);
    void setPixel(uint16_t x, uint16_t y);

```

```

        void clrPixel(uint16_t x, uint16_t y);
        void invPixel(uint16_t x, uint16_t y);
        void invertText(bool mode);
        void print(char *st, int x, int y);
        void print(const char *st, int x, int y);
//        void print(String st, int x, int y);
        void printNumI(long num, int x, int y, int length=0, char
filler=' ');
        void printNumF(double num, byte dec, int x, int y, char
divider='.', int length=0, char filler=' ');
        void setFont(uint8_t* font);
        void drawBitmap(int x, int y, uint8_t* bitmap, int sx, int
sy);

        void drawLine(int x1, int y1, int x2, int y2);
        void clrLine(int x1, int y1, int x2, int y2);
        void drawRect(int x1, int y1, int x2, int y2);
        void clrRect(int x1, int y1, int x2, int y2);
        void drawRoundRect(int x1, int y1, int x2, int y2);
        void clrRoundRect(int x1, int y1, int x2, int y2);
        void drawCircle(int x, int y, int radius);
        void clrCircle(int x, int y, int radius);

protected:
        GPIO_TypeDef      *SPI_Port, *LCD_Port;
        int                Pin_MOSI, Pin_SCK, Pin_CD, Pin_RST,
Pin_CS;
        _current_font      cfont;
        uint8_t            scrbuf[504];
        bool               _sleep;
        int                _contrast;

        int abs (int x);
        void _LCD_Write(unsigned char data, unsigned char mode);
        void _print_char(unsigned char c, int x, int row);
        void _convert_float(char *buf, double num, int width, byte
prec);

        void drawHLine(int x, int y, int l);
        void clrHLine(int x, int y, int l);
        void drawVLine(int x, int y, int l);
        void clrVLine(int x, int y, int l);
};

#endif

```

LCD5110_Graph.cpp

```
#include "LCD5110_Graph.h"

LCD5110::LCD5110(GPIO_TypeDef *_SPI_Port,int SCK, int MOSI,
GPIO_TypeDef *_LCD_Port, int DC, int RST, int CS)
{
    SPI_Port = _SPI_Port;
    LCD_Port = _LCD_Port;
    Pin_CD = DC;
    Pin_CS = CS;
    Pin_MOSI = MOSI;
    Pin_RST = RST;
    Pin_SCK = SCK;
}

void LCD5110::_convert_float(char *buf, double num, int width, byte
prec)
{
    char format[10];

    sprintf(format, "%%i.%%if", width, prec);
    sprintf(buf, format, num);
}

int LCD5110::abs (int x){
    if(x < 0){
        return (-x);
    }
    return (x);
}

void LCD5110::_LCD_Write(unsigned char data, unsigned char mode)
{
    GPIO_ResetBits(LCD_Port, Pin_CS);

    if (mode==LCD_COMMAND)
        GPIO_ResetBits(LCD_Port, Pin_CD);
    else
        GPIO_SetBits(LCD_Port, Pin_CD);
}
```

```

        Fn_SPIx_Transfer(data);
        GPIO_SetBits(LCD_Port, Pin_CS);
    }

void LCD5110::InitLCD(int contrast)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO | RCC_APB2Periph_GPIOA
| RCC_APB2Periph_GPIOE, ENABLE);
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0 | GPIO_Pin_1 | GPIO_Pin_2 |
GPIO_Pin_3;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_10MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_8 | GPIO_Pin_9 | GPIO_Pin_10
| GPIO_Pin_11;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_2MHz;
    GPIO_Init(GPIOE, &GPIO_InitStructure);

    Fn_SPI_Init();
    Fn_DELAY_ms(10);
    Fn_SPIx_Transfer(0x00);

    if (contrast>0x7F)
        contrast=0x7F;
    if (contrast<0)
        contrast=0;

    //resetLCD;
    GPIO_ResetBits(LCD_Port, Pin_RST);
    Fn_DELAY_ms(100);
    GPIO_SetBits(LCD_Port, Pin_RST);
    Fn_DELAY_ms(100);

    _LCD_Write(PCD8544_FUNCTIONSET | PCD8544_EXTENDEDINSTRUCTION,
LCD_COMMAND);
    _LCD_Write(PCD8544_SETVOP | contrast, LCD_COMMAND);
    _LCD_Write(PCD8544_SETTEMP | LCD_TEMP, LCD_COMMAND);
    _LCD_Write(PCD8544_SETBIAS | LCD_BIAS, LCD_COMMAND);
    _LCD_Write(PCD8544_FUNCTIONSET, LCD_COMMAND);
    _LCD_Write(PCD8544_SETYADDR, LCD_COMMAND);

```

```

        _LCD_Write(PCD8544_SETXADDR, LCD_COMMAND);
        _LCD_Write(PCD8544_DISPLAYCONTROL | PCD8544_DISPLAYNORMAL,
LCD_COMMAND);

        clrScr();
        update();
        cfont.font=0;
        _sleep=false;
        _contrast=contrast;
    }

void LCD5110::setContrast(int contrast)
{
    if (contrast>0x7F)
        contrast=0x7F;
    if (contrast<0)
        contrast=0;
    _LCD_Write(PCD8544_FUNCTIONSET | PCD8544_EXTENDEDINSTRUCTION,
LCD_COMMAND);
    _LCD_Write(PCD8544_SETVOP | contrast, LCD_COMMAND);
    _LCD_Write(PCD8544_FUNCTIONSET, LCD_COMMAND);
    _contrast=contrast;
}

void LCD5110::enableSleep()
{
    _sleep = true;
    _LCD_Write(PCD8544_SETYADDR, LCD_COMMAND);
    _LCD_Write(PCD8544_SETXADDR, LCD_COMMAND);
    for (int b=0; b<504; b++)
        _LCD_Write(0, LCD_DATA);
    _LCD_Write(PCD8544_FUNCTIONSET | PCD8544_POWERDOWN, LCD_COMMAND);
}

void LCD5110::disableSleep()
{
    _sleep = false;
    _LCD_Write(PCD8544_FUNCTIONSET | PCD8544_EXTENDEDINSTRUCTION,
LCD_COMMAND);
    _LCD_Write(PCD8544_SETVOP | _contrast, LCD_COMMAND);
    _LCD_Write(PCD8544_SETTEMP | LCD_TEMP, LCD_COMMAND);
    _LCD_Write(PCD8544_SETBIAS | LCD_BIAS, LCD_COMMAND);
    _LCD_Write(PCD8544_FUNCTIONSET, LCD_COMMAND);
}

```



```

        _LCD_Write(PCD8544_DISPLAYCONTROL | PCD8544_DISPLAYNORMAL,
LCD_COMMAND);
        update();
    }

void LCD5110::update()
{
    if (_sleep==false)
    {
        _LCD_Write(PCD8544_SETYADDR, LCD_COMMAND);
        _LCD_Write(PCD8544_SETXADDR, LCD_COMMAND);
        for (int b=0; b<504; b++)
            _LCD_Write(scrbuf[b], LCD_DATA);
    }
}

void LCD5110::clrScr()
{
    for (int c=0; c<504; c++)
        scrbuf[c]=0;
}

void LCD5110::fillScr()
{
    for (int c=0; c<504; c++)
        scrbuf[c]=255;
}

void LCD5110::invert(bool mode)
{
    if (mode==true)
        _LCD_Write(PCD8544_DISPLAYCONTROL |
PCD8544_DISPLAYINVERTED, LCD_COMMAND);
    else
        _LCD_Write(PCD8544_DISPLAYCONTROL | PCD8544_DISPLAYNORMAL,
LCD_COMMAND);
}

void LCD5110::setPixel(uint16_t x, uint16_t y)
{
    int by, bi;

    if ((x>=0) and (x<84) and (y>=0) and (y<48))

```

```

    {
        by=((y/8)*84)+x;
        bi=y % 8;

        scrbuf[by]=scrbuf[by] | (1<<bi);
    }
}

void LCD5110::clrPixel(uint16_t x, uint16_t y)
{
    int by, bi;

    if ((x>=0) and (x<84) and (y>=0) and (y<48))
    {
        by=((y/8)*84)+x;
        bi=y % 8;

        scrbuf[by]=scrbuf[by] & ~(1<<bi);
    }
}

void LCD5110::invPixel(uint16_t x, uint16_t y)
{
    int by, bi;

    if ((x>=0) and (x<84) and (y>=0) and (y<48))
    {
        by=((y/8)*84)+x;
        bi=y % 8;

        if ((scrbuf[by] & (1<<bi))==0)
            scrbuf[by]=scrbuf[by] | (1<<bi);
        else
            scrbuf[by]=scrbuf[by] & ~(1<<bi);
    }
}

void LCD5110::invertText(bool mode)
{
    if (mode==true)
        cfont.inverted=1;
    else
        cfont.inverted=0;
}

```

```

}

void LCD5110::print(char *st, int x, int y)
{
    unsigned char ch;
    int stl;

    stl = strlen(st);
    if (x == RIGHT)
        x = 84-(stl*cfont.x_size);
    if (x == CENTER)
        x = (84-(stl*cfont.x_size))/2;

    for (int cnt=0; cnt<stl; cnt++)
        _print_char(*st++, x + (cnt*(cfont.x_size)), y);
}

void LCD5110::print(const char *st, int x, int y)
{
    unsigned char ch;
    int stl;

    stl = strlen(st);
    if (x == RIGHT)
        x = 84-(stl*cfont.x_size);
    if (x == CENTER)
        x = (84-(stl*cfont.x_size))/2;

    for (int cnt=0; cnt<stl; cnt++)
        _print_char(*st++, x + (cnt*(cfont.x_size)), y);
}

void LCD5110::printNumI(long num, int x, int y, int length, char
filler)
{
    char buf[25];
    char st[27];
    bool neg=false;
    int c=0, f=0;

    if (num==0)
    {

```

```

        if (length!=0)
        {
            for (c=0; c<(length-1); c++)
                st[c]=filler;
            st[c]=48;
            st[c+1]=0;
        }
        else
        {
            st[0]=48;
            st[1]=0;
        }
    }
    else
    {
        if (num<0)
        {
            neg=true;
            num=-num;
        }

        while (num>0)
        {
            buf[c]=48+(num % 10);
            c++;
            num=(num-(num % 10))/10;
        }
        buf[c]=0;

        if (neg)
        {
            st[0]=45;
        }

        if (length>(c+neg))
        {
            for (int i=0; i<(length-c-neg); i++)
            {
                st[i+neg]=filler;
                f++;
            }
        }
    }

```

```

        for (int i=0; i<c; i++)
        {
            st[i+neg+f]=buf[c-i-1];
        }
        st[c+neg+f]=0;

    }

    print(st,x,y);
}

void LCD5110::printNumF(double num, byte dec, int x, int y, char
divider, int length, char filler)
{
    char st[27];
    bool neg=false;

    if (num<0)
        neg = true;

    _convert_float(st, num, length, dec);

    if (divider != '.')
    {
        for (int i=0; i<sizeof(st); i++)
            if (st[i]=='.')
                st[i]=divider;
    }

    if (filler != ' ')
    {
        if (neg)
        {
            st[0]='-';
            for (int i=1; i<sizeof(st); i++)
                if ((st[i]==' ') || (st[i]=='-'))
                    st[i]=filler;
        }
        else
        {
            for (int i=0; i<sizeof(st); i++)
                if (st[i]==' ')
                    st[i]=filler;
        }
    }
}

```

```

    }
}

print(st,x,y);
}

void LCD5110::_print_char(unsigned char c, int x, int y)
{
    if ((cfont.y_size % 8) == 0)
    {
        int font_idx = ((c -
cfont.offset)*(cfont.x_size*(cfont.y_size/8)))+4;
        for (int rowcnt=0; rowcnt<(cfont.y_size/8); rowcnt++)
        {
            for(int cnt=0; cnt<cfont.x_size; cnt++)
            {
                for (int b=0; b<8; b++)
                {
                    if
((fontbyte(font_idx+cnt+(rowcnt*cfont.x_size)) & (1<<b))!=0)
                    if (cfont.inverted==0)
                        setPixel(x+cnt,
y+(rowcnt*8)+b);
                    else
                        clrPixel(x+cnt,
y+(rowcnt*8)+b);
                    else
                        if (cfont.inverted==0)
                            clrPixel(x+cnt,
y+(rowcnt*8)+b);
                        else
                            setPixel(x+cnt,
y+(rowcnt*8)+b);
                }
            }
        }
    }
    else
    {
        int font_idx = ((c -
cfont.offset)*((cfont.x_size*cfont.y_size/8)))+4;
        int cbyte=fontbyte(font_idx);
        int cbit=7;
        for (int cx=0; cx<cfont.x_size; cx++)
        {

```

```

        for (int cy=0; cy<cfont.y_size; cy++)
        {
            if ((cbyte & (1<<cbit)) != 0)
                if (cfont.inverted==0)
                    setPixel(x+cx, y+cy);
                else
                    clrPixel(x+cx, y+cy);
            else
                if (cfont.inverted==0)
                    clrPixel(x+cx, y+cy);
                else
                    setPixel(x+cx, y+cy);
            cbit--;
            if (cbit<0)
            {
                cbit=7;
                font_idx++;
                cbyte=fontbyte(font_idx);
            }
        }
    }
}

void LCD5110::setFont(uint8_t* font)
{
    cfont.font=font;
    cfont.x_size=fontbyte(0);
    cfont.y_size=fontbyte(1);
    cfont.offset=fontbyte(2);
    cfont.numchars=fontbyte(3);
    cfont.inverted=0;
}

void LCD5110::drawHLine(int x, int y, int l)
{
    int by, bi;

    if ((x>=0) and (x<84) and (y>=0) and (y<48))
    {
        for (int cx=0; cx<l; cx++)
        {
            by=((y/8)*84)+x;

```

```

        bi=y % 8;

        scrbuf[by+cx] |= (1<<bi);
    }
}

void LCD5110::clrHLine(int x, int y, int l)
{
    int by, bi;

    if ((x>=0) and (x<84) and (y>=0) and (y<48))
    {
        for (int cx=0; cx<l; cx++)
        {
            by=((y/8)*84)+x;
            bi=y % 8;

            scrbuf[by+cx] &= ~(1<<bi);
        }
    }
}

void LCD5110::drawVLine(int x, int y, int l)
{
    int by, bi;

    if ((x>=0) and (x<84) and (y>=0) and (y<48))
    {
        for (int cy=0; cy<l; cy++)
        {
            setPixel(x, y+cy);
        }
    }
}

void LCD5110::clrVLine(int x, int y, int l)
{
    int by, bi;

    if ((x>=0) and (x<84) and (y>=0) and (y<48))
    {
        for (int cy=0; cy<l; cy++)

```



```

        {
            clrPixel(x, y+cy);
        }
    }
}

void LCD5110::drawLine(int x1, int y1, int x2, int y2)
{
    int tmp;
    double delta, tx, ty;
    double m, b, dx, dy;

    if (((x2-x1)<0))
    {
        tmp=x1;
        x1=x2;
        x2=tmp;
        tmp=y1;
        y1=y2;
        y2=tmp;
    }
    if (((y2-y1)<0))
    {
        tmp=x1;
        x1=x2;
        x2=tmp;
        tmp=y1;
        y1=y2;
        y2=tmp;
    }

    if (y1==y2)
    {
        if (x1>x2)
        {
            tmp=x1;
            x1=x2;
            x2=tmp;
        }
        drawHLine(x1, y1, x2-x1);
    }
    else if (x1==x2)
    {

```

```

        if (y1>y2)
        {
            tmp=y1;
            y1=y2;
            y2=tmp;
        }
        drawVLine(x1, y1, y2-y1);
    }
    else if (abs(x2-x1)>abs(y2-y1))
    {
        delta=(double(y2-y1)/double(x2-x1));
        ty=double(y1);
        if (x1>x2)
        {
            for (int i=x1; i>=x2; i--)
            {
                setPixel(i, int(ty+0.5));
            }
            ty=ty-delta;
        }
        else
        {
            for (int i=x1; i<=x2; i++)
            {
                setPixel(i, int(ty+0.5));
            }
            ty=ty+delta;
        }
    }
    else
    {
        delta=(float(x2-x1)/float(y2-y1));
        tx=float(x1);
        if (y1>y2)
        {
            for (int i=y2+1; i>y1; i--)
            {
                setPixel(int(tx+0.5), i);
            }
            tx=tx+delta;
        }
        else
        {

```

```

        for (int i=y1; i<y2+1; i++)
        {
            setPixel(int(tx+0.5), i);
            tx=tx+delta;
        }
    }

}

void LCD5110::clrLine(int x1, int y1, int x2, int y2)
{
    int tmp;
    double delta, tx, ty;
    double m, b, dx, dy;

    if (((x2-x1)<0))
    {
        tmp=x1;
        x1=x2;
        x2=tmp;
        tmp=y1;
        y1=y2;
        y2=tmp;
    }
    if (((y2-y1)<0))
    {
        tmp=x1;
        x1=x2;
        x2=tmp;
        tmp=y1;
        y1=y2;
        y2=tmp;
    }

    if (y1==y2)
    {
        if (x1>x2)
        {
            tmp=x1;
            x1=x2;
            x2=tmp;
        }
    }
}

```

```

        clrHLine(x1, y1, x2-x1);
    }
    else if (x1==x2)
    {
        if (y1>y2)
        {
            tmp=y1;
            y1=y2;
            y2=tmp;
        }
        clrVLine(x1, y1, y2-y1);
    }
    else if (abs(x2-x1)>abs(y2-y1))
    {
        delta=(double(y2-y1)/double(x2-x1));
        ty=double(y1);
        if (x1>x2)
        {
            for (int i=x1; i>=x2; i--)
            {
                clrPixel(i, int(ty+0.5));
            }
            ty=ty-delta;
        }
        else
        {
            for (int i=x1; i<=x2; i++)
            {
                clrPixel(i, int(ty+0.5));
            }
            ty=ty+delta;
        }
    }
    else
    {
        delta=(float(x2-x1)/float(y2-y1));
        tx=float(x1);
        if (y1>y2)
        {
            for (int i=y2+1; i>y1; i--)
            {
                clrPixel(int(tx+0.5), i);
            }
            tx=tx+delta;
        }
    }
}

```

```

    }
}
else
{
    for (int i=y1; i<y2+1; i++)
    {
        clrPixel(int(tx+0.5), i);
        tx=tx+delta;
    }
}
}

void LCD5110::drawRect(int x1, int y1, int x2, int y2)
{
    int tmp;

    if (x1>x2)
    {
        tmp=x1;
        x1=x2;
        x2=tmp;
    }
    if (y1>y2)
    {
        tmp=y1;
        y1=y2;
        y2=tmp;
    }

    drawHLine(x1, y1, x2-x1);
    drawHLine(x1, y2, x2-x1);
    drawVLine(x1, y1, y2-y1);
    drawVLine(x2, y1, y2-y1+1);
}

void LCD5110::clrRect(int x1, int y1, int x2, int y2)
{
    int tmp;

    if (x1>x2)
    {

```

```

        tmp=x1;
        x1=x2;
        x2=tmp;
    }
    if (y1>y2)
    {
        tmp=y1;
        y1=y2;
        y2=tmp;
    }

    clrHLine(x1, y1, x2-x1);
    clrHLine(x1, y2, x2-x1);
    clrVLine(x1, y1, y2-y1);
    clrVLine(x2, y1, y2-y1+1);
}

void LCD5110::drawRoundRect(int x1, int y1, int x2, int y2)
{
    int tmp;

    if (x1>x2)
    {
        tmp=x1;
        x1=x2;
        x2=tmp;
    }
    if (y1>y2)
    {
        tmp=y1;
        y1=y2;
        y2=tmp;
    }
    if ((x2-x1)>4 && (y2-y1)>4)
    {
        setPixel(x1+1,y1+1);
        setPixel(x2-1,y1+1);
        setPixel(x1+1,y2-1);
        setPixel(x2-1,y2-1);
        drawHLine(x1+2, y1, x2-x1-3);
        drawHLine(x1+2, y2, x2-x1-3);
        drawVLine(x1, y1+2, y2-y1-3);
        drawVLine(x2, y1+2, y2-y1-3);
    }
}

```

```

    }
}

void LCD5110::clrRoundRect(int x1, int y1, int x2, int y2)
{
    int tmp;

    if (x1>x2)
    {
        tmp=x1;
        x1=x2;
        x2=tmp;
    }
    if (y1>y2)
    {
        tmp=y1;
        y1=y2;
        y2=tmp;
    }
    if ((x2-x1)>4 && (y2-y1)>4)
    {
        clrPixel(x1+1,y1+1);
        clrPixel(x2-1,y1+1);
        clrPixel(x1+1,y2-1);
        clrPixel(x2-1,y2-1);
        clrHLine(x1+2, y1, x2-x1-3);
        clrHLine(x1+2, y2, x2-x1-3);
        clrVLine(x1, y1+2, y2-y1-3);
        clrVLine(x2, y1+2, y2-y1-3);
    }
}

void LCD5110::drawCircle(int x, int y, int radius)
{
    int f = 1 - radius;
    int ddF_x = 1;
    int ddF_y = -2 * radius;
    int x1 = 0;
    int y1 = radius;
    char ch, cl;

    setPixel(x, y + radius);
    setPixel(x, y - radius);

```

```

        setPixel(x + radius, y);
        setPixel(x - radius, y);

    while(x1 < y1)
    {
        if(f >= 0)
        {
            y1--;
            ddF_y += 2;
            f += ddF_y;
        }
        x1++;
        ddF_x += 2;
        f += ddF_x;
        setPixel(x + x1, y + y1);
        setPixel(x - x1, y + y1);
        setPixel(x + x1, y - y1);
        setPixel(x - x1, y - y1);
        setPixel(x + y1, y + x1);
        setPixel(x - y1, y + x1);
        setPixel(x + y1, y - x1);
        setPixel(x - y1, y - x1);
    }
}

void LCD5110::clrCircle(int x, int y, int radius)
{
    int f = 1 - radius;
    int ddF_x = 1;
    int ddF_y = -2 * radius;
    int x1 = 0;
    int y1 = radius;
    char ch, cl;

    clrPixel(x, y + radius);
    clrPixel(x, y - radius);
    clrPixel(x + radius, y);
    clrPixel(x - radius, y);

    while(x1 < y1)
    {
        if(f >= 0)
        {

```



```

        y1--;
        ddF_y += 2;
        f += ddF_y;
    }
    x1++;
    ddF_x += 2;
    f += ddF_x;
    clrPixel(x + x1, y + y1);
    clrPixel(x - x1, y + y1);
    clrPixel(x + x1, y - y1);
    clrPixel(x - x1, y - y1);
    clrPixel(x + y1, y + x1);
    clrPixel(x - y1, y + x1);
    clrPixel(x + y1, y - x1);
    clrPixel(x - y1, y - x1);
}
}

void LCD5110::drawBitmap(int x, int y, uint8_t* bitmap, int sx, int sy)
{
    int bit;
    byte data;

    for (int cy=0; cy<sy; cy++)
    {
        bit= cy % 8;
        for(int cx=0; cx<sx; cx++)
        {
            data=bitmapbyte(cx+((cy/8)*sx));
            if ((data & (1<<bit))>0)
                setPixel(x+cx, y+cy);
            else
                clrPixel(x+cx, y+cy);
        }
    }
}

```

DefaultFonts.c

```

unsigned char SmallFont[] =
{
    0x06, 0x08, 0x20, 0x5f,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // sp
    0x00, 0x00, 0x00, 0x2f, 0x00, 0x00, // !

```

0x00, 0x00, 0x07, 0x00, 0x07, 0x00, // "
0x00, 0x14, 0x7f, 0x14, 0x7f, 0x14, // #
0x00, 0x24, 0x2a, 0x7f, 0x2a, 0x12, // \$
0x00, 0x23, 0x13, 0x08, 0x64, 0x62, // %
0x00, 0x36, 0x49, 0x55, 0x22, 0x50, // &
0x00, 0x00, 0x05, 0x03, 0x00, 0x00, // '
0x00, 0x00, 0x1c, 0x22, 0x41, 0x00, // (
0x00, 0x00, 0x41, 0x22, 0x1c, 0x00, //)
0x00, 0x14, 0x08, 0x3E, 0x08, 0x14, // *
0x00, 0x08, 0x08, 0x3E, 0x08, 0x08, // +
0x00, 0x00, 0x00, 0xA0, 0x60, 0x00, // ,
0x00, 0x08, 0x08, 0x08, 0x08, 0x08, // -
0x00, 0x00, 0x60, 0x60, 0x00, 0x00, // .
0x00, 0x20, 0x10, 0x08, 0x04, 0x02, // /

0x00, 0x3E, 0x51, 0x49, 0x45, 0x3E, // 0
0x00, 0x00, 0x42, 0x7F, 0x40, 0x00, // 1
0x00, 0x42, 0x61, 0x51, 0x49, 0x46, // 2
0x00, 0x21, 0x41, 0x45, 0x4B, 0x31, // 3
0x00, 0x18, 0x14, 0x12, 0x7F, 0x10, // 4
0x00, 0x27, 0x45, 0x45, 0x45, 0x39, // 5
0x00, 0x3C, 0x4A, 0x49, 0x49, 0x30, // 6
0x00, 0x01, 0x71, 0x09, 0x05, 0x03, // 7
0x00, 0x36, 0x49, 0x49, 0x49, 0x36, // 8
0x00, 0x06, 0x49, 0x49, 0x29, 0x1E, // 9
0x00, 0x00, 0x36, 0x36, 0x00, 0x00, // :
0x00, 0x00, 0x56, 0x36, 0x00, 0x00, // ;
0x00, 0x08, 0x14, 0x22, 0x41, 0x00, // <
0x00, 0x14, 0x14, 0x14, 0x14, 0x14, // =
0x00, 0x00, 0x41, 0x22, 0x14, 0x08, // >
0x00, 0x02, 0x01, 0x51, 0x09, 0x06, // ?

0x00, 0x32, 0x49, 0x59, 0x51, 0x3E, // @
0x00, 0x7C, 0x12, 0x11, 0x12, 0x7C, // A
0x00, 0x7F, 0x49, 0x49, 0x49, 0x36, // B
0x00, 0x3E, 0x41, 0x41, 0x41, 0x22, // C
0x00, 0x7F, 0x41, 0x41, 0x22, 0x1C, // D
0x00, 0x7F, 0x49, 0x49, 0x49, 0x41, // E
0x00, 0x7F, 0x09, 0x09, 0x09, 0x01, // F
0x00, 0x3E, 0x41, 0x49, 0x49, 0x7A, // G
0x00, 0x7F, 0x08, 0x08, 0x08, 0x7F, // H
0x00, 0x00, 0x41, 0x7F, 0x41, 0x00, // I
0x00, 0x20, 0x40, 0x41, 0x3F, 0x01, // J

```

0x00, 0x7F, 0x08, 0x14, 0x22, 0x41, // K
0x00, 0x7F, 0x40, 0x40, 0x40, 0x40, // L
0x00, 0x7F, 0x02, 0x0C, 0x02, 0x7F, // M
0x00, 0x7F, 0x04, 0x08, 0x10, 0x7F, // N
0x00, 0x3E, 0x41, 0x41, 0x41, 0x3E, // O

0x00, 0x7F, 0x09, 0x09, 0x09, 0x06, // P
0x00, 0x3E, 0x41, 0x51, 0x21, 0x5E, // Q
0x00, 0x7F, 0x09, 0x19, 0x29, 0x46, // R
0x00, 0x46, 0x49, 0x49, 0x49, 0x31, // S
0x00, 0x01, 0x01, 0x7F, 0x01, 0x01, // T
0x00, 0x3F, 0x40, 0x40, 0x40, 0x3F, // U
0x00, 0x1F, 0x20, 0x40, 0x20, 0x1F, // V
0x00, 0x3F, 0x40, 0x38, 0x40, 0x3F, // W
0x00, 0x63, 0x14, 0x08, 0x14, 0x63, // X
0x00, 0x07, 0x08, 0x70, 0x08, 0x07, // Y
0x00, 0x61, 0x51, 0x49, 0x45, 0x43, // Z
0x00, 0x00, 0x7F, 0x41, 0x41, 0x00, // [
0xAA, 0x55, 0xAA, 0x55, 0xAA, 0x55, // Backslash (Checker pattern)
0x00, 0x00, 0x41, 0x41, 0x7F, 0x00, // ]
0x00, 0x04, 0x02, 0x01, 0x02, 0x04, // ^
0x00, 0x40, 0x40, 0x40, 0x40, 0x40, // _

0x00, 0x00, 0x03, 0x05, 0x00, 0x00, // `
0x00, 0x20, 0x54, 0x54, 0x54, 0x78, // a
0x00, 0x7F, 0x48, 0x44, 0x44, 0x38, // b
0x00, 0x38, 0x44, 0x44, 0x44, 0x20, // c
0x00, 0x38, 0x44, 0x44, 0x48, 0x7F, // d
0x00, 0x38, 0x54, 0x54, 0x54, 0x18, // e
0x00, 0x08, 0x7E, 0x09, 0x01, 0x02, // f
0x00, 0x18, 0xA4, 0xA4, 0xA4, 0x7C, // g
0x00, 0x7F, 0x08, 0x04, 0x04, 0x78, // h
0x00, 0x00, 0x44, 0x7D, 0x40, 0x00, // i
0x00, 0x40, 0x80, 0x84, 0x7D, 0x00, // j
0x00, 0x7F, 0x10, 0x28, 0x44, 0x00, // k
0x00, 0x00, 0x41, 0x7F, 0x40, 0x00, // l
0x00, 0x7C, 0x04, 0x18, 0x04, 0x78, // m
0x00, 0x7C, 0x08, 0x04, 0x04, 0x78, // n
0x00, 0x38, 0x44, 0x44, 0x44, 0x38, // o

0x00, 0xFC, 0x24, 0x24, 0x24, 0x18, // p
0x00, 0x18, 0x24, 0x24, 0x18, 0xFC, // q
0x00, 0x7C, 0x08, 0x04, 0x04, 0x08, // r

```

```

0x00, 0x48, 0x54, 0x54, 0x54, 0x20, // s
0x00, 0x04, 0x3F, 0x44, 0x40, 0x20, // t
0x00, 0x3C, 0x40, 0x40, 0x20, 0x7C, // u
0x00, 0x1C, 0x20, 0x40, 0x20, 0x1C, // v
0x00, 0x3C, 0x40, 0x30, 0x40, 0x3C, // w
0x00, 0x44, 0x28, 0x10, 0x28, 0x44, // x
0x00, 0x1C, 0xA0, 0xA0, 0xA0, 0x7C, // y
0x00, 0x44, 0x64, 0x54, 0x4C, 0x44, // z
0x00, 0x00, 0x10, 0x7C, 0x82, 0x00, // {
0x00, 0x00, 0x00, 0xFF, 0x00, 0x00, // |
0x00, 0x00, 0x82, 0x7C, 0x10, 0x00, // }
0x00, 0x00, 0x06, 0x09, 0x09, 0x06 // ~ (Degrees)
};

const unsigned char MediumNumbers[] =
{
0x0c, 0x10, 0x2d, 0x0d,
0x00, 0x00, 0x00, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x00, 0x00, 0x00,
0x00, 0x00, 0x01, 0x03, 0x03, 0x03, 0x03, 0x03, 0x03, 0x01, 0x00, 0x00,
// -
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0xc0, 0xc0, 0x00, 0x00, 0x00, 0x00, 0x00,
// .
0x00, 0x00, 0x02, 0x86, 0x86, 0x86, 0x86, 0x86, 0x86, 0x02, 0x00, 0x00,
0x00, 0x00, 0x81, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0x81, 0x00, 0x00,
// /
0x00, 0xfc, 0x7a, 0x06, 0x06, 0x06, 0x06, 0x06, 0x06, 0x7a, 0xfc, 0x00,
0x00, 0x7e, 0xbc, 0xc0, 0xc0, 0xc0, 0xc0, 0xc0, 0xc0, 0xbc, 0x7e, 0x00,
// 0
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x78, 0xfc, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x3c, 0x7e, 0x00,
// 1
0x00, 0x00, 0x02, 0x86, 0x86, 0x86, 0x86, 0x86, 0x86, 0x7a, 0xfc, 0x00,
0x00, 0x7e, 0xbd, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0x81, 0x00, 0x00,
// 2
0x00, 0x00, 0x02, 0x86, 0x86, 0x86, 0x86, 0x86, 0x86, 0x7a, 0xfc, 0x00,
0x00, 0x00, 0x81, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xbd, 0x7e, 0x00,
// 3
0x00, 0xfc, 0x78, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x78, 0xfc, 0x00,
0x00, 0x00, 0x01, 0x03, 0x03, 0x03, 0x03, 0x03, 0x03, 0x3d, 0x7e, 0x00,
// 4
0x00, 0xfc, 0x7a, 0x86, 0x86, 0x86, 0x86, 0x86, 0x86, 0x02, 0x00, 0x00,
0x00, 0x00, 0x81, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xbd, 0x7e, 0x00,

```

```

// 5
0x00, 0xfc, 0x7a, 0x86, 0x86, 0x86, 0x86, 0x86, 0x86, 0x86, 0x02, 0x00, 0x00,
0x00, 0x7e, 0xbd, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xbd, 0x7e, 0x00,
// 6
0x00, 0x00, 0x02, 0x06, 0x06, 0x06, 0x06, 0x06, 0x06, 0x06, 0x7a, 0xfc, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x3c, 0x7e, 0x00,
// 7
0x00, 0xfc, 0x7a, 0x86, 0x86, 0x86, 0x86, 0x86, 0x86, 0x86, 0x7a, 0xfc, 0x00,
0x00, 0x7e, 0xbd, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xbd, 0x7e, 0x00,
// 8
0x00, 0xfc, 0x7a, 0x86, 0x86, 0x86, 0x86, 0x86, 0x86, 0x86, 0x7a, 0xfc, 0x00,
0x00, 0x00, 0x81, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xc3, 0xbd, 0x7e, 0x00,
// 9
};

const unsigned char BigNumbers[] =
{
0x0e, 0x18, 0x2d, 0x0d,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x10, 0x38, 0x38, 0x38, 0x38, 0x38, 0x38, 0x38,
0x38, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // -
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x40, 0xe0, 0xe0,
0x40, 0x00, 0x00, 0x00, 0x00, 0x00, // .
0x00, 0x00, 0x02, 0x06, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0x06, 0x02,
0x00, 0x00, 0x00, 0x00, 0x10, 0x38, 0x38, 0x38, 0x38, 0x38, 0x38, 0x38,
0x38, 0x10, 0x00, 0x00, 0x00, 0x00, 0x80, 0xc0, 0xe0, 0xe0, 0xe0, 0xe0,
0xe0, 0xe0, 0xc0, 0x80, 0x00, 0x00, // /
0x00, 0xfc, 0xfa, 0xf6, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0xf6, 0xfa,
0xfc, 0x00, 0x00, 0xef, 0xc7, 0x83, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x83, 0xc7, 0xef, 0x00, 0x00, 0x7f, 0xbf, 0xdf, 0xe0, 0xe0, 0xe0, 0xe0,
0xe0, 0xe0, 0xdf, 0xbf, 0x7f, 0x00, // 0
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xf0, 0xf8,
0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x83, 0xc7, 0xef, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x1f, 0x3f, 0x7f, 0x00, // 1
0x00, 0x00, 0x02, 0x06, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0xf6, 0xfa,
0xfc, 0x00, 0x00, 0xe0, 0xd0, 0xb8, 0x38, 0x38, 0x38, 0x38, 0x38, 0x38,
0x3b, 0x17, 0x0f, 0x00, 0x00, 0x7f, 0xbf, 0xdf, 0xe0, 0xe0, 0xe0, 0xe0,
0xe0, 0xe0, 0xc0, 0x80, 0x00, 0x00, // 2
0x00, 0x00, 0x02, 0x06, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0xf6, 0xfa,

```

```

0xfc, 0x00, 0x00, 0x00, 0x10, 0x38, 0x38, 0x38, 0x38, 0x38, 0x38, 0x38,
0xbb, 0xd7, 0xef, 0x00, 0x00, 0x00, 0x80, 0xc0, 0xe0, 0xe0, 0xe0, 0xe0,
0xe0, 0xe0, 0xdf, 0xbf, 0x7f, 0x00, // 3
0x00, 0xfc, 0xf8, 0xf0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xf0, 0xf8,
0xfc, 0x00, 0x00, 0x0f, 0x17, 0x3b, 0x38, 0x38, 0x38, 0x38, 0x38, 0x38,
0xbb, 0xd7, 0xef, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x1f, 0x3f, 0x7f, 0x00, // 4
0x00, 0xfc, 0xfa, 0xf6, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0x06, 0x02,
0x00, 0x00, 0x00, 0x0f, 0x17, 0x3b, 0x38, 0x38, 0x38, 0x38, 0x38, 0x38,
0xb8, 0xd0, 0xe0, 0x00, 0x00, 0x00, 0x80, 0xc0, 0xe0, 0xe0, 0xe0, 0xe0,
0xe0, 0xe0, 0xdf, 0xbf, 0x7f, 0x00, // 5
0x00, 0xfc, 0xfa, 0xf6, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0x06, 0x02,
0x00, 0x00, 0x00, 0xef, 0xd7, 0xbb, 0x38, 0x38, 0x38, 0x38, 0x38, 0x38,
0xb8, 0xd0, 0xe0, 0x00, 0x00, 0x7f, 0xbf, 0xdf, 0xe0, 0xe0, 0xe0, 0xe0,
0xe0, 0xe0, 0xdf, 0xbf, 0x7f, 0x00, // 6
0x00, 0x00, 0x02, 0x06, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0xf6, 0xfa,
0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x83, 0xc7, 0xef, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x1f, 0x3f, 0x7f, 0x00, // 7
0x00, 0xfc, 0xfa, 0xf6, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0xf6, 0xfa,
0xfc, 0x00, 0x00, 0xef, 0xd7, 0xbb, 0x38, 0x38, 0x38, 0x38, 0x38, 0x38,
0xbb, 0xd7, 0xef, 0x00, 0x00, 0x7f, 0xbf, 0xdf, 0xe0, 0xe0, 0xe0, 0xe0,
0xe0, 0xe0, 0xdf, 0xbf, 0x7f, 0x00, // 8
0x00, 0xfc, 0xfa, 0xf6, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0x0e, 0xf6, 0xfa,
0xfc, 0x00, 0x00, 0x0f, 0x17, 0x3b, 0x38, 0x38, 0x38, 0x38, 0x38, 0x38,
0xbb, 0xd7, 0xef, 0x00, 0x00, 0x00, 0x80, 0xc0, 0xe0, 0xe0, 0xe0, 0xe0,
0xe0, 0xe0, 0xdf, 0xbf, 0x7f, 0x00, // 9
};

```

```

unsigned char TinyFont[] =
{
0x04, 0x06, 0x20, 0x5f,
0x00, 0x00, 0x00, 0x03, 0xa0, 0x00, 0xc0, 0x0c, 0x00, 0xf9, 0x4f, 0x80,
0x6b, 0xeb, 0x00, 0x98, 0x8c, 0x80, 0x52, 0xa5, 0x80, 0x03, 0x00, 0x00,
// Space, !"#$$%&'
0x01, 0xc8, 0x80, 0x89, 0xc0, 0x00, 0x50, 0x85, 0x00, 0x21, 0xc2, 0x00,
0x08, 0x40, 0x00, 0x20, 0x82, 0x00, 0x00, 0x20, 0x00, 0x18, 0x8c, 0x00,
// ()*+,-./
0xfa, 0x2f, 0x80, 0x4b, 0xe0, 0x80, 0x5a, 0x66, 0x80, 0x8a, 0xa5, 0x00,
0xe0, 0x8f, 0x80, 0xea, 0xab, 0x00, 0x72, 0xa9, 0x00, 0x9a, 0x8c, 0x00,
// 01234567
0xfa, 0xaf, 0x80, 0x4a, 0xa7, 0x00, 0x01, 0x40, 0x00, 0x09, 0x40, 0x00,
0x21, 0x48, 0x80, 0x51, 0x45, 0x00, 0x89, 0x42, 0x00, 0x42, 0x66, 0x00,

```

```

// 89:;<=>?
0x72, 0xa6, 0x80, 0x7a, 0x87, 0x80, 0xfa, 0xa5, 0x00, 0x72, 0x25, 0x00,
0xfa, 0x27, 0x00, 0xfa, 0xa8, 0x80, 0xfa, 0x88, 0x00, 0x72, 0x2b, 0x00,
// @ABCDEFGF
0xf8, 0x8f, 0x80, 0x8b, 0xe8, 0x80, 0x8b, 0xe8, 0x00, 0xf8, 0x8d, 0x80,
0xf8, 0x20, 0x80, 0xf9, 0x0f, 0x80, 0xf9, 0xcf, 0x80, 0x72, 0x27, 0x00,
// HIJKLMNO
0xfa, 0x84, 0x00, 0x72, 0x27, 0x40, 0xfa, 0x85, 0x80, 0x4a, 0xa9, 0x00,
0x83, 0xe8, 0x00, 0xf0, 0x2f, 0x00, 0xe0, 0x6e, 0x00, 0xf0, 0xef, 0x00,
// PQRSTUVW
0xd8, 0x8d, 0x80, 0xc0, 0xec, 0x00, 0x9a, 0xac, 0x80, 0x03, 0xe8, 0x80,
0xc0, 0x81, 0x80, 0x8b, 0xe0, 0x00, 0x42, 0x04, 0x00, 0x08, 0x20, 0x80,
// XYZ[\]^_
0x02, 0x04, 0x00, 0x31, 0x23, 0x80, 0xf9, 0x23, 0x00, 0x31, 0x24, 0x80,
0x31, 0x2f, 0x80, 0x31, 0x62, 0x80, 0x23, 0xea, 0x00, 0x25, 0x53, 0x80,
// `abcdefg
0xf9, 0x03, 0x80, 0x02, 0xe0, 0x00, 0x06, 0xe0, 0x00, 0xf8, 0x42, 0x80,
0x03, 0xe0, 0x00, 0x79, 0x87, 0x80, 0x39, 0x03, 0x80, 0x31, 0x23, 0x00,
// hijklmno
0x7d, 0x23, 0x00, 0x31, 0x27, 0xc0, 0x78, 0x84, 0x00, 0x29, 0x40, 0x00,
0x43, 0xe4, 0x00, 0x70, 0x27, 0x00, 0x60, 0x66, 0x00, 0x70, 0x67, 0x00,
// pqrstuvw
0x48, 0xc4, 0x80, 0x74, 0x57, 0x80, 0x59, 0xe6, 0x80, 0x23, 0xe8, 0x80,
0x03, 0x60, 0x00, 0x8b, 0xe2, 0x00, 0x61, 0x0c, 0x00 // zyx{|}~
};

```

TÀI LIỆU THAM KHẢO

- [1]. *Embedded Systems Architecture: A Comprehensive Guide for Engineers and Programmers*, Tammy Noergaard, Newnes, 2005.
- [2]. *Embedded Systems Design*, Steve Heath,,Second Edition, Newnes, 2002 .
- [3]. *Embedded Systems- Architecture, Programming and Design*, Raj Kamal, McGraw Hill, 2003
- [4]. *Embedded Microprocessor system*,2nd ed., Stuart R. Ball, Newnes, 2001.
- [5]. *Embedded_Microcomputer Systems: Real Time Interfacing*, 2nd Edition, ISBN 0534551629, Thomson 2006, by J. W. Valvano.
- [6]. *Embedded System Design: A unified Hardware/Software Introduction*, Vahid/Givargis, John Wiley & Sons INC, 2002.
- [7].*Co-Synthesis of Hardware and Software for Digital Embedded Systems*, G.D. Micheli, Illinois University at Urbana Champaign, 2000.
- [8].*Co-Synthesis of Hardware and Software for Digital Embedded Systems*, G.D. Micheli, Illinois University at Urbana Champaign, 2000.
- [9].*Embedded Linux System design and development*, TP. Raghavan , Amol Lad ,Sriram Neelakandan, Auerbach Publications, 2006.
- [10]. *The Insider's Guide To The Philips ARM®7 Based Microcontrollers*, Hitek, 2008.
- [11]. *Embedded Linux Primer: A Practical, Real-World Approach*, Christopher Hallinan, Prentice Hall, 2006.
- [12]. *Building Embedded Linux Systems*, Karim Yaghmour, O'Reilly, 2003